

Traceability und Evidence

Nachweisketten der Meilensteine M0 bis M9 und Audit-Belege

Die Beweisebene: Traceability-Berichte und die vollständige Evidence-Sammlung zu Meilensteinen, CI-Härtung, Supabase- und Stripe-Sicherheit, Kill-Switch-Drills und Provider-Cutovers. Jeder Eintrag belegt, dass eine geplante Massnahme tatsächlich umgesetzt und geprüft wurde.

Repository: SvenKulesa/Finance

Stand: claude/pdf-dokumente-notebook-l1m-l6kcxq @ 4eadf9d

Erzeugt am: 16.08.2026

Enthaltene Dokumente: 73

Umfang: ca. 45.821 Wörter

Dieses PDF ist eine Quelle für NotebookLM. Jedes eingebettete Dokument beginnt auf einer neuen Seite und nennt seinen Repository-Pfad, damit erzeugte Podcasts, Video-Skripte und Zusammenfassungen ihre Aussagen exakt auf die Originaldatei zurückführen können.

Inhalt dieses Quellpakets

1. P0 Main Protection Recovery — 2026-08-14	7
docs/evidence/ci/P0_MAIN_PROTECTION_RECOVERY_2026-08-14.md	
2. PR #236 Bootstrap Incident und Recovery	10
docs/evidence/ci/PR236_BOOTSTRAP_INCIDENT_2026-08-13.md	
3. Evidence — PR-Vorlagenvertrag: Claim-Evidence only (2026-08-16)	15
docs/evidence/governance/PR_TEMPLATE_CONTRACT_CLAIM_EVIDENCE_ONLY_2026-08-16.md	
4. PR-Vorlagenvertrag — Durable Evidence Fix (2026-08-16)	17
docs/evidence/governance/PR_TEMPLATE_CONTRACT_DURABLE_EVIDENCE_2026-08-16.md	
5. Domain & Threat-Reputation Evidence	19
docs/evidence/m0/DOMAIN_THREAT_REPUTATION_EVIDENCE.md	
6. GitHub Enforcement State — M0 Evidence	20
docs/evidence/m0/GITHUB_ENFORCEMENT_STATE_2026-08-10.md	
7. M0 Deep Research – Plugin Evidence	22
docs/evidence/m0/M0_DEEP_RESEARCH_PLUGIN_EVIDENCE.md	
8. M0 Exit Status — Evidence Baseline	24
docs/evidence/m0/M0_EXIT_STATUS_2026-08-10.md	
9. Plugin Evidence Traceability	26
docs/evidence/m0/PLUGIN_EVIDENCE_TRACEABILITY.md	
10. Render Deployment Evidence	28
docs/evidence/m0/RENDER_DEPLOYMENT_EVIDENCE.md	
11. SPDX-SBOM-Baseline — M0 Evidence	30
docs/evidence/m0/SBOM_BASELINE_2026-08-10.md	
12. Stripe Integration Security Evidence	32
docs/evidence/m0/STRIPE_INTEGRATION_SECURITY_EVIDENCE.md	
13. Supabase Security Evidence	33
docs/evidence/m0/SUPABASE_SECURITY_EVIDENCE.md	
14. Workflow- und Action-Inventar — M0 Evidence	34
docs/evidence/m0/WORKFLOW_ACTION_INVENTORY_2026-08-10.md	
15. M3 CI Hardening Evidence	37
docs/evidence/m3/M3_CI_HARDENING_EVIDENCE.md	
16. M3 – Docs-only Fast-Path Proof	40
docs/evidence/m3/M3_FAST_PATH_PROOF_2026-08-11.md	
17. M4 Agent IAM — Implementation Evidence	41
docs/evidence/m4/M4_AGENT_IAM_EVIDENCE.md	

18. M5 — Application Audit Integration Evidence	43
docs/evidence/m5/M5_APP_AUDIT_INTEGRATION_EVIDENCE.md	
19. M5 Observability / Telemetry / Audit — Gap Assessment	47
docs/evidence/m5/M5_OBSERVABILITY_AUDIT_GAP_ASSESSMENT.md	
20. M5 Supabase Agent Audit Mutation Evidence	49
docs/evidence/m5/M5_SUPABASE_AGENT_AUDIT_MUTATION_EVIDENCE.md	
21. M5 — Application Corrective Verification Closure Evidence	52
docs/evidence/m5/M5_VERIFIED_PASS_CLOSURE_EVIDENCE.md	
22. M5A — Repository-Implementierung: Supabase Native TOTP MFA / AAL2	55
docs/evidence/m5a/M5A_REPOSITORY_IMPLEMENTATION_EVIDENCE.md	
23. M5A — Supabase TOTP MFA / AAL2 Baseline Evidence	61
docs/evidence/m5a/M5A_SUPABASE_TOTP_AAL2_BASELINE.md	
24. M5A — Native Factor Enrollment Closure Evidence	67
docs/evidence/m5a/M5A_VERIFIED_PASS_CLOSURE_EVIDENCE.md	
25. M6 — Supply Chain Provenance: Repository Implementation Evidence	70
docs/evidence/m6/M6_REPOSITORY_IMPLEMENTATION_EVIDENCE.md	
26. M7 — Deployment Identity: Phase-0-Preflight und Repository-Controls Evidence	77
docs/evidence/m7/M7_PHASE0_AND_REPOSITORY_CONTROLS_EVIDENCE.md	
27. M7 — Deployment Identity: Required Negative Tests Evidence	84
docs/evidence/m7/M7_REQUIRED_NEGATIVE_TESTS_EVIDENCE.md	
28. M8 — Audit-Korrelation Exit-Gate-Punkt 7	89
docs/evidence/m8/M8_AUDIT_CORRELATION_EXIT_GATE_7_EVIDENCE.md	
29. M8 — Closure Evidence (COMPLETE / VERIFIED PASS)	91
docs/evidence/m8/M8_CLOSURE_EVIDENCE.md	
30. M8 Exit-Gate-Punkt 2 — Scope-Entscheidung (Owner-Proposal)	96
docs/evidence/m8/M8_EXIT_GATE_ITEM2_SCOPE_DECISION.md	
31. M8 — externalHostConfigurationVerified (chatgpt-github-connector)	100
docs/evidence/m8/M8_EXTERNAL_HOST_CONFIGURATION_VERIFIED_EVIDENCE.md	
32. M8 (Integrated Roadmap Phase I1) — Cutover-Readiness-Matrix und Exit-Gate-Sync	105
docs/evidence/m8/M8_I1_CUTOVER_READINESS_MATRIX_AND_EXIT_GATE_SYNC_EVIDENCE.md	
33. M8 — Provider-neutral Agent Cutover: Phase-0-Preflight und Provider-Profile-Paket Evidence	109
docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md	
34. M8 — Provider-spezifische Bypass-Route: Vollständiges Router-Audit (Exit-Gate-Punkt 4)	116
docs/evidence/m8/M8_PROVIDER_BYPASS_ROUTE_AUDIT_EXIT_GATE_4_EVIDENCE.md	
35. M8 — Provider Cutover Readiness Gate (ohne Gemini-Runtime)	120
docs/evidence/m8/M8_PROVIDER_CUTOVER_READINESS_NO_GEMINI_EVIDENCE.md	
36. M8 Provider Set Correction — Owner 2026-08-16	122

docs/evidence/m8/M8_PROVIDER_SET_CORRECTION_2026-08-16.md

37. M8 SA-P05 – Cutover-Simulator Evidence 124

docs/evidence/m8/M8_SA_P05_CUTOVER_SIMULATOR_EVIDENCE.md

38. M9 — I2 Entry: Read-only Inventory & Gap Analysis (2026-08-16) 125

docs/evidence/m9/M9_I2_ENTRY_INVENTORY_AND_GAP_ANALYSIS_2026-08-16.md

39. M9 — Kill-Switch Live-Drill Evidence (2026-08-16) 130

docs/evidence/m9/M9_KILL_SWITCH_LIVE_DRILL_2026-08-16.md

40. P2-1 — Datenqualitätsmetriken: Read-only Bestandsaufnahme (SA-P07 Vorstufe) 133

docs/evidence/p2-1/P2_1_SA_P07_DATA_QUALITY_LINEAGE_INVENTORY.md

41. R-101 — Outbox-Produktionsmigration Evidence 143

docs/evidence/r101/R101_OUTBOX_PRODUCTION_MIGRATION_EVIDENCE.md

42. SA1 — REM Validator / Control-Plane Enforcement Evidence 146

docs/evidence/sa1/SA1_REM_VALIDATOR_EVIDENCE.md

43. SA2 Chat Execution Profile Evidence 151

docs/evidence/sa2/SA2_CHAT_EXECUTION_PROFILE_EVIDENCE.md

44. SA3 Systemadmin Audit Correlation Evidence 156

docs/evidence/sa3/SA3_SYSTEMADMIN_AUDIT_CORRELATION_EVIDENCE.md

45. SA3B Execution Host Binding Evidence 160

docs/evidence/sa3b/SA3B_EXECUTION_HOST_BINDING_EVIDENCE.md

46. SA4 Bootstrap Evidence 165

docs/evidence/sa4/SA4_BOOTSTRAP_EVIDENCE.md

47. SA4 First Autonomous Work Package Evidence 170

docs/evidence/sa4/SA4_FIRST_AUTONOMOUS_WORK_PACKAGE.md

48. Evidence — SC-1 Orchestrator Wiring 171

docs/evidence/sc-md/SC1_ORCHESTRATOR_WIRING_2026-08-15.md

49. SC-1 / SC-3 Baseline Inventory — 2026-08-15 173

docs/evidence/sc-md/SC1_SC3_BASELINE_INVENTORY_2026-08-15.md

50. Evidence — SC-1 ClassificationService Table Expansion 175

docs/evidence/sc-md/SC1_TABLE_EXPANSION_2026-08-15.md

51. Evidence — SC-4 Phase A Gateway Hardening & Provider Matrix 177

docs/evidence/sc-md/SC4_GATEWAY_HARDENING_PROVIDER_MATRIX_2026-08-16.md

52. Evidence — SC-5 Live Coverage Matrix 179

docs/evidence/sc-md/SC5_LIVE_COVERAGE_MATRIX_2026-08-16.md

53. Evidence — SC-5 Phase B Multi-Field Matrix Guards 181

docs/evidence/sc-md/SC5_PHASE_B_MULTIFIELD_MATRIX_GUARDS_2026-08-16.md

54. Evidence — SC-7 Phase B Orchestrator Composite Wiring 183

docs/evidence/sc-md/SC7_PHASE_B_ORCHESTRATOR_WIRING_2026-08-16.md

55. Evidence — SC-7 Phase A Ranking Composite Opt-In	185
docs/evidence/sc-md/SC7_RANKING_COMPOSITE_OPT_IN_2026-08-16.md	
56. GitGuardian-/Snyk-Integrationsbaseline — 2026-08-14	186
docs/evidence/security/GITGUARDIAN_SNYK_BASELINE_2026-08-14.md	
57. Sicherheitsaudit 2026-08-14 — ADR-0069 Trust Root, DevelopmentChain und Systemadmin-Prototyp	189
docs/evidence/security/SECURITY_AUDIT_2026-08-14_ADR0069_DEVELOPMENT_CHAIN.md	
58. Snyk Required-Check-Demotion Evidence	201
docs/evidence/security/SNYK_REQUIRED_CHECK_DEMOTION_2026-08-15.md	
59. Evidence — D3 Soft-404 Production Gap (2026-08-16)	203
docs/evidence/seo/D3_SOFT_404_PROD_GAP_2026-08-16.md	
60. Systemadmin Work-Package Catalog — Generalization Proof	205
docs/evidence/systemadmin-work-packages/WORK_PACKAGE_GENERALIZATION_PROOF.md	
61. DEVELOPMENT Chain Phase Evidence Template	206
docs/evidence/templates/DEVELOPMENT_CHAIN_PHASE_EVIDENCE_TEMPLATE.md	
62. AI Agent M0-M10 Traceability Matrix	211
docs/traceability/AI_AGENT_M0_M9_TRACEABILITY_MATRIX.md	
63. Traceability Coverage Report	221
docs/traceability/COVERAGE_REPORT.md	
64. DEVELOPMENT Chain Document Traceability Matrix	224
docs/traceability/DEVELOPMENT_CHAIN_DOCUMENT_TRACEABILITY_MATRIX.md	
65. Enterprise Traceability Matrix — Übersicht	233
docs/traceability/ETM.md	
66. Marketing Agent Traceability & Gate Matrix	236
docs/traceability/MARKETING_AGENT_TRACEABILITY_MATRIX.md	
67. Systemadmin Agent Traceability Matrix	242
docs/traceability/SYSTEMADMIN_AGENT_TRACEABILITY_MATRIX.md	
68. Traceability Architecture — Betriebssicht	249
docs/traceability/Traceability-Architecture.md	
69. Traceability Governance	252
docs/traceability/Traceability-Governance.md	
70. Traceability Lifecycle	254
docs/traceability/Traceability-Lifecycle.md	
71. Traceability Reports	256
docs/traceability/Traceability-Reports.md	
72. Traceability Validation	258
docs/traceability/Traceability-Validation.md	

73. Traceability Versioning

260

`docs/traceability/Traceability-Versioning.md`

P0 Main Protection Recovery — 2026-08-14

Quelle: docs/evidence/ci/P0_MAIN_PROTECTION_RECOVERY_2026-08-14.md

P0 Main Protection Recovery — 2026-08-14

Status: VERIFIED PASS — RETIREMENT AUTHORIZED 2026-08-15 Repository: `SvenKulesa/Finance` Baseline: `main@3fb7e668ab997909b2c34a020f08d701dc7a3409` Authority: `ADR-0069-human-owner-comment-gate-and-dispatched-pr-ci.md`

Ausgangslage

Nach dem Human-Merge von Recovery-PR #247 wurde das zuvor blockierende GitHub Ruleset entfernt. Der Post-Merge-Main-CI-Lauf war erfolgreich, `main` ist jedoch serverseitig aktuell nicht durch ein Ruleset geschützt.

Der frühere Ruleset-Sollvertrag verlangte den generischen Kontext `build-and-test`. Dieser Name war im PR-#236-#240-Incident sowohl für einen echten Actions-Job als auch für synthetisch erzeugte Check-Evidence verwendet worden und ist deshalb nicht mehr als eindeutige Trust-Root-Identität geeignet.

P0 Entscheidung

Der bestehende funktionierende `.github/workflows/ci.yml` bleibt während der Migration unverändert.

Zusätzlich wird ein nicht privilegierter technischer Shadow-Workflow eingeführt:

- Workflow: `.github/workflows/capital-ai-ci-shadow.yml`
- Check-Kontext: `capital-ai-ci`
- `contents: read` בלבד
- kein `checks: write`
- kein `pull-requests: write`
- kein `issues: write`
- kein Deployment
- kein synthetischer Check-Reporter
- exakter PR-Head-Checkout
- `persist-credentials: false`
- Repository-Integrität, Dependency Audit, Lint, Tests, Build, CSP/Predeploy und bei Runtime-/Workflow-Scope Docker-Hardening plus Image-Invarianten

Promotion Gate

Ein neues GitHub Ruleset darf `capital-ai-ci` erst als Required Check verwenden, wenn mindestens ein realer Pull Request diesen Check auf seinem aktuellen Head erfolgreich ausgeführt hat.

Zielzustand des späteren Rulesets:

- Pull Request required
- Required Check `capital-ai-ci`
- strict status checks
- non-fast-forward protection
- deletion protection
- keine Bypass Actors

Die Ruleset-Aktivierung ist eine separate Human/Owner-Admin-Mutation nach erfolgreichem Shadow-PASS.

Fail-Closed Bedingungen

Keine Promotion, wenn `capital-ai-ci`:

- fehlt;
- unerwartet `skipped` ist;
- fehlschlägt;
- einen falschen Head prüft;
- nicht eindeutig GitHub Actions zugeordnet werden kann;
- durch einen synthetischen Reporter gleichen Namens erzeugt wird.

Exit Gate P0

1. Shadow-PR auf aktuellem `main` geöffnet;
2. `capital-ai-ci` auf aktuellem PR-Head PASS;
3. bestehender Legacy-CI-Pfad weiterhin funktionsfähig;
4. GitHub Ruleset anschließend separat mit `capital-ai-ci` aktiviert;
5. Test-PR beweist Merge-Protection ohne Bypass;
6. Evidence auf `VERIFIED PASS` aktualisiert;
7. P0-Branch nach Human-Merge gelöscht.

Abschluss-Evidence 2026-08-15

Die Shadow-Phase hat ihren Zweck erfüllt:

Pull Request	build-and-test	capital-ai-ci Shadow	Ergebnis
#308	Run <code>31862280230</code> PASS	Run <code>31862280254</code> PASS	

Pull Request	build-and-test	capital-ai-ci Shadow	Ergebnis
			unabhängige parallele technische Evidence
#309	Run 31863283061 PASS	Run 31863283041 PASS	Application-/Test-Scope; Governance ebenfalls PASS

Owner-Entscheidung: Konsolidierungsoption 1. `build-and-test` bleibt technischer Trust Root, `capital-ai-ci` wird stillgelegt. Die Stilllegung erfolgt über die Trigger-Fläche des Shadow-Workflows (nur noch `workflow_dispatch`); die Workflow-Datei wird nicht gelöscht, weil `scripts/security/verifyChangedWorkflowSecurity.mjs` Workflow-Löschungen fail-closed untersagt. Damit die Stilllegung keinen dauerhaft fehlenden Required Check erzeugt, gilt folgende zwingende Reihenfolge:

1. Live-Ruleset `main-production-protection` öffnen.
2. Required Check `capital-ai-ci` entfernen.
3. `build-and-test` und `GitGuardian Security Checks` unverändert Required lassen.
4. Speichern und verifizieren, dass keine Bypass Actors hinzugefügt wurden.
5. Erst danach den Workflow-Retirement-PR mergen.
6. Mit dem nächsten realen PR nachweisen, dass Merge ohne `build-and-test` weiterhin blockiert.

Rollback vor Merge: Ruleset-Änderung rückgängig machen und diesen PR nicht mergen.
 Rollback nach Merge: frischer Recovery-Branch, `pull_request`-Trigger des Shadow-Workflows wieder eintragen, anschließend `capital-ai-ci` erst nach PASS erneut Required setzen.

PR #236 Bootstrap Incident und Recovery

Quelle: docs/evidence/ci/PR236_BOOTSTRAP_INCIDENT_2026-08-13.md

PR #236 Bootstrap Incident und Recovery

Status: RECOVERY PREPARED Datum: 2026-08-13 Repository: SvenKulessa/Finance Owner: SvenKulessa Referenz: docs/adr/ADR-0069-human-owner-comment-gate-and-dispatched-pr-ci.md

1. Zusammenfassung

Zwischen PR #236 und PR #240 wurde die Pull-Request-CI von einem direkt auf dem PR-Head laufenden `build-and-test` auf einen trusted-main, dispatch-basierten Human/Owner-Gate-Pfad umgestellt. Das Sicherheitsziel war korrekt: Kandidatencode sollte keine Schreibrechte erhalten und Human-Evidence sollte an `(PR, Head-SHA)` gebunden werden.

Die Umsetzung erzeugte jedoch einen selbstreferenziellen Trust Root: Das aktive GitHub-Ruleset verlangte weiterhin den Required Check `build-and-test`, während dieser Check nun synthetisch vom Human-Gate reserviert und durch einen Workflow aktualisiert wurde, der ausschließlich aus `main` geladen wurde. Ein Defekt in diesem trusted-main Workflow konnte dadurch seine eigene Reparatur blockieren.

2. Verifizierte Baselines

Letzter Stand vor PR #236

- `main`: 5bd5f4d78b87a89258126d0453eaf5e4bc6b6125
- Tree: 5d95c7e21b7dcdd2fb023047e01ca629b37b75d
- PR #236 Base: exakt dieser Commit

In diesem Stand enthält `.github/workflows/ci.yml`:

- `pull_request`: `types: [edited]`;
- Human-/Owner-Vorprüfung direkt im PR-Workflow;
- current-head Review `/ okay`;
- `build-and-test` als normalen GitHub-Actions-Job auf dem PR-Head;
- keinen separaten trusted-main `workflow_dispatch` für den Required Check;
- keinen synthetischen `build-and-test` Check-Run.

Fehlerhafter main-Stand

- `main`: 74f4eb98345f994854f56e7296b9491241718b57
- Merge PR #240

Dort waren zusätzlich aktiv:

- `.github/workflows/human-owner-comment-gate.yml`

- `.github/workflows/pr-build-and-test.yml`
- `.github/workflows/pr-auto-classification.yml`
- `docs/adr/ADR-0069-human-owner-comment-gate-and-dispatched-pr-ci.md`

3. Ereigniskette

PR #236

Führte den trusted-main Human/Owner-Kommentar-Gate-Bootstrap ein. Der PR dokumentierte selbst, dass der neue Mechanismus erst nach seinem Merge live wirksam werden könne.

PR #237 / #238 / #239

Mehrere Folgefehler mussten korrigiert werden:

- GitHub API `403 Resource not accessible by integration`;
- Check-Reporting auf falschem SHA;
- Rechtepartition zwischen Seed/Verification;
- YAML-/Shell-Fehler im Dispatch.

Diese Folge-PRs zeigen, dass der Bootstrap eine kritische Control-Plane während des laufenden Betriebs ersetzte, ohne unabhängigen Recovery-Kanal.

PR #240

Konsolidierte die neue ADR-0069-Architektur auf `main`. Dabei entstand bzw. blieb im trusted Preflight der fehlerhafte Cross-Checkout-Befehl:

```
git fetch --no-tags ../policy main:refs/remotes/origin/main
```

Das `policy`-Checkout befand sich auf einem konkreten Commit im detached-HEAD-Zustand. Der lokale Pfad stellte deshalb keinen fetchbaren Remote-Ref `main` bereit. Ergebnis:

```
fatal: couldn't find remote ref main
```

PR #243

Enthielt bereits den korrekten Fix:

- Human-Evidence und PR-Preflight als getrennte Jobs;
- Entfernung des Cross-Checkout-Fetches;
- serielle Kette `Governance → Human Gate → PR Build/Test → Auto-Status`.

PR #243 wurde jedoch am 2026-08-12 geschlossen und nicht gemerged.

PR #245 / #246

Der verbliebene Defekt blockierte anschließend normale Weiterentwicklung, obwohl Human-/Owner-Evidence, GitGuardian und Snyk erfolgreich waren. PR #246 reproduzierte den gleichen Fehler erneut.

4. Zweite Regression: PR-Template-Synchronisierung

PR Auto-Status verglich `github.event.workflow_run.name` mit statischen Workflow-Namen. Durch dynamische `run-name`-Werte entsprach der Eventname nicht mehr der Bedingung; der Job wurde `skipped`.

Folge:

- `{{...}}`-Platzhalter blieben in manuell erzeugten PRs stehen;
- Machine-Evidence konnte inkonsistent werden;
- der PR-Body war nicht zuverlässig kanonisch synchronisiert.

5. Connector-Schreibblockaden

Zusätzlich wurden einzelne ChatGPT-GitHub-Connector-Schreiboperationen vor GitHub durch eine Plattform-Sicherheitsklassifizierung blockiert, unter anderem PR-Erstellung und einzelne Workflow-Schreibvorgänge. Diese Sperren sind vom GitHub-Ruleset getrennt und dürfen nicht durch Repository-Bypass-Regeln kompensiert werden.

6. Recovery-Entscheidung

Der Recovery-Branch stellt zunächst den vollständigen Repository-Tree auf den verifizierten Stand unmittelbar vor PR #236 zurück.

Recovery-Commit:

- Commit: `55c08679cc0f184a898466597aeaf5ba7f9168c3`
- Parent: `74f4eb98345f994854f56e7296b9491241718b57`
- Tree: `5d95c7e21b7dcded2fb023047e01ca629b37b75d`

Damit bleibt die Git-Historie vollständig erhalten. Es gibt keinen Force-Push und keine direkte Mutation von `main`.

Auf diesem Snapshot werden ausschließlich diese Recovery-Dokumentation und eine korrigierte ADR-0069 ergänzt.

7. Verifikation des Rollback-Ziels

Statisch verifiziert:

1. der Recovery-Tree ist byte-/SHA-identisch zum Tree des PR-#236-Base-Commits;
2. der Pre-#236-Workflow besitzt einen normalen `build-and-test` Job direkt im PR-Workflow;

3. der Required Check hängt damit nicht von einem separat aus `main` dispatchten Workflow ab;
4. `human-owner-comment-gate.yml` und `pr-build-and-test.yml` existieren in diesem Snapshot nicht;
5. der konkrete Cross-Checkout-Fehler kann dort nicht auftreten.

Eine endgültige GitHub-seitige Laufzeitverifikation ist erst möglich, wenn der Recovery-PR gegen `main` geöffnet wird und GitHub die Checks für dessen Head ausführt.

8. Dauerhafte Invarianten nach Recovery

- Ein Required Check darf nicht ausschließlich von der Version eines Workflows abhängen, die durch denselben Required Check geschützt und erst nach Merge aktiv würde.
- Bootstrap darf keine aktive Merge-/CI-Control-Plane ersetzen, bevor eine unabhängige Recovery- und Abnahme-Stufe existiert.
- Neue Governance-Control-Planes müssen im Shadow-/Observation-Modus validiert werden, bevor sie Required-Check-Autorität übernehmen.
- Der bestehende funktionierende Required Check bleibt während einer Migration aktiv.
- Erst nach nachgewiesenem Parallel-PASS darf ein neuer Check serverseitig Required werden.
- Rollback eines Governance-Checks muss über normalen Branch/PR/Revert möglich bleiben.
- Keine Agenten- oder Workflow-Logik darf selbständig Rulesets oder Bypass-Actors abschwächen.

9. Auswirkungen auf Roadmap

Die PR-#236-#240-Bootstrapserie war dokumentiert und Human-autorisiert, wurde aber als Governance-/SA4B-Zwischenarbeit in die laufende DEVELOPMENT Chain eingeschoben. Die kanonische Roadmap wurde danach nicht vollständig auf den finalen `main@74f4eb...` synchronisiert.

Nach Recovery müssen Roadmap und Traceability den Vorfall als Governance-Incident referenzieren. Der fachliche Systemadministrator-/DevelopmentChain-Stand darf erst nach erfolgreicher Recovery-CI neu bewertet werden; verloren gegangene funktionale Änderungen nach PR #236 werden anschließend gezielt und einzeln aus der Historie wieder eingeführt, nicht durch erneuten Voll-Bootstrap.

10. Definition of Done

Recovery ist abgeschlossen, wenn:

1. Recovery-PR Human-reviewed und gemerged ist;
2. der normale PR-Head `build-and-test` ohne synthetischen trusted-main Reporter PASS liefert;

3. Main-CI nach Merge PASS liefert;
4. keine Bootstrap-Workflows aus PR #236–#240 mehr aktiv sind;
5. ADR-0069 in der korrigierten Fassung gilt;
6. GitGuardian/Snyk anschließend in einem frischen Branch erneut integriert werden;
7. Roadmap/Traceability auf den tatsächlich wiederhergestellten Stand synchronisiert werden;
8. der Recovery-Branch nach Human-Merge gelöscht wird.

Evidence — PR-Vorlagenvertrag: Claim-Evidence only (2026-08-16)

Quelle: docs/evidence/governance/PR_TEMPLATE_CONTRACT_CLAIM_EVIDENCE_ONLY_2026-08-16.md

Evidence — PR-Vorlagenvertrag: Claim-Evidence only (2026-08-16)

Date: 2026-08-16

Related: #368 (durable — no ephemeral head/main SHA body tokens), this follow-up

Problem

After #368, claim-PRs still required `baseline.production.version` and `baseline.production.commitSha` to appear in the PR body.

Agents fill the body at PR-creation time, before CI preflight has queried production health. Common placeholders (`N/A – CI fills`) therefore failed the evidence match and caused recurring **PR-Vorlagenvertrag** failures even when the rest of the template was correct.

This is the same class of timing/ephemerality fragility that #368 fixed for head/main SHAs.

Decision

For claim-PRs, body evidence is reduced to:

- `claim.claimId`
- claim file path (`.ai/work-claims/...`)

No longer required in the body:

- `production.version`
- `production.commitSha`
- head SHA / main SHA (already removed in #368)

Production identity continues to be enforced by:

1. `productionPreflight.mjs` (or the minimal body-edit baseline)
2. Presence of `CAPITAL_AI_PRODUCTION_BASELINE_START` / `END` markers

Fail-closed retained

- Canonical marker `CAPITAL_AI_PR_TEMPLATE_VERSION: 1.4.0`
- All 15 required sections (dash-tolerant)
- Baseline governance IDs
- No unresolved `{{PLACEHOLDER}}`
- At most one new work-claim

- Explicit Human-/CODEOWNER merge line

Validator

`scripts/pr/validatePrBody.mjs` — durableTokens for claim path now only claimId + claim-Path.

Non-goals

- No weakening of marker/section/placeholder rules
- No change to preflight rebase gate on code pushes
- No change to Human-only merge policy

PR-Vorlagenvertrag — Durable Evidence Fix (2026-08-16)

Quelle: docs/evidence/governance/PR_TEMPLATE_CONTRACT_DURABLE_EVIDENCE_2026-08-16.md

PR-Vorlagenvertrag — Durable Evidence Fix (2026-08-16)

Document ID: GOV-PR-TEMPLATE-DURABLE-2026-08-16
Status: ACTIVE
Authority: Owner request — dauerhafte Korrektur der PR-Vorlagenvertrag-Prüfung

Problem

Der PR-Vorlagenvertrag schlug systematisch fehl, obwohl die Vorlage inhaltlich korrekt war:

- 1. **Ephemere SHAs im Body** — `validatePrBody.mjs` verlangte `baseline.head.sha` und `baseline.main.sha` als Substring im PR-Body. Jeder Push (Fix-Commit) änderte den Head-SHA → Body sofort veraltet → Check rot, bis der Body manuell nachgezogen wurde.
- 2. **Kein `edited`-Trigger** — Body-Korrekturen allein starteten den Workflow nicht neu; leere Force-Commits waren nötig.
- 3. **Dash-Mismatch** — Abschnitt 3 mit ASCII-`-` statt Unicode-`—` wurde als fehlend gewertet.
- 4. **Policy vs. Candidate** — Validator lief aus dem PR-Head; ein PR konnte die Prüfung abschwächen.

Decision (fail-closed, durable)

Token / Check	Vorher	Nachher
Claim-ID + Claim-Pfad	Pflicht	Pflicht (unverändert)
Produktionsversion	Pflicht	Pflicht (unverändert)
Produktions-Commit	Pflicht (40 Zeichen)	Pflicht (40 Zeichen oder 12-Zeichen-Präfix)
<code>baseline.head.sha</code>	Pflicht	entfernt (ephemer)
<code>baseline.main.sha</code>	Pflicht	entfernt (ephemer bei Rebase)
Abschnitts-Matching	exact string	dash-tolerant (<code>—</code> / <code>-</code> / <code>-</code>)
Workflow-Trigger	<code>opened/synchronize/...</code>	+ <code>edited</code> (Body-only Re-Check)
Validator-Quelle	candidate	prefer main policy

Non-goals

- Rebase-Gate (`branch contains current main`) bleibt bei Code-Pushes (`synchronize`) aktiv.
- Human-/CODEOWNER-Freigabe-Zeile bleibt Pflicht.

- Template-Marker v1.4.0 und Governance-IDs bleiben Pflicht.

Files

- `scripts/pr/validatePrBody.mjs`
- `.github/workflows/pr-governance.yml`

Verification

Nach Merge: neuer PR mit Claim muss Claim-ID/Pfad + Produktionsversion/Commit im Body tragen; Push eines Fix-Commits darf den Vorlagenvertrag **nicht** allein wegen geändertem Head-SHA rot färben.

Domain & Threat-Reputation Evidence

Quelle: docs/evidence/m0/DOMAIN_THREAT_REPUTATION_EVIDENCE.md

Domain & Threat-Reputation Evidence

Stand: 2026-08-10

Geprüfte Endpunkte

- capital-ai.online
- Render-Origin des produktiven Finance-Service

Bitdefender

Bitdefender Link Checker lieferte für beide geprüften HTTPS-Endpunkte zum Prüfzeitpunkt den Status `so_far_so_good`.

Das ist ein punktueller Threat-Intelligence-Befund und kein Nachweis vollständiger Anwendungssicherheit.

Malwarebytes

Malwarebytes lieferte für beide Endpunkte den Verdict `unknown`, nicht `malicious` oder `suspicious`.

Für `capital-ai.online` weist Malwarebytes auf eine kurze Domainhistorie hin. Das ist für eine neu gestartete Produktdomain erwartbar, reduziert aber die verfügbare Reputation-Historie.

WHOIS / Domain Evidence

- Domain: `capital-ai.online`
- Registrar: IONOS SE
- Registriert: 2026-07-02
- Ablauf laut Abfrage: 2027-07-02
- Nameserver: IONOS/ui-dns-Familie

Interpretation

Die Ergebnisse dürfen nicht als Zertifizierung oder Malware-Freiheitsgarantie interpretiert werden. Für M9 Assurance sollen Domain-/URL-Reputation, TLS, Security Header, Dependency-/Container-Scanning und Runtime-Security als getrennte Evidence-Klassen behandelt werden.

GitHub Enforcement State — M0 Evidence

Quelle: docs/evidence/m0/GITHUB_ENFORCEMENT_STATE_2026-08-10.md

GitHub Enforcement State — M0 Evidence

Stand: 2026-08-10 Repository: `SvenKulessa/Finance` Baseline:

`f615cf4062f60eff07772c948c461025729a89dd`

Verifiziertes Ruleset

Aktiv ist das Repository-Ruleset `main-production-protection` (ID `20609723`). Es gilt für den Default-Branch und explizit `refs/heads/main`.

Aktive Regeln:

- Pull Request erforderlich.
- Required Status Check: `build-and-test`.
- Strict required status checks policy: aktiv.
- Non-fast-forward-Schutz: aktiv.
- Branch-Löschschutz: aktiv.
- Bypass Actors: keine.
- Current user can bypass: `never`.

Verifizierte M1-Gaps

Das Ruleset verlangt derzeit **keine** approving review (`required_approving_review_count = 0`). Weiterhin fehlen:

- `required_review_thread_resolution`
- `require_code_owner_review`
- `require_last_push_approval`
- Stale-review dismissal

Diese Punkte sind keine M0-Evidence-Lücken mehr, sondern konkrete M1-Hardening-Aufgaben.

GitHub Environments

Die Repository-API listet 15 Environments. Die sichtbaren Environments sind überwiegend historische/PR-bezogene Deployment-Environments. Sie besitzen keine Protection Rules und `can_admins_bypass=true`.

Ein kanonisches, geschütztes `production`-Environment mit Deployment-Approval wurde in der abgefragten Evidence nicht gefunden.

Folge: Definition eines dedizierten Production Environments ist M1/M7-Scope.

Secret Metadata

Die GitHub-App besitzt für `GET /repos/SvenKulessa/Finance/actions/secrets` keine ausreichende Berechtigung (`403 Resource not accessible by integration`). Secret-Werte wurden und werden nicht abgefragt.

Damit kann M0 ausschließlich dokumentieren:

- Secret-Metadaten sind mit dem aktuellen Connector nicht evidence-grade inventarisierbar.
- Die fehlende Leseberechtigung ist sicherheitlich vorzuziehen gegenüber einer Ausweitung nur für Dokumentationszwecke.
- Secret-Scope und Rotation müssen später über einen minimal privilegierten Owner-/Admin-Evidence-Prozess bestätigt werden.

M0 Entscheidung

M0-B03 GitHub Enforcement State : PASS.

M0-B04 Secret Metadata : PARTIAL / CONNECTOR-LIMITATION dokumentiert; keine Erweiterung der Connector-Rechte allein für M0 empfohlen.

M0 Deep Research – Plugin Evidence

Quelle: docs/evidence/m0/M0_DEEP_RESEARCH_PLUGIN_EVIDENCE.md

M0 Deep Research – Plugin Evidence

Stand: 2026-08-10 Branch: `agent/m0-deep-research-evidence-bundle` Modus: Evidence-first; externe Systeme wurden für die Bestandsaufnahme nur gelesen.

Ziel

Dieses Dokument ergänzt das M0-Abschlussartefakt um direkt über die angebundenen CAPITAL-AI-Plugins erhobene Evidence aus GitHub, Render, Supabase, Stripe, Bitdefender und Malwarebytes.

Executive Findings

1. GitHub: `SvenKulessa/Finance` ist privat, Default-Branch `main`; Auto-Merge ist deaktiviert. Merge Commit, Squash und Rebase sind erlaubt.
2. Render: Service `Finance` läuft als Docker-Web-Service in Frankfurt auf Starter. `main` ist der Produktionsbranch; Render-Auto-Deploy ist deaktiviert. Der aktuelle Live-Deploy referenziert Commit `f615cf4062f60eff07772c948c461025729a89dd`.
3. Supabase: Projekt `AIFINANCIAL` (`ryzywoktpmyhwzxmstyu`) ist `ACTIVE_HEALTHY`, Region `eu-west-1`, PostgreSQL 17.6.1. Alle im Plugin-Inventar sichtbaren Public-Tabellen haben RLS aktiviert.
4. Supabase Security Advisor meldet zwei Warnungen: Leaked Password Protection deaktiviert und zu wenige MFA-Optionen aktiviert.
5. Stripe: Produktionskonto `CAPITAL-AI` ist verbunden. Für die Zielarchitektur gelten Restricted API Keys als bevorzugtes Key-Modell; Webhook-Signaturen, Idempotenz und kontrollierte API-Versionierung bleiben Pflichtbestandteile.
6. Bitdefender meldet für `capital-ai.online` und die Render-Origin zum Prüfzeitpunkt keinen erkannten schädlichen Linkbefund (`so_far_so_good`).
7. Malwarebytes klassifiziert beide geprüften URLs als `unknown`, nicht als malicious/suspicious. Für `capital-ai.online` ist die geringe Domainhistorie ein Reputationsfaktor, kein Malware-Nachweis.
8. WHOIS: `capital-ai.online` wurde am 2026-07-02 über IONOS SE registriert und läuft laut Abfrage bis 2027-07-02; Nameserver liegen bei `ui-dns`.

Gate-Auswirkung

Die Plugin-Evidence schließt Teile von M0-B04 und M0-B05, aber nicht die vollständige GitHub-Enforcement-, Secret-Metadata- und SBOM-Inventur. M0 bleibt bis zum vollständigen Evidence Bundle formal `BLOCKED`.

Sicherheitsprinzip

Keine Secret-Werte, API-Keys, Tokens, Webhook-Secrets oder personenbezogenen Rohdaten werden in diesem Evidence Bundle gespeichert.

M0 Exit Status — Evidence Baseline

Quelle: docs/evidence/m0/M0_EXIT_STATUS_2026-08-10.md

M0 Exit Status — Evidence Baseline

Stand: 2026-08-10 Repository: `SvenKulesa/Finance`

Gate-Entscheidung

`M0 = PASS WITH DOCUMENTED CONNECTOR LIMITATION`

Die M0-Baseline ist ausreichend vollständig, um M1 kontrolliert zu starten. Nicht verifizierbare Secret-Metadaten werden **nicht** durch eine Ausweitung der GitHub-App-Berechtigungen nur für Dokumentationszwecke erzwungen.

Exit Criteria

- [x] Repository und Default-Branch erfasst.
- [x] offene PRs und Agentenbranches erfasst.
- [x] direkte Dependencies erfasst.
- [x] GitHub Dependency Graph SPDX-SBOM verfügbar und verifiziert.
- [x] alle sieben GitHub-Workflows inventarisiert.
- [x] Kern-Actions auf SHA-Pinning und Permissions geprüft.
- [x] aktives `main-production-protection` Ruleset evidence-grade erfasst.
- [x] Required Check `build-and-test` verifiziert.
- [x] Bypass Actors: keine.
- [x] GitHub Environments inventarisiert; fehlendes kanonisches Production Environment als Gap erfasst.
- [x] Render Service/Deployment-Baseline extern verifiziert.
- [x] Supabase Projekt-/RLS-/Security-Advisor-Baseline extern verifiziert.
- [x] Stripe Account-/Security-Control-Baseline extern verifiziert.
- [x] Domain-/Threat-Reputation-Baseline extern verifiziert.
- [x] Secret-Metadaten-Abfrage versucht und Connector-Limit (`403`) dokumentiert.

Residual Controls — nicht mehr M0-blockierend

M1 Git Guardrails

- approving review requirement ergänzen.
- review thread resolution ergänzen.
- ggf. CODEOWNERS-/last-push-approval risikobasiert aktivieren.
- kanonisches `production`-Environment definieren.

M3 CI Hardening

- vollständige Action-/Permission-Policy als Code erzwingen.
- privilegierte Trigger explizit verbieten/allowlisten.
- Git-Toolchain-Download kryptographisch absichern oder durch vertrauenswürdigen Runner-/Container-Image ersetzen.

M4 Agent IAM

- providerneutrale Actor-/Agent-Identity.
- Capability Grants und Risk Gates.
- Agent darf eigenen HIGH/CRITICAL-Change nicht selbst freigeben.

M5 Observability / Telemetry / Audit

- W3C Trace Context / OpenTelemetry Korrelation.
- unveränderbare Security-Audit-Evidence.
- Redaction vor Export.

M6 Supply Chain

- vollständige SBOM als CI-Artefakt exportieren.
- SBOM-Digest und Build-Provenance erzeugen.
- Artifact Attestation / SLSA Mapping.

M7 Deployment

- dediziertes Production Environment.
- kurzlebige Deployment-Identity/OIDC, soweit Render-Zielmodell dies unterstützt; andernfalls dokumentierter Secret-Bridge-Mechanismus mit Rotation.

M8 Agent Cutover

- produktive AI-Agent-Ausführung nur über Agent Control Plane und Capability Policy.

M9 Assurance

- Negative Tests, Incident/Break-Glass Drill, Traceability-Abschluss und unabhängige Kontrollprüfung.

Freeze Point

Die Baseline für M0 ist der beobachtete `main`-SHA:

```
f615cf4062f60eff07772c948c461025729a89dd
```

Spätere Änderungen werden als Delta gegen diesen Freeze Point oder einen explizit aktualisierten Baseline-Snapshot betrachtet.

Plugin Evidence Traceability

Quelle: docs/evidence/m0/PLUGIN_EVIDENCE_TRACEABILITY.md

Plugin Evidence Traceability

Stand: 2026-08-10

Evidence Domain	Quelle	Befund	M-Meilenstein
Repository	GitHub Plugin	privates Repo, <code>main</code> Default	M0/M1
Merge-Konfiguration	GitHub Plugin	Auto-Merge aus; mehrere Merge-Methoden erlaubt	M1
Production Service	Render Plugin	Docker/Frankfurt/Starter/ <code>main</code>	M0/M7
Deploy State	Render Plugin	SHA <code>f615cf...</code> live	M0/M5/M7
Auto Deploy	Render Plugin	deaktiviert	M1/M7
Supabase Health	Supabase Plugin	ACTIVE_HEALTHY	M0
PostgreSQL	Supabase Plugin	17.6.1 / Engine 17	M0/M6
RLS	Supabase Plugin	alle sichtbaren Public-Ta-bellen RLS-enabled	M0/M4
Auth Advisor	Supabase Plugin	leaked-password protection WARN	M4
Auth Advisor	Supabase Plugin	insufficient MFA WARN	M4
Billing Account	Stripe Plugin	CAPITAL-AI verbunden	M0/M4
Stripe Key Model	Stripe Skill/Docs	Restricted Keys bevorzugt	M4/M6
Stripe Webhooks	Stripe Docs	Signatur + Idempotenz erforderlich	M3/M4
Domain Reputation	Bitdefender	kein erkannter schädli-cher Befund	M9
Domain Reputation	Malwarebytes	unknown, nicht malicious/suspicious	M9
Domain Registration	Malwarebytes WHOIS	IONOS, 2026-07-02	M0/M9

Noch nicht geschlossen

- vollständige GitHub Ruleset-/Branch-Protection-Evidence
- vollständiges Workflow-/Action-Inventar
- Secret-Metadaten nach Scope/Consumer ohne Secret-Werte
- transitive Dependency-SBOM
- OIDC Deployment Identity

- vollständige E2E Trace-Korrelation

Diese Matrix ist ein Ergänzungsartefakt und ersetzt nicht die zentrale M0 Traceability Matrix.

Render Deployment Evidence

Quelle: docs/evidence/m0/RENDER_DEPLOYMENT_EVIDENCE.md

Render Deployment Evidence

Stand: 2026-08-10 Workspace: AICapital Service: Finance Service-ID: srv-d91o1o9o3t8c73e-di55g Region: Frankfurt Runtime: Docker Plan: Starter Branch: main Healthcheck: /healthz

Verifizierte Konfiguration

- Repository: SvenKulessa/Finance
- Dockerfile: ./Dockerfile
- Docker Context: .
- Auto Deploy: no
- Auto Deploy Trigger: off
- PR Previews: aus
- Instanzen: 1
- Port: TCP 10000
- öffentliche IP-Allowlist: 0.0.0.0/0
- Registry Credential: GitHub Container Registry Credential vorhanden

Deployment Evidence

Aktueller Live-Deploy zum Prüfzeitpunkt:

- Commit: f615cf4062f60eff07772c948c461025729a89dd
- Commit-Kontext: Merge PR #183, Observability-/Telemetry-O1-Baseline
- Trigger: API
- Status: live

Vorherige Deployments zeigen Deploy-Hook-, API- und manuelle Trigger. Damit ist die produktive Deployment-Kette funktionsfähig, aber noch nicht auf ein ausschließlich kurzlebiges OIDC-Identity-Modell reduziert.

Bewertung

Positiv: - Produktionsbranch eindeutig main . - Auto-Deploy ist abgeschaltet. - Healthcheck ist konfiguriert. - Docker-basierte reproduzierbare Runtime.

Offene Controls: - OIDC statt langlebigem Deploy-Hook-/Secret-Modell prüfen. - Deployment Authorization und Artifact Digest bis zur Runtime korrelieren. - öffentliche Netzwerexposition gegen tatsächlichen Web-Service-Bedarf dokumentieren. - Deployment-Trigger auf eine einzige kontrollierte CI-Authority konsolidieren.

M7 bleibt der vorgesehene Meilenstein für OIDC/Deployment-Hardening.

SPDX-SBOM-Baseline — M0 Evidence

Quelle: docs/evidence/m0/SBOM_BASELINE_2026-08-10.md

SPDX-SBOM-Baseline — M0 Evidence

Stand: 2026-08-10 Repository: SvenKulesa/Finance Baseline-SHA:

f615cf4062f60eff07772c948c461025729a89dd

Quelle

Die GitHub Dependency Graph API liefert für dieses Repository erfolgreich ein Software Bill of Materials über:

```
GET /repos/SvenKulesa/Finance/dependency-graph/sbom
```

Verifizierte Metadaten

- SPDX Version: `SPDX-2.3`
- Data License: `CC0-1.0`
- Dokumentname: `com.github.SvenKulesa/Finance`
- Erzeuger: GitHub Dependency Graph / protobom
- Erzeugungszeitpunkt der abgefragten Evidence: 2026-08-10T21:54:13Z
- Paketobjekte enthalten Package URLs (`url`), Versionsinformationen und Lizenzmetadaten, soweit GitHub diese bestimmen kann.

Damit ist nachgewiesen, dass GitHub eine transitive Dependency-Graph-basierte SPDX-SBOM für den aktuellen Repository-Zustand erzeugen kann.

Grenzen der M0-Erfassung

Der angebundene GitHub-Connector liefert große API-Antworten gekürzt an den Chat-Client. Deshalb wird die vollständige SBOM in M0 nicht manuell aus einer gekürzten Toolantwort rekonstruiert. Dies vermeidet eine unvollständige oder verfälschte Evidence-Datei.

Stattdessen wird M6 einen reproduzierbaren CI-Schritt einführen, der die vollständige SBOM als Build-/Release-Evidence exportiert, hasht und als Artefakt mit Source SHA und Build Run verknüpft.

Sicherheitsentscheidung

M0 bewertet die Existenz und Abrufbarkeit der vollständigen GitHub Dependency Graph SBOM als ausreichende Baseline-Evidence. Die kryptographische Persistierung und Attestation ist ausdrücklich M6-Scope.

`M0-B01 Dependency/SBOM-Baseline` : PASS.

Residual Controls:

- M6: vollständiger SPDX/CycloneDX Export im CI.
- M6: SHA-256 Digest der SBOM.
- M6: Verknüpfung `source_sha → sbom_digest → artifact_digest → provenance`.

Stripe Integration Security Evidence

Quelle: docs/evidence/m0/STRIPE_INTEGRATION_SECURITY_EVIDENCE.md

Stripe Integration Security Evidence

Stand: 2026-08-10 Konto: CAPITAL-AI

Verbundener Zustand

Das Stripe-Plugin bestätigt ein verbundenes CAPITAL-AI-Konto. Dieses Dokument speichert bewusst keine Schlüssel oder Secret-Werte.

Aktuelle Integrationsleitplanken

Für CAPITAL-AI als Subscription-SaaS gilt:

- Recurring Billing über Stripe Billing + Checkout Sessions.
- Restricted API Keys (`rk_`) sind gegenüber breit berechtigten Secret Keys zu bevorzugen, soweit die benötigten Operationen damit abbildbar sind.
- Webhook-Signaturen müssen vor Verarbeitung verifiziert werden.
- Webhook-Verarbeitung muss idempotent sein; die vorhandene `stripe_event_inbox` in Supabase ist hierfür ein relevanter bestehender Baustein.
- API-/SDK-Versionen werden kontrolliert aktualisiert; der aktuelle Stripe-Skill nennt `2026-06-24.dahlia` als aktuelle API-Version zum Prüfzeitpunkt.
- Secrets ausschließlich serverseitig; keine Secret Keys im Browser oder Repository.
- Checkout-/Webhook-Metadaten müssen eine belastbare interne Identität referenzieren, nicht nur E-Mail-Adressen.

CAPITAL-AI Zielkette

Checkout Session → signierter Stripe Event → durable Event Inbox → idempotente Verarbeitung
→ Subscription State → Audit/Trace → Nutzerberechtigung

Deep-Research-Fokus für M3/M4/M6

1. Key-Scope und Restricted-Key-Eignung.
2. Webhook Signature Verification.
3. Replay-/Duplicate-Schutz.
4. Event-Reconciliation für failed/stuck Events.
5. API-Version und SDK-Version als Supply-Chain-Evidence.
6. Billing-Mutation als HIGH/CRITICAL Capability klassifizieren.
7. Keine Agentenmutation an Stripe ohne explizite Capability + Approval Artifact.

Supabase Security Evidence

Quelle: docs/evidence/m0/SUPABASE_SECURITY_EVIDENCE.md

Supabase Security Evidence

Stand: 2026-08-10 Projekt: AIFINANCIAL / ryzywoktpmyhwzxmstyu Region: eu-west-1 Status: ACTIVE_HEALTHY PostgreSQL: 17.6.1 / Engine 17

RLS-Inventar

Der Plugin-Snapshot zeigt RLS als aktiviert für alle sichtbaren Tabellen im Schema `public`, darunter `users`, `usage_log`, `subscriptions`, `profiles`, IAM-/Step-up-/Break-glass-Tabellen, `security_events`, `compliance_*`, `score_snapshots`, `stripe_event_inbox`, `capability_grants` und `agent_action_approvals`.

RLS-Aktivierung allein beweist noch nicht die fachliche Korrektheit jeder Policy. M1/M4 müssen daher Ownership-Prädikate, `USING / WITH CHECK`, Service-Role-Grenzen, Views und privilegierte Funktionen separat prüfen.

Security Advisor

Aktuelle Warnungen:

1. `auth_leaked_password_protection`: Leaked Password Protection ist deaktiviert.
2. `auth_insufficient_mfa_options`: Es sind zu wenige MFA-Optionen aktiviert.

Zielkontrollen

- Keine Autorisierung über user-editierbare `user_metadata`.
- Autorisierungsdaten nur über vertrauenswürdige Claims/App-Metadata oder serverseitige Capability-Entscheidungen.
- Kein `service_role` /Secret Key im Client.
- Exponierte Tabellen: RLS plus fachlich enge Policies.
- UPDATE-Policies: `USING` und `WITH CHECK`.
- `SECURITY DEFINER` nur begründet, nicht als Permission-Workaround.
- Agentenrechte über explizite `capability_grants`; keine Wildcards.
- Approval-Artefakte einmalig und an Plan-/Request-Hash binden.

Priorität

Die beiden Auth-Advisor-Warnungen werden als HIGH für IAM-Hardening klassifiziert. Änderungen an produktiver Auth-Konfiguration erfolgen nicht in M0, sondern nach dokumentierter Freigabe.

Workflow- und Action-Inventar — M0 Evidence

Quelle: docs/evidence/m0/WORKFLOW_ACTION_INVENTORY_2026-08-10.md

Workflow- und Action-Inventar — M0 Evidence

Stand: 2026-08-10 Repository: `SvenKulesa/Finance`

Vollständige Workflow-Liste auf `main`

1. `.github/workflows/ci.yml`
2. `.github/workflows/document-hygiene-evidence-migration.yml`
3. `.github/workflows/document-hygiene-evidence-once.yml`
4. `.github/workflows/google-marketing-protected-change.yml`
5. `.github/workflows/lockfile-remediation.yml`
6. `.github/workflows/open-agent-draft-pr.yml`
7. `.github/workflows/pr-governance.yml`

Klassifikation

`ci.yml`

Trigger: `push` auf `main`, `pull_request`.

Permissions: `contents: read`.

Externe Actions:

- `actions/checkout` auf vollständige Commit-SHA gepinnt.
- `actions/setup-node` auf vollständige Commit-SHA gepinnt.

Weitere Supply-Chain Inputs:

- Git 2.55.0 wird von kernel.org geladen und lokal gebaut.
- `npm ci` installiert den Lockfile-Stand.
- `npm audit --omit=dev --audit-level=high` ist aktiv.
- Docker-Image wird für Runtime-/Dependency-/Deployment-relevante Änderungen gebaut.

M0/M3-Gap: Der Git-Source-Download prüft zwar `xz --test`, aber keinen separat verankerten kryptographischen Upstream-Hash bzw. Signaturbeweis. Dieses Thema ist M3/M6 zu behandeln.

`pr-governance.yml`

Trigger: `pull_request` gegen `main`, `merge_group`.

Permissions: `contents: read`.

Controls:

- Policy-Checkout aus vertrauenswürdigen `main`.
- Kandidat wird separat ausgecheckt.
- geänderte Workflows werden mit `verifyChangedWorkflowSecurity.mjs` geprüft.
- GitHub-Token wird bei Repository-Konventionsprüfungen explizit geleert.

Actions sind auf vollständige Commit-SHAs gepinnt.

`open-agent-draft-pr.yml`

Trigger: ausschließlich `workflow_dispatch`.

Permissions:

- `contents: read`
- `pull-requests: write`

Controls:

- erlaubte Agenten-Branch-Präfixe: `agent/`, `claude/`, `gemini/`, `copilot/`, `ai/`.
- Policy wird aus `main` geladen.
- Kandidatenbranch separat.
- Production Preflight.
- Work-Claim-Validierung.
- kanonische PR-Body-Erzeugung.
- Duplikatprüfung.

Actions sind auf vollständige Commit-SHAs gepinnt.

`google-marketing-protected-change.yml`

Trigger: path-scoped PR/Push auf `main`, zusätzlich `workflow_dispatch`.

Permissions: `contents: read`.

Control: statische Prüfung des Marketing-/Consent-Schutzvertrags; keine zweite vollständige Testpipeline.

`actions/checkout` ist SHA-gepinnt.

`document-hygiene-evidence-migration.yml`

Nur `workflow_dispatch`, `contents: read`. Der Workflow führt keine Migration mehr aus, sondern ist als deaktivierter Audit Record erhalten.

`document-hygiene-evidence-once.yml`

Nur `workflow_dispatch`, `contents: read`. Führt keine Repository-Mutation aus.

lockfile-remediation.yml

Nur `workflow_dispatch`, `contents: read`, Job zusätzlich `if: false`. Historischer Audit Record ohne aktive Mutation.

M0 Entscheidung

Die vollständige Menge der sieben Workflows wurde erfasst und nach Trigger, Permission, externer Action-Nutzung und Privileg klassifiziert.

M0-B02 vollständige Workflow-/Action-Inventur : PASS mit zwei Folgethemen für M3/M6:

1. Toolchain-Download-Authentizität für Git Source Build härten.
2. Action-Pinning und Workflow-Permissions weiterhin policy-as-code erzwingen, nicht nur dokumentieren.

M3 CI Hardening Evidence

Quelle: docs/evidence/m3/M3_CI_HARDENING_EVIDENCE.md

M3 CI Hardening Evidence

Status: IN PROGRESS Date: 2026-08-11 Baseline: `main@de879d7d060e769b-f9240c891f75b5ac6a1556bf` (M2G merge) Authority: ADR-0060, ADR-0053, ESS-0019, GitHub governance

1. Ausgangsbefund

Die M2G-Validierung von PR #194 war ein reiner Dokumentations-PR. Trotzdem lief im Required Check `build-and-test` vor dem Checkout ein vollständiger Build von Git 2.55.0 aus Source. Docker wurde bereits korrekt scope-basiert übersprungen. Damit waren zwei M3-Gaps bestätigt:

1. Der Git-Tarball wurde nur strukturell mit `xz --test`, nicht gegen einen repository-gepinnten kryptografischen Digest geprüft.
2. Reine Dokumentations-PRs verbrauchten weiterhin unnötig Actions-Minuten für Git-Source-Build, Node/npm, Tests und Production Build.

2. Implementierte Controls

M3-C01 — stabiler Required Check

`build-and-test` bleibt unverändert die technische Job-ID und damit der Required-Check-Contract.

M3-C02 — immutable Checkout / keine persistierten Credentials

Scope-Ermittlung beginnt mit dem bereits SHA-gepinnten `actions/checkout` und `persist-credentials: false`.

M3-C03 — conservative Docs-only Fast Path

Nur PRs, deren geänderte Dateien vollständig in `docs/**`, `.ai/**` oder Markdown-Dateien fallen, dürfen `full_validation=false` erhalten. Jede andere Datei fällt fail-closed auf vollständige Validierung zurück. `push` auf `main` ist immer `full_validation=true`.

M3-C04 — kein Repository-Code im Scope-Preflight

Vor der Scope-Entscheidung werden keine Package-Manager-, Test-, Build- oder Repository-Skripte ausgeführt. Es werden ausschließlich Git-Metadaten/Diffs gelesen und der Commit mit `git rev-parse` verifiziert.

M3-C05 — kryptografischer Git-Source-Pin

Für vollständige Validierung wird `git-2.55.0.tar.xz` weiterhin über HTTPS von kernel.org geladen. Vor `xz --test`, Extraction und Build wird der exakte SHA-256-Digest geprüft:

```
457fdb04dc8728e007d4688695e6912e6f680727920f2a40bf11eacc17505357
```

Mismatch, Downloadfehler oder Buildfehler blockieren CI fail-closed.

M3-C06 — vollständige Validierung für relevante Änderungen

Bei Code, Workflow, Dependencies, Konfiguration, Runtime, Docker oder Deployment bleiben Node Setup, `npm ci`, Audit, Production Config, Docker Hardening, TypeScript, Unit Tests, Production Build, CSP, Predeploy und gegebenenfalls Docker-Build verpflichtend.

M3-C07 — main bleibt Full Validation

Jeder Push nach `main` läuft vollständig. Die Kostenoptimierung verändert damit nicht den bestehenden Production-Deploy-Gate.

3. Nicht Teil von M3

- kein GitHub `production` Environment;
- keine Render-/Supabase-/Stripe-Mutation;
- keine SLSA-Attestation oder Release-Provenance-Erzeugung (M6);
- keine Änderung am Agent IAM (M4);
- keine OTel/Audit-Erweiterung (M5).

4. Validierungsmatrix

Evidence	Erwartung	Status
geänderter <code>ci.yml</code> PR	Full Validation	PENDING PR-CI
Git 2.55 SHA-256 Check	PASS vor Extraction	PENDING PR-CI
Workflow Security Validator	SHA pinning/minimal permissions/ concurrency PASS	PENDING PR-CI
TypeScript + Unit Tests + Build	PASS	PENDING PR-CI
PR Production Deploy	Render-Trigger skipped	PENDING PR-CI
nachgelagerter docs-only PR	Fast Path, <code>build-and-test</code> success	PENDING POST-MERGE EVIDENCE
main push	Full Validation	PENDING POST-MERGE EVIDENCE

5. M3 Exit

M3 darf erst `COMPLETE` werden, wenn:

1. der M3-Implementierungs-PR vollständig grün ist;
2. die Git-Tarball-Digestprüfung nachweislich im Full Path ausgeführt wurde;

3. Governance den geänderten Workflow akzeptiert;
4. nach Merge ein `main`-Run den Full Path bestätigt;
5. ein reiner Dokumentations-PR den Fast Path erfolgreich demonstriert;
6. `docs/architecture/ROADMAP.md`, die M0-M9-Roadmap und Traceability mit der M3-Evidence aktualisiert sind.

6. Rollback

Bei Fehlklassifikation oder CI-Regression wird der Docs-only Fast Path entfernt und Full Validation für alle PRs wiederhergestellt. Der SHA-256-Pin für den Git-Tarball bleibt erhalten und darf durch den Kosten-Rollback nicht zurückgenommen werden.

M3 – Docs-only Fast-Path Proof

Quelle: docs/evidence/m3/M3_FAST_PATH_PROOF_2026-08-11.md

M3 – Docs-only Fast-Path Proof

Status: VALIDATION PR Datum: 2026-08-11 Baseline: 45e81c59ac47cf63d6961f5a7a41b-f9a3011d9b8

Zweck

Dieses ausschließlich dokumentarische Artefakt dient als kontrollierter Post-Merge-Nachweis für den in M3 eingeführten Docs-only Fast Path.

Erwartetes CI-Verhalten

Für diesen Pull Request MUSS der Required Check `build-and-test` erfolgreich bleiben, während die kostenintensiven Software-Gates übersprungen werden, da ausschließlich `docs/**` geändert wird.

Erwartet übersprungen:

- Git-2.55.0-Source-Build
- Node.js-/npm-Toolchain
- Dependency Installation und Audit
- TypeScript
- Unit Tests
- Production Build
- CSP-/Predeploy-Prüfungen
- Docker Build und Runtime-Metadaten

Sicherheitsinvarianten

- Der Fast Path gilt ausschließlich für Pull Requests mit erlaubtem Dokumentationsscope.
- Änderungen an Code, Workflows, Dependencies, Runtime, Deployment oder Konfiguration erzwingen weiterhin Full Validation.
- Pushes auf `main` erzwingen weiterhin Full Validation unabhängig vom Dateiscope.
- `build-and-test` bleibt der Required Check; es wird kein Required Gate entfernt oder umbenannt.

Exit-Kriterium

M3 darf erst nach erfolgreichem CI-Lauf dieses PRs als `COMPLETE` dokumentiert werden. Der CI-Lauf muss nachweisen, dass der Docs-only Fast Path aktiviert und der Required Check erfolgreich abgeschlossen wurde.

M4 Agent IAM — Implementation Evidence

Quelle: docs/evidence/m4/M4_AGENT_IAM_EVIDENCE.md

M4 Agent IAM — Implementation Evidence

Status: CLOSURE IN REVIEW Date: 2026-08-11 Baseline: `main@69f719683b60ba6aadc0022381c6-cecc430f0ea5` (PR #198 merge) Authority: ESS-0019, ESS-0018, ADR-0058, ADR-0050, ADR-0051

Objective

Generalize the existing Supabase/tool-specific CapabilityGrant and Approval controls into a provider-neutral Agent IAM decision layer without weakening or replacing the existing domain/tool policies.

Canonical implementation

PR #198 is the canonical M4 implementation. It merged the provider-neutral Agent IAM into `main` and remains the architectural baseline.

Parallel PRs #200 and #201 were created from the older PR-#197 baseline. They are not separate roadmap phases. Their contents were reviewed against ADR-0058 and the current Roadmap. Only one useful missing control from #201 is retained: approval evidence is additionally bound to the exact logical `agentId`.

The conflicting #201 risk mapping is rejected because it diverges from the canonical M4 ladder.

Canonical controls

- explicit capabilities READ, ANALYZE, PLAN, BRANCH, COMMIT, PR, CI_REQUEST, DEPLOY_REQUEST, PRODUCTION_MUTATION;
- no MERGE capability;
- LOW: READ/ANALYZE/PLAN;
- MEDIUM: BRANCH/COMMIT/PR/CI_REQUEST;
- HIGH: DEPLOY_REQUEST;
- CRITICAL: PRODUCTION_MUTATION;
- contextual risk may increase but never reduce minimum risk;
- principal binds human actor, app, logical agent, session, request and credential holder;
- provider/model are metadata only;
- exact non-inheriting grants;
- HIGH requires current Human Approval;
- CRITICAL requires Human Approval plus verified Step-up;

- approval binds human actor + subject agent + capability + target and must be unexpired;
- development -> production mutation is denied;
- mutation kill switch is enforced;
- ESS-0018 tool-specific grants, approvals and PolicyGate checks remain a second independent layer.

Negative-test evidence

`tests/unit/agentIam.test.ts` covers:

- capability non-inheritance;
- MERGE denial;
- incomplete attribution;
- provider/model non-authority;
- minimum-risk enforcement;
- HIGH approval without mandatory step-up;
- CRITICAL approval plus step-up;
- approval subject-agent mismatch;
- target mismatch;
- expired approval;
- self-approval;
- development production boundary;
- kill switch.

Production boundary

This closure PR introduces no direct mutation of Stripe, Supabase or Render configuration, no database migration and no new production secret.

Closure gate

M4 becomes COMPLETE only after this single consolidation/closure PR:

- passes technical/governance CI;
- is fully reviewed under the Human/Owner gate;
- has both Owner attestation checkboxes checked;
- has a current-head ☐ or `okay` Owner review;
- is merged only after a separate explicit Human merge instruction.

After that merge, M5 Observability/Telemetry/Audit becomes the next authorized phase.

M5 — Application Audit Integration Evidence

Quelle: docs/evidence/m5/M5_APP_AUDIT_INTEGRATION_EVIDENCE.md

M5 — Application Audit Integration Evidence

Status: HISTORICAL CI PASS / CORRECTIVE RUNTIME CONTRACT VERIFICATION ACTIVE

Date: 2026-08-12 Original baseline: `main@e39d5370d8b1498e84952535a38a339cc200082f` (PR #210 merge) Corrective baseline: `main@156142102e7d2a97ad466aee0340f758fa4365e5` (PR #220 merge) Authority: ADR-0056, ADR-0059, ESS-0019

Original scope

PR #210 integrated the application-side M5 audited execution path with the already-created Supabase audit authority `public.agent_audit_events`.

The production table, RLS, least-privilege grants and append-only trigger had already been applied and verified. PR #210 changed application code/tests only.

Original controls

1. `server/agentAudit/agentAuditWriter.ts` - backend-only privileged Supabase writer; - writes only to `agent_audit_events`; - returns `supabase:agent_audit_events:<id>` as `auditReference`; - propagates persistence failure (fail closed); - reuses the canonical Telemetry redaction contract; - omits complete prompt/diff/raw-body payload classes before persistence.
2. `server/agentAudit/authorizedAgentExecution.ts` - keeps provider-neutral authorization in the existing PolicyGate; - couples each server-side authorization decision to durable audit persistence; - returns the durable `auditReference` to the caller; - records terminal `SUCCESS` / `ERROR` as a second append-only outcome event.
3. `tests/unit/agentAudit.test.ts` - redaction/omission tests; - persistence failure = fail closed; - authorization/outcome correlation; - mocked Supabase insert chain.

Historical PR #210 validation

Final PR head: `ffeab08c218314edd5292fbaf5ccb77413cee80e`

Merge commit: `e39d5370d8b1498e84952535a38a339cc200082f`

Required CI #892 / run `31559124198` : **PASS**.

Verified in that run: - Human-/Owner-Vorprüfung; - repository integrity; - `npm ci`; - production dependency audit; - TypeScript/lint; - unit tests; - production build; - deployment readiness; - Docker hardening/image verification.

Corrective runtime finding from SA3B

The first real SA3B execution-host probe after PR #220 merge provided the production integration test that the original mocked writer tests did not provide.

Issue #221 / workflow run `31570833507` reached the production broker with:

- exact trusted `main` SHA — PASS;
- strict request validation — PASS;
- trusted REM binding — PASS;
- GitHub Actions OIDC — PASS.

The broker then failed closed during durable M5 persistence with:

```
[AgentAudit][SECURITY] durable audit persistence failed: Unregistered API key
```

The branch side-effect step was skipped and the requested probe branch was confirmed absent.

Independent writer/schema drift found during diagnosis

Read-only comparison of the actual production table, canonical migration and writer showed that the pre-remediation writer used application aliases that do not exist in the M5 table.

Canonical migration/table fields include:

- `human_actor_id`;
- `intent`;
- `scope`;
- `authorization_decision`;
- `approval_reference`;
- `step_up_reference`;
- `tool_name`;
- `branch` / `commit_sha`;
- `pull_request_number`;
- `ci_run_id`;
- `attributes`.

Pre-remediation aliases included:

- `actor_id`;
- `decision`;
- `approval_id`;
- `tool_id`;
- `pr_number`;

- `workflow_run_id`;
- `metadata`.

The original unit tests asserted those application aliases against a mock and therefore could not detect production schema drift.

Corrective contract

Corrective branch: `fix/sa3b-m5-audit-schema-contract`.

The production migration/table remain authoritative. No Supabase schema mutation is required or authorized.

Corrective controls:

1. writer DB row uses only canonical migration column names;
2. `intent` and structured `scope` are explicit application inputs;
3. generic audited execution supplies semantic authorization/outcome intents;
4. Systemadmin audited execution supplies REM/Roadmap/target/path scope;
5. non-UUID external actor identity is preserved in sanitized `attributes` and never fabricated into `human_actor_id uuid`;
6. non-UUID approval/step-up references follow the same rule;
7. runtime unit tests assert canonical keys and reject legacy DB row aliases;
8. a migration↔writer contract test prevents future vocabulary drift.

Production secret boundary

The deployed service currently also has an invalid/unregistered privileged Supabase API key. This is independent from the writer/schema defect.

No production secret value is included in this evidence. Restoring or rotating the backend-only credential is an Owner-controlled production operation.

Security invariants retained

- No `anon` or `authenticated` audit-table access is introduced.
- No new database privilege is introduced.
- No schema rollback/replacement is authorized.
- Full prompts, full diffs and raw request/response bodies remain omitted.
- Secret-like/PII fields remain redacted.
- Only syntactically valid audit references bypass generic authorization-key redaction.
- Audit persistence failure remains fail-closed.
- Authorization and terminal outcome remain separate append-only events.
- External IDs are not coerced into UUID columns.

Corrective exit criteria

Application integration is considered runtime-verified again only after:

1. corrective PR final-head CI PASS;
2. Human merge + branch deletion;
3. corrected `main` deployment;
4. valid privileged Supabase credential available in production;
5. a real audited authorization insert succeeds against `public.agent_audit_events` ;
6. a correlated terminal outcome insert succeeds;
7. SA3B positive host probe proves audit-before-side-effect ordering.

Until then, the historical PR #210 CI result remains valid as code-test evidence, but it must not be treated as proof that the pre-remediation writer matched the actual production schema.

M5 Observability / Telemetry / Audit — Gap Assessment

Quelle: docs/evidence/m5/M5_OBSERVABILITY_AUDIT_GAP_ASSESSMENT.md

M5 Observability / Telemetry / Audit — Gap Assessment

Status: PLAN / PRE-MUTATION Date: 2026-08-12 Baseline: `main@85e9b44ba604e046b7ff-c9e58113a177eb0e908a`

Ziel

M5 erweitert die bestehende Observability-Baseline aus ADR-0056 und die Agent-Korrelation aus ADR-0059, ohne einen zweiten Logging-Stack einzuführen.

Read-only Evidence

Repository: - `src/platform/Telemetry` existiert bereits mit `TelemetryRecord`, Request-/Trace-/Span-Kontext und Redaction. - Operational Telemetry und Audit Evidence sind bereits konzeptionell getrennt. - ADR-0059 verlangt Ende-zu-Ende-Korrelation für AI-assisted Commands.

Supabase-Projekt `AIFINANCIAL`: - `audit_logs_iam` existiert, enthält aber keine vollständige Agent-/Request-/Trace-/PR-/CI-/Deployment-Korrelation. - `iam_access_log` ist auf IAM-Zugriffsevidence fokussiert. - `agent_action_approvals` persistiert Approval-Evidence, aber keine vollständigen Execution-Audit-Events. - keine dedizierte `agent_audit_events` -/Telemetry-Evidence-Tabelle vorhanden.

Gap gegen ADR-0059

Fehlend als dauerhaft korrelierbare Audit-Evidence: - `request_id` / `trace_id` / `span_id` - `app_id` / `agent_id` / `provider` / `model` - `intent` / `scope` / `capability` / `risk_class` - `policy_id` / `policy_version` / `authorization decision` - `approval` / `step-up reference` - `repository` / `branch` / `commit` / `PR` / `CI run` - `artifact digest` / `deployment identity` / `runtime version` - `result` / `error` / `rollback reference`

Entscheidung

Supabase-Mutation für M5: **REQUIRED**, aber erst nach Human/Owner-Freigabe des M5-Planungs-PRs.

Minimaler Zielzustand: 1. eine dedizierte append-only Tabelle `agent_audit_events` für revisionsrelevante Agent-/Command-Evidence; 2. keine Persistenz von Secrets, Tokens, vollständigen Prompts oder vollständigen Diffs; 3. RLS/Grants deny-by-default gegenüber `anon` und `authenticated`; 4. Server-/Service-Pfad schreibt ausschließlich redigierte Evidence; 5. Operational Telemetry bleibt in `src/platform/Telemetry` und referenziert Audit-Evidence nur

per ID; 6. OpenTelemetry/W3C Trace Context dienen der Korrelation, nicht als Datenbankersatz.

Nebenbefund

Die bestehende Policy auf `agent_action_approvals` verwendet noch `auth.role() = 'service_role'`. Nach aktueller Supabase-Guidance ist `auth.role()` für Policies veraltet. Dieser Befund wird als separater IAM-Hardening-Punkt geführt und nicht ungeprüft in die M5-Mutation gemischt.

M5 Exit

M6 bleibt blockiert, bis: - Plan/ADR/Runbook Human-approved sind; - Pre-Mutation-Test PASS ist; - Supabase-Mutation durchgeführt ist; - RLS/Permission- und Append-only-Negativtests PASS sind; - ein synthetisches redigiertes Audit-Event geschrieben und gelesen werden kann; - verbotene Felder nicht persistiert werden; - Evidence und ROADMAP auf `VERIFIED PASS` aktualisiert sind.

M5 Supabase Agent Audit Mutation Evidence

Quelle: docs/evidence/m5/M5_SUPABASE_AGENT_AUDIT_MUTATION_EVIDENCE.md

M5 Supabase Agent Audit Mutation Evidence

Status: VERIFIED PASS — persistence mutation Date: 2026-08-12 Target: Supabase project AIFINANCIAL (rzywoktpmyhwzxmstyu) Roadmap authority: ADR-0056, ADR-0059, docs/run-books/M5_SUPABASE_AGENT_AUDIT_MUTATION.md

Approved trigger

PR #206 was Human/Owner reviewed and merged as planning authority before mutation.
Merge commit: 9322a7e274bfb765a5bbec71a3ce3107bde1d46c .

Pre-mutation baseline

- public.agent_audit_events did not exist.
- Existing audit authorities: audit_logs_iam , iam_access_log , agent_action_approvals , agent_evaluation_runs .
- Existing structures did not provide the complete ADR-0059 request/trace/agent/policy/PR/CI/deployment correlation contract.
- Rollback path was defined before mutation.

Applied production migrations

1. Supabase migration 20260811230540_m5_agent_audit_events - creates public.agent_audit_events ; - enables RLS; - adds correlation indexes; - creates append-only UPDATE/DELETE trigger; - denies anon and authenticated table access; - initially grants server-side access.
2. Supabase migration 20260811230743_m5_agent_audit_events_least_privilege - removes Supabase default extra privileges from service_role ; - leaves only SELECT + INSERT .

The matching repository migration files are recorded under supabase/migrations/ .

Post-mutation verification

RLS

public.agent_audit_events reports relrowsecurity = true .

Effective table privileges

Expected/verified:

- anon : SELECT = false, INSERT = false;

- `authenticated` : SELECT = false, INSERT = false;
- `service_role` : SELECT = true, INSERT = true, UPDATE = false, DELETE = false.

The first verification exposed unintended Supabase default privileges (`TRUNCATE` , `TRIGGER` , `REFERENCES`) on `service_role` ; the second migration removed them before M5 was accepted as PASS.

Synthetic redacted evidence event

A synthetic, non-secret event was persisted successfully:

- id: `5e69abb1-3f50-443e-abeb-214be4a70c88`
- request_id: `m5-verification-2026-08-12`
- trace_id: `00000000000000000000000000000001`
- agent_id: `m5-verification-agent`
- repository: `SvenKulesa/Finance`
- PR reference: `206`
- result: `PASS`
- attributes: `synthetic=true` , `contains_secrets=false`

The synthetic row intentionally remains as immutable audit evidence.

Append-only negative tests

Attempted UPDATE of the synthetic evidence row: **DENIED** with `agent_audit_events` is `append-only` .

Attempted DELETE of the synthetic evidence row: **DENIED** with `agent_audit_events` is `append-only` .

These tests were executed even through the privileged SQL path and therefore prove trigger-level fail-closed behavior in addition to the reduced `service_role` grants.

Supabase Security Advisor

Post-DDL advisor results:

- `RLS Enabled No Policy` for `agent_audit_events` : INFO and intentional. The table is policy-less deny-by-default for client roles; server access is controlled by explicit table grants plus service-role execution.
- `Leaked Password Protection Disabled` : pre-existing Auth warning; not introduced by M5.
- `Insufficient MFA Options` : pre-existing Auth warning; not introduced by M5.

The pre-existing `agent_action_approvals` policy still uses deprecated `auth.role()` semantics. This is recorded as a separate IAM maintenance item and was intentionally not mixed into the M5 observability mutation.

Data minimization

The persisted schema forbids raw secret material by contract. Full prompts, full diffs, tokens, private keys and raw sensitive request bodies are not part of the audit schema. Application integration must apply redaction before INSERT.

Mutation result

Persistence mutation: VERIFIED PASS.

M5 as a roadmap phase remains `IN PROGRESS` until the application writer is integrated and an end-to-end test proves that an authorized AI-assisted command can be reconstructed from request/trace/actor/agent/policy through PR/CI/runtime references without exposing secrets.

Rollback state

No rollback was required because all mandatory persistence checks passed. If application integration later fails, writers must be disabled first; the audit table must not be destructively dropped once productive evidence exists without a separate Owner-approved rollback decision.

M5 — Application Corrective Verification Closure Evidence

Quelle: docs/evidence/m5/M5_VERIFIED_PASS_CLOSURE_EVIDENCE.md

M5 — Application Corrective Verification Closure Evidence

Status: VERIFIED PASS **Date:** 2026-08-14 **Authority:** ADR-0056, ADR-0059, server/agentAudit/*, public.agent_audit_events **Requestor:** Owner ("finalisiere den Systemadministrator Agent um alle dokumentenbasierte Dokumente schrittweise umzusetzen" → SA-P02 aus ROADMAP_CONSOLIDATION_MASTER_INDEX.md §6) **Verifier:** Claude-Code-Sitzung, read-only gegen Produktions-Supabase + GitHub-API

1. Ausgangslage

Seit PR #222 (fix/sa3b-m5-audit-schema-contract , gemerkt 91963f5 am 2026-08-12 07:21:40 UTC) stand M5 auf „PERSISTENCE VERIFIED / APPLICATION CORRECTIVE VERIFICATION ACTIVE" mit der offenen Anforderung: „corrected writer must still deploy and prove a real successful privileged audit insert." Diese Formulierung blieb seit dem 12.08. unverändert in mehreren Dokumenten stehen, obwohl — wie diese Verifikation zeigt — der Nachweis bereits am selben Tag erbracht wurde und nur nie dokumentiert/synchronisiert wurde.

2. Read-only Produktionsnachweis

Abfrage von public.agent_audit_events (der von PR #222 korrigierten Tabelle):

15 Zeilen insgesamt, Zeitraum 2026-08-11 23:09 UTC – 2026-08-12 08:42 UTC

Relevante Kette (SA4-Pilot, zweiter/erfolgreicher Lauf, Branch agent/sa4-pilot-proof-20260812b):

Zeitpunkt (UTC)	Capability	Decision	Result	Event-ID
08:42:13	BRANCH	ALLOW	PENDING	1b4b04cb-a86b-4612-9ef5-308e95a18c95
08:42:14	BRANCH	ALLOW	SUCCESS	e99b9af7-74cc-4693-966f-c9b85102035d
08:42:16	COMMIT	ALLOW	PENDING	3c5916a1-d8c1-4ba1-9de1-d839fcc1bc85
08:42:17	COMMIT	ALLOW	SUCCESS	bc6ecf0b-86db-4b8c-ae95-2c963a0776f5
08:42:18	PR	ALLOW	PENDING	41874d2e-a7b7-48c9-8287-71d75-bea7d05

Zeitpunkt (UTC)	Capability	Decision	Result	Event-ID
08:42:20	PR	ALLOW	SUCCESS	d4722 - de8-3242-4822-ab7d-f353880312ac , pull_request_number: 229

Zusätzlich, zeitlich davor: die SA3B-Branch-Probe (agent/sa3b-host-probe-20260812b , 07:28:32-07:28:33 UTC, BRANCH ALLOW→SUCCESS) und ein erster SA4-Lauf (agent/sa4-pilot-proof-20260812a , 08:27 UTC), dessen PR-Schritt mit result: ERROR endete — die Roadmap-Doku (SA4-Abschnitt) dokumentiert diesen ersten Fehlversuch bereits separat als Teil der Lifecycle-Historie.

3. Unabhängige Kreuzverifikation gegen GitHub

pull_request_number: 229 aus der DB wurde gegen die tatsächliche PR #229 auf GitHub geprüft:

- erstellt von github-actions[bot] (nicht von einem menschlichen oder Chat-Account — korrekte, least-privilege Identität für den SA4-Host);
- created_at: 2026-08-12T08:42:19Z — exakt im Sekundenfenster der DB-Events;
- PR-Body zitiert **exakt dieselben sechs Event-IDs** wie oben, eigenständig aus dem GitHub-Actions-Workflow-Lauf heraus geschrieben, nicht aus der DB kopiert — zwei unabhängige Quellen (Supabase-DB und GitHub-API) korrelieren verlustfrei;
- merged: true , merged_by: SvenKulessa .

4. Deploy-Zeitpunkt-Nachweis (verhindert False-Positive durch Deploy-Lag)

```
git merge-base --is-ancestor 91963f5 (PR #222) f7dfcda (SA4-Pilot Basis-SHA) → true
```

Der SA4-Pilot lief nachweislich auf einem main -Stand, der PR #222s korrigierten Writer bereits enthielt — nicht auf einem veralteten, vor der Korrektur deployten Stand. Kein Deploy-Lag-Risiko.

5. Ergebnis

Anforderung	Ergebnis
Corrected writer (PR #222) deployed	Abschnitt 4
Real successful privileged audit insert	Abschnitt 2 (6 Events, davon 3 SUCCESS)
Unabhängige Korrelation	Abschnitt 3 (DB ↔ GitHub, exakte Event-ID-Übereinstimmung)

M5: VERIFIED PASS. Die Anforderung war bereits am 2026-08-12 technisch erfüllt; dieses Dokument schließt lediglich die seitdem offene Dokumentations-/Traceability-Lücke.

6. Nicht Teil dieser Verifikation

- keine neue Mutation, kein neuer Audit-Insert wurde durch diese Sitzung erzeugt — ausschließlich read-only Auswertung bereits existierender Produktionsdaten und einer bereits gemergten PR;
- autonome Mutation bleibt weiterhin gemäß SA4-Scope (`BRANCH → COMMIT → PR` , kein `CI_REQUEST` , kein Merge, keine Produktionsmutation) begrenzt — diese Verifikation erweitert keine Berechtigung.

M5A — Repository-Implementierung: Supabase Native TOTP MFA / AAL2

Quelle: docs/evidence/m5a/M5A_REPOSITORY_IMPLEMENTATION_EVIDENCE.md

M5A — Repository-Implementierung: Supabase Native TOTP MFA / AAL2

Status: CODE COMPLETE / CI PASS — PRODUKTIONSMUTATION NICHT AUTORISIERT Datum: 2026-08-14 Authority: ESS-0020, ADR-0064, ADR-0003.5, docs/roadmaps/M5A_SYSTE-MADMIN_REPOSITORY_WORK_PACKAGE.md Basis: main nach PR #254

Dieses Dokument ist ausschließlich der Repository-Code-Nachweis (Stage B/C der M5A_SUPABASE_TOTP_AAL2_HARDENING.md -Runbook-Sequenz). Es autorisiert **keine** Produktionsmutation. Kein Supabase Auth-Faktor wurde erstellt, verifiziert, entfernt oder verändert. Kein Owner-Profil wurde mutiert. Keine Secrets, Codes, Faktor-IDs oder TOTP-Payloads wurden gelesen, angezeigt oder in diesem Dokument, Logs oder dem PR-Diff festgehalten.

Umsetzung dieser Session

Diese Implementierung wurde **nicht** über den auditierten Systemadmin-SA4-Ausführungspfad erzeugt — der Audit-Befund P1-2 (docs/evidence/security/SECURITY_AUDIT_2026-08-14_ADR0069_DEVELOPMENT_CHAIN.md) stellte fest, dass der SA4-Host strukturell auf REM-SA4-PIL0T-001 /eine einzelne Zieldatei verdrahtet ist und REM-M5A-REPOSITORY-001 nicht ausführen kann. Der Owner hat sich explizit für die risikoärmere Alternative entschieden: direkte Implementierung durch die aktuelle, Owner-instruierte Claude-Code-Sitzung im normalen Branch → PR → Human-Review → CI → Merge-Zyklus, ohne neue autonome Ausführungsinfrastruktur zu bauen.

Umgesetzte Code-Slices

Slice A — Authority und Service

Datei	Änderung
src/platform/Security/nativeMfa.ts	Neu. Dünner, testbarer Wrapper um supabase.auth.mfa.{enroll,challenge,verify,getAuthenticatorAssuranceLevel,listFactors,unenroll}. Client wird injiziert (kein festes Modul-Import), damit Tests ohne Browser laufen.
src/platform/Security/authMiddleware.ts	Neuer Export requireVerifiedAal2(req) . Fragt supabase.auth.mfa.getAuthenticatorAssuranceLevel(token) direkt mit dem vorliegenden Bearer-Token ab (echter Netzwerk-Roundtrip gegen Supabase

Datei	Änderung
	Auth, Token wird dabei erneut validiert). Fail-closed bei fehlendem Token, ungültigem Token, Lookup-Fehler oder <code>currentLevel !== 'aal2'</code> . Kein Caching zwischen Requests — jede Prüfung ist frisch, daher kein "stale aal2/aal1"-Zustand möglich. <code>requireStepUp()</code> ruft <code>requireVerifiedAal2()</code> jetzt zusätzlich zur bestehenden Token-Prüfung auf.

Exit-Kriterien (verifiziert durch `tests/unit/authMiddlewareAal2.test.ts`, `tests/unit/nativeMfa.test.ts`):

- AAL1 für `requireVerifiedAal2 / requireStepUp` → DENY;
- fehlender/ungültiger Bearer-Token → DENY;
- AAL-Lookup-Fehler (Netzwerk/Auth) → DENY (fail-closed);
- kein Vertrauen in Client-/UI-Marker als Autorität (server-seitige Prüfung liest ausschließlich das signierte JWT-Claim über die Supabase-API, nie einen Client-Header oder sessionStorage-Marker).

Slice B — Enrollment/Login UI

Datei	Änderung
<code>src/components/TotpSettings.tsx</code>	Neuer, additiver Abschnitt „Native Zwei-Faktor-Authentifizierung (empfohlen)“ oberhalb des bestehenden Legacy-Abschnitts (jetzt „(Legacy)“ beschriftet). Enroll → QR/Secret anzeigen → Challenge → Verify. Legacy-Abschnitt unverändert funktionsfähig (ADR-0064 Punkt 5: Migration, keine Löschung).
<code>src/components/LoginStepUpGate.tsx</code>	Prüft beim Laden zuerst den nativen AAL-Status (<code>getCurrentAssuranceLevel</code>). Existiert ein verifizierter nativer Faktor (<code>nextLevel === 'aal2'</code>), hat der native Challenge-Pfad Vorrang vor Passkey/Legacy-TOTP. Ohne nativen Faktor bleibt der bisherige Passkey-/Legacy-Pfad unverändert aktiv. Siehe Nachtrag unten: die ursprünglich hier vorgesehene Break-Glass-Recovery-Option für den nativen Pfad wurde durch explizite Owner-Policy noch vor Merge verworfen und nicht umgesetzt.

Exit-Kriterien (verifiziert durch `tests/integration/nativeMfaAal2.test.ts`, `tests/unit/nativeMfa.test.ts`):

- unterstützter Supabase-nativer Ablauf (enroll → challenge → verify);
- ein `verify()`-Erfolg ohne tatsächliche AAL2-Sitzung zählt NICHT als Erfolg;
- Session/AAL wird nach Verify serverseitig unabhängig neu bewertet, nicht aus dem Client-Ergebnis übernommen;
- kein Secret/Code/Faktor-ID-Leak in Logs oder Evidence (Code enthält keine entsprechenden Log-Aufrufe).

Slice C — Purpose-bound Step-up

Datei	Änderung
<code>server/stepUp.ts</code>	<code>/step-up/verify</code> verlangt jetzt zusätzlich zum bestehenden Legacy-TOTP-Code eine gültige AAL2-Sitzung (<code>requireVerifiedAal2</code>), bevor ein Step-Up-Token ausgestellt wird — HTTP 428 <code>aal2_required</code> sonst. Siehe Nachtrag unten: <code>/break-glass/redeem</code> wurde durch explizite Owner-Policy vollständig entfernt statt (wie ursprünglich hier vorgesehen) um natives MFA erweitert.
<code>tests/unit/totp.test.ts</code>	Unverändert — bleibt als Migrations-/Regressionstest für die weiterhin vorhandene Legacy-RFC-6238-Implementierung bestehen.

Exit-Kriterien (verifiziert durch `tests/unit/authMiddlewareAal2.test.ts`):

- Step-Up-Ausstellung ohne AAL2 → DENY (428);
- Step-Up-Konsum (`requireStepUp`) ohne AAL2 → DENY, auch mit sonst gültigem, ungenutztem Token;
- Legacy-Verhalten (User-Scope, Hash-Match, Unused, Unexpired) unverändert erhalten.

Pflicht-Negativtests (Runbook Stage C, 11 Fälle)

#	Fall	Testdatei
1	Owner/Admin AAL1 verweigert	<code>authMiddlewareAal2.test.ts</code>
2	fehlender Faktor verweigert	<code>nativeMfaAal2.test.ts</code> (kein Faktor → <code>insufficient-aal</code>)
3	unverifizierter Faktor kein Erfolg	<code>nativeMfa.test.ts</code> , <code>nativeMfaAal2.test.ts</code>
4	falscher TOTP-Code verweigert	<code>nativeMfa.test.ts</code> , <code>nativeMfaAal2.test.ts</code>
5	ungültige/abgelaufene Challenge verweigert	<code>nativeMfa.test.ts</code> (Challenge-Fehler → <code>NativeMfaError</code>)
6	stale <code>aal2/aal1</code> verweigert	<code>authMiddlewareAal2.test.ts</code> (kein Cache — jede Prüfung frisch)
7	Auth-/AAL-Lookup-Fehler verweigert	<code>authMiddlewareAal2.test.ts</code> (<code>aal-lookup-failed</code> , <code>internal-error</code>)
8	Step-Up ohne AAL2 verweigert	<code>authMiddlewareAal2.test.ts</code>
9	falscher User/Purpose verweigert	bestehende Logik unverändert (User-Scope in <code>requireStepUp</code> / <code>step_up_tokens</code> -Query)
10	Replay verweigert	bestehende Logik unverändert (<code>used_at</code> IS NULL atomar)

#	Fall	Testdatei
11	unautorisierter Faktor-Reset verweigert	kein neuer serverseitiger Fremd-Reset-Pfad eingeführt; Faktor-Entfernung ausschließlich Self-Service (eigene Session) — kein Break-Glass-Pfad mehr, siehe Nachtrag

Verifikation dieser Session

```

npm run lint          -> tsc --noEmit, Exit 0
npx vitest run        -> siehe unten
npm audit --omit=dev --audit-level=high -> 0 Vulnerabilities
verifyGoogleMarketingInvariants.ts    -> 34/34 PASS
verifyProductionConfigInvariants.ts   -> 11/11 PASS
verifyDockerHardening.mjs             -> PASS
npm run build          -> PASS

```

Testergebnis: 142 bestehende + 3 neue Testdateien, 812 Tests bestanden, 2 architekturell übersprungen (unverändert gegenüber Baseline).

Nicht Teil dieses Work-Packages

- keine Änderung an `checkAdminAccess()` selbst oder an den ~9 weiteren Server-Dateien, die `checkAdminAccess / requireStepUp` verwenden (`adminDiagnostics.ts`, `agentEvaluationRouter.ts`, `ai.ts`, `documentHygiene.ts`, `orchestrator.ts`, `scoreExplainability.ts`, `scoreValidation.ts`, `supervisorRouter.ts`, `systemEvents.ts`) — außerhalb der REM-M5A-REPOSITORY-001-Pfad-Allowlist; `requireStepUp()` ändert sein Verhalten für den einzigen produktiven Aufrufer (`adminDiagnostics.ts`) ausschließlich durch die geänderte gemeinsame Funktion, ohne dass diese Datei selbst angefasst wurde;
- keine Löschung von Legacy-TOTP-Code oder -Spalten (ADR-0064 Punkt 5: Migration, kein sofortiger Cutover). Der Break-Glass-Recovery-Anwendungscode wurde dagegen entfernt (siehe Nachtrag) — die `break_glass_codes`-Produktionsdaten selbst wurden davon nicht berührt;
- keine Supabase-Projektkonfigurations-Mutation;
- keine Owner-Faktor-Registrierung.

Betriebliche Konsequenz — WICHTIG für die Owner-Freigabe

Sobald dieser PR nach `main` gemergt und deployed ist, verlangt `/step-up/verify` serverseitig eine echte AAL2-Sitzung. Da laut `M5A_SUPABASE_TOTP_AAL2_BASELINE.md` aktuell **0 native MFA-Faktoren** in Produktion existieren, können **beide Owner-Profil ab diesem Deploy keine neuen Step-Up-Token mehr erhalten**, bis mindestens ein Owner-Profil native MFA über die neue Oberfläche in `TotpSettings.tsx` eingerichtet hat. Das betrifft ausschließlich Step-Up-geschützte kritische Aktionen (aktuell: `adminDiagnosticsRouter` s `requireOwnerWithStepUp`) — normaler Login, normale App-Nutzung und alle anderen `checkAdminAccess`-geschützten Admin-Zonen sind davon **nicht** betroffen, da deren Code in diesem PR unverändert bleibt.

Das ist eine bewusste, im Runbook (`M5A_SUPABASE_TOTP_AAL2_HARDENING.md` , Abschnitt „Roll-back“) bereits antizipierte Konsequenz eines Hard-Cutovers, keine unbeabsichtigte Nebenwirkung. **Seit dem Nachtrag unten gibt es dafür keinen Break-Glass-Notfallpfad mehr** — Verlust von Passkey und Authenticator gleichzeitig kann nur noch außerhalb der Anwendung (Supabase-Dashboard durch den Owner) behoben werden. Empfehlung: unmittelbar nach Merge mindestens ein Owner-Profil über die neue Oberfläche registrieren, um die Lücke zu schließen — die Dringlichkeit dafür ist durch den Wegfall des Notfallpfads höher als ursprünglich in diesem Dokument dargestellt.

Nächster Schritt

Nach Merge und CI `VERIFIED PASS` : separates, explizites Owner-Mutation-Gate für die native TOTP-Registrierung der beiden Owner-Identitäten gemäß Runbook Stage D-I. Dieses Dokument autorisiert diesen nächsten Schritt nicht selbst.

Nachtrag 2026-08-14 — Entfernung des Break-Glass-Recovery-Pfads (Owner-Policy)

Nach Erstellung der obigen Fassung (noch vor Merge, gleiche PR/Branch) hat der Owner explizit angewiesen: „Es soll kein Notfall-Bypass-Code in der Anwendung stehen (Policy)“, konkretisiert über `AskUserQuestion` als „Break-Glass-Recovery-System komplett entfernen“. Damit sind die Aussagen weiter oben zu einer weiterhin nutzbaren Break-Glass-Recovery-Option **überholt** und durch diesen Nachtrag korrigiert.

Umgesetzt in derselben Session, zusätzlich zu Slice A-C:

Datei	Änderung
<code>server/stepUp.ts</code>	Route <code>/break-glass/redeem</code> vollständig entfernt (~113 Zeilen, inkl. Passkey- und nativer-MFA-Faktor-Bereinigung). <code>/totp/verify-setup</code> erzeugt keine Recovery-Codes mehr und liefert nur noch <code>{ success: true }</code> . Kein <code>break_glass_codes</code> -Zugriff mehr im Anwendungscode.
<code>src/components/TotpSettings.tsx</code>	Stage <code>'recovery-codes'</code> und die zugehörige UI (Codes-Grid, Copy-Button, Bestätigungsbutton) entfernt. Nach erfolgreicher Verifikation direkter Übergang zu <code>stage: 'idle'</code> mit Erfolgsmeldung.
<code>src/components/LoginStepUpGate.tsx</code>	Recovery-Code-Eingabe, zugehörige States und <code>handleRecoveryRedeem</code> entfernt. Statischer Hinweistext: kein automatischer Recovery-Weg, Kontakt zum Owner bei Verlust von Passkey/Authenticator.
<code>src/platform/Security/totp.ts</code>	Zusätzlich, unabhängig von der Break-Glass-Entfernung: <code>TOTP_WINDOW_STEPS = 1</code> als benannte Konstante extrahiert und mit RFC-6238-§5.2-Zitat dokumentiert (Owner-Anfrage „MFA-Faktor auf die empfohlenen Best-Practice-Anzahl ändern“ — Antwort: der bestehende Wert war bereits die von der RFC selbst empfohlene Zahl; keine funktionale Änderung, nur Benennung/Dokumentation).

Ausdrücklich **nicht** Teil dieses Nachtrags: die `break_glass_codes`-Tabelle und ihre 20 unbenutzten Produktions-Zeilen (siehe `M5A_SUPABASE_TOTP_AAL2_BASELINE.md`) wurden **nicht** gelöscht oder verändert. Das ist eine separate, noch zu treffende Produktions-Datenmutations-Entscheidung außerhalb des Anwendungscodes und wird hier nur referenziert, nicht ausgeführt.

Verifikation: `npx tsc --noEmit` (0 Fehler), betroffene Testdateien (`totp.test.ts`, `nativeMfa.test.ts`, `authMiddlewareAal2.test.ts`, `nativeMfaAal2.test.ts`) grün.

M5A — Supabase TOTP MFA / AAL2 Baseline Evidence

Quelle: docs/evidence/m5a/M5A_SUPABASE_TOTP_AAL2_BASELINE.md

M5A — Supabase TOTP MFA / AAL2 Baseline Evidence

Status: READ-ONLY BASELINE COMPLETE / REMEDIATION PLAN IN REVIEW
Date: 2026-08-12
Baseline: `main@ee65ba19f64e7e8ee2d618e16364a658dfe60e4c` (PR #211 merge)
Authority: ESS-0020, ADR-0064, ADR-0003.5
Mutation State: NOT PERFORMED — HUMAN APPROVAL REQUIRED

1. Scope

This evidence records the first M5A gate only:

- repository inspection of TOTP enrollment/login/step-up and IAM enforcement;
- read-only Supabase Auth/database inventory;
- current Supabase documentation comparison;
- gap classification and exact mutation decision.

No Supabase Auth factor was created, challenged, verified, removed or modified. No project Auth setting, database schema, Render setting or Stripe configuration was mutated.

2. Production Supabase baseline

Project: `AIFINANCIAL` (`ryzywoktpmyhwzxmstyu`)
Project health: `ACTIVE_HEALTHY`
Organization plan: `free`

Auth schema presence

Read-only information-schema inspection confirms:

- `auth.mfa_factors` exists;
- `auth.mfa_challenges` exists;
- `auth.sessions` exists.

This means no custom Postgres DDL is required to create Supabase Native MFA storage.

Native factor state

Aggregate-only query of `auth.mfa_factors` :

```
0 native MFA factors
```

No factor IDs, secrets or user identities were read.

Session assurance state

Aggregate-only query of `auth.sessions` :

```
aal1: 2 sessions
aal2: 0 sessions
```

Therefore no current production session demonstrates Supabase Native MFA assurance.

Legacy application MFA state

Aggregate read of `public.profiles` :

```
owner + legacy totp_enabled=true: 2 profiles
user  + legacy totp_enabled=false: 2 profiles
```

Additional aggregate state:

```
legacy break_glass_codes: 20 rows, 20 unused
legacy step_up_tokens: 0 rows, 0 currently valid
```

This proves that the two Owner identities have CAPITAL-AI's historical custom TOTP enabled while Supabase Auth itself has no MFA factors.

Security Advisor

Current relevant warning:

- `auth_insufficient_mfa_options` — project currently has too few MFA options enabled/configured.

Separate warning:

- Leaked Password Protection disabled. This remains a Pro+ plan dependency and is not an M5A TOTP blocker under the current Free plan.

The Advisor must be rerun after any later Auth/config mutation.

3. Repository baseline

`src/platform/Security/totp.ts`

Implements RFC-6238 TOTP locally with Node crypto. This code is independent of Supabase Auth MFA and cannot produce a Supabase `aal2` session.

`server/stepUp.ts`

Current application flow:

1. `/totp/setup` generates a CAPITAL-AI secret;
2. secret is encrypted into `profiles.totp_pending_secret_encrypted` ;
3. `/totp/verify-setup` verifies via local `verifyTotp()` ;

4. verified secret is persisted to `profiles.totp_secret_encrypted` and `totp_enabled=true`;
5. CAPITAL-AI creates 10 own break-glass codes;
6. `/step-up/verify` verifies the local secret and issues an application `step_up_token`.

No `supabase.auth.mfa.enroll`, `challenge` or `verify` is used.

`src/components/TotpSettings.tsx`

Uses the custom server routes and reads `profiles.totp_enabled` as the displayed 2FA state. It does not use native factor state or AAL.

`src/lib/loginStepUp.ts`

Findings:

- Passkey is checked first;
- custom TOTP is checked only through `profiles.totp_enabled`;
- a tab-local `sessionStorage` marker can suppress repeated checks;
- factor/status lookup failures degrade to `none` and allow login to proceed.

The last behavior is unacceptable for privileged Owner/Admin assurance because M5A requires fail-closed behavior.

`src/components/LoginStepUpGate.tsx`

A successful passkey confirmation can complete the current login step-up independently of TOTP. This conflicts with ADR-0003.5's stated Owner requirement that TOTP is mandatory independently of passkey availability.

`src/platform/Security/authMiddleware.ts`

`checkAdminAccess()` correctly verifies Supabase user identity and IAM role, but there is no application-code enforcement of `aal2` today. Repository-wide search for `aal2` returns documentation references rather than runtime enforcement.

Tests

`tests/unit/totp.test.ts` validates the local RFC-6238 implementation. It does not prove Supabase enrollment/challenge/verify, AAL transition, stale-session handling or privileged AAL2 enforcement.

4. Gap matrix

Requirement	Current state	Result
Native TOTP enrollment	custom local TOTP	GAP
Supabase <code>challenge</code> → <code>verify</code>	absent	GAP
AAL2 session produced	0 production AAL2 sessions	GAP
Server rejects Owner/Admin <code>aal1</code>	not implemented	CRITICAL GAP

Requirement	Current state	Result
Stale aal2/aal1 rejected	not implemented	GAP
MFA status from authoritative factor source	profiles.totp_enabled	GAP
Privileged auth failure fail-closed	login status check can fail-open	CRITICAL GAP
Purpose-bound step-up	implemented	KEEP / HARDEN ABOVE AAL2
Recovery	custom codes exist	MIGRATION REQUIRED
Native MFA negative tests	absent	GAP
Security Advisor after mutation	required later	PENDING

5. Root cause

The historical IAM hardening added a technically valid local second factor but placed it **be-side** Supabase Auth instead of **inside** the Supabase session assurance model.

As a result:

```
Supabase identity/session = source A
CAPITAL-AI TOTP state     = source B
CAPITAL-AI step-up token  = source C
```

These sources are correlated by user ID but do not create a single provider-authenticated AAL2 assertion. M5A closes this split-brain authority.

6. Target architecture

```
Primary login / session
  |
  v
Supabase Auth session (aal1)
  |
  +-- native TOTP enroll → challenge → verify
  |
  v
Supabase Auth session (aal2)
  |
  v
Server IAM role check + central AAL2 check
  |
  +-- critical action? → purpose-bound single-use CAPITAL-AI step-up
  |
  v
Privileged operation
  |
  v
Audit evidence
```

The purpose-bound token remains an additional action-control, not an MFA replacement.

7. Mutation decision

Repository code: REQUIRED

Required implementation areas:

- Native TOTP enrollment UI/service;
- native challenge/verify login gate;
- centralized server AAL2 verification;
- privileged Owner/Admin enforcement;
- stale/error fail-closed behavior;
- recovery redesign;
- native MFA unit/integration negative tests.

Supabase Auth factor mutation: REQUIRED, but not yet authorized

Both Owner identities currently have no native MFA factor. To reach M5A `VERIFIED PASS`, they must later enroll and verify native TOTP through supported Supabase Auth APIs.

Supabase project configuration mutation: CONDITIONAL

Current official Supabase guidance states TOTP MFA APIs are available by default, but the available connector does not expose the exact Dashboard Challenge/Verify setting for this project. Therefore no project-setting mutation is justified by assumption.

Pre-mutation gate must determine the actual configuration. If native challenge/verify works without config change, project config mutation is `NOT REQUIRED`.

Postgres DDL: NOT REQUIRED for Native MFA

The native Auth schema already exists.

Legacy cleanup: DEFERRED

Do not remove:

- encrypted legacy TOTP values;
- legacy break-glass rows;
- custom verifier code;

until native factor enrollment, recovery and AAL2 enforcement are verified and a separate cleanup mutation is approved.

8. Required implementation tests

Negative:

- Owner/Admin AAL1 denied;
- enrolled/unverified factor denied;
- invalid TOTP denied;

- invalid/expired challenge denied;
- no factor denied where MFA is required;
- stale AAL2/AAL1 denied;
- Auth/AAL lookup failure denied for privileged access;
- purpose-bound token without AAL2 denied;
- wrong-user/wrong-purpose/replayed step-up denied;
- unauthorized factor-reset denied.

Positive:

- native enroll/challenge/verify reaches AAL2;
- AAL2 Owner/Admin with valid policy/role/action step-up can execute allowed privileged operation;
- audit event is emitted without storing TOTP secret/code.

9. Current gate

M5A state: IN PROGRESS – BASELINE COMPLETE / REMEDIATION PLAN IN REVIEW

Production mutation: BLOCKED – HUMAN/OWNER APPROVAL REQUIRED

Next allowed action after merge of the architecture/baseline PR:

1. implement repository code on a fresh branch;
2. test via CI;
3. then open an explicit Human/Owner production-mutation gate for native Owner factor enrollment.

M6 remains blocked until M5A is VERIFIED PASS .

M5A — Native Factor Enrollment Closure Evidence

Quelle: docs/evidence/m5a/M5A_VERIFIED_PASS_CLOSURE_EVIDENCE.md

M5A — Native Factor Enrollment Closure Evidence

Status: VERIFIED PASS **Date:** 2026-08-14 **Baseline compared against:** M5A_SUPABA-SE_TOTP_AAL2_BASELINE.md (2026-08-12, 0 native factors, 0 aal2 sessions) **Authority:** ESS-0020, ADR-0064, ADR-0003.5, ADR-0071 (Mutation-Proposal-Zustandsmodell) **Requestor:** Owner (direct Chat-Anweisung, native MFA für beide Owner-Profil selbst eingerichtet) **Verifier:** Claude-Code-Sitzung, read-only gegen Produktions-Supabase (AIFINANCIAL / rzyzwoktpmyhwzxmstyu)

Dieses Dokument schließt die in DEVELOPMENT_CHAIN_ROADMAP.md (M5A, Schritte 9–13) und ROADMAP_CONSOLIDATION_MASTER_INDEX.md (Abschnitt 5, „M5A Exit“) offen gelassenen Verifikationsschritte ab. Alle Prüfungen sind read-only; es wurde kein Auth-Faktor, keine Projektkonfiguration und kein Schema durch diese Sitzung mutiert — die Faktor-Registrierung selbst wurde vom Owner interaktiv über TotpSettings.tsx durchgeführt.

1. Aktive Faktorart und Zielidentität (redigiert)

Read-only Abfrage von auth.mfa_factors , JOIN auf public.profiles (nur iam_role='owner'):

Owner-Profil	Faktortyp	Status	Verifiziert seit
Owner-Profil 1	totp (nativ)	verified	2026-08-14 16:49 UTC
Owner-Profil 2	totp (nativ)	verified	2026-08-14 18:05 UTC

Keine Secrets, Codes oder Faktor-IDs wurden gelesen oder hier festgehalten — nur Faktortyp, Status und Zeitstempel.

2. AAL2-Positivtest — PASS

Read-only Abfrage von auth.sessions (Supabase-eigene, serverseitig verwaltete Session-Tabelle, exakt die Quelle, die requireVerifiedAal2() in authMiddleware.ts über getAuthenticatorAssuranceLevel() abfragt):

Owner-Profil	Aktuellste Session-AAL	Aktualisiert
Owner-Profil 1	aal2	2026-08-14 16:50 UTC
Owner-Profil 2	aal2	2026-08-14 18:06 UTC

Beide Owner-Identitäten haben damit eine von Supabase selbst — nicht von der Anwendung — bezeugte AAL2-Sitzung. Vergleich zur Baseline vom 2026-08-12: dort aal1: 2, aal2: 0 .

3. AAL1-/fehlender-Faktor-Negativtest — DENY (bestehende Testabdeckung, Code deployed)

Nicht erneut gegen Produktion provoziert (kein Grund, echte Admin-Endpunkte absichtlich mit ungültigen Sitzungen zu bombardieren, wenn die Logik bereits deterministisch getestet und der Code nachweislich in `main` gemergt/deployed ist). Nachweis stattdessen über:

- `tests/unit/authMiddlewareAal2.test.ts` — deckt fehlenden Token, ungültigen Token, AAL-Lookup- Fehler und `currentLevel !== 'aal2'` ab, alle DENY;
- `tests/integration/nativeMfaAal2.test.ts` — Ende-zu-Ende en-roll→challenge→verify→AAL2-Pfad;
- Code ist Teil von PR #255 (gemerged `main@66da35b`) und PR #256 (gemerged `main@0efeb05`) — beide `VERIFIED PASS` durch CI vor Merge, `requireVerifiedAal2()` ist der einzige Prüfpfad für `/api/auth/step-up/verify`, keine alternative Bypass-Route existiert (Break-Glass wurde in derselben PR-Kette vollständig entfernt).

4. Recovery/Reset-Runbook — geprüft

`docs/runbooks/M5A_SUPABASE_TOTP_AAL2_HARDENING.md`, Stage H, ist bereits mit dem Owner-Policy-Nachtrag 2026-08-14 aktuell: kein In-App-Recovery-Pfad mehr (Break-Glass entfernt), Verlust von Passkey + Authenticator ist ausschließlich über Supabase-Dashboard-Administration durch den Owner lösbar. Beide Owner-Profile haben jetzt zusätzlich mindestens einen Passkey UND einen nativen TOTP-Faktor — das Verlustrisiko ist dadurch strukturell reduziert (zwei unabhängige Faktoren pro Profil), nicht nur dokumentiert.

5. Privilegierter Auditinsert / Korrelation

`auth.audit_log_entries` ist auf diesem Projekt leer (kein separates GoTrue-Audit-Log aktiv) — daher kein Nachweis über diesen Kanal möglich. Die maßgebliche, tatsächlich von der Anwendung genutzte Quelle ist stattdessen direkt herangezogen worden: `auth.mfa_factors` und `auth.sessions` sind exakt die Tabellen, gegen die `requireVerifiedAal2()` / `getAuthenticatorAssuranceLevel()` zur Laufzeit prüfen (siehe Abschnitt 1-2) — das ist die authoritative Quelle selbst, nicht nur ein Audit-Log darüber, und wurde read-only mit Zeitstempel-Korrelation zur Owner-Aktion (Chat-Anweisung „native MFA ist nun aktiviert“ um 2026-08-14) bestätigt.

6. Advisor/Policy — kein unowned HIGH/CRITICAL

`get_advisors(type=security)` erneut ausgeführt (Baseline vom 2026-08-12 zeigte `WARN: auth_insufficient_mfa_options`):

Finding	Level	Status
<code>auth_insufficient_mfa_options</code>	—	behoben — nicht mehr im aktuellen Advisor-Ergebnis
	INFO	

Finding	Level	Status
<code>rls_enabled_no_policy (public.agent_audit_events)</code>		vorbestehend, unabhängig von M5A
<code>auth_leaked_password_protection</code>	WARN	vorbestehend, unabhängig von M5A (Free-Plan-Limitierung, bereits in der Baseline dokumentiert)

Kein neues, kein HIGH-, kein CRITICAL-Finding durch diese Änderung.

7. Ergebnis

M5A-Exit-Kriterium (ROADMAP_CONSOLIDATION_MASTER_INDEX.md §5)	Ergebnis
1. aktive Faktorart und Zielidentität redigiert belegt	Abschnitt 1
2. AAL2-Positivtest PASS	Abschnitt 2
3. AAL1-/fehlender-Faktor-Negativtest DENY	Abschnitt 3 (Testabdeckung + deployter Code)
4. Recovery/Reset Runbook geprüft	Abschnitt 4
5. privilegierter Auditinsert erfolgreich korreliert	Abschnitt 5 (authoritative Quelle direkt, kein separates Audit-Log verfügbar)
6. Advisor/Policy ohne unowned HIGH/CRITICAL	Abschnitt 6
7. Roadmap und Traceability synchronisiert	siehe begleitender Commit

M5A: VERIFIED PASS. M6 (Supply Chain Provenance) ist damit laut `DEVELOPMENT_CHAIN_ROADMAP.md` entsperrt — Repository-Code-Implementierung für M6 bleibt trotzdem ein separater, neuer Auftrag (kein automatischer Start durch dieses Dokument).

8. Nicht Teil dieser Verifikation

- keine Faktor-Mutation durch diese Sitzung (Owner hat selbst über die UI eingerichtet);
- keine Legacy-Cleanup-Entscheidung (`totp_secret_encrypted`, `profiles.totp_enabled`, Legacy-Step-up-Code) — bleibt laut Runbook Stage J ein separater, noch zu treffender Mutation-Proposal;
- keine Aussage über Nicht-Owner-Konten (M5A-Scope war explizit auf Owner-/Admin-Identitäten begrenzt).

M6 — Supply Chain Provenance: Repository Implementation Evidence

Quelle: docs/evidence/m6/M6_REPOSITORY_IMPLEMENTATION_EVIDENCE.md

M6 — Supply Chain Provenance: Repository Implementation Evidence

Status: **VERIFIED PASS** (siehe Nachtrag Abschnitt 0.1) Datum: 2026-08-14 (Nachträge: 2026-08-14, selber Tag) Roadmap phase: M6 Authority: ADR-0060, docs/runbooks/M6_SUPPLY_CHAIN_PROVENANCE.md, AI_AGENT_SUPPLY_CHAIN_MODEL.md Executor: direkte Owner-instruierte Claude-Code-Sitzung (kein SA4/neuer autonomer Host — siehe Owner-Entscheidung zu M5A, dieselbe Vorgehensweise für M6 bestätigt) Standards-Baseline: SLSA v1.2 (Provenance-Prädikat), NIST SSDF SP 800-218 v1.1 als stabile Referenz — kein Draft-Requirement als final dargestellt.

0. Nachtrag — erster echter push -Lauf, realer Blocker, Fix

Nach Merge dieses PRs löste der erste push -Lauf auf main (Merge-Commit 7c73d552b9d2994d8df9c64f554fb07a87f5330c, Run 31832197746) den Job supply-chain-attestation real aus. build-and-test und der Render-Deploy liefen PASS; der Attestation-Schritt selbst schlug fehl:

```
##[error]Error: Failed to persist attestation: Feature not available for user-owned private repositories.  
To enable this feature, please make this repository public.
```

Root Cause (verifiziert per git clone gegen actions/attest-build-provenance@v4.2.2 und actions/attest@508db95d... — den vom Composite-Wrapper intern verwendeten Node-Action —, inkl. Lesen von dist/index.js): actions/attest ruft GitHubs Attestations-API (writeAttestation) **unbedingt** auf, sobald die Sigstore-Signatur erzeugt wurde; die zugrunde liegende @actions/attest-Bibliothek kennt zwar ein skipWrite-Flag, aber die Action exponiert es nicht als Input — es gibt also keinen Weg, mit diesem Werkzeug die Persistierung zu umgehen. GitHubs Attestations-API ist laut GitHub-Dokumentation nur für öffentliche Repositories oder für private/interne Repositories **unter einer Organisation** (GitHub Team/Enterprise Cloud) verfügbar — nicht für private Repositories unter einem persönlichen Account. SvenKulessa/Finance ist genau das: ein privates Repository unter einem persönlichen Account. Dies ist ein struktureller Plattform-Blocker, kein Fehler in diesem Repository-Code — genau der Fall, den Abschnitt 1 dieses Dokuments ursprünglich offen gelassen hatte, statt vorzeitig „VERIFIED PASS“ zu behaupten.

Owner-Entscheidung (AskUserQuestion, 2026-08-14): von drei vorgelegten Optionen — (a) unabhängige Sigstore-Signatur ohne GitHub-Attestations-API, (b) Repo-Sichtbarkeit-/Ownership ändern (öffentlich machen oder in eine Organisation übertragen — bewusst nicht eigenständig vorgenommen, da Business-/Sicherheitsentscheidung), (c) Signing vorerst zurückstellen — wurde **(a) gewählt**.

Fix: `supply-chain-attestation` signiert `dist/security/provenance.json` jetzt per `cosign sign-blob --yes --bundle ... keyless` gegen die öffentliche Sigstore-Infrastruktur (Fulcio-Kurzzeit-Zertifikat gebunden an die GitHub-Actions-OIDC-Identität des Workflows, Eintrag im öffentlichen Rekord-Transparency-Log) — vollständig unabhängig von GitHub-Repository-Sichtbarkeit/-Ownership. Ein unmittelbarer `cosign verify-blob`-Schritt im selben Job verifiziert die Signatur gegen die erwartete Zertifikats-Identität (<https://github.com/<repo>/github/workflows/ci.yml@refs/heads/main>) und den erwarteten OIDC-Issuer (<https://token.actions.githubusercontent.com>) — das im Runbook geforderte „Attestation verifiziert mit erwarteter Builder-Identität“ wird also real im selben Lauf geprüft, nicht nur behauptet. Das signierte Bundle plus SBOM/Provenance/Release-Manifest werden als Workflow-Artefakt (`actions/upload-artifact` , 90 Tage Aufbewahrung) abgelegt statt im GitHub-Attestations-Tab.

`sigstore/cosign-installer@6f9f17788090df1f26f669e9d70d6ae9567deba6` (Tag `v4.1.2`) und `actions/upload-artifact@043fb46d1a93c77aae656e7c1c64a875d1fc6a0a` (Tag `v7.0.1`) wurden beide per `git clone --branch <tag>` direkt gegen ihre Ursprungs-Repositories verifiziert, nicht nur aus einer gescrapten Seite übernommen — derselbe Verifikationsstandard wie bei `actions/attest-build-provenance` zuvor. Die `attestations: write`-Berechtigung wurde entfernt (nicht mehr benötigt); `id-token: write` bleibt für die OIDC-Anfrage an Fulcio erforderlich.

Der Fix wurde zum Zeitpunkt dieses ersten Nachtrags lokal vollständig implementiert und verifiziert (Abschnitt 6), aber noch nicht gemergt — Status blieb bewusst unterhalb `VERIFIED PASS`. Siehe Abschnitt 0.1 für den realen Nachweis nach Merge.

0.1 Nachtrag — realer `push`-Lauf nach dem `cosign`-Fix: erfolgreich signiert und verifiziert

PR #262 (der `cosign`-Fix aus Abschnitt 0) wurde vom Owner per Review () freigegeben und gemerged (Merge-Commit `c10a232a72a1c9fc73f3aaa4821649aff08e3389`). Der dadurch ausgelöste reale `push`-Lauf auf `main`, Run [31834114193](#), lief vollständig **PASS** — alle vier Jobs (`Human-/Owner-Vorprüfung` , `build-and-test` , `Deployment verifiziert` / `Render-Produktion` , `Supply-Chain-Provenance` / `Attestation`) mit `conclusion: success`.

Job-Log des Attestation-Schritts (Job-ID `94877020826`, wörtlich):

```
$ cosign sign-blob --yes --bundle dist/security/provenance.json.sigstore.json dist/security/provenance.json
Generating ephemeral keys...
Using payload from: dist/security/provenance.json
Signing artifact...
Wrote bundle to file dist/security/provenance.json.sigstore.json

$ cosign verify-blob --bundle dist/security/provenance.json.sigstore.json \
  --certificate-identity "https://github.com/SvenKullessa/Finance/.github/workflows/ci.yml@refs/heads/main" \
  --certificate-oidc-issuer "https://token.actions.githubusercontent.com" \
  dist/security/provenance.json
Verified OK
```

Die Signatur wurde also nicht nur erzeugt, sondern **im selben Lauf real gegen die exakt erwartete Zertifikats-Identität und den erwarteten OIDC-Issuer verifiziert** —

genau das im Runbook geforderte „Attestation verifiziert mit erwarteter Builder-Identität“. `actions/upload-artifact` legte das signierte Bundle plus SBOM/Provenance/Release-Manifest ab: Artefakt `supply-chain-provenance-c10a232a72a1c9fc73f3aaa4821649aff08e3389`, Artifact-ID `9231890684`, SHA-256 des hochgeladenen Zip-Archivs `2522c28f52e378079b-c80a45af8b3eec2449fccbee2c85ef104dbb3dd054ec5f`, abrufbar unter <https://github.com/SvenKulesa/Finance/actions/runs/31834114193/artifacts/9231890684> (90 Tage Aufbewahrung ab 2026-08-14).

Grenzen dieses Nachweises, offen benannt: Der eingebettete Rekor-Transparency-Log-Index/-URI innerhalb der `.sigstore.json`-Bundle-Datei selbst wurde nicht zusätzlich aus der Rohdatei extrahiert — der Download-Link des Artefakts zeigt auf Azure Blob Storage, das der Egress-Proxy dieser Sandbox-Sitzung blockiert (`CONNECT tunnel failed, response 403`), dieselbe bereits mehrfach in dieser Sitzung dokumentierte Netzwerkbeschränkung. Die Evidence stützt sich stattdessen auf die GitHub-Actions-Job-Logs selbst als authoritative, von GitHub erzeugte Aufzeichnung (`cosign verify-blob` meldet nur bei tatsächlich gültiger, gegen Fulcio/Rekor geprüfter Signatur `Verified OK` — ein Fake-Erfolg ohne echte Signatur ist damit ausgeschlossen). Das signierte Bundle selbst bleibt für die Aufbewahrungsdauer über den obigen Artifact-Link unabhängig nachprüfbar (`cosign verify-blob --bundle ...` durch jeden Dritten mit Repo-Zugriff).

Damit sind alle im Runbook geforderten Kettenglieder — `source` → `lockfile` → `SBOM` → `artifact` → `provenance/attestation` — nicht nur lokal, sondern auf dem tatsächlichen, gehosteten `main`-Build-Pfad real erzeugt und verifiziert. M6 wird auf **VERIFIED PASS** gehoben.

1. Ursprüngliche Begründung (historisch) für den Zwischenstatus „nicht VERIFIED PASS“

Der M6-Runbook-Exit-Gate verlangt explizit: „exact source → lockfile → SBOM → artifact → provenance/attestation chain verifies“ — die **Attestation** selbst kann per Definition nur von einem echten, gehosteten CI-Lauf auf `main` erzeugt werden (nicht lokal, nicht von einem Agenten). Der Workflow-Job `supply-chain-attestation` in `ci.yml` läuft ausschließlich auf `push + main` (`if: github.event_name == 'push' && github.ref == 'refs/heads/main'`). Der erste reale Lauf (Nachtrag Abschnitt 0) deckte einen strukturellen Plattform-Blocker in der ursprünglich gewählten Mechanik auf; der zweite reale Lauf nach dem `cosign`-Fix (Nachtrag Abschnitt 0.1) lief vollständig PASS inkl. realer Signaturverifikation — der oben beschriebene Zwischenstatus ist damit aufgelöst.

2. Umgesetzte Kette

```
source SHA (git HEAD / GITHUB_SHA)
→ package-lock.json (SHA-256 gebunden)
→ CycloneDX-SBOM (dist/security/sbom.cdx.json, an source+lock gebunden)
→ Release-Manifest (dist/control-plane/release-manifest.json, bereits vorhanden aus M5A/früherer Arbeit)
→ in-toto/SLSA-v1-Provenance-Statement (dist/security/provenance.json)
→ cosign keyless signing (Sigstore Fulcio/Rekor, GitHub-0IDC-Identität) – siehe Nachtrag Abschnitt 0
```

Neue Dateien

Datei	Zweck
<code>scripts/automation/sourceIdentity.ts</code>	gemeinsame <code>resolveSourceCommit()</code> — verhindert, dass Release-Manifest, SBOM und Provenance unabhängig voneinander (und potenziell inkonsistent) den Quell-Commit ableiten.
<code>scripts/automation/buildSupplyChainProvenance.ts</code>	erzeugt <code>dist/security/provenance.json</code> aus bereits vorhandenem Release-Manifest + SBOM; bricht fail-closed ab, wenn deren Quell-Commits nicht exakt übereinstimmen.
<code>scripts/security/verifySupplyChainProvenance.ts</code>	Gate-Skript: prüft Konsistenz der gesamten Kette (Quell-Commit, Lockfile-Digest, Artefakt-Digests, <code>--require-ci</code> -Modus für Builder-Identität). Exportiert die reine Prüf-Funktion <code>verifySupplyChainConsistency()</code> für Tests.

Geänderte Dateien

Datei	Änderung
<code>scripts/automation/dependencySecurity.ts</code>	<code>buildCycloneDxSbom / writeCycloneDxSbom</code> binden jetzt optional Quell-Commit + Lockfile-Digest in <code>metadata.properties</code> der SBOM ein.
<code>scripts/automation/buildRuntimeReleaseManifest.ts</code>	nutzt jetzt die gemeinsame <code>resolveSourceCommit()</code> statt einer eigenen Kopie.
<code>package.json</code>	neue Skripte <code>supplychain:provenance</code> , <code>supplychain:verify</code> ; beide in <code>predeploy:check</code> verkettet.
<code>.github/workflows/ci.yml</code>	neuer Job <code>supply-chain-attestation</code> (nur <code>push</code> + <code>main</code> , eigene minimale Berechtigungen <code>contents: read</code> , <code>id-token: write</code> — NICHT workflow-weit, damit PR-Läufe diese Rechte nie erhalten); signiert per <code>sigstore/cosign-installer</code> + <code>cosign sign-blob / cosign verify-blob</code> , legt Bundle+Artefakte per <code>actions/upload-artifact</code> ab (siehe Nachtrag Abschnitt 0).

3. Provenance-Format

In-toto Statement v1 (<https://in-toto.io/Statement/v1>) mit SLSA-Provenance-v1-Prädikat (<https://slsa.dev/provenance/v1>). `subject` bindet `release-manifest.json` und `sbom.cdx.json` per SHA-256. `predicate.buildDefinition.resolvedDependencies` bindet den exakten Git-Commit und den Lockfile-Digest. `predicate.runDetails.builder.id` ist die GitHub-Actions-Workflow-URL (nicht irgendeine Agenten-/Modell-/Provider-Identität) — außerhalb von GitHub Actions wird explizit `local-developer-build` gesetzt, was `--require-ci` gezielt ablehnt.

Bewusst **kein** selbstgebautes „Signieren“: die Vertrauensbasis ist ausschließlich die öffentliche Sigstore-Infrastruktur (Fulcio/Rekor) über `sigstore/cosign-installer@6f9f17788090df1f26f669e9d70d6ae9567deba6` (Tag `v4.1.2`) und `cosign sign-blob --yes --`

bundle ... keyless signing, gebunden an die GitHub-Actions-OIDC-Identität des Workflows — beide Action-SHAs verifiziert per `git clone --branch <tag>` direkt gegen ihre Ursprungs-Repositories, nicht nur aus einer gescrapten Seite übernommen. Ursprünglich war `actions/attest-build-provenance` vorgesehen; der erste reale `push`-Lauf deckte auf, dass dessen GitHub-Attestations-API für private, unter einem persönlichen Account gehaltene Repositories nicht verfügbar ist (siehe Nachtrag Abschnitt 0) — cosign ist davon unabhängig, da es ausschließlich gegen die öffentliche Sigstore-Infrastruktur signiert, nicht gegen ein GitHub-internes, plan-/ownership-gegatetes Feature. Ein lokal erzeugtes „Signat“ wäre weiterhin wertlos, da kein vertrauenswürdiges Schlüsselmateriale in diesem Repository oder auf einem Entwickler-Laptop liegt — keyless signing gegen Fulcio erfordert die echte GitHub-Actions-OIDC-Identität und ist lokal nicht reproduzierbar.

4. Required Positive Tests (Runbook) — Nachweis

Test	Nachweis
SBOM für exakten Quell-/Lock-Zustand erzeugt	<code>tests/unit/dependencySecurity.test.ts</code> (2 neue Tests) + lokaler Ende-zu-Ende-Lauf (Abschnitt 5)
SBOM enthält erwartete Top-Level-Identität + Dependency-Inventar	bestehender Test, unverändert grün
Provenance-Subject-Digest == gebautes Artefakt	<code>tests/unit/verifySupplyChainProvenance.test.ts</code> „PASS: consistent chain“ + lokaler Lauf
Quellreferenz == exakter Commit-SHA	dito
Attestation verifiziert mit erwarteter Builder-Identität	Real bestätigt im <code>push</code> -Lauf <code>31834114193</code> : <code>cosign verify-blob</code> meldet <code>Verified OK</code> gegen die exakt erwartete Zertifikats-Identität + OIDC-Issuer — siehe Nachtrag Abschnitt 0.1
Rollback-Artefakt per Digest abrufbar	Release-Manifest (<code>buildIdentity</code>) bereits als immutable Referenz etabliert (M5A/vorherige Arbeit); keine neue Mutation hier
Workflow-Security-Checks bestehen	<code>scripts/security/verifyChangedWorkflowSecurity.mjs</code> — siehe Abschnitt 6

5. Required Negative Tests (Runbook) — Nachweis

Alle in `tests/unit/verifySupplyChainProvenance.test.ts` (10 Tests) abgedeckt:

- geänderter Quell-Commit bei veralteter Provenance/Manifest → DENY;
- geändertes Lockfile bei veralteter SBOM → DENY;
- Artefakt-Digest-Mismatch (Release-Manifest) → DENY;
- Artefakt-Digest-Mismatch (SBOM) → DENY;
- unauflösbarer aktueller Quell-Commit → DENY;
- nicht vertrauenswürdige/lokale Builder-Identität bei `--require-ci` → DENY;
- fehlende Builder-Identität bei `--require-ci` → DENY;

- frei behauptete Agenten-/Provider-Identität in `runDetails` wird ignoriert (keine Vertrauens-Elevation über ein Freitext-Feld möglich) — nur `builder.id` und Digests zählen.

Zusätzlich lokal Ende-zu-Ende bestätigt: `verifySupplyChainProvenance.ts --require-ci` lehnt einen lokalen (Nicht-CI-)Lauf korrekt mit `builder.id: local-developer-build` ab (Exit-Code 1).

6. Lokaler Ende-zu-Ende-Nachweis dieser Sitzung

Erster Implementierungslauf (vor dem Merge, vor Entdeckung des Attestations-API-Blockers):

```
npm run build                -> PASS (Release-Manifest erzeugt)
npm run predeploy:check      -> PASS (SBOM, Provenance, Verify-Kette gruen, 0 violations)
npx tsx scripts/security/verifySupplyChainProvenance.ts --require-ci
                             -> FAIL-CLOSED wie erwartet (lokaler Build, keine CI-Identitaet)
npm run lint                 -> PASS (tsc --noEmit)
npx vitest run               -> 148 Testdateien, 845 Tests PASS (12 neu: 2 SBOM-Bindung,
                             3 sourceIdentity, 10 verifySupplyChainProvenance
minus                         bereits vorhandene ueberschneidende Zaehlung,
siehe Testlauf)
```

Nachtrag-Lauf (nach dem cosign-Fix, nach Rebase auf `main@7c73d55`):

```
npm run lint                 -> PASS (tsc --noEmit)
npm run build                -> PASS (Release-Manifest erzeugt)
npm run predeploy:check      -> PASS (SBOM, Provenance, Verify-Kette gruen, 0 violations)
npx vitest run               -> 150 Testdateien, 854 Tests PASS
```

`cosign sign-blob / cosign verify-blob` selbst sind lokal **nicht** ausführbar (keyless signing erfordert die echte GitHub-Actions-OIDC-Identität, die außerhalb eines echten Workflow-Laufs nicht existiert) — das ist der gesamte Punkt von keyless signing und kein Testlücke. Der reale Nachweis erfolgte im CI-Job selbst, siehe Nachtrag Abschnitt 0.1: `Signing artifact... Wrote bundle to file ...` gefolgt von `cosign verify-blob ... Verified OK`.

7. Nicht Teil dieser Implementierung

- keine Docker-Image-Digest-Bindung an eine Registry (das CI-Docker-Build in `ci.yml` ist ein reiner Validierungs-Build, kein Push in eine Registry; das eigentliche Produktions-Artefakt wird von Render separat aus demselben Quellstand gebaut — Deployment-Identitäts-Korrelation ist explizit M7-Scope laut Runbook, nicht M6);
- keine externe Plattform-Mutation (Render/Stripe/Supabase) — `NOT REQUIRED` laut Runbook, auch nicht durchgeführt;
- keine Änderung an bestehenden, bereits gepinnten Actions über die eine neue hinzugefügte Zeile hinaus.

8. Nächster Schritt

M6 ist mit diesem Dokument **VERIFIED PASS** — der reale `push`-Lauf `31834114193` hat die vollständige Kette `source` → `lockfile` → `SBOM` → `artifact` → `provenance/attestation` auf dem tatsächlichen, gehosteten Build-Pfad erzeugt und verifiziert (Nachtrag Abschnitt 0.1). M7 (Deployment Identity) ist damit gemäß `docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md` das nächste zulässige Gate; M7 erfordert laut eigenem Runbook explizite Owner-Mutation-Approval für jede externe Plattformänderung und wird nicht ohne separate Anweisung begonnen. Optional (nicht blockierend): das eingebettete Rekor-Log-Index/-URI aus der `.sigstore.json`-Bundle-Datei könnte bei künftigem Zugriff auf das Artefakt-Storage (außerhalb der aktuellen Sandbox-Egress-Beschränkung) noch zusätzlich extrahiert und hier nachgetragen werden — nicht erforderlich für den bereits erbrachten VERIFIED-PASS-Nachweis.

M7 — Deployment Identity: Phase-0-Preflight und Repository-Controls Evidence

Quelle: docs/evidence/m7/M7_PHASE0_AND_REPOSITORY_CONTROLS_EVIDENCE.md

M7 — Deployment Identity: Phase-0-Preflight und Repository-Controls Evidence

Status: PHASE 0 COMPLETE, REPOSITORY-CONTROLS-PAKET **VERIFIED PASS** auf dem realen Build-/Deploy-Pfad (siehe Nachtrag Abschnitt 0.2) — M7 als Ganzes bleibt PLANNED, keine externe Mutation autorisiert Datum: 2026-08-14 (Nachträge: 2026-08-14, selber Tag) Roadmap phase: M7 Authority: ADR-0061, docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md, docs/architecture/ai-agent/AI_AGENT_DEPLOYMENT_IDENTITY.md Executor: direkte Owner-instruierte Claude-Code-Sitzung (kein SA4/neuer autonomer Host — dieselbe Vorgehensweise wie M5A/M6, vom Owner nach Rückfrage bestätigt)

0. Explizit NICHT Teil dieser Sitzung

Diese Sitzung führt **keine** echte Render-, Stripe- oder Supabase-Mutation durch. Kein Deploy wurde manuell ausgelöst, keine Umgebungsvariable/kein Secret geändert, kein Service erstellt oder gelöscht. Alle Render-Zugriffe in Abschnitt 1 sind ausschließlich lesend (`list_workspaces`, `select_workspace`, `list_services`, `list_deploys`) über die Render-MCP-Tools dieser Sitzung. Der M7-Runbook-Owner-Mutation-Gate (Abschnitt „Owner Mutation Gate“) wurde nicht durchlaufen und ist für diese Sitzung nicht relevant, da keine externe Mutation stattfindet.

0.1 Nachtrag — erster echter `push`-Lauf: `verify-deployment-identity` schlug real fehl, Root Cause gefunden und behoben

Nach Merge von PR #265 löste der resultierende `push`-Lauf auf `main` (`31841526883`, Commit `c0bfa4d63bab7a770bb4d4b79a0c87f44d6290eb`) den vollständigen neuen Trust Chain real aus: `build-and-test` PASS, `supply-chain-attestation` PASS, `deploy-production` PASS (bestätigt: wartete korrekt auf `supply-chain-attestation`, wie in Abschnitt 2.1 vorgesehen) — aber der neue Job `verify-deployment-identity` schlug nach dem vollen 5-Minuten-Timeout (29 Poll-Versuche) fehl:

```
[deployment-verify] FAIL-CLOSED nach 29 Versuch(en), 300609ms: Deployment liefert keinen gueltigen 40-stelligen Commit-SHA (ADR-0036 Header/Feld fehlt oder ungueltig).
```

Root Cause (verifiziert durch Lesen der tatsächlichen Endpoint-Quelle, `server/routes/health.ts` und `server/logger.ts`): `/healthz` liefert **keine** `deployment`-Eigenschaft im JSON-Body — die ADR-0036-Deployment-Identität wird ausschließlich als Response-Header gesetzt (`x-capital-ai-version` / `-commit` / `-branch` / `-repo` / `-provider`, `server/logger.ts` Zeilen 84-89). Die ursprüngliche Behauptung in Abschnitt 1 („Laufzeit-Endpoint liefert u. a. ADR-0036-Deployment-Identität“) war also ungenau formuliert — richtig ist: nur über Hea-

der, nicht im Body. `scripts/pr/productionPreflight.mjs` kennt diesen Fallback bereits (`payload?.deployment || headerDeployment`); die erste Version von `scripts/deployment/verifyDeploymentIdentity.ts` hatte diesen Fallback **nicht** übernommen und las ausschließlich den JSON-Body — das Skript konnte dadurch grundsätzlich nie erfolgreich sein, unabhängig davon, wie lange es pollte. Ein reiner Timing-Bug (Render-Deploy noch nicht fertig) wurde geprüft und ausgeschlossen: Der Commit-Mismatch-Violation-Text taucht nirgends auf, sondern durchgehend „kein gültiger 40-stelliger Commit-SHA“ — konsistent mit einer komplett fehlenden `deployment`-Eigenschaft im Body, nicht mit einem alten, aber gültigen Commit.

Fix: `fetchHealth()` liest jetzt zusätzlich die Response-Header und baut daraus einen Fallback `headerDeployment`, exakt nach demselben Muster wie `productionPreflight.mjs`. Die Merge-Logik wurde als eigene, exportierte, reine Funktion `mergeHealthPayloadWithHeaders()` herausgezogen und mit 3 neuen Tests abgesichert (`tests/unit/verifyDeploymentIdentity.test.ts`, jetzt 13 Tests gesamt) — genau die Art Test, die diesen Bug von Anfang an gefangen hätte. Zusätzlich lokal end-to-end gegen einen echten HTTP-Mock-Server bestätigt (Header-only-Response, exakt wie die reale Produktions-Antwort): PASS nach 1 Versuch, 61ms.

Dies ist eine reale, im ersten echten Lauf gefundene und behobene Lücke — kein hypothetisches Risiko. Genau deshalb blieb der Status bewusst unterhalb „VERIFIED PASS“, bis ein echter `push`-Lauf das Skript tatsächlich geprüft hat.

0.2 Nachtrag — zweiter echter `push`-Lauf nach dem Header-Fallback-Fix: vollständig PASS

PR #267 (der Header-Fallback-Fix aus Nachtrag 0.1) wurde vom Owner per Review () freigegeben und gemerged (Merge-Commit `fd52af95449ef4ee13114dba0649976126d0db8d`). Der dadurch ausgelöste reale `push`-Lauf auf `main`, Run `31843242491`, lief vollständig **PASS** — alle fünf Jobs (`Human-/Owner-Vorprüfung`, `build-and-test`, `Supply-Chain-Provenance / Attestation`, `Deployment verifiziert / Render-Produktion`, `Deployment-Identität post-verifizieren`) mit `conclusion: success`.

Job-Log des Post-Deploy-Verifikationsschritts (Job-ID `94904984229`), wörtlich:

```
$ npx tsx scripts/deployment/verifyDeploymentIdentity.ts
[deployment-verify] PASS nach 5 Versuch(en), 40871ms: Commit fd52af95449ef4ee13114dba0649976126d0db8d
live und healthy.
```

5 Poll-Versuche über ~41 Sekunden — plausibel für die tatsächliche Render-Rollout-Zeit (deutlich unter dem 5-Minuten-Timeout). Der Job prüfte damit real: HTTP 2xx, `status: "ok"`, exakter Commit-Match (`fd52af95...` == der auslösende `main`-Commit), Repository-Match, Branch == `main` — alles über die zuvor gefundene Header-Quelle, nicht mehr über den leeren JSON-Body. Redigierte Evidence wurde per `actions/upload-artifact` abgelegt: Artefakt `deployment-identity-evidence-fd52af95449ef4ee13114dba0649976126d0db8d`, Artifact-ID `9235084959`, SHA-256 des Zip-Archivs `6b55dd3868b888e300c143edf259-ce43370986cd9a5852c8c3c705a7a4eb5e1e`, abrufbar unter <https://github.com/SvenKulesa/Fi->

[nance/actions/runs/31843242491/artifacts/9235084959](#) (90 Tage Aufbewahrung ab 2026-08-14).

Damit ist das in diesem Dokument beschriebene Repository-Controls-Paket (Provenance-Gate + Post-Deploy-Verifikation) auf dem tatsächlichen, gehosteten `main`-Build-/Deploy-Pfad **vollständig verifiziert** — nicht nur lokal, sondern im realen CI-Lauf gegen den echten Render-Deploy. Das rechtfertigt `VERIFIED PASS` für **dieses Paket**, ändert aber nichts am Gesamtstatus von M7: der Runbook-Exit-Gate (`docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md` , Abschnitt „Exit Gate“) verlangt zusätzlich jede erforderliche externe Mutation separat Human-genehmigt und verifiziert, bewiesenes Rollback und vollständige Negativtests über den gesamten M7-Scope — nichts davon war Teil dieses Pakets, da keine externe Mutation eingeführt wurde. M7 als Ganzes bleibt **PLANNED**.

1. Phase 0 — Read-only Preflight (Runbook-Abschnitt „Phase 0“)

Runbook-Schritt	Befund
M6 <code>COMPLETE</code> / <code>VERIFIED PASS</code> bestätigen	Bestätigt — <code>docs/evidence/m6/M6_REPOSITORY_IMPLEMENTATION_EVIDENCE.md</code> , realer signierter+verifizierter <code>push</code> -Lauf 31834114193 .
aktuellen <code>main</code> + M6-Evidence auflösen	<code>main@6d611ddda387bfe620e826d110f9a5623dce4378</code> (Merge PR #263) zum Zeitpunkt dieser Preflight.
exakte(n) Render-Service(s)/Environment(s)/Deploy-Pfad/aktuell deployten Commit identifizieren	Genau ein Service: <code>srv-d91010903t8c73edi55g</code> ("Finance", <code>web_service</code> , Docker-Runtime, Region Frankfurt, Plan starter, 1 Instanz, <code>https://finance-7clq.onrender.com</code>). Aktueller live Deploy zum Preflight-Zeitpunkt: <code>dep-d9vnagpt0d-sc73e2gnhg</code> , Status <code>live</code> , Commit <code>6d611ddda387bfe620e826d110f9a5623dce4378</code> — identisch mit <code>main</code> zu diesem Zeitpunkt, keine Drift.
Credential-/Hook-Ownership, Rotation, Revocation ohne Wertexposition inspizieren	Bestehender Mechanismus: <code>REN-DEPLOY_HOOK_URL</code> als GitHub-Actions-Secret, referenziert in <code>.github/workflows/ci.yml</code> (<code>deploy-production</code> -Job), Wert nie gelesen/ausgegeben. Render selbst zeigt zusätzlich einen GitHub-Container-Registry-Credential-Eintrag (<code>rgc-d9sa-j5c9v7es73ei60mg</code> , Name „capital-ai-container-registr“, Username <code>SvenKullessa</code>) — nur Metadaten eingesehen, kein Secret-Wert. Rotation/Revocation-Pfad dieser beiden Credentials liegt weiterhin ausschließlich im Render-Dashboard/GitHub-Secrets, nicht in dieser Sitzung geändert.
GitHub OIDC direkt nutzbar?	Render bietet keine direkte GitHub-OIDC-Föderation für Deploy-Trigger (Stand dieser Prüfung) — der bestehende Secret-Bridge-Mechanismus (Deploy-Hook-URL als Secret) bleibt der einzige verfügbare Weg; dokumentiert hier, nicht geändert.
Health-/Readiness-/Rollback-Mechanismus inspizieren	<code>render.yaml</code> : <code>healthCheckPath</code> : <code>/healthz</code> ; Laufzeit-Endpoint <code>server/routes/health.ts</code> liefert

Runbook-Schritt	Befund
	u. a. ADR-0036-Deployment-Identität (<code>server/deploymentIdentity.ts: version, commitSha, branch, repoSlug, provider</code>) — bereits vor M7 vorhanden, von <code>scripts/pr/productionPreflight.mjs</code> bereits für PR-Baselines genutzt. Automatisierte Post-Deploy -Prüfung existierte vor dieser Sitzung nicht — <code>deploy-production</code> löste den Deploy-Hook aus und endete sofort, ohne je zu prüfen, ob der erwartete Commit tatsächlich live ging. Das ist die konkrete Lücke, die Abschnitt 2 schließt.
jede vorgeschlagene Plattform-Mutation einzeln klassifizieren	Für diesen Schritt vorgeschlagene Mutation: keine . Die einzige bereits bestehende Mutation (Render-Deploy-Hook-Aufruf) bleibt unverändert in ihrer Wirkung; diese Sitzung ändert nur, wann sie ausgelöst wird (fail-closed hinter M6-Attestation) und ergänzt eine rein lesende Nachverifikation.
DevelopmentChain Mutation Handoff je extern mutierendem Ziel vorbereiten	Nicht erforderlich für diesen Schritt — keine neue externe Mutation eingeführt.
Last-known-good Deploy + Rollback-Trigger definieren	Letzter bekannter guter Deploy zum Preflight-Zeitpunkt: <code>dep-d9vnagpt0dsc73e2gnhg</code> (Commit <code>6d611ddd...</code> , <code>live</code>). Rollback-Trigger bleibt der bestehende Render-Mechanismus (vorheriger Deploy in der Render-Historie); kein neuer Rollback-Automatismus in diesem Schritt eingeführt.
aktive Deploy-/Config-Writer prüfen	Kein Hinweis auf einen aktiven, parallelen Deploy-/Config-Writer zum Preflight-Zeitpunkt; <code>list_deploys</code> zeigte eine konsistente, monoton fortschreitende Deploy-Historie ohne konkurrierende Läufe.

Zusätzlicher Befund (nicht Teil des Runbook-Checklisten-Items, aber sicherheitsrelevant genug für Aufnahme hier): `render.yaml` deklariert `autoDeployTrigger: checksPass`, der live Service meldet laut `list_services` jedoch `"autoDeployTrigger": "off"`. Diese Sitzung hat das **nicht** geändert und keine Ursache angenommen — laut Owner-Rückfrage bleibt dies offen für spätere Klärung (könnte eine bewusste manuelle Owner-Einstellung im Render-Dashboard sein). Der bestehende explizite Deploy-Hook-Aufruf in `deploy-production` funktioniert unabhängig von diesem Auto-Deploy-Setting.

2. Repository Implementation (Runbook-Abschnitt „Repository Implementation“)

Vom Owner bestätigter Scope: **Provenance-Gate + Post-Deploy-Verifikation** (empfohlene Option).

2.1 Provenance-Gate

`deploy-production` hängt jetzt zusätzlich von `supply-chain-attestation` (M6) ab: `needs: [build-and-test, supply-chain-attestation]`, `if:` prüft zusätzlich `needs.supply-chain-attestation.result == 'success'`. Schließt die Lücke im Required Trust Chain („M6 verified artifact/

provenance → protected deployment request"): vorher konnte ein Deploy ausgelöst werden, auch wenn die Attestation fehlschlug, da beide Jobs nur parallel von `build-and-test` abhingen.

2.2 Post-Deploy-Verifikation

Neuer Job `verify-deployment-identity` (`needs: [deploy-production]` , nur `push + main` , eigene minimale Berechtigung `contents: read`): pollt `https://capital-ai.online/healthz` (per `scripts/deployment/verifyDeploymentIdentity.ts`), bis der ausgelöste Commit als `live + healthy` gemeldet wird, oder schlägt nach Timeout (Standard 5 Minuten, 10s-Intervall) `fail-closed` fehl. Prüft: HTTP 2xx, `status: "ok"` , exakter Commit-Match (case-insensitive), Repository-Match, Branch `== main` . Redigierte Evidence (kein Secret-Wert) wird sowohl bei PASS als auch bei FAIL/Timeout nach `artifacts/deployment/deployment-identity-evidence.json` geschrieben und per `actions/upload-artifact@043fb46d1a93c77aae656e7c1c64a875d1fc6a0a` (Tag `v7.0.1` , bereits in M6 gegen den Ursprungs-Repository verifiziert) 90 Tage aufbewahrt.

Neue/geänderte Dateien

Datei	Zweck
<code>scripts/deployment/verifyDeploymentIdentity.ts</code> (neu)	Exportiert reine <code>verifyDeploymentIdentity()</code> - Prüf-Funktion + CLI mit Poll-Loop, fail-closed bei Timeout/Mismatch.
<code>tests/unit/verifyDeploymentIdentity.test.ts</code> (neu)	13 Tests: 1 Positiv-Fall (exakter Match), 1 Positiv-Fall (kein Repository-Constraint), 1 Case-Insensitivitäts-Fall, 7 Negativ-Fälle, 3 Tests für <code>mergeHealthPayload-WithHeaders</code> (Nachtrag 0.1 — Header-Fallback exakt für den real gefundenen Bug).
<code>.github/workflows/ci.yml</code> (geändert)	<code>deploy-production</code> jetzt zusätzlich von <code>supply-chain-attestation</code> abhängig; neuer Job <code>verify-deployment-identity</code> .
<code>docs/evidence/m7/M7_PHA-SE0_AND_REPOSITORY_CONTROLS_EVIDENCE.md</code> (dieses Dokument, neu)	Phase-0- und Repository-Controls-Evidence.

3. Required Positive/Negative Tests (Runbook-relevant, für dieses Skript)

Die Runbook-Liste „Required Negative Tests“ enthält mehrere Fälle außerhalb des Scopes dieses reinen Post-Verifikations-Skripts (z. B. Handoff-Target-Mismatch, abgelaufene Owner-Approval, Replay-Dedupe — diese betreffen die noch nicht implementierte externe Mutationsausführung, siehe Abschnitt 4). Die für `verifyDeploymentIdentity.ts` tatsächlich relevanten Fälle sind alle in `tests/unit/verifyDeploymentIdentity.test.ts` abgedeckt:

- PASS: exakter Commit-/Repository-/Branch-Match auf gesunder Antwort;
- PASS: `expectedRepository: null` überspringt die Repository-Prüfung (für lokale/Test-Läufe ohne `GITHUB_REPOSITORY`);
- PASS: Groß-/Kleinschreibung des erwarteten SHA wird case-insensitive behandelt;

- DENY: HTTP-Antwort nicht 2xx → „health/readiness failure" aus der Runbook-Liste;
- DENY: `status != "ok"` ;
- DENY: fehlender/ungültiger Commit im Deployment-Payload;
- DENY: deployter Commit $\neq$ erwarteter Commit → „source/provenance/artifact mismatch" aus der Runbook-Liste;
- DENY: Repository-Mismatch;
- DENY: Branch $\neq$ `main` → „deploy request from non-main or unverified source" aus der Runbook-Liste;
- DENY: erwarteter Commit selbst kein gültiger 40-Zeichen-SHA.

Zusätzlich abgedeckt (`mergeHealthPayloadWithHeaders` , 3 Tests): Header-Fallback greift, wenn der JSON-Body keine `deployment` -Eigenschaft enthält (der reale Produktionsfall); ein vorhandenes `deployment` -Feld im Body hat weiterhin Vorrang vor Headern; ein unparsbarer/ `null` -Body fällt vollständig auf Header zurück.

Zusätzlich lokal bestätigt (siehe Abschnitt 5): CLI schlägt korrekt fehl, wenn `VERIFIED_COMMIT_SHA` fehlt, und schreibt bei Timeout gegen eine unerreichbare URL korrekt eine `FAILED / TIMEOUT` -Evidence nach 3 Versuchen (Poll-Intervall 1s, Timeout 3s im Test). Nach dem Fix (Nachtrag 0.1) zusätzlich gegen einen lokalen HTTP-Mock-Server bestätigt, der die reale Produktionsantwort nachbildet (Header-only, kein `deployment` im Body): PASS nach 1 Versuch, 61ms.

4. Warum M7 als Ganzes (trotz Paket-VERIFIED-PASS) weiterhin **PLANNED** bleibt

Der erste echte `push` -Lauf nach Merge von PR #265 fand einen realen Bug (Nachtrag Abschnitt 0.1) — `verify-deployment-identity` konnte grundsätzlich nie erfolgreich sein, weil das Skript die Deployment-Identität ausschließlich im JSON-Body suchte, während die Produktion sie nur über Header liefert. Der Fix wurde gemergt, und der zweite echte `push` -Lauf lief vollständig PASS inkl. realer Post-Deploy-Verifikation (Nachtrag Abschnitt 0.2) — das Repository-Controls-Paket dieses Dokuments ist damit **VERIFIED PASS** auf dem tatsächlichen Build-/Deploy-Pfad.

Das ändert nichts an M7 als Ganzem: dies ist **nur der erste** von mehreren möglichen M7-Repository-Implementation-Paketen, und es führt **keine** externe Mutation ein. Der M7-Exit-Gate selbst (`docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md` , Abschnitt „Exit Gate") verlangt zusätzlich: jede erforderliche externe Mutation separat Human-genehmigt und `VERIFIED PASS` /sicher `FAILED / ROLLED BACK` , bewiesenes Rollback, vollständige Negativtests über den gesamten Scope. Keines davon ist Teil dieses Pakets — M7 als Ganzes bleibt **PLANNED**, nicht `VERIFIED PASS` .

5. Lokaler Nachweis dieser Sitzung

Erster Implementierungslauf (vor Merge von PR #265, vor Entdeckung des Header-vs-Body-Bugs):

```

npm run lint                                -> PASS (tsc --noEmit)
npx vitest run tests/unit/verifyDeploymentIdentity.test.ts
                                              -> 10/10 PASS
npx vitest run                              -> 151 Testdateien, 864 Tests PASS
npm run build                              -> PASS
npm run predeploy:check                    -> PASS (0 violations)
npm audit --omit=dev --audit-level=high    -> 0 Vulnerabilities

npx tsx scripts/deployment/verifyDeploymentIdentity.ts (ohne VERIFIED_COMMIT_SHA)
                                              -> FAIL-CLOSED wie erwartet, Exit 1

CAPITAL_AI_PRODUCTION_HEALTH_URL=https://127.0.0.1:1/healthz \
VERIFIED_COMMIT_SHA=aaaa...aaaa (40 Zeichen) \
DEPLOYMENT_VERIFY_TIMEOUT_MS=3000 DEPLOYMENT_VERIFY_POLL_INTERVAL_MS=1000 \
npx tsx scripts/deployment/verifyDeploymentIdentity.ts
                                              -> FAIL-CLOSED nach 3 Versuchen/3045ms wie erwartet,
                                              Evidence-Datei mit result: "FAILED / TIMEOUT"

korrekt
                                              geschrieben, Exit 1

```

Nachtrag-Lauf (nach dem Header-Fallback-Fix, Abschnitt 0.1):

```

npm run lint                                -> PASS (tsc --noEmit)
npx vitest run tests/unit/verifyDeploymentIdentity.test.ts
                                              -> 13/13 PASS (3 neu: mergeHealthPayloadWithHeaders)
npx vitest run                              -> 151 Testdateien, 867 Tests PASS

Lokaler HTTP-Mock-Server (Node http, Port 38123) bildet die reale Produktionsantwort exakt nach:
Header x-capital-ai-* gesetzt, KEIN deployment-Feld im JSON-Body.

CAPITAL_AI_PRODUCTION_HEALTH_URL=http://127.0.0.1:38123/healthz \
VERIFIED_COMMIT_SHA=cccc...cccc (40 Zeichen) GITHUB_REPOSITORY=SvenKulesa/Finance \
npx tsx scripts/deployment/verifyDeploymentIdentity.ts
                                              -> PASS nach 1 Versuch, 61ms, Evidence-Datei mit
                                              result: "VERIFIED PASS" korrekt geschrieben, Exit
0

```

6. Nicht Teil dieser Implementierung

- keine echte Render-/Stripe-/Supabase-Mutation (siehe Abschnitt 0);
- keine Änderung an `render.yaml`, an Render-Dashboard-Einstellungen oder an bestehenden Secrets;
- keine Klärung/Korrektur des in Abschnitt 1 dokumentierten `autoDeployTrigger`-Drifts — bewusst dem Owner überlassen;
- kein Rollback-Automatismus über den bestehenden Render-Mechanismus hinaus;
- keine GitHub-OIDC-Föderation zu Render (laut Preflight nicht verfügbar).

7. Nächster Schritt

Dieses Repository-Controls-Paket (Provenance-Gate + Post-Deploy-Verifikation) ist mit Nachtrag 0.2 abgeschlossen — real auf dem gehosteten Build-/Deploy-Pfad verifiziert, kein weiterer Nachtrag hierzu erforderlich. Weitere M7-Repository-Implementation-Pakete (z. B. Rollback-Tooling, explizite Mutation-Handoff-Vorlage für einen konkreten zukünftigen Render-Mutationsbedarf) bleiben eigene, separat zu beauftragende Schritte — keine externe Mutation ohne die im Runbook beschriebene separate Owner-Mutation-Approval.

M7 — Deployment Identity: Required Negative Tests Evidence

Quelle: docs/evidence/m7/M7_REQUIRED_NEGATIVE_TESTS_EVIDENCE.md

M7 — Deployment Identity: Required Negative Tests Evidence

Status: alle 10 im Runbook gelisteten Required Negative Tests sind real, automatisiert und reproduzierbar **VERIFIED PASS** — M7-Exit-Gate-Punkt 6 ist damit geschlossen Datum: 2026-08-14 Roadmap phase: M7 Authority: ADR-0061, docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md Abschnitt „Required Negative Tests“, docs/contracts/DEVELOPMENT_CHAIN_MUTATION_HANDOFF_CONTRACT.md, .ai/contracts/development-chain-mutation-handoff.schema.json Executor: direkte Owner-instruierte Claude-Code-Sitzung (Owner-Anweisung „fang mit der Required Negative Tests Liste an“)

0. Ausgangslage

docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md listet 10 Required Negative Tests als Voraussetzung für den M7-Exit-Gate-Punkt 6. Vor dieser Sitzung existierte dafür kein Code: die DEVELOPMENT Chain Mutation Handoff Contract (.ai/contracts/development-chain-mutation-handoff.schema.json) ist ein reines Dokumentationsartefakt ohne Validator, und executor-AgentId ist fest auf den SA3B/GitHub-Roadmap-Executor gebunden — dieses Schema war für einen hypothetischen, für Render nie gebauten automatisierten Executor gedacht, nicht für die tatsächlich genutzte Owner-manuelle Mutationsausführung (Deploy-Hook-Rotation, Rollback, Roll-Forward — alle drei real durchgeführt und verifiziert, siehe docs/runbooks/M7_DEPLOY_HOOK_ROTATION_HANDOFF.md und docs/runbooks/M7_ROLLBACK_VERIFICATION_HANDOFF.md).

Diese Sitzung schließt diese Lücke, indem sie für jeden der 10 Punkte einen konkreten, automatisiert laufenden Test schreibt, der beweist, dass das jeweilige DENY-Verhalten tatsächlich eintritt — nicht nur, dass es dokumentiert ist.

1. Neue Implementierung

- src/platform/Security/developmentChainMutationHandoff.ts (neu): strukturvalidierender validateMutationHandoff() (spiegelt die JSON-Schema-Regeln plus eine neue, strengere Regel: bei platform: RENDER muss targetResource exakt dem bekannten Finance-Service srv-d91o1o9o3t8c73edi55g entsprechen) sowie ein Ausführungszeit-Gate evaluateMutationHandoffExecution(), das zusätzlich Akteur-, Ziel-, Ablauf-, Replay- und Audit-Permit-Prüfungen durchführt. Folgt exakt dem bereits bestehenden Muster von src/platform/Security/roadmapExecutionMandate.ts (validateRoadmapExecutionMandate / evaluateSystemadminRoadmapAuthorization), das für den SA3B/SA4-Pfad bereits produktiv ist.
- **Wichtig, ehrlich benannt:** Dieses Gate ist **nicht** an einen echten Render-Executor angeschlossen — keiner existiert. Es beweist, dass die DENY-Logik korrekt ist und ein-

satzbereit wäre, falls jemals ein automatisierter Render-Executor gebaut wird. Alle drei bisherigen echten M7-Render-Mutationen blieben Owner-manuell und wurden ausschließlich nachträglich über Renders eigene API verifiziert (siehe die beiden oben verlinkten Handoff-Dokumente) — das bleibt das akzeptierte Betriebsmodell für M7, dieses Gate ändert daran nichts.

- `tests/unit/ciDeploymentGuards.test.ts` (neu): statische Assertions gegen `.github/workflows/ci.yml`, dass `deploy-production`, `supply-chain-attestation` und `verify-deployment-identity` ausschließlich bei `push` auf `refs/heads/main` laufen und niemals `pull_request` / `workflow_dispatch` erwähnen.
- `tests/integration/verifyDeploymentIdentityCli.test.ts` (neu): startet `scripts/deployment/verifyDeploymentIdentity.ts` als echten Subprozess gegen einen lokalen Mock-Health-Server, der entweder einen falschen Commit oder HTTP 503 liefert, mit sehr kurzem Poll-/Timeout-Intervall, und beweist Exit-Code $\neq 0$ plus `result: "FAILED / TIMEOUT"` im Evidence-Artefakt.

2. Mapping — jeder Punkt der Required-Negative-Tests-Liste

#	Runbook-Punkt	Nachweis	Ergebnis
1	deploy request from non-main or unverified source → DENY	<code>developmentChainMutationHandoff.test.ts</code> : „rejects a non-main baseBranch“; <code>ciDeploymentGuards.test.ts</code> : alle drei Deploy-Jobs tragen <code>github.event_name == 'push' && github.ref == 'refs/heads/main'</code> und erwähnen nirgends <code>pull_request</code>	PASS
2	source/provenance/artifact mismatch → DENY	<code>developmentChainMutationHandoff.test.ts</code> : „rejects an invalid baseSha“; ergänzend bereits bestehend: <code>verifyDeploymentIdentity.test.ts</code> (Commit-Mismatch, Repo-Mismatch) und <code>scripts/security/verifySupplyChainProvenance.ts</code> (M6, cosign-Signaturprüfung)	PASS
3	wrong Render service/environment → DENY	<code>developmentChainMutationHandoff.test.ts</code> : „rejects a Render targetResource that is not the known Finance service“ — neue	PASS

#	Runbook-Punkt	Nachweis	Ergebnis
		Regel pinnt <code>targetRe-source</code> bei <code>platform: RENDER</code> hart auf <code>srv-d91o1o9o3t8c73edi55g</code>	
4	expired/missing Owner mutation approval → DENY	<code>developmentChainMutationHandoff.test.ts</code> : „requires approvalEvidence-Ref..." (fehlend) und „denies execution once the Handoff/approval has expired" (<code>expiresAt</code> in der Vergangenheit)	PASS
5	Handoff target mismatch → DENY	<code>developmentChainMutationHandoff.test.ts</code> : „denies when the attempted target does not match the Handoff target"	PASS
6	duplicate/replayed mutation request → DENY/DE-DUPE	<code>developmentChainMutationHandoff.test.ts</code> : „denies a replayed/duplicate idempotency key" (<code>seenIdempotencyKeys</code>); ergänzend bereits bestehend für den SA3B-Pfad: <code>roadmapExecutionMandate.ts</code> 's <code>unchanged-HeadAlreadyValidated</code> -De-dupe, getestet in <code>roadmapExecutionMandate.test.ts</code>	PASS
7	execution without durable audit permit → DENY	<code>developmentChainMutationHandoff.test.ts</code> : „denies execution without a durable audit permit reference"; ergänzend bereits bestehend für den SA3B-Pfad: <code>systemadminExecutionHostWorkflow.test.ts</code> 's „places durable authorization before the sole branch side effect"	PASS
8	agent attempts <code>MERGE</code> → DENY	<code>developmentChainMutationHandoff.test.ts</code> : „denies an attempted <code>MERGE</code> operation" und „rejects al-	PASS

#	Runbook-Punkt	Nachweis	Ergebnis
		lowedOperations that contain MERGE"; bereits bestehend generisch für jede Agent-Capability: <code>agentIam.test.ts</code> 's „never exposes MERGE as an agent capability" und <code>policyGate.test.ts</code> 's „keeps MERGE outside the agent runtime policy surface"	
9	arbitrary platform operation outside Handoff → DENY	<code>developmentChainMutationHandoff.test.ts</code> : „denies an arbitrary operation outside the Handoff allow-list" und „denies an operation explicitly listed in forbiddenOperations beyond the always-forbidden set"	PASS
10	health/readiness failure → rollback/STOP	<code>verifyDeploymentIdentityCli.test.ts</code> : echter Subprozesslauf gegen falschen Commit und gegen HTTP 503 — beide enden mit Exit-Code $\neq 0$ und <code>FAILED / TIMEOUT</code> -Evidence; real bereits einmal in Produktion beobachtet (siehe <code>M7_PHA-SE0_AND_REPOSITORY_CON-TROLS_EVIDENCE.md</code> Abschnitt 0.1: der erste echte <code>push</code> -Lauf schlug 5 Minuten lang korrekt fail-closed fehl, bevor der Root-Cause-Fix griff)	PASS

3. Testlauf

- `npx vitest run` (voller Suite-Lauf, nicht nur neue Dateien): **901 Tests, 158 Dateien, alle PASS** (davon neu: 20 in `developmentChainMutationHandoff.test.ts`, 5 in `ciDeploymentGuards.test.ts`, 2 in `verifyDeploymentIdentityCli.test.ts`).
- `npm run lint ( tsc --noEmit )`: PASS.
- `node --test scripts/pr/*.test.mjs` : 1 vorbestehender, von dieser Änderung unabhängiger Fehler in `runtimeArtifactImmutability.test.mjs` (Fixture vergleicht gegen einen festen Platzhalter-Commit, nicht gegen den aktuellen `main` -HEAD, der sich durch

parallele Agenten-PRs seither weiterbewegt hat) — reproduziert identisch mit und ohne diese Änderung, außerhalb des Scopes dieser Aufgabe, nicht angefasst.

4. Was dieses Ergebnis NICHT bedeutet

- Es gibt weiterhin **keinen** automatisierten Render-Execution-Host. Alle drei bisherigen M7-Render-Mutationen bleiben Owner-manuell durchgeführt und nachträglich verifiziert. Das neue Gate (`evaluateMutationHandoffExecution`) ist bereit, falls ein solcher Host jemals gebaut wird, ersetzt aber nicht die bestehende Owner-Handoff-Verifikations-Praxis.
- M7 als Ganzes ist mit diesem Punkt **nicht automatisch** `COMPLETE / VERIFIED PASS` — siehe die vollständige Exit-Gate-Neubewertung in den Roadmap-/Traceability-Dokumenten für den Status aller 9 Punkte.

5. Geänderte/neue Dateien

- `src/platform/Security/developmentChainMutationHandoff.ts` (neu)
- `tests/unit/developmentChainMutationHandoff.test.ts` (neu, 20 Tests)
- `tests/unit/ciDeploymentGuards.test.ts` (neu, 5 Tests)
- `tests/integration/verifyDeploymentIdentityCli.test.ts` (neu, 2 Tests)
- `docs/evidence/m7/M7_REQUIRED_NEGATIVE_TESTS_EVIDENCE.md` (diese Datei)

M8 — Audit-Korrelation Exit-Gate-Punkt 7

Quelle: docs/evidence/m8/M8_AUDIT_CORRELATION_EXIT_GATE_7_EVIDENCE.md

M8 — Audit-Korrelation Exit-Gate-Punkt 7

Status: VERIFIED PASS — PR #308 MERGED

Datum: 2026-08-15

Baseline: `main@fafba177101ff0a19d71c6ddea3605df93d9036d`

Authority: ADR-0059, ADR-0062, ESS-0019, `docs/runbooks/M8_AGENT_CUTOVER.md`

Ziel

M8-Exit-Gate-Punkt 7 verlangt eine vollständige, manipulationsgeschützte Korrelation zwischen der Autorisierungsentscheidung und dem terminalen Ausführungsergebnis.

Read-only Befund

Der produktive SA3B-Pfad schrieb bereits zwei append-only Ereignisse und verband das Outcome über `authorizationAuditReference`. Nicht vollständig technisch gebunden waren jedoch:

- die gemeinsame Identität aus Request-, Trace- und Session-ID;
- Provider-/Modellattribution im terminalen Outcome;
- Toolattribution im terminalen Outcome;
- ein expliziter Fail-closed-Test gegen manipulierte Korrelation.

Die regulären Produktclients `server/anthropicClient.ts`, `server/openaiClient.ts` und `src/services/agentModelRouting.ts` sind keine privilegierten Bypass-Pfade und wurden deshalb nicht gelöscht. Die interaktiven Connector-Berechtigungen liegen außerhalb des Repository-Codes.

Umsetzung

`server/agentAudit/systemadminAuditedExecution.ts` :

1. erzeugt vor dem Autorisierungsinsert eine deterministische `auditCorrelationId` aus `requestId + traceId + sessionId`;
2. verweigert fehlende Korrelationsbestandteile vor jeder Evidence-Erzeugung;
3. bindet die Korrelations-ID an Autorisierung, Permit und Outcome;
4. prüft vor dem Outcome, dass Referenz und Korrelations-ID unverändert sind;
5. übernimmt Provider, Modell und Tool vom autorisierten Kontext in das Outcome;
6. schreibt Authorization-Referenz und vollständige Korrelationsattribute in das zweite append-only Ereignis.

Negative Tests

`tests/unit/systemadminAuditedExecution.test.ts` beweist:

- manipulierte `auditCorrelationId` → DENY vor Outcome-Insert;
- fehlende Session-ID → fail closed vor Autorisierungsinsert;
- Authorization und Outcome besitzen dieselbe Korrelations-ID;
- Provider, Modell und Tool bleiben durch beide Ereignisse erhalten;
- sensible Prompt-/Response-Payloads bleiben weiterhin ausgelassen.

Bypass-Bewertung

Es wurde kein löschbarer privilegierter In-Repo-Bypass nachgewiesen. Der einzige kanonische Repository-Mutationspfad bleibt SA3B/SA4 mit Provider-Profile-, REM-/IAM- und Audit-Gates. Externe interaktive Tool-Grants können nur auf Host-/Connector-Ebene deaktiviert werden und werden nicht durch das Löschen regulärer Finanzanalyseclients vorgetäuscht.

Validierung

- TypeScript/Lint: PASS
- Unit Tests: PASS
- Production Build: PASS
- Governance/Security: PASS
- `build-and-test` : PASS — Run `31862280230`
- Shadow CI: PASS — Run `31862280254`
- Human Merge: PASS — PR #308, Merge-Commit `2fa15c8d3b5bce-b32a26e34e1ae51cf9d3ec14b9`

M8-Exit-Gate-Punkt 7 ist damit `VERIFIED PASS`. Der allgemeine M8-Branch-Cleanup aus Exit-Gate-Punkt 9 bleibt separat offen; diese Evidence behauptet keine Löschung, solange die Remote-Banches nicht nachweislich entfernt wurden.

M8 — Closure Evidence (COMPLETE / VERIFIED PASS)

Quelle: docs/evidence/m8/M8_CLOSURE_EVIDENCE.md

M8 — Closure Evidence (COMPLETE / VERIFIED PASS)

Status: **COMPLETE / VERIFIED PASS** Datum: 2026-08-16 Roadmap phase: M8 (Development Chain), Integrated Roadmap Phase I1 (ROADMAP-INTEGRATED-DC-SA-0001) Authority: docs/runbooks/M8_AGENT_CUTOVER.md (## Exit Gate), ADR-0062, ESS-0019, docs/evidence/m8/M8_PROVIDER_SET_CORRECTION_2026-08-16.md (PR #365, kanonisches Provider-Set) Executor: Owner-instruierte Claude-Code-Sitzung, unmittelbar nach expliziter Owner-Entscheidung ("ACCEPT (empfohlen)", via AskUserQuestion) ZU docs/evidence/m8/M8_EXIT_GATE_ITEM2_SCOPE_DECISION.md

0. Zweck

Dieses Dokument schließt M8 formal ab, nachdem der Owner die in M8_EXIT_GATE_ITEM2_SCOPE_DECISION.md §3 vorgeschlagene Scope-Entscheidung für Exit-Gate-Punkt 2 explizit akzeptiert hat (§5 dort, Status OWNER_ACCEPTED). Es fasst alle 9 Exit-Gate-Kriterien mit ihrer jeweiligen Evidence-Quelle zusammen — es erzeugt keine neue Evidence, sondern konsolidiert die bereits über mehrere PRs (#308, #316, #328, #334, #339, #362, #365) erarbeitete.

Nachtrag noch vor Erstveröffentlichung: Zwischen der Owner-ACCEPT-Entscheidung und dem Schreiben dieses Dokuments hat der Owner das kanonische Provider-Set korrigiert (PR #365, docs/evidence/m8/M8_PROVIDER_SET_CORRECTION_2026-08-16.md): **Google AI Studio ist nicht Teil der DEVELOPMENT/AI-Wertschöpfungskette.** Kanonisch sind **ChatGPT, Claude, Grok** (chatgpt-github-connector , claude-code-cli , grok-xai-connector); google-ai-studio , notebooklm , gemini sind RETIRED_PROVIDER_ALIASES (DENY im Control Plane, evaluateProviderCutoverReadiness() → RETIRED). Dieses Dokument verwendet durchgehend bereits das korrigierte Drei-Provider-Set — es wurde nie mit dem alten Vier-Provider-Stand veröffentlicht.

1. Vollständiger Exit-Gate-Status (9/9, M8_AGENT_CUTOVER.md ## Exit Gate)

#	Kriterium	Status	Evidence
1	M7 ist verifiziert	PASS	M7 COMPLETE / VERIFIED PASS (2026-08-14), docs/evidence/m7/M7_PHASE0_AND_REPOSITORY_CONTROLS_EVIDENCE.md
2			

#	Kriterium	Status	Evidence
	Privilegierte unterstützte Provider nutzen den provider-neutralen Control Plane als kanonischen Pfad	PASS (unter Owner-akzeptiertem Scope)	docs/evidence/m8/M8_E-XIT_GA-TE_I-TEM2_SC0-PE_DECISION.md (OWNER_ACCEPTED , 2026-08-16) + docs/evidence/m8/M8_EXTER-NAL_HOST_CONFIGURATION_VERIFIED_E (chatgpt-github-connector READY, alle 6 Provider-CutoverEvidence -Felder true)
3	Policy-Equivalence-Tests bestehen	PASS	tests/unit/provider-Profile.test.ts (kanonisches Drei-Provider-Set + RETIRED-DENY-Fälle), docs/evidence/m8/M8_PHA-SE0_AND_PROVI-DER_PROFI-LE_EVIDENCE.md
4	Direkte provider-spezifische privilegierte Bypässe sind verweigert/deaktiviert	PASS	docs/evidence/m8/M8_PROVI-DER_BY-PASS_ROU-TE_AU-DIT_E-XIT_GA-TE_4_EVIDENCE.md , tests/unit/adminRouterBypassAudit.test.ts (PR #328)
5	Research-Profile scheitern an Mutationstests	PASS	tests/unit/provider-Profile.test.ts (research-plane constructor-enforced + Negative Tests)
6	Rollback-zu-read-only ist bewiesen	PASS	docs/evidence/m8/M8_PHA-SE0_AND_PROVI-DER_PROFI-LE_EVIDENCE.md §7 — zwei unabhängige Hebel (IAM-Kill-Switch, Provider-Profil-Registry-Rollback), End-to-End durch die reale SA3B-Kette

#	Kriterium	Status	Evidence
7	Audit-Korrelation ist vollständig	PASS	docs/evidence/m8/M8_AU-DIT_CORRELATION_E-XIT_GA-TE_7_EVIDENCE.md (PR #308)
8	Evidence + Roadmap/Traceability sind synchronisiert	PASS (dieser PR)	Dieses Dokument + Roadmap-/Traceability-Updates im selben PR
9	Arbeitsbranches sind gelöscht	PASS / N/A	git ls-remote --heads origin zum Zeitpunkt dieses PRs: kein offener M8-spezifischer Arbeitsbranch

Ergebnis: 9 von 9 Punkten **PASS** . M8 ist **COMPLETE / VERIFIED PASS** .

2. Cutover-Readiness-Matrix (kanonisches Drei-Provider-Set, Stand 2026-08-16)

Provider	Mutierend	Readiness	Grund
chatgpt-github-connector	Ja	READY	Realer, code-adressierbarer Aufrufer (SA3B/SA4/Work-Package-GitHub-Actions-Host); alle 6 ProviderCutoverEvidence-Felder true, inkl. externalHostConfigurationVerified (M8_EXTERNAL_HOST_CONFIGURATION_VERIFIED_E
claude-code-cli	Ja	BLOCKED	Strukturell — die interaktive CCR-Sitzung wird nicht durch agentI-am.ts vermittelt; kein code-adressierbarer Aufrufer (docs/architecture/M8_CLAUDE_CODE_RE-AL_CALLER_DESIGN.md)
grok-xai-connector	Ja	BLOCKED	Dieselbe strukturelle Lücke wie claude-code-cli — laut provider-Profile.ts s eigener Beschreibung ist die Authentisierungsquelle

Provider	Mutierend	Readiness	Grund
			"Grok GitHub Connector / SuperGrok session (xAI); repository grants are host-side, not model identity", d. h. eine interaktive, host-vermittelte Sitzung ohne code-adres-sierbaren, OIDC-gegate-ten Execution-Host analog SA3B. Diese Sitzung hat keinen neuen Aufrufer für Grok gebaut oder verifi-ziert — der Status folgt di-rekt aus der bereits für Claude Code etablierten Analyse, symmetrisch an-gewendet

`google-ai-studio` / `notebooklm` / `gemini` sind nicht mehr Teil der Matrix — `RETIRED`
(`evaluateProviderCutoverReadiness()` liefert für diese Aliase jetzt `{ status: 'RETIRED' }`, kein `BLOCKED` / `NOT_APPLICABLE` mehr).

Exit-Gate-Punkt 2 unter dem Owner-akzeptierten Scope ("privileged supported providers" = mutierende Provider mit produktivem Execution-Host): weiterhin ausschließlich `chatgpt-github-connector` erfüllt diesen Scope. Die Owner-Entscheidung selbst ändert sich durch die Provider-Set-Korrektur nicht — sie skaliert unverändert auf drei statt vier Profile, mit demselben Ergebnis (ein `READY`, alle übrigen mutierenden Profile `BLOCKED`).

3. Was `COMPLETE` / `VERIFIED PASS` hier konkret bedeutet — und was nicht

Bedeutet: - Der provider-neutrale Control Plane (SA3 → SA2 → SA1 → M4 + M8 Provider-Profil-Gate) ist der einzige kanonische privilegierte Pfad im Repository für den einen real verdrahteten mutierenden Provider (`chatgpt-github-connector`). - Kein provider-spezifischer Admin-Bypass existiert (Exit-Gate 4). - Rollback-zu-read-only und Audit-Korrelation sind für diesen Pfad bewiesen (Exit-Gate 6, 7). - Retired Provider-Aliase (`google-ai-studio` , `notebooklm` , `gemini`) sind fail-closed als `DENY` / `RETIRED` markiert — sie benötigen und erhalten keinen privilegierten Cutover. - `claude-code-cli` **und** `grok-xai-connector` bleiben bewusst außerhalb des unterstützten privilegierten Produktionsumfangs — laut Owner-akzeptierter Scope-Entscheidung **kein** offener Restpunkt, sondern der korrekte, sicherheitsarchitekto-nisch begründete Zustand für beide interaktiven Connector-Sitzungen, bis ein separates ADR einen tragfähigen realen Aufrufer definiert.

Bedeutet NICHT: - Keine Aktivierung, kein neuer Aufrufer, keine Mutation für Claude Code, Grok, Google AI Studio, NotebookLM oder Gemini. - Keine automatische Freigabe von M9-Drills — Integrated Roadmap Phase I2 (M9 Assurance) ist ab jetzt **nicht mehr durch M8 blockiert**, aber jeder einzelne M9-Drill (Prompt/Tool-Injection, Authorization-Bypass, Replay, Exfiltration, Audit-Outage, Kill-Switch, Break-Glass, Rollback) bleibt eine eigene, separat vom Owner zu autorisierende Handlung (`docs/roadmaps/INTEGRA-`

TED_DEVELOPMENT_SYSTEMADMIN_ROADMAP.md §11: "Nicht ausführbar ohne separate Owner-Freigabe". - Keine Schwächung von IAM, REM, Kill-Switch, Human-only-MERGE, oder der Owner-Entscheidungspflicht für Produktions-/IAM-/Secret-/Deploy-Mutation.

4. Autorisierungskette (vollständig, nachvollziehbar)

1. Grok-Sitzung (Owner-Auftrag "Punkt a bis c abarbeiten"): Evidence + Scope-Proposal erstellt, PR #362, vom Owner selbst gemergt (merged_by: SvenKulessa).
2. Claude-Code-Sitzung: Owner-Frage explizit gestellt ("Wie entscheidest du?" mit ACCEPT/REJECT/DEFER als getrennte Optionen, via AskUserQuestion).
3. Owner-Antwort: **"ACCEPT (empfohlen)"** — explizite, unzweideutige Wahl einer von drei klar unterscheidbaren Optionen, keine generische "mach weiter"-Anweisung.
4. Vor Veröffentlichung dieses Dokuments: Owner korrigiert das kanonische Provider-Set (PR #365, Google AI Studio raus, Grok rein). Diese Sitzung prüfte main erneut, bevor sie pushte, fand den noch offenen PR #365 und wartete auf dessen Entscheidung, statt ein bereits veraltetes Closure-Dokument zu veröffentlichen.
5. Diese Sitzung: Status-Flip in M8_EXIT_GATE_ITEM2_SCOPE_DECISION.md auf OWNER_ACCEPTED, dieses Closure-Dokument (bereits mit dem korrigierten Drei-Provider-Set), Roadmap-/Traceability-Sync — alles additiv, keine Selbstfreigabe.

Verwandte Dokumente

- docs/evidence/m8/M8_EXIT_GATE_ITEM2_SCOPE_DECISION.md
- docs/evidence/m8/M8_EXTERNAL_HOST_CONFIGURATION_VERIFIED_EVIDENCE.md
- docs/evidence/m8/M8_I1_CUTOVER_READINESS_MATRIX_AND_EXIT_GATE_SYNC_EVIDENCE.md
- docs/evidence/m8/M8_PROVIDER_BYPASS_ROUTE_AUDIT_EXIT_GATE_4_EVIDENCE.md
- docs/evidence/m8/M8_AUDIT_CORRELATION_EXIT_GATE_7_EVIDENCE.md
- docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md
- docs/evidence/m8/M8_PROVIDER_SET_CORRECTION_2026-08-16.md
- docs/architecture/M8_CLAUDE_CODE_REAL_CALLER_DESIGN.md
- docs/runbooks/M8_AGENT_CUTOVER.md
- docs/roadmaps/INTEGRATED_DEVELOPMENT_SYSTEMADMIN_ROADMAP.md

M8 Exit-Gate-Punkt 2 — Scope-Entscheidung (Owner-Proposal)

Quelle: docs/evidence/m8/M8_EXIT_GATE_ITEM2_SCOPE_DECISION.md

M8 Exit-Gate-Punkt 2 — Scope-Entscheidung (Owner-Proposal)

Status: **OWNER_ACCEPTED — 2026-08-16, direkte Owner-Anweisung in Claude-Code-Sitzung** Datum: 2026-08-16 Roadmap phase: M8 / Integrated Roadmap I1 Authority: docs/runbooks/M8_AGENT_CUTOVER.md Exit Gate Punkt 2, ADR-0062, ESS-0019, docs/architecture/M8_CLAUDE_CODE_REAL_CALLER_DESIGN.md Executor: Owner-instruierte Grok-Sitzung (Punkt A-C, Proposal-Erstellung); Owner-Bestätigung in einer separaten Claude-Code-Sitzung (Status-Flip, Closure) Baseline: main@fc03e327a01df458595b1210d74d109fd895b717

0. Zweck

Exit-Gate-Punkt 2 lautet:

privileged supported providers use the provider-neutral Control Plane as the canonical path

Historisch wurde das so gelesen, dass **alle vier** in PROVIDER_PROFILES eingetragenen Provider einen realen Aufrufer brauchen. Die Evidence zeigt jedoch:

- nur **ein** mutierendes Profil hat einen realen, repository-geaten Host;
- zwei Profile sind bewusst non-mutating (NOT_APPLICABLE);
- ein mutierendes Profil ist **strukturell** ohne vertretbaren realen Aufrufer.

Dieses Dokument formuliert die Scope-Entscheidung, mit der Punkt 2 **ohne Sicherheitsabsenkung** geschlossen werden kann — oder abgelehnt wird. **Nur der Owner** kann den Status auf OWNER_ACCEPTED setzen.

1. Ist-Zustand (Evidence-gebunden)

Provider	Plane / Mutation	realer Aufrufer	Cutover-Readiness
chatgpt-github-connector	mutierend	Ja (SA3B / SA4 / Work-Package-Host)	READY nach M8_EXTERNAL_HOST_CONFIGURATION_VERIFIED_Evidence-Felder)
claude-code-cli	mutierend (Profil)	Nein — strukturell blockiert	BLOCKED (M8_CLAUDE_CODE_REAL_CALLER_DESIGN.md)
google-ai-studio	non-mutating	N/A	NOT_APPLICABLE

Provider	Plane / Mutation	realer Aufrufer	Cutover-Readiness
notebooklm	research-only	N/A	NOT_APPLICABLE

2. Warum „alle vier Caller“ falsch skaliert

1. **google-ai-studio** und **notebooklm** besitzen keine mutierende Capability. Ein privilegierten Cutover ist weder erforderlich noch möglich (`evaluateProviderCutoverReadiness` → `NOT_APPLICABLE`).
2. **claude-code-cli**: Die interaktive CCR-Sitzung wird nicht durch `agentIam.ts` vermittelt. Ein echter agentischer Host (Modell B) erfordert eine Tool-Call-Vermittlungsschicht, die heute nicht existiert und ein **eigenes ADR** bräuchte. Weg 1 (Content-Generator in Modell-A-Work-Package) bedient das mutierende Profil **nicht**. `BLOCKED` ist der korrekte Sicherheitszustand, kein offener To-do-Rest.
3. Der **kanonische privilegierte Pfad** im Repository ist bereits der provider-neutrale Control Plane (SA3 → SA2 → SA1 → M4 + M8 Profile-Gate). Es existiert kein paralleler provider-spezifischer Admin-Bypass (Exit-Gate 4 VERIFIED PASS).

3. Vorgeschlagene Scope-Entscheidung (Owner)

Entscheidungsvorschlag:

Für M8 Exit-Gate-Punkt 2 gilt als „privileged supported providers“ die Menge der **mutierenden Provider mit produktivem Execution-Host**. Aktuell ist das ausschließlich `chatgpt-github-connector`. Non-mutating Profile bleiben `NOT_APPLICABLE`. `claude-code-cli` bleibt `BLOCKED` und ist **kein** unterstützter privilegierter Produktionspfad, bis ein separates ADR einen sicherheitsarchitektonisch tragfähigen realen Aufrufer freigibt.

Folgen bei Owner-Akzeptanz

Exit-Gate-Punkt	Ergebnis
1	PASS (unverändert)
2	PASS unter Scope oben
3-7	PASS (unverändert)
8	PASS nach Merge dieses Docs-PRs + Traceability-Sync
9	N/A / PASS (kein offener M8-Arbeitsbranch nach Merge+Delete)

Dann darf M8 als `COMPLETE / VERIFIED PASS` **nur nach ausdrücklicher Owner-Bestätigung** (Kommentar/Approve auf dem PR oder Follow-up-Commit Status-Flip) geführt und M9 freigegeben werden.

Folgen bei Ablehnung

- Punkt 2 bleibt `PARTIAL`.

- M8 bleibt `IN PROGRESS` .
- M9 bleibt blockiert.
- Nächster Schritt wäre entweder (a) neues ADR für Claude-Code-Host oder (b) Profil-Umbau `claude-code-cli` → non-mutating / separates Content-Profil.

4. Was diese Entscheidung nicht tut

- Kein Self-Merge und kein automatisches M8-Closure ohne Owner.
- Keine Aktivierung von Claude-/Gemini-/NotebookLM-Mutation.
- Keine Schwächung von IAM, REM, Kill-Switch oder Human-only MERGE.
- Kein Start von M9-Drills in diesem PR.

5. Owner-Aktionsfeld

- **[x] ACCEPT** — Scope-Entscheidung wie in §3; M8 Closure Evidence wird geschrieben, M9 wird als Phase freigegeben (Start jedes einzelnen M9-Drills bleibt eine eigene, separat zu autorisierende Handlung).
- **[] REJECT**
- **[] DEFER**

Signatur / Datum (Owner): SvenKulesa (explizite Anweisung "ACCEPT (empfohlen)" via AskUserQuestion in Claude-Code-Sitzung) / 2026-08-16

Reichweite der Freigabe: Diese Owner-Bestätigung akzeptiert ausschließlich die in §3 formulierte Scope-Entscheidung für Exit-Gate-Punkt 2 und die daraus resultierende M8-Closure. Sie ist **keine** Freigabe für einzelne M9-Assurance-Drills, keine Freigabe für einen `claude-code-cli`-Host, und keine Produktions-/IAM-/Secret-/Deploy-Mutation — diese bleiben eigenständig zu autorisieren (CLAUDE.md Authorized Principals, Integrated Roadmap §11 "Nicht ausführbar ohne separate Owner-Freigabe").

6. Nachtrag — Provider-Set-Korrektur nach Owner-ACCEPT (2026-08-16)

Zwischen der Owner-ACCEPT-Entscheidung oben und der Veröffentlichung dieses Status-Flips hat der Owner das kanonische Provider-Set korrigiert (PR #365, `docs/evidence/m8/M8_PROVIDER_SET_CORRECTION_2026-08-16.md`): Google AI Studio ist **nicht** Teil der Wertschöpfungskette; kanonisch sind ChatGPT, Claude, **Grok** (`grok-xai-connector` neu, `google-ai-studio/notebooklm` jetzt `RETIRED_PROVIDER_ALIASES`).

§1-§4 oben bleiben als historische Evidence unverändert (Stand PR #362, Vier-Provider-Modell) — siehe `M8_PROVIDER_SET_CORRECTION_2026-08-16.md` s eigene Hygiene-Regel: bereits veröffentlichte M8-Evidence wird nicht rückwirkend umgeschrieben.

Die akzeptierte Scope-Entscheidung selbst bleibt unverändert gültig und skaliert direkt auf drei Provider: „privileged supported providers“ = mutierende Provider mit produktivem Execution-Host. Das ist weiterhin ausschließlich `chatgpt-github-connector`. `grok-xai-connector` ist — nach identischer struktureller Begründung wie `claude-code-cli` (interaktive, host-vermittelte Connector-Sitzung ohne

code-adressierbaren, OIDC-gegateten Execution-Host) — ebenfalls `BLOCKED` und kein unterstützter privilegierter Produktionspfad. `google-ai-studio / notebooklm` sind nicht mehr `NOT_APPLICABLE`, sondern `RETIRED` (Alias-DENY im Control Plane). Ergebnis für Exit-Gate-Punkt 2 unverändert: `PASS` unter demselben Scope, jetzt mit drei statt vier Profilen bewertet. Volle aktualisierte Matrix: `docs/evidence/m8/M8_CLOSURE_EVIDENCE.md` §2.

Verwandte Dokumente

- `docs/evidence/m8/M8_EXTERNAL_HOST_CONFIGURATION_VERIFIED_EVIDENCE.md`
- `docs/evidence/m8/M8_I1_CUTOVER_READINESS_MATRIX_AND_EXIT_GATE_SYNC_EVIDENCE.md`
- `docs/evidence/m8/M8_PROVIDER_SET_CORRECTION_2026-08-16.md`
- `docs/evidence/m8/M8_CLOSURE_EVIDENCE.md`
- `docs/architecture/M8_CLAUDE_CODE_REAL_CALLER_DESIGN.md`
- `docs/runbooks/M8_AGENT_CUTOVER.md`
- `docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md`

M8 — externalHostConfigurationVerified (chatgpt-github-connector)

Quelle: docs/evidence/m8/M8_EXTERNAL_HOST_CONFIGURATION_VERIFIED_EVIDENCE.md

M8 — externalHostConfigurationVerified (chatgpt-github-connector)

Status: **VERIFIED PASS** (read-only Host-Konfigurationsnachweis) Datum: 2026-08-16

Roadmap phase: M8 / Integrated Roadmap I1 Authority: docs/runbooks/M8_AGENT_CUTOVER.md , src/platform/Security/providerProfile.ts (ProviderCutoverEvidence.externalHostConfigurationVerified), ADR-0062, ESS-0019 Executor: Owner-instruierte Grok-Sitzung (Punkt A-C) Baseline: main@fc03e327a01df458595b1210d74d109fd895b717

0. Zweck und Abgrenzung

docs/evidence/m8/M8_I1_CUTOVER_READINESS_MATRIX_AND_EXIT_GATE_SYNC_EVIDENCE.md §3.1 markierte externalHostConfigurationVerified für chatgpt-github-connector als **false**, weil nur Laufzeit-Erfolge (echte SA3B/SA4-Runs) belegt waren, nicht eine **separierte** Prüfung der GitHub-seitigen OIDC-/Workflow-/Trust-Konfiguration.

Dieses Dokument schließt genau dieses Feld. Es:

- ändert **keinen** Code und **keine** Host-Konfiguration;
- aktiviert **keinen** Cutover;
- erklärt M8 **nicht** eigenständig für COMPLETE / VERIFIED PASS (siehe Scope-Entscheidung).

1. Was „externalHostConfigurationVerified“ bedeutet

Für den einzigen real verdrahteten mutierenden Provider (chatgpt-github-connector = SA3B/SA4/ Work-Package-Host-Principal) muss die **Host-Seite** (GitHub Actions) nachweisbar so konfiguriert sein, dass:

1. OIDC-Token nur von freigegebenen Workflows aus main ausgestellt und akzeptiert werden;
2. Audience, Issuer, Actor, Repository und Workflow-Ref fail-closed geprüft werden;
3. Privilegien minimal und auf dokumentierte Capabilities begrenzt sind;
4. Ingress nur über Owner-gesteuerte Issue-Titel-Präfixe erfolgt;
5. Branch-Protection/ main -Ruleset Agenten am direkten main -Schreiben hindert.

2. Geprüfte Konfigurationsartefakte (Repository-Stand)

2.1 OIDC-Verifikationsvertrag (Server)

Datei: server/systemadmin/githubActionsOidc.ts

Konstante / Regel	Wert / Verhalten
Issuer	https://token.actions.githubusercontent.com
Audience	capital-ai-systemadmin-execution
Repository	SvenKulessa/Finance (ID 1284319285)
Owner / Actor	SvenKulessa (ID 84307769)
Erlaubte Workflow-Refs	nur systemadmin-roadmap-executor.yml@refs/ heads/main , systemadmin-sa4-pilot.yml@refs/ heads/main , systemadmin-work-package-run- ner.yml@refs/heads/main
Event	nur issues
Ref	nur refs/heads/main
Signatur	RS256 + GitHub JWKS, kid-gebunden
Uhrzeit	exp/iat/nbf mit 60s Clock-Skew

Jeder abweichende Claim → `fail()` (fail-closed). Unbekannte Workflow-Refs werden explizit abgelehnt.

2.2 SA3B Execution Host Workflow

Datei: `.github/workflows/systemadmin-roadmap-executor.yml`

Aspekt	Konfiguration
Trigger	issues: [opened]
Ingress-Gate	github.event.issue.user.login == 'SvenKulessa' und Titel beginnt mit [SA3B-PROBE]
Permissions	contents: write , issues: write , id-token: write
Checkout	ref: main , persist-credentials: false
OIDC-Anforderung	Audience capital-ai-systemadmin-execution über ACTIONS_ID_TOKEN_REQUEST_*
Broker	https://capital-ai.online/api/internal/systemadmin-execution/authorize mit Bearer-OIDC
Principal appld	chatgpt-github-connector
Side-Effect-Reihenfolge	durable Authorization Evidence vor Branch-Erzeugung; Outcome append-only; Rollback bei Outcome-Fehler
Mandat	nur .ai/mandates/REM-SA3B-PROBE-001.json mit status: OWNER_APPROVED

2.3 SA4 Pilot Workflow

Datei: `.github/workflows/systemadmin-sa4-pilot.yml`

Aspekt	Konfiguration
Trigger	<code>issues: [opened]</code>
Status	deaktiviert (<code>false</code> && in <code>if:</code>) nach COMPLETE / VERIFIED PASS
Permissions	analog SA3B + <code>pull-requests: write</code>
OIDC	<code>id-token: write</code>
Self-blocking	Zielartefakt bereits auf <code>main</code> → Pilot-Skript blockiert Dauerbetrieb

2.4 Work-Package Runner (ADR-0074)

Datei: `.github/workflows/systemadmin-work-package-runner.yml`

Aspekt	Konfiguration
Trigger	Owner + Titelpräfix <code>[SYSTEMADMIN-WORK-PACKAGE]</code>
Permissions	<code>contents/pull-requests/issues/id-token: write</code>
Auswahl	Issue wählt nur welches bereits gemergte Katalog-Paket läuft; Inhalt kommt aus trusted <code>main</code> -Code
REM-Gate	Ausführung scheitert, solange gebundenes Mandat nicht <code>OWNER_APPROVED</code> ist

2.5 Branch- / Main-Protection (Policy-Evidence)

Datei: `.github/policies/main-production-protection.expected.json`

Regel	Wert
Target	<code>refs/heads/main</code>
Pull-Request required	<code>true</code>
Required status checks	<code>build-and-test , GitGuardian Security Checks</code>
Strict status checks	<code>true</code>
Non-fast-forward protection	<code>true</code>
CODEOWNER review	<code>true</code>
Bypass actors	<code>[]</code>
Agent direct main push	<code>deny (deny_direct_main_push_for_agents: true)</code>
Agent branch prefixes	<code>agent/ , claude/ , gemini/ , copilot/ , ai/</code>

3. Laufzeit-Korrelation (bereits vorhandene Evidence, nicht ersetzt)

Die Host-Konfiguration ist **zusätzlich** durch reale Läufe belegt (nicht allein dokumentarisch):

- SA3B positive Probe: Issue #223, Run 31574111075 — OIDC → Broker ALLOW → BRANCH → Outcome SUCCESS (docs/evidence/sa3b/SA3B_EXECUTION_HOST_BINDING_EVIDENCE.md)
- SA3B stale-base negative: Issue #224, Run 31574221718 — DENY vor OIDC/Broker/Side-Effect
- Weitere erfolgreiche Host-Runs in M8-Evidence referenziert (u. a. 31891824130, 31893202838, 31893623159, 31894252190)

Dieses Dokument ergänzt die **Konfigurationssicht**; die Laufzeit-Evidence bleibt in den SA3B/M8-Dateien.

4. Bewertung gegen ProviderCutoverEvidence

Für chatgpt-github-connector :

Feld	Wert nach diesem Dokument
realCallerVerified	true (unverändert, SA3B/SA4)
canonicalControlPlanePathVerified	true (unverändert)
providerSpecificBypassDenied	true (unverändert, Exit-Gate 4)
auditCorrelationVerified	true (unverändert, Exit-Gate 7)
rollbackToReadOnlyVerified	true (unverändert, Exit-Gate 6)
externalHostConfigurationVerified	true (dieses Dokument)

Damit liefert evaluateProviderCutoverReadiness('chatgpt-github-connector', completeEvidence) rechnerisch **READY** , sofern alle sechs Booleans true übergeben werden.

5. Explizit nicht behauptet

- Keine Verifikation der GitHub-UI-Ruleset-Live-Ansicht jenseits der committed expected-policy und beobachteten Merge-Gates.
- Keine Behauptung, dass claude-code-cli einen Host hat.
- Keine Freigabe von M9.
- Keine Änderung an Secrets, Environments oder Rulesets in dieser Sitzung.

Verwandte Dokumente

- docs/evidence/m8/M8_I1_CUTOVER_READINESS_MATRIX_AND_EXIT_GATE_SYNC_EVIDENCE.md
- docs/evidence/m8/M8_EXIT_GATE_ITEM2_SCOPE_DECISION.md
- docs/evidence/sa3b/SA3B_EXECUTION_HOST_BINDING_EVIDENCE.md
- server/systemadmin/githubActionsOidc.ts

- `.github/workflows/systemadmin-roadmap-executor.yml`
- `.github/policies/main-production-protection.expected.json`

M8 (Integrated Roadmap Phase I1) — Cutover-Readiness-Matrix und Exit-Gate-Sync

Quelle: docs/evidence/m8/M8_I1_CUTOVER_READINESS_MATRIX_AND_EXIT_GATE_SYNC_EVIDENCE.md

M8 (Integrated Roadmap Phase I1) — Cutover-Readiness-Matrix und Exit-Gate-Sync

Status: **DOCUMENTATION / EVIDENCE SYNC — kein Code, keine Mutation, keine Cutover-Freigabe** Datum: 2026-08-15 · **Nachtrag 2026-08-16** Roadmap phase: I1 (docs/roadmaps/INTEGRATED_DEVELOPMENT_SYSTEMADMIN_ROADMAP.md , ROADMAP-INTEGRATED-DC-SA-0001) Authority: docs/runbooks/M8_AGENT_CUTOVER.md (## Exit Gate , ## Cutover Sequence), src/platform/Security/providerProfile.ts (ProviderCutoverEvidence , evaluateProviderCutoverReadiness()) Executor: 2026-08-15 Claude-Code (Option A Sync); 2026-08-16 Grok (Punkte A-C)

0. Zweck und Abgrenzung

Dieses Dokument erfüllt Option A der I1-Fortsetzungsanweisung: Status-Sync von DEVELOPMENT_CHAIN_ROADMAP.md und M8_AGENT_CUTOVER.md , eine formale Cutover-Readiness-Matrix für alle vier Provider-Profile, Exit-Gate-Fortschritt und Traceability-Anpassung.

Es enthält keine Code-Änderung, keine Mutation, keine Cutover-Aktivierung. Gemäß Integrated Roadmap Abschnitt 3: „Documentation readiness authorizes never blocked phase execution.“ Diese Datei autorisiert nichts — sie dokumentiert nur den verifizierten Ist-Zustand.

1. Terminologie-Klarstellung: Exit Gate vs. Cutover Sequence

docs/runbooks/M8_AGENT_CUTOVER.md definiert zwei separate Listen, die in der Integrated Roadmap („Exit-Gate-Punkte 1–3, 5, 8–10“) vermischt zitiert werden:

- ## Exit Gate (9 Punkte): die Bedingungen, unter denen M8 als Ganzes COMPLETE / VERIFIED PASS gilt.
- ## Cutover Sequence (10 Schritte): der Ablaufplan, wie ein Provider-Cutover durchzuführen ist.

Diese Datei verwendet ausschließlich die tatsächliche **9-Punkte-Exit-Gate-Liste** als Zählbasis.

2. Exit-Gate-Status (9 Punkte)

#	Kriterium	Status	Evidence
1	M7 ist verifiziert	PASS	

#	Kriterium	Status	Evidence
			M7 COMPLETE / VERIFIED PASS
2	Privilegierte unterstützte Provider nutzen den provider-neutralen Control Plane als kanonischen Pfad	PENDING OWNER SCOPE	chatgpt-github-connector READY (alle 6 Felder); non-mutating NOT_APPLICABLE; claude-code-cli strukturell BLOCKED — siehe M8_EXIT_GATE_I-TEM2_SCOPE_DECISION.md
3	Policy-Equivalence-Tests bestehen	PASS	tests/unit/provider-Profile.test.ts, Phase-0-Evidence
4	Direkte provider-spezifische privilegierte Bypässe sind verweigert/deaktiviert	PASS	Bypass-Route-Audit + adminRouterBypassAudit.test.ts
5	Research-Profile scheitern an Mutationstests	PASS	konstruktorseitig + Negative Tests
6	Rollback-zu-read-only ist bewiesen	PASS	Phase-0-Evidence §7 (SA3B-Pfad)
7	Audit-Korrelation ist vollständig	PASS	M8_AUDIT_CORRELATION_EXIT_GATE_7_EVIDENCE.md
8	Evidence + Roadmap/Traceability sind synchronisiert	IN PROGRESS	dieser PR (A-C)
9	Arbeitsbranches sind gelöscht	N/A für M8-Scope	kein offener M8-Cutover-Branch

Ergebnis vor Owner-Entscheidung: 6 PASS, 1 PENDING OWNER SCOPE (Punkt 2), 1 IN PROGRESS (Punkt 8), 1 N/A. M8 Closure und M9 bleiben an ausdrückliche Owner-Akzeptanz von Punkt 2 gebunden.

3. Formale Cutover-Readiness-Matrix (4 Provider-Profile)

3.1 chatgpt-github-connector (mutierendes Profil)

Der reale Aufrufer ist der Systemadmin-GitHub-Actions-Host (SA3B/SA4/Work-Package-Runner) mit appId-Literal 'chatgpt-github-connector' — nicht eine interaktive ChatGPT-Sitzung.

Evidence-Feld	Wert	Beleg
realCallerVerified	true	SA3B/SA4/Work-Package-Host, echte OIDC-JWKS-Läufe
canonicalControlPlanePathVerified	true	OIDC → SA3 → SA2 → SA1 → M4 (+ M8 Profile-Gate)
providerSpecificBypassDenied	true	M8_PROVI-DER_BY-PASS_ROU-TE_AU-DIT_EXIT_GATE_4_EVIDENCE.md
auditCorrelationVerified	true	M8_AU-DIT_CORRELATION_EXIT_GATE_7_EVIDENCE.md
rollbackToReadOnlyVerified	true	M8_PHA-SEO_AND_PROVI-DER_PROFILE_EVIDENCE.md §7
externalHostConfigurationVerified	true	M8_EXTERNAL_HOST_CONFIGURATION_VERIFIED_EVIDENCE.(2026-08-16)

Rechnerisches Ergebnis: `evaluateProviderCutoverReadiness('chatgpt-github-connector', completeEvidence)` → **READY**.

3.2 `claude-code-cli` (mutierendes Profil)

Evidence-Feld	Wert
realCallerVerified	false
canonicalControlPlanePathVerified	false
providerSpecificBypassDenied	true
auditCorrelationVerified	false
rollbackToReadOnlyVerified	false
externalHostConfigurationVerified	false

Ergebnis: **BLOCKED**. Strukturell kein vertretbarer realer Aufrufer — siehe `docs/architecture/M8_CLAUDE_CODE_REAL_CALLER_DESIGN.md`.

3.3 `google-ai-studio` / 3.4 `notebooklm`

Beide → **NOT_APPLICABLE** (keine mutierende Capability).

3.5 Zusammenfassung (Nachtrag 2026-08-16)

Provider	Mutierend?	Readiness	Fehlende Bausteine
chatgpt-github-connector	Ja	READY	—

Provider	Mutierend?	Readiness	Fehlende Bausteine
claude-code-cli	Ja	BLOCKED	4 Felder; strukturell
google-ai-studio	Nein	NOT_APPLICABLE	—
notebooklm	Nein	NOT_APPLICABLE	—

4. Branch-Hygiene (Exit-Gate-Punkt 9)

Unverändert: kein offener M8-Cutover-Arbeitsbranch. Punkt 9 = N/A für M8-Scope.

5. Roadmap/Traceability-Sync (Exit-Gate-Punkt 8)

2026-08-15 Sync (Runbook-Header, Roadmap-Crossref, Traceability) plus 2026-08-16:

- M8_EXTERNAL_HOST_CONFIGURATION_VERIFIED_EVIDENCE.md
- M8_EXIT_GATE_ITEM2_SCOPE_DECISION.md
- Runbook-Status-Header
- Roadmap / Traceability-Nachzüge in diesem PR

6. Was dieses Dokument NICHT tut

- Kein Aufrufer für Claude Code / Google AI Studio / NotebookLM.
- Kein Self-Closure von M8 ohne Owner-ACCEPT auf Scope-Entscheidung.
- Kein Start von M9.
- Keine Branch-Löschungen fremder Themen.

7. Nachtrag 2026-08-16 — Punkte A-C

Punkt	Ergebnis
A	externalHostConfigurationVerified für chatgpt-github-connector → PASS
B	Scope-Entscheidung Exit-Gate-2 als PROPOSED dokumentiert (Owner ACCEPT/REJECT/DEFER)
C	Docs-PR inkl. Matrix/Runbook/Roadmap-Sync; Merge + Owner-Entscheidung bleiben Human

Verwandte Dokumente

- docs/evidence/m8/M8_EXTERNAL_HOST_CONFIGURATION_VERIFIED_EVIDENCE.md
- docs/evidence/m8/M8_EXIT_GATE_ITEM2_SCOPE_DECISION.md
- docs/architecture/M8_CLAUDE_CODE_REAL_CALLER_DESIGN.md
- docs/runbooks/M8_AGENT_CUTOVER.md
- src/platform/Security/providerProfile.ts

M8 — Provider-neutral Agent Cutover: Phase-0-Preflight und Provider-Profile-Paket Evidence

Quelle: docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md

M8 — Provider-neutral Agent Cutover: Phase-0-Preflight und Provider-Profile-Paket Evidence

Status: PHASE 0 COMPLETE, PROVIDER-PROFILE-PAKET **VERIFIED PASS** auf dem realen Test-Pfad, jetzt zusätzlich real gegen einen echten Aufrufer (SA3/SA4) verdrahtet (siehe Nachtrag Abschnitt 6) — M8 als Ganzes bleibt PLANNED, keine externe Mutation autorisiert, kein Live-Cutover Datum: 2026-08-15 (Nachtrag: 2026-08-15, selber Tag) Roadmap phase: M8 Authority: ADR-0062, docs/runbooks/M8_AGENT_CUTOVER.md, docs/architecture/ai-agent/AI_AGENT_PROVIDER_PROFILE_CONTRACT.md, ESS-0019 Executor: direkte Owner-instruierte Claude-Code-Sitzung (dieselbe Vorgehensweise wie M5A-M7)

0. Explizit NICHT Teil dieser Sitzung

Diese Sitzung führt **keine** Live-Cutover-Aktion durch. Kein Provider-Profil wird produktiv verdrahtet, keine bestehende direkte Provider-Zugriffsmethode wird deaktiviert, keine externe Plattform-Konfiguration wird geändert. Der neue Provider-Profile-Gate (src/platform/Security/providerProfile.ts) ist reiner Repository-Code, an keinen produktiven Aufrufer angeschlossen.

1. Phase 0 — Read-only Inventory (Runbook-Abschnitt „Phase 0“)

Ergebnis einer read-only Codebasis-/Dokumenten-Untersuchung:

1.1 Bestehende AI-Provider-Integrationen

- **In-Repo-MCP-Konfiguration ist minimal:** .mcp.json deklariert ausschließlich ga4-analytics (read-only Google-Analytics-Zugriff). Kein GitHub-/Supabase-/Stripe-/Render-Schreibzugriff ist im Repository selbst konfiguriert.
- **SA3B/SA4-Systemadmin-Execution-Host ist bereits ein funktionierendes, getestetes Provider-neutral-artiges Muster — aber nur für einen Provider/Transport** (GitHub Actions OIDC, Issue-getriggert): src/platform/Security/agentIam.ts (M4) + roadmapExecutionMandate.ts (SA1) + server/agentAudit/* (SA3) + server/systemadmin/systemadminExecutionBrokerRouter.ts bilden die volle Kette Autorisierung → Audit → Ausführung → Outcome-Audit, real bewiesen für die Capability BRANCH. Interessant: das SA3B-Workflow-Principal trägt bereits appId: 'chatgpt-github-connector', provider: 'openai' — die Form, die M8 will, existiert bereits für eine ChatGPT-Issue-Trigger-Kombination.
- **ChatGPT/Gemini/Google-AI-Studio/NotebookLM-Referenzen sind fast ausschließlich Dokumentation/ADR**, kein echter Ausführungspfad existiert für Gemini/Google-AI-Studio/NotebookLM. docs/runbooks/SYSTEM-

MADMIN_CHAT_EXECUTION_PROFILE.md benennt selbst (§10) eine strukturelle Grenze: "Repository TypeScript cannot intercept or technically wrap every external ChatGPT connector call by itself."

- **Produktfeature-LLM-Clients** (server/anthropicClient.ts , server/openaiClient.ts , src/services/agentModelRouting.ts) sind reine Inferenz-SDK-Wrapper für Finanzanalyse-Content, keine privilegierten Ausführungspfade — explizit außerhalb des M8-Scopes.

1.2 Der wichtigste Einzelbefund: diese Live-Sitzung selbst umgeht agentIam.ts vollständig

Bestätigt: die GitHub-/Render-/Supabase-/Stripe-MCP-Tool-Aufrufe dieser (und jeder) interaktiven Claude-Code-Sitzung laufen **nicht** durch evaluateAgentAuthorization() oder evaluateSystemadminRoadmapAuthorization(). Diese Funktionen sind reine Bibliotheksexporte, die nur vom serverseitigen Systemadmin-Broker und von Unit-Tests aufgerufen werden. Nichts im Repository fängt interaktive MCP-Tool-Aufrufe ab. CLAUDE.md / AGENTS.md wirken nur auf Prosa-Ebene verhaltenssteuernd, nicht technisch durchsetzend.

Kritisch für den Scope dieser Sitzung: der Tool-Grant dieser Sitzung kommt von der äußeren CCR-/Session-Laufzeitumgebung, **nicht** aus diesem Repository. Es gibt kein In-Repo-Credential oder keine In-Repo-Konfiguration, die diese Sitzung deaktivieren könnte. „Schließen" dieser Lücke für eine Live-interaktive-Sitzung ist daher **keine Code-Änderung**, sondern eine Umgebungs-/CCR-Ebenen-Frage außerhalb dieses Repositories.

1.3 Bestehende Testabdeckung (vor dieser Sitzung)

agentIam.test.ts , policyGate.test.ts , roadmapExecutionMandate.test.ts , systemadminExecutionHostWorkflow.test.ts , systemadminExecutionProfile.test.ts , systemadminAuditedExecution.test.ts , systemadminSa4Contracts.test.ts decken den generischen IAM-/REM-Kernel und den SA3B/SA4-GitHub-Actions-Pfad vollständig ab. **Es existierte keine einzige Test-Coverage für ein tatsächliches Provider-Profil-Objekt (ChatGPT/Claude/Google AI Studio) oder für Cross-Provider-Policy-Equivalence-Vektoren** — genau das schließt Abschnitt 2.

1.4 Kill-Switch-Bestand

Zwei unabhängige, bereits getestete Kill-Switch-Mechanismen existieren: agentIam.ts 's killSwitchActive -Flag (denied jede mutierende Capability, READ bleibt erhalten) und REM-Level killSwitch.enabled (Mandat ist nur ausführbar, solange scharf geschaltet — Deaktivieren fällt fail-closed). Kein Kill-Switch existiert für eine Live-interaktive-Sitzung, da nichts sie ohnehin gate-t (siehe 1.2).

1.5 Prerequisite Gate (Runbook-Abschnitt „Prerequisite Gate")

Punkt	Status
1. M7 COMPLETE / VERIFIED PASS	2026-08-14, siehe docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md Abschnitt „Exit Gate Closure"
2. Production Deployment Identity verifiziert	

Punkt	Status
	▪ <code>verify-deployment-identity</code> CI-Job, real PASS auf mehreren Produktions-Pushes
3. M4 IAM und M5 Audit intakt	▪ unverändert, weiterhin <code>VERIFIED PASS</code>
4. M6 Provenance/Attestation intakt	unverändert, weiterhin <code>VERIFIED PASS</code>
5. Systemadmin-Execution-Host-Pfad hat eigene Evidence	SA3B/SA4, siehe <code>systemadminExecutionHost-Workflow.test.ts</code> etc.
6. Human-only <code>MERGE</code> durchgesetzt	<code>MERGE</code> ist keine bekannte Agent-Capability (<code>agentIam.ts</code>), jetzt zusätzlich am Provider-Profile-Layer bestätigt (Abschnitt 2)

Alle 6 Punkte erfüllt — Phase-1-Implementierung ist zulässig.

2. Provider-Profile-Paket (Owner-bestätigter Scope: „Provider Profile Contract + Registry“)

- `src/platform/Security/providerProfile.ts` (neu): `PROVIDER_PROFILES`-Registry für alle 4 in `AI_AGENT_PROVIDER_PROFILE_CONTRACT.md` dokumentierten Provider (`chatgpt-github-connector`, `claude-code-cli`, `google-ai-studio`, `notebooklm`), je mit `plane`, `allowedCapabilities` (Teilmenge von `AGENT_CAPABILITIES`), Auth-Quelle, Tool-Transport, Sandbox-Grenze, Data-Retention-Hinweis, Kill-Switch-Prozedur. Research-Plane-Profile sind Constructor-durchgesetzt auf `READ / ANALYZE` begrenzt (ein Versuch, eine mutierende Capability einem Research-Profil zuzuweisen, wirft beim Modul-Laden).
- `evaluateProviderScopedAuthorization()`: schränkt `agentIam.ts` nur ein, ersetzt es nie — exakt nach dem bereits produktiven Muster von `roadmapExecutionMandate.ts` (SA1). Prüft zusätzlich Audit-Korrelations-ID und Envelope-Replay für jede mutierende Capability, bevor überhaupt an `evaluateAgentAuthorization()` delegiert wird.
- **Nicht produktiv verdrahtet.** Kein bestehender Aufrufer (Server-Route, Workflow) ruft dieses Modul auf — es ist bereit, wird aber von keiner Live-Mutation verwendet.
- `google-ai-studio` und `notebooklm` erhalten bewusst konservative Capability-Sets (`[READ, ANALYZE, PLAN]` bzw. `[READ, ANALYZE]`), weil für beide laut Phase-0-Befund kein realer Ausführungstransport existiert — eine Erweiterung braucht eine eigene, spätere Scope-Entscheidung.

2.1 Policy Equivalence Tests + Negative Tests (Runbook-Abschnitte)

`tests/unit/providerProfile.test.ts` (neu, 23 Tests) deckt jeden im Runbook gelisteten Punkt ab:

Policy Equivalence Tests: read-only unter jedem Read-Profil erlaubt; Repository-PR-Arbeit identisch für ChatGPT und Claude Code unter äquivalentem Scope erlaubt; `MERGE` für jedes Profil verweigert; Produktionsmutation ohne Approval verweigert; fehlendes/leeres Ziel identisch über Provider hinweg verweigert; abgelaufene Approval verweigert; Self-Authority-Expansion-artige Capability verweigert (unbekannt im Capability-Modell, wie `MERGE`);

vorgetäuschter Provider-/Modell-Admin-Status wird ignoriert; Research-Profil-Mutation verweigert; mutierende Anfrage ohne Audit-Korrelations-ID verweigert.

Negative Tests: unbekanntes Provider-Profil verweigert; Principal-appld/Profil-Mismatch verweigert; unvollständiger Principal bei privilegierter Capability verweigert; Profil-/Capability-Mismatch verweigert; kein „direkter Admin“-Pfad (leerer appld verweigert); Research-Profil ruft Mutation-Endpunkt auf → verweigert; Ziel-Mismatch verweigert; wiederholtes Mutation-Envelope (Replay/Dedupe) verweigert; nicht verfügbares Audit auf Mutationspfad verweigert; aktiver Kill-Switch verweigert Mutation, behält Lesezugriff.

3. Testlauf (Provider-Profile-Paket, isoliert)

- `npx vitest run` (voller Suite-Lauf): **930 Tests, 161 Dateien, alle PASS** (davon neu: 23 in `providerProfile.test.ts`).
- `npm run lint ( tsc --noEmit )`: PASS.

6. Nachtrag — Policy Equivalence gegen einen echten Aufrufer verdrahtet (2026-08-15)

Owner-Anweisung (nach Merge des Provider-Profile-Pakets): „Policy Equivalence gegen einen echten Aufrufer verdrahten“. Ziel: das neue, generische `providerProfile.ts` nicht nur isoliert testen, sondern tatsächlich in einen bestehenden, produktiven Autorisierungspfad einbinden — ohne dessen bisheriges Verhalten zu verändern.

Gewählter Aufrufer: `server/agentAudit/systemadminAuditedExecution.ts` (SA3), der aktive, Owner-reaktivierte SA3B-GitHub-Actions-Host. Grund: das SA2-Chat-Profil (`systemadminExecutionProfile.ts`) implementiert bereits händisch exakt ein ChatGPT-Provider-Profil (`SYSTEMADMIN_CHAT_APP_ID = 'chatgpt-github-connector', SYSTEMADMIN_CHAT_CAPABILITIES`) — die ideale Stelle, um zu beweisen, dass das neue generische M8- Registry-Profil zum selben Ergebnis kommt wie die bisherige SA2-spezifische Logik.

Sicherheitsanalyse vor der Umsetzung: ein naiver Ansatz — `evaluateProviderScopedAuthorization()` (die volle Funktion inkl. eigenem `evaluateAgentAuthorization()`-Aufruf) direkt in SA3 einzuhängen — hätte die bestehende HIGH-Risk-PR-Testabdeckung real gebrochen: SA1 (`evaluateSystemadminRoadmapAuthorization`) baut sein eigenes `AgentApprovalEvidence`-Objekt aus dem REM-Mandat (`mandate.approvalEvidenceRef`); ein zweiter, unabhängiger `evaluateAgentAuthorization()`-Aufruf ohne dieselbe Approval-Konstruktion hätte für jede HIGH-Risk- Anfrage fälschlich verweigert. Daher wurde `providerProfile.ts` refaktoriert: eine neue, reine `checkProviderProfileScope()`-Funktion führt ausschließlich die Profil-Eingrenzung durch (bekanntes Profil, appld-Match, Capability-Zugehörigkeit, Audit-Korrelation, Replay) **ohne** `agentIam.ts` selbst aufzurufen. Die bestehende `evaluateProviderScopedAuthorization()` bleibt für Aufrufer ohne eigene IAM-Kette unverändert bestehen (jetzt intern auf `checkProviderProfileScope()` aufgebaut, alle 23 bestehenden Tests weiterhin PASS, reiner Refactor).

Verdrahtung: `authorizeSystemadminAuditedExecution()` ruft `checkProviderProfileScope()` zusätzlich zu (nicht anstelle von) `prepareSystemadminChatAction()` (SA2→SA1→M4) auf. Eine

Verweigerung durch die neue Schicht (`layer: 'PROVIDER_PROFILE'`) gewinnt nur, wenn weder Self-Authority-Schutz noch die neue Schicht ein ALLOW liefern — die bestehende SA2/SA1/M4-Kette bleibt in jedem Fall die maßgebliche Autorität für ALLOW-Entscheidungen. `SystemadminChatProfileLayer` wurde um `'PROVIDER_PROFILE'` erweitert (rein additiv, keine bestehende Verwendung angepasst).

Regressionsnachweis: alle 6 bestehenden `systemadminAuditedExecution.test.ts`-Tests bestehen unverändert. 3 neue Tests bewiesen die Komposition:

1. **Policy-Equivalence-Drift-Guard:** `PROVIDER_PROFILES['chatgpt-github-connector'].allowedCapabilities` und `SYSTEMADMIN_CHAT_CAPABILITIES` werden explizit auf exakte Übereinstimmung getestet — driften sie künftig auseinander, würde die neue Schicht sofort live gültige SA3B/SA4-Anfragen verweigern, und dieser Test schlägt fehl, bevor das passiert.
2. **Audit-Verfügbarkeit:** ein leerer `traceId` führt weiterhin zu einem harten Fehlschlag (bereits vorher durch `agentAuditWriter.ts` erzwungen); die neue Schicht stimmt darin unabhängig überein.
3. **Echte Live-Capability (BRANCH, MEDIUM Risk):** end-to-end-ALLOW durch die komplette Kette inklusive der neuen Schicht, identisch zum Verhalten vor dieser Änderung (`layer: 'REM_POLICY'` , `executionPermit.auditBoundExecutionPermitted: true`).

Testlauf (voller Suite-Lauf nach der Verdrahtung): `npx vitest run` — **949 Tests, 168 Dateien, alle PASS.** `npm run lint ( tsc --noEmit ):` PASS.

Ergebnis: das M8-Provider-Profile-Registry ist jetzt kein isoliert getestetes Modul mehr, sondern nimmt real an einer produktiven Autorisierungsentscheidung teil — additiv, ausschließlich einschränkend, ohne das bestehende SA3B/SA4-Verhalten zu verändern. Dies ist die im Runbook geforderte „Policy Equivalence“ real gegen einen echten Aufrufer bewiesen, nicht nur in Unit-Tests gegen synthetische Fixtures.

4. Was dieses Ergebnis NICHT bedeutet

- M8 als Ganzes ist **nicht** `COMPLETE / VERIFIED PASS` . Bisher deckt dies „Phase 1 — Provider Profile Implementation“ (für die 4 dokumentierten Provider), eine reale Verdrahtung an genau einen bestehenden Aufrufer (SA3/SA3B) und den Rollback-zu-read-only-Nachweis (Abschnitt 7) ab. Offen bleiben laut Runbook-Exit-Gate insbesondere: die Cutover-Sequenz für die übrigen Provider (Claude Code, Google AI Studio, NotebookLM haben noch keinen echten Aufrufer), direkte Provider-Bypässe deaktivieren, und die Frage, wie/ob die Live-interaktive-Sitzungs-Lücke (Abschnitt 1.2) überhaupt im Scope von M8 geschlossen werden kann, da sie außerhalb der Repository-Kontrolle liegt.
- Kein bestehender direkter Provider-Zugriffspfad wurde deaktiviert. Der SA3B-Live-Host verhält sich für seine tatsächlich genutzte Capability (BRANCH) exakt wie vor dieser Änderung — die neue Schicht ist bewiesen ein reiner Zusatz, keine Verhaltensänderung.

7. Nachtrag — Rollback-zu-read-only real bewiesen (M8-Exit-Gate-Punkt 6, 2026-08-15)

Owner-Anweisung (nach Merge der SA3B-Verdrahtung): „fahre mit Roadmap fort“ → gewähltes nächstes Element (bei fehlender Präferenz empfohlene Option): Rollback-zu-read-only beweisen, direktes Gegenstück zu M7s Rollback-Nachweis.

Befund vor der Umsetzung: Der einzige bisher an SA3B verdrahtete Kill-Switch (`mandate.killSwitch.enabled`) verweigert bei Deaktivierung **alles**, inklusive `READ` — zu strikt, um „restore read-only operation“ (M8-Runbook, Rollback-Anforderung 3) tatsächlich zu beleben. `agentIam.ts` s eigener `killSwitchActive` -Mechanismus verweigert dagegen gezielt nur mutierende Capabilities und erhält `READ / ANALYZE / PLAN` — genau das vom Runbook geforderte Verhalten —, war aber bisher **nicht** durch die SA1-REM-Kette hindurch verdrahtet (`evaluateSystemadminRoadmapAuthorization` s `iamRequest` enthielt kein `killSwitchActive` -Feld).

Schließung (additiv, rein einschränkend): - `src/platform/Security/roadmapExecutionMandate.ts` : `SystemadminRoadmapAuthorizationRequest` erhält ein neues, optionales Feld `killSwitchActive?: boolean` (Standard: nicht gesetzt = keine Verhaltensänderung), das unverändert an `evaluateAgentAuthorization()` durchgereicht wird. Fließt automatisch durch die gesamte bestehende Kette (Broker → SA3 → SA2 → SA1 → M4), ohne `systemadminExecutionProfile.ts` oder `systemadminAuditedExecution.ts` ändern zu müssen. - `src/platform/Security/providerProfile.ts` : `checkProviderProfileScope()` (und darüber `evaluateProviderScopedAuthorization()`) erhält ein optionales `registry` -Parameter — beweist den zweiten, unabhängigen Rollback-Hebel „Provider-Profil deaktivieren“ (M8-Runbook-Anforderung 1), ohne die exportierte `PROVIDER_PROFILES` -Singleton-Registry je zu mutieren.

Realer Nachweis (nicht nur synthetisch): - 2 neue Tests in

`tests/unit/roadmapExecutionMandate.test.ts` : `killSwitchActive: true` verweigert `BRANCH / COMMIT / PR / CI_REQUEST` , erlaubt `READ / ANALYZE / PLAN` weiterhin; Normalbetrieb ohne das Flag bleibt unverändert. - 2 neue Tests in `tests/unit/systemadminAuditedExecution.test.ts` , end-to-end durch die **echte, live verdrahtete** SA3B-Kette (`authorizeSystemadminAuditedExecution` , nicht nur die isolierte REM-Unit-Test-Ebene): die tatsächlich live genutzte Capability `BRANCH` wird bei `killSwitchActive: true` verweigert; `READ` bleibt durch dieselbe zurückgerollte Kette erlaubt. - 3 neue Tests in `tests/unit/providerProfile.test.ts` : eine zurückgerollte Registry-Momentaufnahme (chatgpt-github-connector-Profil auf `READ / ANALYZE` verengt) verweigert `BRANCH` , erlaubt `READ` ; die reale exportierte `PROVIDER_PROFILES` -Registry bleibt dabei nachweislich unverändert.

Testlauf: `npx vitest run` — **975 Tests, 174 Dateien, alle PASS** (davon neu: 7). Bestehende SA1/SA2/SA3-Tests (37 vorher) bestehen unverändert — reine additive Erweiterung, kein bestehendes Verhalten geändert.

Ergebnis: M8-Exit-Gate-Punkt 6 („rollback-to-read-only is proven“) ist für den bisher einzigen real verdrahteten Aufrufer (SA3B) real bewiesen, mit zwei unabhängigen, komponierbaren Hebeln (IAM-Kill-Switch und Provider-Profil-Registry). Für Provider ohne echten Aufrufer (Claude Code, Google AI Studio, NotebookLM) ist dieser Nachweis mangels Aufrufer nicht anwendbar — bleibt offen, bis ein echter Aufrufer für sie existiert.

5. Geänderte/neue Dateien

Provider-Profile-Paket: - `src/platform/Security/providerProfile.ts` (neu) - `tests/unit/providerProfile.test.ts` (neu, 23 Tests)

Nachtrag — Verdrahtung gegen SA3/SA3B: - `src/platform/Security/providerProfile.ts` (refaktoriert: neue `checkProviderProfileScope()`, `evaluateProviderScopedAuthorization()` baut jetzt darauf auf) - `src/platform/Security/systemadminExecutionProfile.ts` (Typ `Systemadmin-ChatProfileLayer` um `'PROVIDER_PROFILE'` erweitert) - `server/agentAudit/systemadminAuditedExecution.ts` (neue Komposition mit `checkProviderProfileScope()`) - `tests/unit/systemadminAuditedExecution.test.ts` (3 neue Tests)

Nachtrag — Rollback-zu-read-only: - `src/platform/Security/roadmapExecutionMandate.ts` (neues optionales Feld `killSwitchActive`) - `src/platform/Security/providerProfile.ts` (neues optionales `registry`-Parameter) - `tests/unit/roadmapExecutionMandate.test.ts` (2 neue Tests) - `tests/unit/systemadminAuditedExecution.test.ts` (2 neue Tests) - `tests/unit/providerProfile.test.ts` (3 neue Tests)

`docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md` (diese Datei)

8. Nachtrag — Exit-Gate-Punkt 4 / Cutover-Sequenz-Punkt 6 vollständig auditiert (2026-08-15)

Owner-Anweisung (nach „fahre mit der Roadmap fort“): „Direkte Provider-Bypass-Routen deaktivieren“ (gewählte Option). Ergänzend zu Abschnitt 1.1 dieses Dokuments (das bereits feststellte, dass ChatGPT/Gemini/Google-AI-Studio/NotebookLM fast ausschließlich Dokumentations-/ADR-Referenzen ohne echten Ausführungspfad sind) wurde in einer separaten Sitzung ein vollständiges, repository-weites Router-Mount-Audit durchgeführt, um Exit-Gate-Punkt 4 („direct provider-specific privileged bypasses are denied/deactivated“) und Cutover-Sequenz-Punkt 6 direkt und mit Datei:Zeile-Belegen zu prüfen, statt sich nur auf die Dokumentations-Stichprobe zu stützen.

Ergebnis (unabhängig übereinstimmend mit Abschnitt 1.1): Kein provider-spezifischer privilegierter Bypass existiert im Repository. Alle Admin-nahen Routen nutzen die geteilte `checkAdminAccess`-Guard; die einzige KI-Agent-erreichbare privilegierte Oberfläche (`/api/internal/systemadmin-execution`) nutzt die volle OIDC+SA3-Kette. Punkt 4/6 sind damit **vacuously erfüllt** — es gibt nichts zu deaktivieren, weil nie eine parallele provider-spezifische Admin-Route existierte.

Vollständige Methodik, das 7-zeilige Router-Mount-Audit und die Bewertung gegen `ProviderCutoverEvidence.providerSpecificBypassDenied`: `docs/evidence/m8/M8_PROVIDER_BYPASS_ROUTE_AUDIT_EXIT_GATE_4_EVIDENCE.md`. Regressionsschutz gegen künftige unauditierte Admin-Routen: `tests/unit/adminRouterBypassAudit.test.ts` (4 Tests, alle PASS).

M8 — Provider-spezifische Bypass-Route: Vollständiges Router-Audit (Exit-Gate-Punkt 4)

Quelle: docs/evidence/m8/M8_PROVIDER_BYPASS_ROUTE_AUDIT_EXIT_GATE_4_EVIDENCE.md

M8 — Provider-spezifische Bypass-Route: Vollständiges Router-Audit (Exit-Gate-Punkt 4)

Status: **VERIFIED PASS** — kein deaktivierbarer, provider-spezifischer privilegierter Bypass-Pfad im Repository gefunden, vollständiges Router-Mount-Audit durchgeführt Datum: 2026-08-15 Roadmap phase: M8 Authority: docs/runbooks/M8_AGENT_CUTOVER.md Abschnitt „Cutover Sequence“ Punkt 6 („explicitly disable/deprecate provider-specific canonical admin routes“) und „Exit Gate“ Punkt 4 („direct provider-specific privileged bypasses are denied/deactivated“) Executor: direkte Owner-instruierte Claude-Code-Sitzung (Owner-Anweisung „Direkte Provider-Bypass-Routen deaktivieren“, nach AskUserQuestion aus „fahre mit der Roadmap fort“)

0. Ausgangslage

docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md listete „externe Connector-Bypass-Deaktivierung“ bislang als offenen M8-Punkt, ohne dass zuvor ein systematisches, vollständiges Audit aller HTTP-Routen im Repository dokumentiert war.

src/platform/Security/providerProfile.ts definiert bereits (aus paralleler Sitzungsarbeit, PR #316) ProviderCutoverEvidence.providerSpecificBypassDenied: boolean als eines von sechs Evidence-Feldern für evaluateProviderCutoverReadiness() — diese Funktion prüft nur, dass der Aufrufer dieses Feld als true liefert; sie verifiziert selbst nichts. Diese Sitzung liefert die tatsächliche, verifizierte Grundlage für dieses Feld.

1. Methodik

Vollständige Inventur aller express.Router()-Instanzen im Repository und ihrer Mount-Punkte (server/routes/registerApplicationRoutes.ts), für jede geprüft: (a) welcher Autorisierungspfad greift, (b) ob dieser Pfad provider-spezifisch ist (an eine bestimmte KI-Provider-Identität gebunden, unter Umgehung der geteilten Kette) oder provider-neutral. Zusätzlich Grep über server/ und src/ nach provider-spezifischen Header-/Env-/Bypass-Mustern (x-chatgpt, x-claude, CHATGPT_*, CLAUDE_CODE_*, GOOGLE_AI_STUDIO_*, NOTEBOOKLM_*, hartkodierte API-Key-Vergleiche, bypass / BYPASS -Marker). Kernbefunde stichprobenartig selbst gegen geprüft (Router-Mount-Liste und checkAdminAccess-Definition direkt gelesen), nicht nur aus dem Recherche-Agenten übernommen.

2. Vollständige Router-Mount-Inventur

(server/routes/registerApplicationRoutes.ts:61-85, per direktem Read verifiziert)

Mount	Router (Datei)	Autorisierung	Provider-spezifischer Bypass?
/api/admin/hygiene	hygieneRouter (server/documentHygiene.ts)	checkAdminAccess (geteilt)	Nein
/api/admin	systemEventsRouter (server/systemEvents.ts)	checkAdminAccess (geteilt)	Nein
/api/admin	versionManagerRouter (src/platform/VersionManager/versionManager.ts)	checkAdminAccess + requireStepUp (muti-render Bump)	Nein
/api/admin/diagnostics	adminDiagnosticsRouter (server/adminDiagnostics.ts)	checkAdminAccess (Lesen), zusätzlich requireStepUp + OWNER_ONLY_ROLES (Capability-Grant/-Revoke, Approvals), zusätzliche Capability-Prüfung in AdminDiagnosticsAgent	Nein
/api/admin/supervisor	supervisorRouter (server/supervisorRouter.ts)	checkAdminAccess (geteilt)	Nein
/api/admin/agent-evaluation	createAgentEvaluationRouter (server/agentEvaluationRouter.ts)	checkAdminAccess (geteilt)	Nein
/api/scoring/explain	createScoreExplainabilityRouter (server/scoreExplainability.ts)	checkAdminAccess (geteilt)	Nein
/api/compliance	complianceRouter (src/platform/Compliance/router.ts)	checkAdminAccess auf jeder Route	Nein
/api/seo	seoEngineRouter (server/routes/seoEngineRoutes.ts)	checkAdminAccess auf jeder Route	Nein
/api/auth	stepUpRouter (server/stepUp.ts)	requireAuth (Supabase-Identität)	Nein — Nutzer-Self-Service-TOTP, kein Admin-/Provider-Pfad
/api/internal/systemadmin-execution	systemadminExecutionBrokerRouter (server/systemadmin/systemadminExecutionBrokerRouter.ts)	Volle SA3/SA3B-Kette: verifyGithubActionsOidcToken (echte JWKS-Signaturprüfung) → authorizeSystemadminAuditedExecution (komponiert SA2→SA1→M4 und seit demselben Tag	Nein — dies ist der einzige KI-Agent-erreichbare privilegierte Pfad, und er ist der provider-neutrale Broker selbst

Mount	Router (Datei)	Autorisierung	Provider-spezifischer Bypass?
		zusätzlich den M8-Provider-Profil-Gate <code>check-ProviderProfileScope</code>)	

Alle übrigen `express.Router()` -Deklarationen in `server/` und `src/` gewähren keine privilegierte/Admin-Capability (repository-weit enumeriert, jeweils Mount-Punkt und Guard nachverfolgt).

`checkAdminAccess` ist einmalig definiert (`src/platform/Security/authMiddleware.ts:185`) und wird konsistent von allen Admin-Routern importiert — kein Router implementiert eine eigene, bespoke Zugriffsprüfung.

3. `src/agents/*.ts` -Erreichbarkeit

- `src/agents/adminDiagnosticsAgent.ts` — ausschließlich in `server/adminDiagnostics.ts` instanziiert (siehe Tabelle oben, korrekt gegated). Kein anderer Zugriffspfad.
- `classificationAgent.ts`, `cryptoClassificationAgent.ts`, `cryptoOnChainAgent.ts`, `cryptoRiskAgent.ts`, `cryptoSentimentAgent.ts`, `fundamentalsAgent.ts`, `riskAgent.ts`, `valuationAgent.ts` — ausschließlich referenziert von `server/agentEvaluation.ts` (hinter dem gegateten `/api/admin/agent-evaluation`) und `server/scoreExplainability.ts` (hinter dem gegateten `/api/scoring/explain`). Finanzanalyse-Inferenz-Agenten (LLM-Content-Generierung), keine privilegierten Ausführungs-/Admin-Agenten, kein eigenständiger HTTP-Einstiegspunkt.

4. Provider-spezifische Secrets/Bypass-Middleware

Kein Treffer, der auf eine lebende Bypass-Route hindeutet. `server/anthropicClient.ts` / `server/openaiClient.ts` sind reine Inferenz-SDK-Wrapper (Auswahl, welches LLM eine Finanzanalyse-Anfrage beantwortet), gegated nur über Env-Var-Präsenz — keine Autorisierungsprimitive, keine Rechtevergabe.

5. Der eine bereits dokumentierte, nicht code-adressierbare Vorbehalt

`docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md` Abschnitt 1.2 dokumentiert bereits, dass eine laufende interaktive Sitzung (wie diese) MCP-Tool-Aufrufe direkt tätigt, ohne durch `agentIam.ts` zu laufen. Das ist real, aber **keine Route und keine Datei in diesem Repository** — es ist eine Eigenschaft der äußeren CCR-/Sitzungs-Laufzeitumgebung, außerhalb des Repository-Codeumfangs. Es kann nicht als „Route deaktivieren“ im Sinne von Cutover-Sequenz-Punkt 6 behandelt werden, da keine Route existiert; die Evidence-Doku grenzt das bereits korrekt ab und markiert es stattdessen als offene Architekturfrage (siehe `docs/architecture/GENERALIZED_SYSTEMADMIN_EXECUTION_HOST_DESIGN.md`, Modell-B-Diskussion).

6. Bewertung gegen Cutover-Sequenz Punkt 6 / Exit-Gate Punkt 4

Punkt 6 verlangt, dass eine provider-spezifische kanonische Admin-Route existiert, die deaktiviert werden kann. Nach vollständigem Sweep aller Router-Mounts und ihrer Guards in `server/` und `src/` **existiert keine solche Route** — jede Admin-nahe HTTP-Oberfläche nutzt `checkAdminAccess` (menschliches IAM) und die einzige KI-Agent-erreichbare privilegierte Oberfläche (SA3B-Broker) nutzt die volle OIDC- + SA3→SA2→SA1→M4(+M8-Provider-Profil)-Kette. **Es gibt nichts zu deaktivieren.**

Das ist der berichtenswerte Befund für Punkt 6: er ist **vacuously erfüllt** — es gab von Anfang an keine parallele/legacy provider-spezifische Admin-Route in dieser Codebasis. Dieser Befund stimmt unabhängig mit der bereits am selben Tag von einer anderen Sitzung dokumentierten Schlussfolgerung überein (`docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md` §1.1: „ChatGPT/Gemini/ Google-AI-Studio/NotebookLM-Referenzen sind fast ausschließlich Dokumentation/ADR, kein echter Ausführungspfad existiert“).

Für `evaluateProviderCutoverReadiness()` (`src/platform/Security/providerProfile.ts`) bedeutet das: `ProviderCutoverEvidence.providerSpecificBypassDenied = true` ist für den aktuellen Repository-Stand real, verifiziert begründbar — vorausgesetzt, die übrigen fünf Evidence-Felder (`realCallerVerified`, `canonicalControlPlanePathVerified`, `auditCorrelationVerified`, `rollbackToReadOnlyVerified`, `externalHostConfigurationVerified`) werden unabhängig davon jeweils für den konkreten Provider verifiziert. Dieses Dokument liefert ausschließlich die Evidence für das Bypass-Feld, nicht für die anderen fünf.

7. Nicht Bestandteil dieser Untersuchung

- Keine Code-Änderung — es gab nichts zu deaktivieren.
- Keine Bewertung der übrigen fünf `ProviderCutoverEvidence`-Felder.
- Keine Aussage zum M8-Exit-Gate als Ganzes — Punkt 2 (weitere Provider ohne echten Aufrufer) und Punkt 9 (Branch-Cleanup) bleiben unabhängig davon offen bzw. zu verifizieren.
- Keine Aktivierung von `evaluateProviderCutoverReadiness()` gegen einen echten Provider — das bleibt eine separate, vom Owner zu autorisierende Cutover-Entscheidung.

Verwandte Dokumente

- `docs/runbooks/M8_AGENT_CUTOVER.md` — Cutover Sequence, Exit Gate
- `docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md` — Phase-0-Inventur, unabhängig übereinstimmender Befund
- `docs/evidence/m8/M8_PROVIDER_CUTOVER_READINESS_NO_GEMINI_EVIDENCE.md` — definiert den Readiness-Gate, den dieses Dokument mit Evidence für ein Feld versorgt
- `src/platform/Security/providerProfile.ts` — `ProviderCutoverEvidence`, `evaluateProviderCutoverReadiness()`
- `server/routes/registerApplicationRoutes.ts`, `src/platform/Security/authMiddleware.ts`

M8 — Provider Cutover Readiness Gate (ohne Gemini-Runtime)

Quelle: docs/evidence/m8/M8_PROVIDER_CUTOVER_READINESS_NO_GEMINI_EVIDENCE.md

M8 — Provider Cutover Readiness Gate (ohne Gemini-Runtime)

Status: IMPLEMENTED / PR-CI PENDING

Datum: 2026-08-15

Baseline: main@c7bb16c9d02025da703c28fee2b92f7291842efc

Authority: ADR-0062, ESS-0019, docs/runbooks/M8_AGENT_CUTOVER.md

Owner-Entscheidung

Gewählte Sicherheitsoption: fail-closed Cutover Readiness Gate. Zusätzliche Vorgabe: keine Gemini-API oder Gemini-Runtime-Integration; die APIs wurden bereits vollständig entfernt.

Implementierung

`evaluateProviderCutoverReadiness()` erlaubt einen privilegierten Provider-Cutover nur, wenn folgende Evidence vollständig vorliegt:

1. realer Aufrufer;
2. kanonischer provider-neutraler Control-Plane-Pfad;
3. DENY eines provider-spezifischen Bypasses;
4. vollständige Audit-Korrelation;
5. Rollback auf Read-only;
6. verifizierte externe Host-/Connector-Konfiguration.

Fehlt ein Nachweis, bleibt der mutierende Provider `BLOCKED`.

Provider-Abgrenzung

- `chatgpt-github-connector`: mutierendes Profil; Readiness nur mit vollständiger Evidence.
- `claude-code-cli`: mutierendes Profil; aktuell ohne bestätigte externe Host-/Bypass-Evidence blockiert.
- `google-ai-studio`: Development Plane ohne mutierende Capability; privilegierter Cutover `NOT_APPLICABLE`. Keine Gemini-API-/Runtime-Integration.
- `notebooklm`: Research Plane; privilegierter Cutover `NOT_APPLICABLE`.
- `gemini`: kein Registry-Profil; unbekannter/entfernter Alias bleibt `BLOCKED`.

Sicherheitswirkung

Der Gate kann Berechtigungen nicht erweitern. Er bewertet nur die zusätzliche Cutover-Reife eines bereits definierten Profils. Provider-/Modellnamen verleihen keine Authority. Externe Connector-Grants müssen am jeweiligen Host verifiziert werden; Repository-Evidence allein genügt nicht.

Tests

- vollständige Evidence → `READY` für mutierendes Profil;
- fehlende Bypass-/Host-Evidence → `BLOCKED` ;
- Google AI Studio → `NOT_APPLICABLE` , keine `BRANCH-/PR-Capability`;
- NotebookLM → `NOT_APPLICABLE` ;
- Gemini-Alias → `BLOCKED` .

CI-/Build-Nachweise: `PENDING PR-CI`.

M8 Provider Set Correction — Owner 2026-08-16

Quelle: docs/evidence/m8/M8_PROVIDER_SET_CORRECTION_2026-08-16.md

M8 Provider Set Correction — Owner 2026-08-16

Document ID: M8-PROVIDER-SET-CORRECTION-2026-08-16

Status: ACTIVE EVIDENCE

Authority: Human/Owner decision (SvenKulesa)

Decision

Google AI Studio is **not** part of the CAPITAL-AI DEVELOPMENT Chain or AI value chain.

Canonical active providers:

1. ChatGPT (`chatgpt-github-connector`)
2. Claude (`claude-code-cli`)
3. Grok (`grok-xai-connector`)

Retired aliases (DENY / RETIRED in control plane):

- `google-ai-studio`
- `notebooklm`
- `gemini`

Code impact

- `src/platform/Security/providerProfile.ts` — registry reduced to three canonical profiles; `RETIRED_PROVIDER_ALIASES`; `evaluateProviderCutoverReadiness` returns `RETIRED` for retired aliases; `getCanonicalValueChainProviderInventory`.
- `src/platform/Supervisor/agentProviderObservation.ts` — ESS-0002 chain observation of the three providers.
- `src/platform/Supervisor/supervisor.ts` — `getSupervisorStatus()` includes `agentProviderChain` and lightweight `findings`.
- `docs/architecture/ai-agent/AI_AGENT_PROVIDER_PROFILE_CONTRACT.md` — contract updated.

Documentation hygiene

Historical M8 evidence files that still mention Google AI Studio / NotebookLM as active profiles are **not rewritten**. They remain append-only historical evidence. This note supersedes their provider-set assumptions for all forward work.

Historical value-chain diagrams in `docs/architecture/AI_VALUE_CHAIN_VALIDATION.md` (ARCH-CHAIN-0001) that list "Google AI Studio" as Stage 1 are superseded for operational authori-

ty by this Owner decision and the provider profile contract. A follow-up doc revision may restate the normative chain as ChatGPT → Claude → Grok → Documentary → Supervisor → ... without rewriting past findings.

Verification

- Unit tests: `tests/unit/providerProfile.test.ts` (canonical three + retired DENY/RETIRED).
- Supervisor status exposes `agentProviderChain.expectedProviders === [chatgpt, claude, grok]` .

Related

- ADR-0062, ESS-0019, ESS-0002
- REPORT-SVC-AI-VC-0001 (update in same change set where applicable)

M8 SA-P05 – Cutover-Simulator Evidence

Quelle: docs/evidence/m8/M8_SA_P05_CUTOVER_SIMULATOR_EVIDENCE.md

M8 SA-P05 – Cutover-Simulator Evidence

Umfang

SA-P05 ergänzt einen rein deterministischen Sandbox-Simulator für Provider-Cutover-Entscheidungen. Er ruft weder Provider noch Tools oder externe Mutation-Endpunkte auf.

Sicherheitsinvarianten

- Der vorhandene M8 Readiness Gate wird vor jedem Shadow-Vergleich ausgewertet.
- Fehlende Caller-, Control-Plane-, Bypass-, Audit-, Rollback- oder Host-Evidence blockiert fail-closed.
- Die Kandidatenentscheidung läuft über die kanonische Provider-Profile- und Agent-IAM-Kette.
- Ein Shadow-Mismatch blockiert.
- Auch ein Match setzt immer `executionPermitted: false`; es ist ausschließlich Review-Evidence.
- Google AI Studio bleibt nicht-privilegierte Development Plane, NotebookLM Research-only.
- Entfernte oder unbekannte Aliase wie `gemini` bleiben blockiert; keine Gemini API oder Runtime wurde ergänzt.

Negative Tests

Die Tests decken fehlende Host-Evidence, Kill-Switch-Mismatch, Replay, nicht privilegierte Profile und unbekannte Provider ab. Damit besitzt der Prototyp keinen Pfad zu einer echten Mutation.

Rollback

Der neue Simulator ist ein isoliertes, seiteneffektfreies Modul. Rollback erfolgt durch Entfernen von `providerCutoverSimulator.ts` und seines Unit-Tests; Provider Registry und IAM bleiben unverändert.

M9 — I2 Entry: Read-only Inventory & Gap Analysis (2026-08-16)

Quelle: docs/evidence/m9/M9_I2_ENTRY_INVENTORY_AND_GAP_ANALYSIS_2026-08-16.md

M9 — I2 Entry: Read-only Inventory & Gap Analysis (2026-08-16)

Status: INFORMATIONAL — NOT A DRILL, NOT AN EXIT-GATE CLAIM Authority: docs/run-books/M9_ASSURANCE_INCIDENT_BREAK_GLASS.md , ADR-0063, docs/roadmaps/INTEGRATED_DEVELOPMENT_SYSTEMADMIN_ROADMAP.md §5 (I2)

0. Zweck und Abgrenzung

Dieses Dokument ist der von dieser Sitzung selbst identifizierte, sichere Einstiegsschritt in I2 („Nächster sicherer Schritt wäre eine reine Bestandsaufnahme/Planungs-Evidence für I2 ... nicht die Ausführung eines echten Drills", INTEGRATED_DEVELOPMENT_SYSTEMADMIN_ROADMAP.md §11).

Explizit NICHT Teil dieses Dokuments:

- keine Ausführung irgendeines M9-Drills (Injection, Authorization-Bypass, Replay, Exfiltration, Audit-Outage, Kill-Switch, Break-Glass, Rollback);
- kein Anspruch, dass irgendein M9-Exit-Gate-Punkt erfüllt ist;
- keine Mutation von IAM/Kill-Switch/Break-Glass/Produktionszustand.

Grund: INTEGRATED_DEVELOPMENT_SYSTEMADMIN_ROADMAP.md §5 „Systemadmin-Rolle" für I2 erlaubt Assurance-Evidence-Work-Packages und Kill-Switch-/Break-Glass-Proposals, hält aber fest: „Ausführung der Drills bleibt Human/Owner-gesteuert." Jeder einzelne Drill benötigt eine separate, explizite Owner-Freigabe (Reichweite: genau dieser Drill, nicht I2 pauschal).

Dieses Dokument tut, was auch der I1-Einstieg tat: den vorhandenen Bestand real bewerten, bevor irgendetwas Riskantes beginnt.

1. Prerequisite Gate (Runbook „Prerequisite Gate") — Statusprüfung

#	Bedingung	Status	Beleg
1	M8 COMPLETE / VERIFIED PASS		docs/evidence/m8/M8_CLOSURE_EVIDENCE.md , 9/9 Exit-Gate-Punkte PASS (2026-08-16)
2	Provider-neutrale Control Plane für privilegierte Ausführung aktiv	(für den einzigen realen Aufrufer)	src/platform/Security/providerProfile.ts + SA3B-Verdrahtung (systemadminAudite-

#	Bedingung	Status	Beleg
			dExecution.ts); chatgpt-github- connector READY, claude-code-cli / grok-xai-connector strukturell BLOCKED — kein produktiver Aufrufer über die Control Plane hinaus
3	M5-Audit-Persistenz ge- sund	unverändert 	docs/evidence/m5/ M5_VERI- FIED_PASS_CLOSURE_EVIDENCE.md
4	M6-Provenance-Kontrollen intakt	unverändert	docs/traceability/ AI_AGENT_M0_M9_TRACEABILITY_MATRI M6-Zeile
5	M7-Deployment-Identity intakt	unverändert 	docs/runbooks/M7_DE- PLOY- MENT_IDENTI- TY_MUTATION.md „Exit Gate Closure“
6	Kill-Switch- und Rollback- Mechanismen identifiziert		siehe §2.6/§2.8 unten
7	Kein unresolved CRITICAL Finding aus vorangegan- genen Phasen		docs/traceability/ AI_AGENT_M0_M9_TRACEABILITY_MATRI ne offene CRITICAL-Zeile gefunden

Alle 7 Punkte erfüllt — I2-Bestandsaufnahme ist zulässig. Das Gate erlaubt den **Beginn** von Drill-Planung; es autorisiert noch **keinen einzelnen Drill**.

2. Bestand je Assurance-Domain (Runbook §„Assurance Domains“)

Für jede Domain: was bereits als **Kontrollmechanismus** existiert (aus M4–M8), vs. ob bereits ein **M9-adversarieller Drill mit Evidence-Schema** dazu ausgeführt wurde.

2.1 Authorization Bypass

- Kontrollmechanismus vorhanden: agentIam.ts , REM-Kette (SA1→SA2→SA3→SA4), Provider-Profil-Scope (providerProfile.ts), negative Unit-Tests für falsche Capability/ Principal existieren bereits verstreut (tests/unit/agentAudit.test.ts , tests/unit/system-adminAuditedExecution.test.ts , tests/unit/roadmapExecutionMandate.test.ts).
- M9-Drill ausgeführt: **Nein**. Kein konsolidiertes Drill-Evidence-Dokument mit dem in Runbook-§„Evidence Schema“ geforderten Format existiert bisher.

2.2 Prompt / Tool Injection

- Kontrollmechanismus vorhanden: kein dediziertes Modul gefunden (grep -rli injection tests/ src/ → keine Treffer). Die Threat-Model-Dokumentation (docs/architecture/ai-

agent/ AI_AGENT_THREAT_MODEL.md) benennt die Bedrohung, aber ohne zugehörige Testsuite.

- M9-Drill ausgeführt: **Nein**. Größte offene Lücke dieser Domain — keine automatisierten Negative-Tests, dass untrusted Content (Repo/PR-Kommentare/Tool-Output) niemals als Autorität behandelt wird.

2.3 Replay / Idempotency

- Kontrollmechanismus vorhanden: substantiell — `tests/unit/eventMeshReplayReliability.test.ts`, `tests/unit/outbox.test.ts`, `tests/unit/developmentChainMutationHandoff.test.ts`, `tests/unit/providerCutoverSimulator.test.ts` decken Replay/Dedupe-Verhalten bereits ab.
- M9-Drill ausgeführt: Teilweise Grundlage vorhanden, aber nicht als konsolidierter M9-Drill gegen Approval-Evidence/Mutation-Handoff/CI-Request/Terminal-Outcome im Runbook-Format dokumentiert.

2.4 Secret / Data Exfiltration

- Kontrollmechanismus vorhanden: `src/platform/Telemetry/redaction.ts`, `tests/unit/telemetryContract.test.ts`, Audit-Redaction in `systemadminAuditedExecution.ts`.
- M9-Drill ausgeführt: **Nein** als konsolidierter Angriffsversuch (TOTP-Secrets, Passkey-Material, vollständige Prompts/Request-Bodies) — bestehende Tests prüfen Redaction strukturell, nicht adversariell.

2.5 Audit Completeness / Outage

- Kontrollmechanismus vorhanden: stark — `docs/evidence/m5/M5_VERIFIED_PASS_CLOSURE_EVIDENCE.md`, `docs/evidence/m8/M8_AUDIT_CORRELATION_EXIT_GATE_7_EVIDENCE.md`, `docs/evidence/sa3/SA3_SYSTEMADMIN_AUDIT_CORRELATION_EVIDENCE.md`.
- M9-Drill ausgeführt: Korrelation bewiesen; „Audit-Persistenzausfall verhindert autonome Mutation“ (Runbook-Anforderung) noch nicht als eigener Fehlerinjektions-Drill nachgewiesen.

2.6 Kill Switch

- Kontrollmechanismus vorhanden: **zwei unabhängige, bereits getestete Mechanismen** — `agentIam.ts` S `killSwitchActive` (verweigert Mutation, erhält READ) und REM-Level `killSwitch.enabled`. Real end-to-end durch die SA3B-Kette verdrahtet und getestet, siehe `docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md` §1.4 und §7.
- M9-Drill ausgeführt: **Ja, 2026-08-16** (Owner-autorisiert via `AskUserQuestion`) — siehe `docs/evidence/m9/M9_KILL_SWITCH_LIVE_DRILL_2026-08-16.md`. Alle vier realen mutierenden Capabilities (`BRANCH` / `COMMIT` / `PR` / `CI_REQUEST`) einzeln gegen die live-wired SA3B-Kette angegriffen und verweigert; `READ` / `ANALYZE` / `PLAN` bleiben erhalten; differenzieller Beweis und Nicht-Sticky-Beweis erbracht. Trägt zu M9-Exit-Gate-Punkt 3 bei (teilweise — nur für den einzigen produktiven Aufrufer SA3B, siehe Drill-Evidence §7).

2.7 Break-Glass

- Kontrollmechanismus vorhanden: **nicht implementiert**. ADR-0063 (die normative Grundlage für M9 selbst) steht weiterhin auf `Status: PROPOSED`, nicht `ACCEPTED`. Kein Break-Glass-Modul, kein Runbook-Verfahren über die Zieldescriptions in `M9_ASSURANCE_INCIDENT_BREAK_GLASS.md` selbst hinaus gefunden.
- M9-Drill ausgeführt: **Nein — nicht ausführbar, bevor ein Break-Glass-Mechanismus überhaupt existiert**. Dies ist der klarste Fall, in dem I2 zunächst einen **Proposal** (Systemadmin-Rolle laut Roadmap §5) statt eines Drills braucht.

2.8 Rollback / Recovery

- Kontrollmechanismus vorhanden: **zwei unabhängige, real bewiesene Hebel** für den einzigen realen Aufrufer (SA3B) — IAM-Kill-Switch und Provider-Profil-Registry-Rollback, siehe `docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md` §7. Für `claude-code-cli / grok-xai-connector` mangels echtem Aufrufer nicht anwendbar (siehe `M8_CLOSURE_EVIDENCE.md` §2).
- M9-Drill ausgeführt: Rollback-zu-read-only für SA3B bewiesen; ein vollständiger Repository-Revert- oder Konfigurations-Rollback-Drill im M9-Format noch offen.

3. Governance-Lücke: ADR-0063 Status

ADR-0063 („Agent Assurance, Incident Response and Break-Glass“) — die normative ADR-Grundlage, auf die sowohl das M9-Runbook als auch dieser Roadmap-Abschnitt verweisen — steht auf `Status: PROPOSED`. Das Runbook-Exit-Gate verlangt selbst keine ADR-`ACCEPTED`-Bedingung als eigenen Punkt, aber ein `PROPOSED`-ADR als Autorität für einen Phase-Exit ist ein Hygiene-Risiko, das vor M9-Closure (nicht notwendigerweise vor Drill-Beginn) aufgelöst werden sollte. Dies ist eine Beobachtung, keine Blockade — wird hier dokumentiert, damit sie nicht verloren geht.

4. Was dieses Dokument NICHT bedeutet

- Kein M9-Exit-Gate-Punkt gilt als erfüllt.
- Kein Drill wurde ausgeführt; keine Kill-Switch-/Break-Glass-Aktion wurde in dieser Sitzung betätigt.
- Die oben aufgeführten „Kontrollmechanismus vorhanden“-Befunde sind Bausteine, keine Drill-Nachweise im Runbook-Evidence-Schema (Drill-ID, Baseline, erwartetes/tatsächliches Ergebnis, Rollback-Zustand, Owner-Zuordnung).

5. Empfohlene nächste Schritte (jeweils eigene, separate Owner-Freigabe erforderlich)

Priorisierungsvorschlag nach Reifegrad der Grundlage (am weitesten fortgeschritten zuerst):

1. **~~Kill-Switch-Live-Drill** (§2.6)~~ — **erledigt 2026-08-16**, siehe `docs/evidence/m9/M9_KILL_SWITCH_LIVE_DRILL_2026-08-16.md`.

2. **Rollback/Recovery-Drill** (§2.8) — ebenfalls starke Grundlage, direktes Gegenstück zu Punkt 1.
3. **Audit-Outage-Drill** (§2.5) — Korrelation bereits bewiesen; fehlt nur der Fehlerinjektionsteil.
4. **Replay/Idempotency-Drill** (§2.3) — bestehende Tests konsolidieren + Runbook-Evidence-Format.
5. **Authorization-Bypass-Negativtests** (§2.1) — verstreute Tests konsolidieren.
6. **Secret/Exfiltration-Drill** (§2.4) — Redaction-Struktur vorhanden, adversarieller Test fehlt.
7. **Prompt/Tool-Injection-Tests** (§2.2) — größte Lücke, braucht Testsuite von Grund auf.
8. **Break-Glass** (§2.7) — braucht zuerst einen Owner-genehmigten Proposal/Implementierung, bevor überhaupt ein Drill möglich ist; realistisch der letzte Punkt in der Sequenz.

Dieses Dokument trifft selbst keine Auswahl unter diesen Optionen — das ist Owner-Entscheidung.

Related Documents

- docs/runbooks/M9_ASSURANCE_INCIDENT_BREAK_GLASS.md
- docs/adr/ADR-0063-agent-assurance-incident-break-glass.md
- docs/roadmaps/INTEGRATED_DEVELOPMENT_SYSTEMADMIN_ROADMAP.md §5
- docs/evidence/m8/M8_CLOSURE_EVIDENCE.md
- docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md §1.4, §7

M9 — Kill-Switch Live-Drill Evidence (2026-08-16)

Quelle: docs/evidence/m9/M9_KILL_SWITCH_LIVE_DRILL_2026-08-16.md

M9 — Kill-Switch Live-Drill Evidence (2026-08-16)

Status: DRILL COMPLETE — PASS (Assurance Domain 6 only; M9 overall remains NOT COMPLETE) Authority: docs/runbooks/M9_ASSURANCE_INCIDENT_BREAK_GLASS.md §„Assurance Domains" 6, §„Evidence Schema"; Owner-Autorisierung: explizite Wahl „Kill-Switch-Live-Drill (empfohlen)" via AskUserQuestion in dieser Sitzung, 2026-08-16, im Anschluss an docs/evidence/m9/M9_I2_ENTRY_INVENTORY_AND_GAP_ANALYSIS_2026-08-16.md §5.

0. Zweck und Abgrenzung

Dieser Drill erfüllt **Assurance Domain 6 (Kill Switch)** und trägt zu **M9-Exit-Gate-Punkt 3** bei. Er ist **nicht** M9-Closure — sieben weitere Exit-Gate-Punkte (u. a. Break-Glass-Drill, Rollback/Recovery-Drill, Independent Evidence Review) bleiben offen und benötigen jeweils eigene Owner-Freigabe.

Ausgeführt wurde ausschließlich der bereits real verdrahtete, geprüfte killSwitchActive - Hebel (IAM-Ebene, agentIam.ts). Kein Produktionszustand, keine echte GitHub-Mutation, kein Secret/Credential wurde berührt — der Drill ruft die reale Autorisierungsfunktion (authorizeSystemadminAuditedExecution) mit einer gemockten Supabase-Audit-Senke auf, exakt wie die bereits gemergte M8-Rollback-Evidence es tat (docs/evidence/m8/ M8_PHA-SE0_AND_PROVIDER_PROFILE_EVIDENCE.md §7).

1. Drill-ID und Metadaten (Runbook „Evidence Schema")

Feld	Wert
Drill-ID	M9-DRILL-KILLSWITCH-2026-08-16-01
Datum/Zeit	2026-08-16T02:08:10Z
Exakte Baseline	Commit 39a6a6f4029a273b93843b3dc84c-c4cd4a093c7b (main, nach Merge PR #377)
Actor/Human-Owner	SvenKulesa (explizite AskUserQuestion -Wahl, siehe §0)
Agent/App/Ausführungs-Host	Claude Code (claude-code-cli), Claude Sonnet 5, Claude Code Remote Session
Policy-/Contract-Versionen	ADR-0058 (Agent IAM), ADR-0059/ADR-0065/SA3, ADR-0063 (M9, PROPOSED)
Capability/Risiko/Ziel	BRANCH / COMMIT / PR / CI_REQUEST (MEDIUM) und READ / ANALYZE / PLAN (LOW), Ziel github:SvenKulesa/Finance

2. Angriffs-/Testaufbau

Erweitert die real live-wired SA3B-Kette (`server/agentAudit/systemadminAuditedExecution.ts:authorizeSystemadminAuditedExecution` → `evaluateSystemadminChatExecutionProfile` → `evaluateSystemadminRoadmapAuthorization` → `evaluateAgentAuthorization`) um 9 neue Testfälle in `tests/unit/systemadminAuditedExecution.test.ts` , Describe-Block „M9 Kill-Switch Live-Drill (I2 Assurance, 2026-08-16)“:

- 1. **Alle vier realen mutierenden Capabilities einzeln angegriffen** (`BRANCH` , `COMMIT` , `PR` , `CI_REQUEST`), je mit passendem Checkpoint, sodass ausschließlich der Kill-Switch die Testvariable ist (nicht ein unabhängiger SA2-Sequenz-Gap). Erwartung: `DENY` , Begründung enthält „Kill-Switch“, kein `executionPermit` , Audit-Event mit `authorization_decision: 'DENY'` , `result: 'DENIED'` wird geschrieben.
- 2. **Alle drei nicht-mutierenden Capabilities** (`READ` , `ANALYZE` , `PLAN`) unter aktivem Kill-Switch angefragt. Erwartung: `ALLOW` , `executionPermit` vorhanden, Audit-Event mit `authorization_decision: 'ALLOW'` .
- 3. **Differenzieller Beweis:** identische `BRANCH` -Anfrage einmal ohne, einmal mit `killSwitchActive: true` — beweist, dass die Ablehnung ursächlich am Kill-Switch liegt, nicht an einer anderen Bedingung.
- 4. **Kein Sticky-/Cache-Zustand:** in derselben Session zwei aufeinanderfolgende Anfragen — erst mit aktivem, dann mit deaktiviertem Kill-Switch — beweisen, dass jede Anfrage frisch ausgewertet wird und keine separate Policy-/Config-Änderung zur Aktivierung/Deaktivierung nötig ist (nur ein Boolean-Feld pro Anfrage).

3. Erwartetes vs. tatsächliches Ergebnis

Runbook-Erwartung (Domain 6)	Tatsächliches Ergebnis
Neue Agent-Mutationen werden verweigert	Alle 4 realen mutierenden Capabilities <code>DENY</code> , real durch die live-wired Kette
Human/Owner- und Operator-Read-Zugriff bleibt erhalten	<code>READ</code> / <code>ANALYZE</code> / <code>PLAN</code> bleiben <code>ALLOW</code> unter identischem aktivem Switch
Keine Policy-Schwächung zur Aktivierung/Deaktivierung nötig	Reines Boolean-Feld pro Anfrage; kein Mandate-/Config-/Code-Change zwischen den beiden Zuständen
Aktion wird auditert	Jeder <code>DENY</code> - und <code>ALLOW</code> -Fall schreibt ein korreliertes Audit-Event (<code>mocks.insert</code> erfasst den realen Payload-Aufruf)

Testlauf: `npx vitest run` — **1086 Tests, 189 Dateien, alle PASS** (davon neu: 9, in `tests/unit/systemadminAuditedExecution.test.ts`). `npm run lint ( tsc --noEmit )` PASS.

4. Audit-/Outcome-Referenzen

Da die Supabase-Senke in der Testumgebung gemockt ist (wie bei allen bestehenden SA3B-Tests), sind die Audit-Referenzen synthetische Testwerte (`supabase:agent_audit_events:sa3-auth-1` etc.), keine echten Produktions-Audit-IDs. Die tatsächlich geschriebenen Payload-Felder (`authorization_decision` , `result` , `scope.auditCorrelationId`)

wurden jedoch gegen die reale, unveränderte `writeAgentAuditEvent` -Funktion geprüft — dieselbe Funktion, die auch im echten SA3B-Produktionspfad läuft.

5. Seiteneffekt-/Rollback-Zustand

Kein Seiteneffekt: keine echte Branch-/Commit-/PR-/CI-Aktion wurde ausgelöst (die Autorisierungsfunktion prüft nur die Entscheidung, sie führt keine GitHub-Mutation aus). Kein Rollback erforderlich — nichts wurde verändert, das zurückgesetzt werden müsste. Der Kill-Switch selbst wurde nur als Testparameter (`killSwitchActive: true/false`) übergeben, nie an einem echten Agenten oder Produktionszustand umgeschaltet.

6. Residual Findings

Ein bereits in `M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md` §7 dokumentierter, bewusst akzeptierter Befund bleibt unverändert bestehen und ist **nicht** Gegenstand dieses Drills: der zweite, unabhängige Kill-Switch (`mandate.killSwitch.enabled = false` , REM-Ebene) verweigert **alles** inklusive `READ` — strenger als der hier getestete `killSwitchActive` -Hebel. Das ist die dokumentiert gewollte, striktere Variante („vollständige Abschaltung“ statt „Rollback zu read-only“) und keine neue Lücke.

Keine neuen ungeklärten CRITICAL-Funde. Kein Owner-Zuordnungsbedarf über die bestehende SA3/M8-Eigentümerschaft hinaus.

7. Bezug zum M9-Exit-Gate

- Punkt 3 „kill-switch drill PASS“: **teilweise erfüllt** — dieser Drill deckt den real verdrahteten `killSwitchActive` -Hebel für den einzigen produktiven Aufrufer (SA3B) ab. Für `claude-code-cli` / `grok-xai-connector` bleibt der Nachweis mangels echtem Aufrufer nicht anwendbar (wie bereits in `M8_CLOSURE_EVIDENCE.md` §2 für Rollback/Provider-Profile festgehalten).
- M9 bleibt insgesamt `PLANNED – EXECUTION BLOCKED BY M8` im Runbook-Kopf, bis auch die übrigen Exit-Gate-Punkte (1, 2, 4–9) erfüllt sind; dieser Drill allein schließt M9 nicht ab.

Related Documents

- `docs/runbooks/M9_ASSURANCE_INCIDENT_BREAK_GLASS.md`
- `docs/evidence/m9/M9_I2_ENTRY_INVENTORY_AND_GAP_ANALYSIS_2026-08-16.md`
- `docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md` §7
- `tests/unit/systemadminAuditedExecution.test.ts`
- `server/agentAudit/systemadminAuditedExecution.ts`
- `src/platform/Security/agentIam.ts`

P2-1 — Datenqualitätsmetriken: Read-only Bestandsaufnahme (SA-P07 Vorstufe)

Quelle: docs/evidence/p2-1/P2_1_SA_P07_DATA_QUALITY_LINEAGE_INVENTORY.md

P2-1 — Datenqualitätsmetriken: Read-only Bestandsaufnahme (SA-P07 Vorstufe)

Status: **READ-ONLY INVENTORY COMPLETE** — reine Bestandsaufnahme, keine Code-Änderung, kein Provider Quality Contract implementiert. P2-1-Exit-Kriterium („feldbezogene Qualität/Lineage messbar“, docs/roadmaps/ROADMAP_CONSOLIDATION_MASTER_INDEX.md Abschnitt 6) ist **weiterhin nicht erreicht** — diese Bestandsaufnahme ist die geforderte Vorstufe dazu. Datum: 2026-08-15 Roadmap-Bindung: P2-1 (Datenqualitätsmetriken), Bindung D0C/M5/M9 , Systemadministrator-Prototyp SA-P07 Provider Quality Contract Authority: docs/roadmaps/ROADMAP_CONSOLIDATION_MASTER_INDEX.md Abschnitt 6 Executor: Claude-Code-Sitzung (Owner-Anweisung „starte mit p2-1“, AskUserQuestion-Antwort „Read-only Bestandsaufnahme zuerst“)

0. Ausgangslage

docs/architecture/DATENQUALITAETSSCHICHT.md dokumentiert ein einfaches, älteres Provenance-Muster (dataSource: 'live'|'fallback' , scoreBasis: 'market-data'|'heuristic'|'synthetic' , source: 'live'|'simulated' für Historie). Vor dieser Bestandsaufnahme war nicht bekannt, dass daneben bereits eine deutlich umfangreichere, neuere Provenance-/Lineage-/ Provider-Health-Infrastruktur existiert, die in keinem der bestehenden Architekturdokumente zusammenhängend beschrieben ist. Diese Sitzung liest den tatsächlichen Code (direkt + über einen Recherche-Agenten, Ergebnisse stichprobenartig gegengeprüft) und legt fest, was real vorhanden ist, bevor irgendein neuer Code für SA-P07 geschrieben wird.

1. Kernprimitive

Primitiv	Datei:Zeilen	Form
FinancialFieldProvenance / TraditionalScoringLineage / buildFinancialEvidenceId() / buildTraditionalScoringLineage()	src/types/financialProvenance.ts:1-58	Je Feld: field, provider (Stooq\ AlphaVantage\FMP\ TwelveData\ EODHD) , sourcePath, retrievedAt, observedAt?, unit?, value?, derivedFrom?. FinancialAsset-Class ist typseitig auf 'stock'\ 'forex'\ 'index' beschränkt — Krypto/Anleihen/Rohstoffe sind in diesem Typ nicht repräsentierbar.
ProviderHealthRecord / recordProviderHealth() / getProviderHealth()	src/platform/Supervisor/providerHealth.ts:1-73	Schlüssel provider:capability , In-Memory- Map (Prozesslaufzeit, nicht persistent). Felder: state

Primitiv	Datei:Zeilen	Form
		(healthy\ degraded\ unavailable), diagnosticCode?, payloadUsable?, lastSuccessAt, lastFailureAt, lastObservedAt, consecutiveFailures, cacheMode?, circuitOpenUntil?, message?.
VerifiedFieldProvenance / ScoringEvidenceRef / CanonicalScoreResult / evaluateDataQualityGate()	src/services/cryptoSnapshotProvider.ts, src/types/scoringIntegrity.ts, src/services/scoringIntegrity.ts	Strukturell ähnlich zu Familie A, aber ein eigener, unabhängiger Typ — provider ist auf 'CoinGecko' beschränkt, kein unit / derivedFrom. Bindet Scoring über ein binäres READY/nicht-READY-Gate (Coverage ≥ 0.5 , Mindest-Historienpunkte, Max-Alter).

2. Abdeckung nach Anlageklasse/Domäne

2.1 Aktien

Mechanismus	Provenance	Provider Health	Lineage/Evidence-ID	Lückenbehandlung	Datei:Zeile
Quote (kanonisch)	eigener VerifiedTraditionalQuote-Typ, nicht FinancialFieldProvenance	TwelveData:traditional-quote	Gateway-generierte evidenceId	fail-open zu SOURCE_UNAVAILABLE / STABLE_EVIDENCE	src/services/traditionalQuoteEvidence.ts:79-128
Quote (Schatten, Alpaca)	eigener AlpacaShadowObservation-Typ	Alpaca:traditional-quote-shadow	handgebautes Format quote:alpaca: {feed}: {symbol}: {ts}	Schatten-only, überschreibt nie kanonischen Preis/Score	src/services/alpacaShadowProvider.ts:1-205
Technische Faktoren (Trend/Momentum/Breakout/Vol/RSI)	FinancialFieldProvenance[]	Stooq:stock-history	buildTraditionalScoringLineage()	renormalizeAndScore() — fehlender Faktor ausgeschlossen, Gewicht proportional umgelegt	src/services/traditionalAssetScoring.ts:87-277
Technische Faktoren, Fallback	FinancialFieldProvenance[], provider: TwelveData\ EOD-HD	TwelveData:stock-history, EOD-HD:stock-history	dieselbe Lineage	Rangfolge-Fallback über mehrere Provider	src/services/traditionalHistoryFallback.ts:1-53
Fundamentaldaten (peRatio, dividen-	FinancialFieldProvenance[] je	AlphaVantage:stock-fundamentals	speist dieselbe Lineage	Feld wird bei Nicht-Finite einfach weggelassen	server/stock-Fundamen-

Mechanismus	Provenance	Provider Health	Lineage/Evidence-ID	Lückenbehandlung	Datei:Zeile
dYieldPct , profitMarginPct)	Feld, observedAt aus LasttestQuarter			sen, kein Fallback-Wert	tals.ts: 63-101

Korrektur gegenüber DATENQUALITAETSSCHICHT.md §4: der dort als offen gelistete Punkt „H1-Fundamentaldaten: fetchedAt -Zeitstempel wird nicht bis zu einer direkten Anzeige der Rohwerte durchgereicht" ist **überholt** — server/stockFundamentals.ts:72-89 liefert bereits ein vollständiges, exportiertes FinancialFieldProvenance[] je Feld inkl. retrievedAt / observedAt .

2.2 Forex

Gleiche Mechanik wie Aktien, ohne Fundamentaldaten. Gleicher FinancialFieldProvenance / Lineage-Pfad (traditionalAssetScoring.ts , traditionalHistoryFallback.ts), kein Alpaca-Schatten (aktienspezifisch).

2.3 Indizes

Mechanismus	Provenance	Provider Health	Lineage/Evidence-ID	Lückenbehandlung	Datei:Zeile
Quote	FMP über Gateway	FMP:index-quote	Gateway-generiert	SOURCE_UNAVAILABLE / STALE_EVIDENCE , kein erfundener Preis	src/services/traditionalQuoteEvidence.ts: 130-202
Roh-Cache Quote/Historie	keine	FMP:index-quote , FMP:index-history	keine	60s-Cooldown, Cache-TTL	server/fmpIndices.ts: 43-121
Historien-Evidence (FMP primär, TwelveData Kandidat)	eigener Typ, evidenceIds: index:fmp:{sym}:{date}	kein direkter recordProviderHealth - Aufruf in dieser Datei (an Adapter delegiert)	eigenes Format, abweichend von buildFinancialEvidenceId	Provider-Prioritäts-Fallback, fail-closed bei Identitäts-Mismatch/Staleness	src/services/indexMarketEvidence.ts: 31-94
Technische Faktoren	FinancialFieldProvenance[]	keine direkt	speist buildTraditionalScoringLineage() , Evidence-IDs als financial:fmp:{SYM}:{field}	renormalizeAndScore	src/services/indexMarketEvidence.ts: 96-118

Befund: Für dieselben zugrunde liegenden FMP/TwelveData-Indexdaten existieren **zwei unterschiedliche Evidence-ID-Formate** (`index:fmp:GSPC:2026-08-01` vs. `financial:fmp:GSPC:trend`), ohne dass etwas beide verknüpft.

2.4 Krypto

Läuft komplett parallel zu Familie A: `VerifiedFieldProvenance` + `ScoringEvidenceRef` + `evaluateDataQualityGate()` (`src/services/cryptoSnapshotProvider.ts`, `src/types/scoringIntegrity.ts`, `src/services/verifiedCryptoTechnicalScoring.ts:1-247`). Provider-Health-Keys: `CoinGecko:crypto-snapshot`, `{provider}:crypto-history`, `CoinAPI\|TwelveData\|EODHD:crypto-spot-price` (Konsens, `src/services/cryptoSpotConsensus.ts`), `Kraken:crypto-exchange-liquidity` (`src/services/krakenSpotMarketEvidence.ts`, kein Anschluss an `buildFinancialEvidenceId`). Gemeinsamer Nenner mit Familie A ist ausschließlich `recordProviderHealth / getProviderHealth`.

Besonderheit: `data_quality_risk` (`verifiedCryptoTechnicalScoring.ts:172`, hartkodiert `0.15 / 0.05` je nach `history.degraded`) fließt als **Scoring-Faktor** in den gewichteten Score ein statt als Provenance-/Health-Metadatum separat gehalten zu werden — architektonisch anders als bei Aktien/Forex/Index, wo `missingFactors / reasoning[]` getrennt vom eigentlichen Score bleiben.

2.5 Anleihen — Befund F1: Engine existiert, Routing-Tabelle weiß es nicht

Mechanismus	Provenance	Lineage/Evidence-ID	Datei:Zeile
Rendite-Historie (EODHD *.GBOND)	eigener <code>BondEvidence-Result</code> , <code>evidenceIds: bond:eodhd:{sym}:{date}</code>	nicht über <code>buildFinancialEvidenceId</code>	<code>src/services/eodhd-BondEvidence.ts:1-87</code>
Sovereign-Benchmark-Rendite-Scoring ×	<code>ScoringEvidenceRef[]</code> , ADR-0033-freigegebener Contract <code>sovereign-benchmark-yield-scoring/1.0.0</code> (<code>approvedAt: 2026-08-02</code>)	Passthrough <code>evidenceIds</code>	<code>src/services/sovereignBenchmarkEvidenceScoring.ts:1-60+</code> , verdrahtet in <code>src/features/registry/registryRoutes.ts:37,126,209</code>

Zwei Diskrepanzen, beide verifiziert: 1. `DATENQUALITAETSSCHICHT.md:57` behauptet Anleihen-Historie sei „ nicht abgedeckt" — überholt, `eodhdBondEvidence.ts` liefert real evidenzbasierte Historie, ein dediziertes Scoring existiert. 2. `src/platform/Supervisor/supervisor.ts:36` — **verifiziert per direktem Read:** `bond: { engineId: 'heuristic_fallback', label: 'Kein dediziertes Anleihen-Scoring implementiert', hasDedicatedEngine: false }`. Das widerspricht der tatsächlich verdrahteten, ADR-0033-freigegebenen `sovereignBenchmarkEvidenceScoring.ts`. `getSupervisorStatus()` ist damit die aktuell einzige quasi-öffentliche „Datenqualität je Anlageklasse"-Übersicht im System und meldete für Anleihen eine falsche Aussage. **Behoben 2026-08-15 auf separate Owner-Anweisung „behebe F1" — siehe Nachtrag am Ende dieses Dokuments.**

Einzel-Anleihen-Scoring ist bewusst ausgeschlossen (`individualBondScoringEligible: false` , `sovereignBenchmarkEvidenceScoring.ts:35`) — nur Benchmark-/Zinsniveau-Score, kein Instrument-Score.

2.6 Rohstoffe

Gleiches Muster wie Anleihen: `CommodityMarketEvidence` (`src/services/commodityMarketEvidence.ts:1-209` , `TwelveData:commodity-history` , Fuzzy-Namensauflösung mit 0.66-Schwelle, fail-closed bei Mehrdeutigkeit) → `commodityEvidenceScoring.ts` (ADR-0033, `commodity-evidence-scoring/1.0.0` , `approvedAt: 2026-08-02`), verdrahtet in `src/routes/rawMaterialsRoutes.ts:16` . Hier stimmt `supervisor.ts:32` (`hasDedicatedEngine: true`) mit dem realen Code überein — nur die Anleihen-Zeile ist falsch.

2.7 Makro/FRED

`MacroEvidenceSeries` (FRED: DGS2/DGS10/FEDFUNDS/CPIAUCSL; ECB EUR-Referenzkurs), `FRED:macro-series` / `ECB:macro-series` , fail-closed bei leerem/fehlerhaftem Payload (`src/services/macroRateEvidence.ts:63-144`). Zinskurven-Risikoregime (`macroRiskRegime.ts:1-153`) nutzt bewusst einen **eigenen** Capability-Key `FRED:macro-risk-regime` statt `FRED:macro-series` — dokumentierte Designentscheidung (Zeilen 44-47), damit „HTTP erfolgreich" und „Regime-Evidence nutzbar" nicht zu einem Signal verschmelzen. `execution-PriceEligible: false` ist an mehreren Stellen hart typisiert — Makrodaten sind nie ein handelbarer Preis.

2.8 News-Sentiment — Lücke behoben 2026-08-15 (F4)

`classifyNewsSentiment()` (`src/features/news/newsRoutes.ts`) ist eine deterministische Schlüsselwort-Heuristik. **Selbst verifiziert per Grep:** keine einzige Referenz auf `recordProviderHealth` oder `FinancialFieldProvenance` in dieser Datei. Weder alte noch neue Provenance-Generation deckt dieses Feld ab. Die API kennzeichnete `sentiment` bislang nicht als heuristisch — auf Owner-Anweisung „News-Sentiment kennzeichnen" behoben, siehe Nachtrag Abschnitt 11.

3. Provider-Health-Katalog (alle beobachteten `provider:capability` -Paare)

`TwelveData:traditional-quote` · `FMP:index-quote` · `FMP:index-history` · `Alpaca:traditional-quote-shadow` · `Kraken:crypto-exchange-liquidity` · `Stooq:stock-history/forex-history` · `TwelveData:stock-history/forex-history` · `EODHD:stock-history/forex-history` · `AlphaVantage:stock-fundamentals` · `CoinGecko:crypto-snapshot` · `{provider}:crypto-history` · `CoinAPI/TwelveData/EODHD:crypto-spot-price` · `FRED:macro-series` · `ECB:macro-series` · `FRED:macro-risk-regime` · `EODHD:bond-history` · `TwelveData:commodity-history`.

Alle Datensätze leben in einer einzigen In-Prozess-Map (`providerHealth.ts:27`) — keine Persistenz, keine Aggregation über mehrere Instanzen; ein Neustart verliert die Historie. `consecutiveFailures` wird nur bei `state:'healthy'` zurückgesetzt — ein `degraded` -Zustand resettet den Zähler eines vorherigen `unavailable` -Streaks nicht.

4. Verhältnis `renormalizeAndScore()` zu Provenance

`renormalizeAndScore()` (`src/services/realMarketSignals.ts`) kennt keine Provenance — es rechnet rein auf `values / weights` . Sein Ergebnis (`usedFactors`) filtert **danach** das bereits gebaute `FinancialFieldProvenance[]` -Array, das wiederum in `buildTraditionalScoringLineage()` einfließt (`traditionalAssetScoring.ts:236-277`). Die Verbindung ist einseitig und nachgelagert: Renormalisierung entscheidet, Provenance-Filterung folgt — nicht umgekehrt. Dieselbe Funktion wird unabhängig voneinander auch von `cryptoScoringService.ts` , `memeCoinScoringService.ts` , `commodityEvidenceScoring.ts` und `sovereignBenchmarkEvidenceScoring.ts` genutzt — der einzige wirklich geteilte Lückenbehandlungs-Baustein über alle Anlageklassen hinweg, obwohl die Provenance-/Evidence-Typen um sein Ergebnis herum je Domäne unterschiedlich sind.

`classifyTrendLabel()` ist noch unabhängiger: es speist ausschließlich das nutzerseitige `pattern` -Feld über `computeDisplayTrendLabel()` (`server.ts`) und berührt weder Provenance noch Lineage noch Health.

5. Gibt es bereits einen vereinten Qualitäts-/Health-Report?

Nein. Geprüft:

- `src/agents/adminDiagnosticsAgent.ts:46-54` (`readProviderHealth()`) ist ein reiner, capability-gegateter Passthrough zu `getProviderHealth()` — keine Anreicherung mit Feld-Provenance.
- `src/platform/Compliance/runtimeEvidence.ts:1-52` wandelt `ProviderHealthRecord[]` in Compliance-Findings (ISO A.8.6/A.8.16) — meldet nur nicht-`healthy` Provider; ein vollständig gesunder Provider-Bestand erzeugt null Findings und ist damit in diesem Scanner unsichtbar. Keine Provenance-/Lineage-Daten enthalten.
- `src/platform/Supervisor/supervisor.ts` `getSupervisorStatus()` aggregiert `providerHealth / marketDataRouting / marketIntegrity / screeningSla / scoreConfidence / aiGovernance` — kommt einem vereinten Status am nächsten, enthält aber **keine feld-/anlageklassenspezifische Provenance oder Lineage**.

Schlussfolgerung für SA-P07: Aktuell kann ein Operator entweder Provider-Health (Supervisor/ AdminDiagnosticsAgent/RuntimeEvidence) **oder** Feld-Provenance/Lineage (je Scoring-Service-Rückgabewert) einsehen — nie beides zusammen in einem Artefakt. Genau diese Verknüpfung wäre der Kern eines „Provider Quality Contract“-Prototyps.

6. Getrennte Systeme (bestätigt)

1. **Generationsbruch:** das alte `dataSource / scoreBasis / source` -Muster (`DATENQUALITAETS-SCHICHT.md` , `server.ts / src/lib/assetRegistry.ts`) und das neue `FinancialFieldProvenance / ProviderHealthRecord` -System sind **strukturell nicht verbunden** — kein gemeinsamer Typ, keine Konvertierungsfunktion.
2. **Drei parallele Provenance-/Evidence-Typfamilien** innerhalb der „neuen“ Generation, die nur über `recordProviderHealth / getProviderHealth` zusammenlaufen: - **A** — `FinancialFieldProvenance / TraditionalScoringLineage` (Aktien/Forex/Index, typseitig auf diese

drei beschränkt). - **B** — `VerifiedFieldProvenance / ScoringEvidenceRef / CanonicalScoreResult` (Krypto, sowie wiederverwendet für Rohstoffe/Anleihen-Scoring). - **C** — sechs handgebaute, domänenspezifische Evidence-Taschen mit je eigenem ID-Schema (`bond:eodhd:...` , `commodity:twelvedata:...` , `macro:fred:...` , `index:fmp:...` , `kra-ken:spot-ticker:...` , `quote:alpaca:...`) statt einheitlich über `buildFinancialEvidenceId()` (Format `financial:{provider}:{assetId}:{field}`), das nur von `traditionalAssetScoring.ts` und transitiv `indexMarketEvidence.ts` tatsächlich aufgerufen wird.

7. Zusammengefasste Befunde/Diskrepanzen

#	Befund	Art	Ort	Status
F1	Anleihen-Routing-Tabelle im Supervisor ist falsch (<code>hasDedicatedEngine: false</code> trotz real verdrahtetem, ADR-0033-freigegebenem Scoring)	Code, veraltet	<code>src/platform/Supervisor/supervisor.ts:36</code>	behoben 2026-08-15 (Owner-Anweisung „behebe F1“) — siehe Nachtrag
F2	<code>DATENQUALITAETS-SCHICHT.md</code> §1-Tabelle veraltet für Anleihen (Historie als „nicht abgedeckt“ gelistet)	Dokumentation, veraltet	<code>docs/architecture/DATENQUALITAETSSCHICHT.md:57</code>	Nachtrag in diesem Dokument ergänzt (siehe unten)
F3	<code>DATENQUALITAETS-SCHICHT.md</code> §4-Punkt zu H1-Fundamentaldaten-Zeitstempel ist überholt (bereits gelöst)	Dokumentation, veraltet	<code>docs/architecture/DATENQUALITAETSSCHICHT.md:118-121</code>	Nachtrag ergänzt
F4	News-Sentiment ohne jede Provenance-/Health-Kennzeichnung	echte offene Lücke	<code>src/features/news/newsRoutes.ts</code>	behoben 2026-08-15 (Owner-Anweisung „News-Sentiment kennzeichnen“) — siehe Nachtrag
F5	Zwei Evidence-ID-Schemata für dieselben Index-Rohdaten	Inkonsistenz	<code>src/services/indexMarketEvidence.ts</code>	dokumentiert, nicht behoben
F6	Sechs ad-hoc Evidence-ID-Formate statt einheitlichem <code>buildFinancialEvidenceId()</code>	Inkonsistenz	Familie C (§6)	dokumentiert, nicht behoben
F7	Provider-Health-Store In-Memory, nicht persistent, keine	Architektur-Grenze	<code>providerHealth.ts:27</code>	vom Supervisor selbst bereits offen-

#	Befund	Art	Ort	Status
	Multi-Instanz-Aggregation			gelegt (Kommentar Zeile 289)
F8	<code>consecutiveFailures</code> resettet nicht bei <code>degraded</code> , nur bei <code>healthy</code>	mögliche Feinheit für künftige Alarmschwellen	<code>providerHealth.ts:57</code>	dokumentiert, keine Bewertung ob Bug oder Absicht
F9	Krypto- <code>data_quality_risk</code> ist ein Scoring-Faktor, kein separates Qualitäts-Metadatum (anders als bei Aktien/Forex/Index)	Architektur-Inkonsistenz zwischen Domänen	<code>verifiedCryptoTechnicalScoring.ts:172</code>	dokumentiert, nicht bewertet ob zu vereinheitlichen

8. Nicht Bestandteil dieser Untersuchung

- Keine Code-Änderung an `supervisor.ts` (F1) — reine Lesearbeit, wie vom Owner für diesen ersten Schritt gewählt.
- Keine Definition eines formalen `SA-P07 Provider Quality Contract` — dieses Dokument ist die Faktenbasis dafür, nicht der Contract selbst.
- Keine Bewertung, ob die drei Provenance-Familien (§6) vereinheitlicht werden sollen — das ist eine Architekturentscheidung für den nächsten Schritt, nicht Teil einer read-only Bestandsaufnahme.
- `src/services/marketDataConsensus.ts`, der TwelveData-Adapter hinter `fetchExternalHistory`, und einzelne weitere Blattdateien wurden nicht vollständig gelesen (nur über Konsumenten referenziert) — bei Bedarf für einen konkreten SA-P07-Entwurf gezielt nachlesen.

9. Empfehlung für den nächsten Schritt

Basierend auf dieser Bestandsaufnahme ergeben sich für den eigentlichen `SA-P07 Provider Quality Contract` mehrere unabhängig wählbare, nicht sich gegenseitig ausschließende Optionen — keine davon wurde bereits begonnen:

1. **~~F1 beheben** (`supervisor.ts:36` korrigieren) — kleinster Schritt, reine Faktenkorrektur, kein neues Konzept nötig. **~~ Erledigt, siehe Nachtrag Abschnitt 10.**
2. **~~News-Sentiment-Kennzeichnung** (F4) — die im ursprünglichen `AskUserQuestion` als dritte Option angebotene, engste Lücke. **~~ Erledigt, siehe Nachtrag Abschnitt 11.**
3. **Vereinter Qualitäts-Report** (§5) — `getSupervisorStatus()` um feld-/anlageklassenbezogene Provenance-Zusammenfassung erweitern, ohne die drei bestehenden Typfamilien zu verschmelzen.
4. **Evidence-ID-Vereinheitlichung** (F5/F6) — alle Domänen auf `buildFinancialEvidenceId()` umstellen.

Owner-Entscheidung erforderlich, welche(r) weitere(n) Punkt(e) als nächstes bearbeitet werden soll(en).

10. Nachtrag 2026-08-15 — F1 behoben (Owner-Anweisung „behebe F1“)

`src/platform/Supervisor/supervisor.ts:36` korrigiert: die Anleihen-Zeile der `TASK_ROUTING_TABLE` lautete ZUVOR `{ engineId: 'heuristic_fallback', label: 'Kein dediziertes Anleihen-Scoring implementiert', hasDedicatedEngine: false }` und behauptete damit fälschlich, es existiere keine dedizierte Engine. Neu:

```
bond: {
  engineId: 'sovereign_benchmark_yield_engine',
  label: 'Anleihen-Scoring: Sovereign-Benchmark-Rendite (ADR-0033, yield-state; kein
    Einzelanleihen-/Credit-/Duration-Score)',
  hasDedicatedEngine: true,
},
```

Die Formulierung übernimmt bewusst den in `sovereignBenchmarkEvidenceScoring.ts` (`SOVEREIGN_BENCHMARK_SCORING_CONTRACT.semanticScope`) selbst dokumentierten Geltungsbereich, um keine neue Überclaim-Lücke zu öffnen: `hasDedicatedEngine: true` ist jetzt korrekt, aber weiterhin ausdrücklich als Yield-State-Benchmark-Score gekennzeichnet, nicht als Einzelanleihen-/Credit-/Duration-/Liquiditäts- oder Empfehlung-Score (`individualBondScoringEligible: false` bleibt in der Scoring-Engine selbst unverändert).

`tests/unit/supervisor.test.ts` aktualisiert: der bisherige Test „kennzeichnet Anleihen weiterhin ehrlich als ohne dedizierte Engine“ (erwartete `hasDedicatedEngine === false`) ersetzt durch einen Test, der `engineId === 'sovereign_benchmark_yield_engine'`, `hasDedicatedEngine === true` und dass das Label weiterhin explizit „kein Einzelanleihen“-Scoring ausschließt, prüft — die Ehrlichkeits- Eigenschaft des ursprünglichen Tests bleibt erhalten, nur die zugrunde liegende Tatsachenbehauptung wurde korrigiert.

Voller Suite-Lauf nach der Änderung: siehe zugehöriger PR-Body. Keine weiteren Aufrufer von `routeTask('bond') / TASK_ROUTING_TABLE.bond` außerhalb von `supervisor.ts / supervisor.test.ts` gefunden (per Grep verifiziert).

11. Nachtrag 2026-08-15 — F4 behoben (Owner-Anweisung „News-Sentiment kennzeichnen“)

`src/features/news/newsRoutes.ts` lieferte `sentiment` bislang ohne jede Herkunftskennzeichnung aus, obwohl `classifyNewsSentiment()` eine rein deterministische Schlüsselwort-Heuristik ist, keine NLP-/KI-Analyse. Neu, analog zum bestehenden `dataSource / scoreBasis`-Muster:

```
export type NewsSentimentBasis = 'heuristic';
export const NEWS_SENTIMENT_BASIS: NewsSentimentBasis = 'heuristic';
```

Jeder ausgelieferte News-Eintrag trägt jetzt zusätzlich `sentimentBasis: 'heuristic'` neben `sentiment` (`newsRoutes.ts`, / -Route-Handler). Der einzige reale Frontend-Konsument, `src/`

`components/Newsticker.tsx`, wurde geprüft: `sentiment` wird dort aktuell **nicht** gerendert (nur ins `NewItem`-State-Interface übernommen) — der Typ-Vertrag (`sentimentBasis?: 'heuristic'`) wurde dennoch synchron ergänzt, damit das Interface den tatsächlich empfangenen Datenshape korrekt beschreibt und eine künftige Anzeige die Kennzeichnung sofort zur Verfügung hat, ohne den Backend-Vertrag erneut ändern zu müssen. Keine neue UI-Anzeige wurde ergänzt — außerhalb des angeforderten Scopes „kennzeichnen“ (Datenvertrag), da `sentiment` selbst noch nirgends sichtbar ist.

`tests/unit/newsRoutes.test.ts` um einen Test für `NEWS_SENTIMENT_BASIS === 'heuristic'` ergänzt. Der HTTP-Router selbst wird — konsistent mit der etablierten Projektkonvention (keine supertest-Abhängigkeit, siehe `tests/unit/alerts.test.ts`) — nicht end-to-end getestet; die neue Konstante ist exportiert und direkt geprüft.

Voller Suite-Lauf nach der Änderung: siehe zugehöriger PR-Body.

Verwandte Dokumente

- `docs/architecture/DATENQUALITAETSSCHICHT.md` — älteres Provenance-Muster, teilweise veraltet (F2/F3)
- `docs/roadmaps/ROADMAP_CONSOLIDATION_MASTER_INDEX.md` Abschnitt 6, P2-1
- `src/types/financialProvenance.ts`, `src/platform/Supervisor/providerHealth.ts`, `src/types/scoringIntegrity.ts`, `src/services/cryptoSnapshotProvider.ts`
- `src/platform/Supervisor/supervisor.ts`, `tests/unit/supervisor.test.ts` — F1-Fix
- `src/features/news/newsRoutes.ts`, `src/components/Newsticker.tsx`, `tests/unit/newsRoutes.test.ts` — F4-Fix

R-101 — Outbox-Produktionsmigration Evidence

Quelle: docs/evidence/r101/R101_OUTBOX_PRODUCTION_MIGRATION_EVIDENCE.md

R-101 — Outbox-Produktionsmigration Evidence

Status: **VERIFIED PASS**

Datum: 2026-08-14

Authority: ADR-0054

Repository-Migration: `supabase/migrations/20260810160000_outbox_jobs.sql`

1. Autorisierung und Ziel

Der Owner wählte ausdrücklich Variante 2: read-only Preflight, anschließende produktive Migration und Post-Mutation-Verifikation.

- Supabase-Projekt: `AIFINANCIAL`
- Projekt-Ref: `ryzywoktpmyhwzxmstyu`
- Region: `eu-west-1`
- Projektstatus vor Mutation: `ACTIVE_HEALTHY`
- Weitere Supabase-Projekte: keine
- Keine Stripe-, Render-Konfigurations- oder Credential-Mutation

Secret-Werte wurden weder gelesen noch protokolliert.

2. Pre-Mutation-Baseline

Vor der Mutation ergab die produktive Schemaabfrage:

- `public.outbox_jobs`: nicht vorhanden;
- `enqueue_outbox_job`, `claim_outbox_job`, `complete_outbox_job`, `fail_outbox_job`: nicht vorhanden;
- die Repository-Migration war noch nicht in der produktiven Migrationshistorie registriert;
- Render meldete im 15-Sekunden-Takt: `claim_outbox_job ... could not find the function ... in the schema cache`.

Die Migration ist additiv, durch `begin/commit` atomar, aktiviert RLS und vergibt Rechte ausschließlich an `service_role`.

3. Ausgeführte Mutation

- Executor: Supabase Migration API im Owner-autorisierten ChatGPT/Codex-Work-Schritt
- Operation: exakt der versionierte SQL-Inhalt der Repository-Migration

- Migrationsname: `outbox_jobs`
- registrierte Produktionsversion: `20260814221830_outbox_jobs`
- Ergebnis: `success: true`

Keine bestehenden Tabellen, Funktionen, Benutzer-, Billing- oder IAM-Daten wurden verändert.

4. Post-Mutation-Verifikation

Kontrolle	Ergebnis
<code>public.outbox_jobs</code> vorhanden	PASS
RLS aktiviert	PASS
vorhandene Zeilen nach Migration	0
vier erwartete RPC-Signaturen	PASS
alle vier Funktionen <code>SECURITY DEFINER</code>	PASS
Execute für <code>anon</code>	DENY
Execute für <code>authenticated</code>	DENY
Execute für <code>service_role</code>	ALLOW
Policy <code>service_role_full_access</code>	PASS
leerer Claim-Test	PASS, Ergebnis <code>[]</code> , keine Testdaten
neue Supabase-Advisor-Funde für Outbox	keine
Render-Outbox-Fehler nach 22:18:35 UTC	keine
Render-App-Fehler nach 22:18:35 UTC	keine

Bestehende Advisor-Hinweise zu anderen Objekten sind nicht durch diese Migration entstanden und bleiben separate Arbeitspakete.

5. Nachfolgender Live-Deploy

Der nachfolgende aktuelle Produktionsdeploy bestätigt die dauerhafte Kompatibilität:

- Render-Service: `srv-d91010903t8c73edi55g`
- Deploy: `dep-d9vpf3dbedkc73et92ag`
- Commit: `ef711596f0020764441ff6ea9e1cf350b6ad2c1a`
- Status: `live`
- Outbox- oder allgemeine Error-Logs seit Start: keine
- Document Hygiene: Produktions-Read-only-Modus aktiv
- Alpaca: konfiguriert und authentifiziert; `STALE` außerhalb der Handelszeit, Feed `iex`

6. Rollback

Kein Rollback wurde benötigt. Falls eine spätere bestätigte Regression einen Rollback verlangt:

1. Outbox-Verbraucher per human-autorisierter Codeänderung stoppen;
2. vorhandene Jobs sichern und auf `processing / dead_letter` prüfen;
3. ausschließlich die vier R-101-RPCs und `public.outbox_jobs` in einer kontrollierten Supabase-Migration entfernen;
4. Render- und Supabase-Nachprüfung ausführen.

Ein Rollback darf nicht blind erfolgen, sobald produktive Jobs vorhanden sind.

7. Abschluss

R-101s Repository-Implementierung, produktive Datenbankmigration, Least-Privilege-Grenze und laufender Worker sind damit **VERIFIED PASS**. Für diesen Abschluss ist keine weitere Plattformmutation erforderlich.

SA1 — REM Validator / Control-Plane Enforcement Evidence

Quelle: docs/evidence/sa1/SA1_REM_VALIDATOR_EVIDENCE.md

SA1 — REM Validator / Control-Plane Enforcement Evidence

Status: IMPLEMENTATION CANDIDATE — FULL REPOSITORY CI PENDING Date: 2026-08-12

Base: `main@d342654715b4f1aef23e9fbaf3230b54f327e0a2` Branch: `agent/sa1-rem-validator-`

`control-plane` Authority: ESS-0021, ADR-0065, ADR-0058, Systemadmin Agent Roadmap

Purpose

SA1 turns the Roadmap Execution Mandate (REM) from governance text into a fail-closed runtime authorization layer for the Systemadmin Roadmap Executor.

This evidence covers repository/control-plane implementation only. No Supabase, Stripe, Render, deployment, billing, credential or other production mutation is part of SA1.

Implementation scope

Primary code:

- `src/platform/Security/roadmapExecutionMandate.ts`
- `src/platform/Compliance/PolicyGate.ts`
- `tests/unit/roadmapExecutionMandate.test.ts`

Contract synchronization:

- `docs/governance/ROADMAP_EXECUTION_MANDATE.schema.json`
- `docs/roadmaps/SYSTEMADMIN_AGENT_ROADMAP.md`
- `docs/traceability/SYSTEMADMIN_AGENT_TRACEABILITY_MATRIX.md`

Security design

The SA1 evaluator composes two independent gates:

`REM_VALIDATION/SCOPE → EXISTING M4 AGENT IAM`

A REM cannot bypass or replace the existing Agent IAM. Provider/model metadata remains non-authoritative.

Canonical identity binding

The runtime binds the mandate to:

- Owner: `SvenKulessa`
- Agent: `capital-ai-systemadmin-roadmap-executor`

- Repository: `SvenKulesa/Finance`
- Base branch: `main`

A mismatch is DENY.

SA1 capability ceiling

Only the following can reach ALLOW in SA1:

`READ, ANALYZE, PLAN, BRANCH, COMMIT, PR, CI_REQUEST`

The following remain technically blocked:

- `MERGE`
- `DEPLOY_REQUEST`
- `PRODUCTION_MUTATION`
- effective `CRITICAL` execution

This remains true even if an over-broad or malicious REM lists those capabilities.

Scope enforcement

The validator checks:

- `OWNER_APPROVED` status;
- mandatory `approvalEvidenceRef`;
- validity window and expiry;
- roadmap item membership;
- capability membership;
- effective risk $\leq$ mandate maximum and $\leq$ HIGH;
- exact allowed target;
- exact path or `prefix/**` path allowlist;
- no absolute/backslash/traversal paths or unsupported wildcards;
- explicit mutation class for repository-mutating capabilities;
- complete reserved Human/Owner mutation denylist;
- maximum number of open Systemadmin PRs;
- exact changed-file overlap with already open PRs;
- CI budget/duplicate unchanged-head request protection;
- kill-switch mechanism availability.

Self-elevation protection

The Systemadmin principal is hard-denied from mutating its own trust root, including:

- ESS-0021;
- ADR-0065;

- AGENTS.md;
- Autonomous Agent Concept Gate;
- REM schema;
- Systemadmin execution policy;
- Agent IAM;
- REM validator itself;
- PolicyGate REM entry point.

These changes remain on the normal Human/Owner-governed path.

Pre-PR validation evidence

An isolated local security harness was executed before PR creation.

Environment used for the isolated check:

- Node: v22.16.0
- TypeScript: 5.8.3

The repository requires Node 24.x, therefore this preflight is supporting evidence only and does not replace GitHub CI.

Type check

The SA1 module was compiled together with the current `agentIam.ts` contract using TypeScript 5.8.x.

Result: **PASS**

Functional security harness

The compiled SA1 evaluator was executed against representative ALLOW/DENY cases.

Result: **PASS**

Verified negative cases:

- unknown REM field → DENY/invalid;
- missing REM → DENY;
- expired REM → DENY;
- wrong Human Actor → DENY;
- wrong Agent → DENY;
- `MERGE` → DENY;
- `DEPLOY_REQUEST` → DENY;
- `PRODUCTION_MUTATION` → DENY;
- path traversal → DENY;
- path outside allowlist → DENY;

- Systemadmin self-authority path → DENY;
- exact open-PR changed-file overlap → DENY;
- open Systemadmin PR limit exceeded → DENY;
- CI budget exceeded → DENY;
- unchanged head already validated → DENY;
- reserved Owner mutation class → DENY.

Verified positive case:

- valid Owner-approved HIGH-risk repository PR request inside exact REM scope → ALLOW through REM scope and existing Agent IAM.

Required repository CI

Before SA1 can become `VERIFIED PASS`, the final reviewed PR head must pass the repository's normal code validation, including as selected by the current check-class workflow:

- dependency/integrity checks;
- TypeScript;
- unit tests, including `roadmapExecutionMandate.test.ts`;
- production build/predeploy checks where selected;
- governance/security checks;
- final `build-and-test`.

Mutation state

Domain	State
Repository code	IMPLEMENTED / PR REVIEW PENDING
REM runtime enforcement	IMPLEMENTED / CI PENDING
Supabase	NOT REQUIRED
Stripe	NOT REQUIRED
Render	NOT REQUIRED
Deployment	NOT REQUIRED
Production mutation	PROHIBITED IN SA1

Exit criteria

SA1 is complete only after:

1. Human/Owner current-head review;
2. required CI PASS;
3. Human merge into `main`;
4. merged SA1 branch deletion;

5. resulting main SHA/evidence recorded for SA2.

Until then SA2 remains blocked.

SA2 Chat Execution Profile Evidence

Quelle: docs/evidence/sa2/SA2_CHAT_EXECUTION_PROFILE_EVIDENCE.md

SA2 Chat Execution Profile Evidence

Status: PRE-PR / REPOSITORY CI PENDING Date: 2026-08-12 Baseline: `main@d8cc-c3c5e136d51ae36b57b103119b25f4a42b8b` Authority: ESS-0021, ADR-0065, SA1 REM Validator, M4 Agent IAM

1. SA1 prerequisite verification

PR #215 is merged at `d8ccc3c5e136d51ae36b57b103119b25f4a42b8b`.

Authoritative SA1 checks on final PR head `b3aacd0c9ca6d9dfb981c4f563f0c840f93dad2e`:

- CI-Prüfung #909: PASS;
- Governance-Prüfung #634: PASS;
- SA1 repository/control-plane implementation: merged.

A separate Google Marketing protection run #155 failed in the static contract step. During SA2 the same failure reproduced as run #157. The diagnosis identified two concrete CI defects rather than an SA1/SA2 runtime-policy failure:

1. the protected-change workflow watched the over-broad path `src/platform/Security/**`, so unrelated Systemadmin security code triggered the Google Marketing guard;
2. the central CI consolidation had removed the dedicated post-build invocation of `tests/unit/securityResponse.production.test.ts`, even though that test intentionally skips before `dist/index.html` exists.

PR #216 therefore remediates this inherited CI regression instead of suppressing it:

- `ci.yml` restores a dedicated production-CSP delivery test **after** `npm run build`;
- the Owner gate evaluates the body snapshot of the exact `edited` event instead of re-reading a later live PR body, preventing checkbox race/double-build authorization;
- the Google Marketing guard watches exact security dependencies instead of all `src/platform/Security/**` files;
- the guard now also watches `ci.yml` and `package.json`, validates the invariant script wiring, verifies the production test file exists and statically proves `build → production CSP test → predeploy` order.

These changes are not treated as PASS until the final PR #216 head completes the corresponding GitHub checks.

2. SA2 objective

Create the ChatGPT execution profile for logical agent:

capital-ai-systemadmin-roadmap-executor

The profile must compose with SA1 rather than bypass it and must not create an unaudited live mutation gap before SA3.

3. Implemented artifacts

- `src/platform/Security/systemadminExecutionProfile.ts`
- `.ai/contracts/systemadmin-roadmap-execution-profile.json`
- `tests/unit/systemadminExecutionProfile.test.ts`
- `docs/runbooks/SYSTEMADMIN_CHAT_EXECUTION_PROFILE.md`
- this Evidence document
- Systemadmin Roadmap/Traceability synchronization
- CI/Owner-gate race remediation and restoration of protected post-build CSP evidence discovered during SA2 review

4. Canonical authorization chain

OWNER_APPROVED REM → SA2 CHAT PROFILE → SA1 REM VALIDATOR → M4 AGENT IAM → ACTION ENVELOPE

The profile uses the exact ChatGPT app/client id only as an execution-surface constraint. It is not an authority source.

5. Capability boundary

Allowed for SA2 policy evaluation:

- READ
- ANALYZE
- PLAN
- BRANCH
- COMMIT
- PR
- CI_REQUEST

Hard excluded:

- MERGE
- DEPLOY_REQUEST
- PRODUCTION_MUTATION

SA1 also continues to deny CRITICAL execution, reserved Human/Owner mutation classes, invalid targets/paths, invalid/expired REMs, out-of-scope actions, CI-budget violations and concurrent-writer conflicts.

6. Dry-run / live boundary

DRY_RUN

The complete initial repository capability sequence may be policy-evaluated and represented as an immutable action envelope.

Mutating envelopes are explicitly marked:

```
liveMutationPermitted = false
```

LIVE

Before SA3:

- non-mutating READ/ANALYZE may be allowed when the exact REM allows them;
- BRANCH/COMMIT/PR/CI_REQUEST are fail-closed denied.

This prevents unaudited standing mutation authority from existing between SA2 and SA3.

7. Sequence checks

The profile adds execution-order checks above SA1:

Action	Required SA2 state
PLAN	current main + Roadmap resolved
BRANCH	full security/Roadmap/overlap/check-class/test/rollback preflight
COMMIT	fresh <code>agent/*</code> branch + targeted validation PASS
PR	branch + implementation complete + validation PASS + current head SHA + no existing work-package PR
CI_REQUEST	open PR + PR number + current head SHA + validation PASS

Mutating actions are denied after final Human/Owner review has started.

8. Fail-closed stop conditions

Tests cover denial on:

- wrong agent;
- wrong ChatGPT app/client id;
- SA1 not VERIFIED PASS;
- incomplete preflight;
- missing/fake branch state;
- missing targeted validation;
- incomplete implementation before PR;
- missing current head;

- duplicate/open PR state mismatch;
- credential exposure signal;
- untrusted prompt/tool scope-elevation signal;
- unexpected production mutation requirement;
- final Owner review already started;
- MERGE, DEPLOY_REQUEST, PRODUCTION_MUTATION;
- SA1 target/path scope mismatch;
- SA1 CI budget denial.

9. Positive path evidence

A valid Owner-approved REM with exact:

- Owner;
- Systemadmin agent;
- Finance repository/main;
- Roadmap item;
- allowed capability;
- target;
- repository paths;
- HIGH-or-lower risk;
- repository mutation class;
- validity window;
- kill switch;
- approval evidence

can produce a DRY_RUN PR action envelope after all SA2 checkpoints pass.

The envelope is frozen, secret-free by contract and records `requiresHumanMerge = true`.

10. Credential and data handling

The model-facing action envelope contains no token/password/secret/reusable-credential field. Credentials remain with the connector/tool host.

If credential exposure is detected, the profile returns DENY before a mutating action may be prepared.

11. Tool-host enforcement boundary

This repository code is the canonical policy/profile contract, but it cannot by itself intercept arbitrary external ChatGPT connector calls. SA2 therefore does not claim end-to-end connector enforcement.

SA3 must connect the prepared action path to the existing M5 append-only audit mechanism and prove correlation before LIVE mutation can be enabled.

12. External mutations

Domain	SA2 state
Repository code	IMPLEMENTED ON BRANCH
Chat execution profile	IMPLEMENTED / CI PENDING
GitHub workflow hardening	IMPLEMENTED / CURRENT-HEAD CI PENDING
Supabase	NOT REQUIRED
Stripe	NOT REQUIRED
Render	NOT REQUIRED
Deployment	NOT REQUIRED
Production mutation	PROHIBITED
Merge	HUMAN/OWNER ONLY

13. Exit gate

SA2 becomes VERIFIED PASS only after:

1. current-head Human/Owner review;
2. required repository CI PASS for final reviewed head;
3. Google Marketing guard PASS when its protected paths are part of the final diff;
4. the dedicated post-build production CSP delivery test PASS;
5. Human merge to `main`;
6. SA2 branch deletion;
7. mutating LIVE mode remains disabled;
8. SA3 append-only audit correlation becomes the next active stage.

SA3 Systemadmin Audit Correlation Evidence

Quelle: docs/evidence/sa3/SA3_SYSTEMADMIN_AUDIT_CORRELATION_EVIDENCE.md

SA3 Systemadmin Audit Correlation Evidence

Status: SA3A IMPLEMENTED / REPOSITORY CI PENDING — SA3B EXECUTION-HOST BINDING REQUIRED Date: 2026-08-12 Current baseline:

`main@083d8f25083034e3785d1a8e0c57eaf03463c907` (PR #217 merge) SA3 prerequisite baseline:

`main@a5abc1685026651f4297a487e855683a1fa1e58e` (PR #216 merge) Authority: ESS-0021, ADR-0065, ADR-0059, M5 verified append-only audit persistence

Prerequisite verification

SA2 is complete and merged through PR #216.

Final SA2 evidence on head `14b9ec25dd60a34ae78a852f0a5b689b4811832b`:

- CI run #919 / `31566058836` : SUCCESS;
- Human-/Owner-Vorprüfung: SUCCESS;
- TypeScript: SUCCESS;
- Unit tests: SUCCESS;
- production build: SUCCESS;
- post-build production CSP test: SUCCESS;
- Docker/runtime validation: SUCCESS;
- Google-Marketing guard #164: SUCCESS;
- Governance #643: SUCCESS;
- merge commit: `a5abc1685026651f4297a487e855683a1fa1e58e` ;
- former SA2 branch is deleted.

PR #217 subsequently merged the M10 passkey-only target architecture at `083d8f25083034e3785d1a8e0c57eaf03463c907` . It does not activate M10 runtime enforcement; M10 remains sequentially blocked until M9.

Therefore SA3 remains the next active Systemadmin stage.

Existing M5 control reused

SA3 does not create a second audit database or a new production schema.

Existing M5 controls are reused:

- `public.agent_audit_events` is already production-verified append-only;
- server-only privileged writer: `server/agentAudit/agentAuditWriter.ts` ;

- authorization persistence failure propagates as an exception;
- terminal outcome is a second append-only event rather than UPDATE;
- central metadata sanitization omits prompts, full diffs and raw bodies and redacts credential-like values.

Mutation state for Supabase/Stripe/Render: `NOT REQUIRED` for this repository implementation.

SA3A — repository audit adapter

Implemented artifacts:

- `server/agentAudit/systemadminAuditedExecution.ts`;
- `.ai/contracts/systemadmin-audit-execution-profile.json`;
- `tests/unit/systemadminAuditedExecution.test.ts`;
- `docs/runbooks/SYSTEMADMIN_AUDITED_EXECUTION.md`.

The adapter keeps direct mutating SA2 LIVE evaluation disabled. It performs:

`SA2 DRY_RUN ALLOW → SA3 self-authority check → durable authorization INSERT → auditReference → audit-bound execution permit`

The existing SA2 envelope retains `liveMutationPermitted=false`. The SA3 server result adds a distinct `auditBoundExecutionPermitted=true` only after the durable audit insert returned a syntactically valid `supabase:agent_audit_events:<id>` reference.

This separation prevents an old or unaudited SA2 envelope from being interpreted as repository mutation authority.

SA3B — execution-host binding gap

The repository implementation cannot by itself intercept external ChatGPT GitHub connector calls. This limitation was already documented by the SA2 runbook and remains materially relevant.

Therefore repository CI/merge alone cannot prove end-to-end autonomous enforcement.

Before SA3 may become `COMPLETE / VERIFIED PASS`, one actual execution mode must be proven:

1. the execution tool host consumes and validates the SA3 permit before the mutating GitHub action and rejects bypass/direct mutation; or
2. a CAPITAL-AI execution gateway consumes the SA3 permit and itself performs the exact repository action.

Until that evidence exists, direct mutating ChatGPT→GitHub connector actions are not classified as SA3-enforced autonomous Systemadmin execution and SA4 remains blocked.

Required SA3B evidence:

- actual execution host/gateway identity;
- permit checked before side effect;
- exact capability/target/head binding;
- bypass/direct mutation DENY;
- authorization and terminal outcome references correlated end-to-end.

Correlation contract

Authorization/outcome evidence reconstructs:

```
mandateId → roadmapItem → humanActorId → app/agent/session/request → traceId → capability/  
risk → target/repository/paths → branch/head/PR → policy decision → authorizationAuditRefer-  
ence → commit/PR/workflow → result
```

No reusable credential is part of the execution permit.

Additional SA3 self-authority protection

The audited execution layer denies repository-mutating requests targeting its own SA2/SA3 control plane, including:

- `.ai/contracts/systemadmin-roadmap-execution-profile.json` ;
- `.ai/contracts/systemadmin-audit-execution-profile.json` ;
- `src/platform/Security/systemadminExecutionProfile.ts` ;
- `server/agentAudit/agentAuditWriter.ts` ;
- `server/agentAudit/authorizedAgentExecution.ts` ;
- `server/agentAudit/systemadminAuditedExecution.ts` .

Existing SA1 self-authority protections remain in force independently.

Required repository tests

Repository CI must prove at least:

1. valid scoped PR action receives no permit before audit persistence succeeds;
2. durable audit failure throws before an execution permit is returned;
3. SA2/SA3 self-authority path request is DENY and auditable;
4. MERGE remains DENY;
5. PRODUCTION_MUTATION remains DENY;
6. successful authorization returns a durable reference and audit-bound permit;
7. SUCCESS/ERROR outcome is a second correlated append-only event;
8. outcome without an audited ALLOW permit is rejected;
9. prompt/raw response metadata remains omitted/redacted through the existing writer.

Security boundary

SA3A covers only the already approved repository capability surface:

READ, ANALYZE, PLAN, BRANCH, COMMIT, PR, CI_REQUEST

Still excluded:

- MERGE ;
- DEPLOY_REQUEST ;
- PRODUCTION_MUTATION ;
- CRITICAL/reserved Owner mutation classes;
- self-expansion of the audit/control plane.

Human final review and Human merge authority are unchanged. The current transitional Owner gate remains authoritative until a future M10 implementation reaches VERIFIED PASS and performs the controlled passkey cutover defined by ADR-0066.

Exit gate

SA3 may become COMPLETE / VERIFIED PASS only when all of the following hold:

1. SA3A final PR head passes repository CI and positive/negative tests;
2. SA3A is Human-merged;
3. SA3 work branch is deleted;
4. SA3B actual execution-host binding is identified;
5. permit-before-mutation enforcement is proven;
6. direct/bypass mutation is proven DENY;
7. end-to-end authorization/outcome audit correlation is verified without secret evidence;
8. roadmap/traceability are synchronized.

Only then may SA4 create the first bounded non-production REM pilot.

SA3B Execution Host Binding Evidence

Quelle: docs/evidence/sa3b/SA3B_EXECUTION_HOST_BINDING_EVIDENCE.md

SA3B Execution Host Binding Evidence

Status: **TECHNICAL VERIFIED PASS / LIFECYCLE CLEANUP PENDING** Date:

2026-08-12 Current verified production baseline:

main@91963f59b74c8c3c3c0b33c6a23237a01ac0128e Authority: ADR-0059, ADR-0065, ADR-0067, ESS-0021

Merged implementation

SA3A

PR #218:

- final head 4178f76c1c33b50b957cd073d83ed9eeb0493642 ;
- merge 8de5a538ae9d2f0afc7b2e505ddda427ceb77780 ;
- CI #926 PASS;
- Governance #647 PASS;
- branch deleted.

SA3B host

PR #220:

- final head c4c7d00b33e521dfd12b14ddfdc097288a80f385 ;
- merge 156142102e7d2a97ad466aee0340f758fa4365e5 ;
- CI #936 PASS;
- Governance #653 PASS;
- branch deleted.

M5 writer/schema corrective

PR #222:

- final head 1bb6fa3a3bb30d1671d14ba012462070ccd7293d ;
- merge 91963f59b74c8c3c3c0b33c6a23237a01ac0128e ;
- final CI #951 PASS;
- Governance #656 PASS;
- corrective branch deleted.

Render deploy dep-d9u1v5942hec739bsc6g reached live on the same #222 merge SHA.

Secret-file runtime verification

The server contract loads `/etc/secrets/finance-secrets.env` before privileged database construction. The file contract includes both:

- `SUPABASE_SECRET_KEY` ;
- `SUPABASE_SERVICE_ROLE_KEY` .

`server/db.ts` resolves privileged Supabase credentials in this order:

`SUPABASE_SECRET_KEY` → `SUPABASE_SERVICE_ROLE_KEY`

No credential value is stored in this evidence.

The functionality of the Owner-managed Render secret file was verified through the real SA3B execution path, not by reading or displaying the secret value.

Earlier fail-closed probe

Issue #221 / workflow run `31570833507` proved:

`DURABLE AUDIT FAILURE` → `NO PERMIT` → `NO BRANCH`

The broker returned 503 when Supabase rejected the prior privileged credential. The requested branch `agent/sa3b-host-probe-20260812a` was never created.

This remains valid negative evidence.

Positive live probe after PR #222 + secret repair

Owner Issue #223 triggered the trusted SA3B host on exact:

`main@91963f59b74c8c3c3c0b33c6a23237a01ac0128e`

Request:

- mode `BRANCH_PROBE` ;
- mandate `REM-SA3B-PROBE-001` ;
- roadmap item `SA3B-HOST-PROBE` ;
- branch `agent/sa3b-host-probe-20260812b` .

Workflow run `31574111075` :

Step	Result
Owner/title ingress	PASS
strict Issue JSON validation	PASS
exact current-main + REM binding	PASS
GitHub Actions OIDC	PASS
broker authorization	PASS

Step	Result
durable M5 authorization insert	PASS
audit-bound BRANCH permit	PASS
branch creation	PASS
durable M5 terminal outcome	PASS
Issue evidence correlation	PASS

Authorization reference:

supabase:agent_audit_events:194f1198-492c-4d4f-b1e4-0c13a5d99d20

Outcome reference:

supabase:agent_audit_events:5ab9eefe-f472-4396-a7e7-2a3ceb029e34

The created branch was independently verified to point exactly to:

91963f59b74c8c3c3c0b33c6a23237a01ac0128e

Direct Supabase correlation

Read-only verification of `public.agent_audit_events` confirmed both referenced rows.

Shared correlation:

- request ID `issue-223-run-31574111075` ;
- trace ID `github-actions:31574111075` ;
- agent `capital-ai-systemadmin-roadmap-executor` ;
- app `chatgpt-github-connector` ;
- capability `BRANCH` ;
- risk `MEDIUM` ;
- exact Finance repository and probe branch;
- workflow run `31574111075` .

Authorization row:

- decision `ALLOW` ;
- result `PENDING` ;
- exact current-main SHA;
- workload identity / workflow metadata retained in sanitized attributes.

Outcome row:

- decision `ALLOW` ;
- result `SUCCESS` ;
- `authorizationAuditReference` points to the authorization row;

- `requiresHumanMerge=true` remains preserved.

This is the positive proof that the Render secret-file credential, corrected M5 writer and append-only audit path function together in production.

Separate stale-base negative probe

Owner Issue #224 requested stale base:

`156142102e7d2a97ad466aee0340f758fa4365e5`

Requested branch:

`agent/sa3b-host-probe-negative-20260812c`

Workflow run `31574221718` :

- strict request validation: PASS;
- current-main/REM binding: **DENY as expected**;
- OIDC: SKIPPED;
- broker authorization: SKIPPED;
- audit side effect: SKIPPED;
- branch creation: SKIPPED;
- outcome: SKIPPED.

Independent branch lookup returned 404 for the requested negative branch.

Therefore:

`STALE BASE → ZERO OIDC/BROKER/REPOSITORY SIDE EFFECT`

Proven SA3B security invariants

1. repository side effect occurs only after durable authorization evidence;
2. invalid/stale base fails before side effect;
3. audit persistence outage fails closed;
4. OIDC identity is bound to Owner, repository, main and exact workflow;
5. outcome is bound to the same issue/run/request and prior authorization;
6. no direct ChatGPT→GitHub connector mutation counts as autonomous Systemadmin execution;
7. `MERGE` , deployment and production mutation are not part of SA3B authority.

Remaining lifecycle cleanup

The successful positive probe branch:

`agent/sa3b-host-probe-20260812b`

still exists after evidence capture and must be deleted before SA3B may be recorded as full lifecycle **COMPLETE / VERIFIED PASS** and before the SA4 pilot is allowed to execute.

The connected GitHub tool surface used during this session exposes branch creation/update but no reference-delete mutation, so this cleanup cannot be truthfully recorded as completed by the agent.

SA4 therefore includes an independent activation check that refuses execution while this exact branch exists.

Conclusion

SA3B technical enforcement is VERIFIED PASS. The Render Supabase privileged secret path is functional, positive Authorization → Branch → Outcome correlation is proven in the production audit store, and independent fail-closed behavior is proven.

Lifecycle completion remains pending only on deletion of `agent/sa3b-host-pro-be-20260812b` .

SA4 Bootstrap Evidence

Quelle: docs/evidence/sa4/SA4_BOOTSTRAP_EVIDENCE.md

SA4 Bootstrap Evidence

Status: OWNER-DIRECTED BOOTSTRAP / LIVE PILOT NOT YET EXECUTED Date: 2026-08-12
Base: `main@91963f59b74c8c3c3c0b33c6a23237a01ac0128e` Authority: ADR-0068, ESS-0021,
ADR-0065, ADR-0067

Why this bootstrap exists

SA3B has now proven a real positive and negative execution-host path. SA4 is the first increase in autonomous mutation authority from branch-only proof to a bounded repository work package containing branch, commit and draft PR creation.

The bootstrap itself is Owner-directed normal repository maintenance because it modifies the Systemadmin trust root. It is not evidence that the Systemadmin agent may modify its own authority.

Preflight evidence

PR #222 remediation

- merged: yes;
- final head `1bb6fa3a3bb30d1671d14ba012462070ccd7293d` ;
- merge `91963f59b74c8c3c3c0b33c6a23237a01ac0128e` ;
- CI #951 PASS;
- corrective branch deleted;
- corrected runtime deployed on Render as `dep-d9u1v5942hec739bsc6g` and `live` .

Supabase privileged path

Positive SA3B live probe #223 proves the Owner-managed Render `finance-secrets.env` is functional for privileged M5 persistence. No secret value was read into this evidence.

Real authorization row:

```
supabase:agent_audit_events:194f1198-492c-4d4f-b1e4-0c13a5d99d20
```

Real terminal outcome row:

```
supabase:agent_audit_events:5ab9eefe-f472-4396-a7e7-2a3ceb029e34
```

Both rows were independently verified read-only in `public.agent_audit_events` .

Negative execution evidence

- Issue #221 proved audit persistence failure → no permit → no branch.

- Issue #224 / run 31574221718 proved stale base → deny before OIDC/broker → no branch.

Open PR overlap

At SA4 bootstrap start:

- PR #225 changes DevelopmentChain documentation;
- PR #193 changes cost/monetization documentation;
- exact SA4 pilot path docs/evidence/sa4/SA4_FIRST_AUTONOMOUS_WORK_PACKAGE.md is in neither open PR.

Remaining SA3B lifecycle cleanup

The successful positive probe branch:

agent/sa3b-host-probe-20260812b

still exists and must be deleted after evidence capture.

The connected GitHub connector exposes no ref-delete mutation, so the bootstrap does not falsely claim completion. The new SA4 trusted runner independently checks for this branch and terminates before OIDC if it still exists.

SA4 bootstrap artifacts

Architecture and mandate

- docs/adr/ADR-0068-first-bounded-autonomous-work-package.md
- .ai/mandates/REM-SA4-PILOT-001.json

Execution host

- .github/workflows/systemadmin-sa4-pilot.yml
- scripts/systemadmin/validateSa4PilotIssue.mjs
- scripts/systemadmin/runSa4Pilot.mjs

Control-plane hardening

- src/platform/Security/roadmapExecutionMandate.ts
- server/agentAudit/systemadminAuditedExecution.ts
- server/systemadmin/githubActionsOidc.ts
- server/systemadmin/systemadminExecutionBrokerRouter.ts

Tests

- tests/unit/systemadminSa4PilotIssue.test.ts
- tests/unit/systemadminSa4Contracts.test.ts
- updated tests/unit/githubActionsOidc.test.ts

- updated `tests/unit/systemadminExecutionBroker.test.ts`

Governance synchronization

- `docs/evidence/sa3b/SA3B_EXECUTION_HOST_BINDING_EVIDENCE.md`
- `docs/roadmaps/SYSTEMADMIN_AGENT_ROADMAP.md`
- `docs/traceability/SYSTEMADMIN_AGENT_TRACEABILITY_MATRIX.md`

Authority increase

Before SA4:

`BRANCH` was the only real host side effect proven by SA3B.

SA4 pilot target:

`BRANCH` → `COMMIT` → Draft PR

The increase is constrained by:

- exact Owner issue identity;
- strict six-field JSON ingress;
- exact current-main SHA;
- exact stage-specific OIDC workflow ref;
- workflow↔mandate binding in broker;
- SA1 REM policy;
- SA2 sequence preconditions;
- SA3 durable authorization-before-side-effect;
- one exact allowed evidence path;
- MEDIUM risk ceiling;
- open-PR overlap inventory;
- one open SA4 PR maximum;
- deterministic file content generated only by trusted host code;
- post-commit digest validation;
- separate authorization/outcome pair for `BRANCH`, `COMMIT` and `PR`;
- Human-only final CI authorization and merge.

Explicit non-authority

The SA4 REM does not grant:

- `CI_REQUEST`;
- merge;
- deploy;
- production mutation;

- IAM role changes;
- MFA/break-glass changes;
- secret disclosure/rotation;
- billing/entitlement changes;
- DNS/TLS;
- repository protection weakening;
- control-plane self-modification.

Bootstrap validation required

The bootstrap PR is Check Class **R + Workflow Security** because it modifies Runtime/Security/Audit control-plane code and a write-capable GitHub workflow.

Final reviewed head must pass:

- repository integrity;
- workflow security validator;
- Node 24.18.0;
- production dependency audit;
- TypeScript;
- complete unit/contract suite including SA4 negative tests;
- production build;
- post-build CSP;
- deployment readiness;
- Docker hardening/image where selected by the existing scope resolver.

No redundant expensive rerun should be triggered before final Owner review.

Rollback

Before live pilot activation, bootstrap rollback is repository-only through a protected revert PR.

After a successful Human merge of this bootstrap, Render deployment is required because OIDC/broker runtime changed. No Supabase schema, Stripe, IONOS or other production configuration mutation is part of this bootstrap PR.

Live-pilot activation

The first real [SA4-PILOT] Issue must not be opened until:

1. bootstrap PR is Human-merged;
2. bootstrap branch is deleted;
3. `agent/sa3b-host-probe-20260812b` is deleted;
4. bootstrap merge is deployed and healthy;

5. current `main` SHA is re-resolved;
6. exact target path remains free of concurrent-writer overlap.

Only the resulting real audit rows, commit and draft PR can qualify SA4 as `VERIFIED PASS` .

SA4 First Autonomous Work Package Evidence

Quelle: docs/evidence/sa4/SA4_FIRST_AUTONOMOUS_WORK_PACKAGE.md

SA4 First Autonomous Work Package Evidence

Status: GENERATED BY AUDIT-BOUND SA4 HOST / HUMAN MERGE REQUIRED

This file is the deliberately bounded output of the first CAPITAL-AI autonomous System-admin work package. It was generated by trusted GitHub Actions host code; the triggering Issue cannot supply this Markdown body.

Bound execution

- Mandate: REM-SA4-PILOT-001
- Roadmap item: SA4-FIRST-AUTONOMOUS-WORK-PACKAGE
- Issue: #228
- Workflow run: 31579519025
- Base SHA: f7dfcda36905d9a55d74f57f2140224928960379
- Branch: agent/sa4-pilot-proof-20260812b
- Path: docs/evidence/sa4/SA4_FIRST_AUTONOMOUS_WORK_PACKAGE.md

Pre-side-effect audit evidence

- BRANCH authorization: supabase:agent_audit_events:1b4b04cb-a86b-4612-9ef5-308e95a18c95
- BRANCH outcome: supabase:agent_audit_events:e99b9af7-74cc-4693-966f-c9b85102035d
- COMMIT authorization: supabase:agent_audit_events:3c5916a1-d8c1-4ba1-9de1-d839fcc1bc85

Security boundary

The pilot is documentation-only. It does not request CI, deploy, mutate production, change IAM, expose secrets, alter repository protection or merge a pull request. Final review and merge remain Human/Owner-only.

Evidence — SC-1 Orchestrator Wiring

Quelle: docs/evidence/sc-md/SC1_ORCHESTRATOR_WIRING_2026-08-15.md

Evidence — SC-1 Orchestrator Wiring

SPT: SC-MD-SPT-0001
WP: SC-1 Classification Consolidation
Date: 2026-08-15
Branch: docs/sc-md-scl-orchestrator-wiring-2026-08-15

Change summary

Area	Before	After
Classification construction in <code>cryptoOrchestra-tor.analyzeCrypto</code>	Manual string matching on agent <code>category</code> + average confidence with deterministic	<code>adaptAgentClassification</code> → <code>ensureCanonicalClassification(ClassificationService)</code> → <code>mergeDeterministicAndAgent-Classification</code>
Free-text on ranking path	Possible (agent raw strings partially mapped)	Impossible — only canonical <code>CryptoCategory</code> / <code>CryptoSubCategory</code> exit
Adapter version	<code>classification-adapter/1.0.0</code>	<code>classification-adapter/1.1.0</code> (adds merge)
Ranking / score formulas	unchanged	unchanged
SC-3 composite	not observed	computed as non-mutating observation; <code>scoreImpactEnabled</code> remains false

Merge preference (documented in adapter)

1. Deterministic `category_main` wins when $\neq$ Unknown
2. Else agent canonical when $\neq$ Unknown
3. Sub / `asset_type` / tier: deterministic if known, else agent, else infer
4. Confidence = mean of both (clamped)
5. Reasoning: det first, agent second, provenance tag

Files touched

- `src/services/classificationAdapter.ts`
- `src/orchestrator/cryptoOrchestra.ts`
- `tests/unit/classificationAdapter.test.ts`
- `docs/roadmaps/work-packages/SC-1_CLASSIFICATION_CONSOLIDATION.md`
- `docs/evidence/sc-md/SC1_ORCHESTRATOR_WIRING_2026-08-15.md`

- `.ai/work-claims/SC1-ORCHESTRATOR-WIRING-2026-08-15.json`

Verification intent

- Unit tests cover merge preference, agent fallback, dual-Unknown fail-closed.
- No change to `calculateRankScore` / `isTop10Eligible` formulas.
- Owner merge gate: PR review + CI green.

SC-1 / SC-3 Baseline Inventory — 2026-08-15

Quelle: docs/evidence/sc-md/SC1_SC3_BASELINE_INVENTORY_2026-08-15.md

SC-1 / SC-3 Baseline Inventory — 2026-08-15

Authority: SC-MD-SPT-0001

Baseline: branch work against current `main`

Mutation: none to ranking formula / scoreImpact flags

Classification worlds (SC-1)

Path	Role	Action
<code>src/types/crypto.types.ts</code>	Canonical	KEEP
<code>src/types/crypto.ts</code>	Legacy enterprise payload	ADAPT via classificationAdapter
<code>src/agents/cryptoClassificationAgent.ts</code>	Free-text agent output	ADAPT via classificationAdapter
<code>src/services/classification.service.ts</code>	Deterministic symbol table	EXTEND later
<code>src/orchestrator/cryptoOrchestrator.ts</code>	Merge point	WIRE later

DQ / Confidence (SC-3)

Path	Role	Action
<code>src/platform/MarketData/DataQualityService.ts</code>	Snapshot accept/reject	KEEP
<code>src/platform/MarketData/CompositeDataQuality.ts</code>	NEW composite	FOUNDATION
<code>src/services/ranking.service.ts</code>	Coarse DQ points	UNCHANGED this slice
<code>src/services/screeningEligibility.ts</code>	Fail-closed evidence rules	KEEP
<code>src/services/scoreConfidenceCalibration.ts</code>	Empirical calibration	KEEP (<code>scoreImpactEnabled: false</code>)

Tests added

- `tests/unit/classificationAdapter.test.ts`
- `tests/unit/compositeDataQuality.test.ts`

Next Owner decision

1. Approve wiring of adapter in orchestrator (SC-1 remaining).

2. Approve optional ranking consumer of composite level (SC-3 → SC-7).
3. Keep scoreImpact disabled until backtest evidence (SC-8) supports calibration.

Evidence — SC-1 ClassificationService Table Expansion

Quelle: docs/evidence/sc-md/SC1_TABLE_EXPANSION_2026-08-15.md

Evidence — SC-1 ClassificationService Table Expansion

SPT: SC-MD-SPT-0001

Date: 2026-08-15 / 2026-08-16

Branch: docs/sc-md-scl-table-expansion-2026-08-15

Change

`ClassificationService.classifyAsset` converted from a short if/else chain (~20 symbols) to a **table-driven** map covering core symbols across:

- Layer 1 / Smart Contract Platform
- Layer 2
- DeFi / Derivatives / Liquid Staking / Restaking
- Oracle
- Stablecoin
- Exchange Token
- AI / Data
- Storage / Compute
- Gaming
- Real World Assets
- Privacy
- Payments
- Bridging / Interoperability
- Meme (legacy set retained)

Unlisted symbols remain **Unknown / tier 3 / confidence 0.6** (fail-closed).

Non-goals

- No change to `calculateRankScore` / `isTop10Eligible`
- No `scoreImpactEnabled` flip (SC-3 / SC-7 still gated)
- No synthetic price or score invention

Verification

- Unit: `tests/unit/classificationServiceTable.test.ts`
- Adapter/orchestrator path unchanged (still `ensureCanonical` → merge)

Authority

SC-MD-SPT-0001 SC-1 · DOCUMENTATION_HYGIENE_POLICY

Evidence — SC-4 Phase A Gateway Hardening & Provider Matrix

Quelle: docs/evidence/sc-md/SC4_GATEWAY_HARDENING_PROVIDER_MATRIX_2026-08-16.md

Evidence — SC-4 Phase A Gateway Hardening & Provider Matrix

Date: 2026-08-16
SPT: SC-MD-SPT-0001
Claim: SC4-GATEWAY-HARDENING-PROVIDER-MATRIX-2026-08-16

What changed

Area	Before	After
Rate limits	Global capacity for all provider IDs	<code>RateLimitBudget.perProvider</code> + matrix overrides
Provider inventory	Implicit in adapters	<code>ProviderMatrix</code> with gateway status
Supervisor health	Caller-side only (e.g. traditionalQuote)	Gateway also records snapshot outcomes
Circuit open metadata	Internal only	<code>openedUntilIso</code> → <code>circuit0-penUntil</code>

Gateway adoption (Phase A)

Provider ID	Status
twelvedata	behind_gateway
fmp-index	behind_gateway
alpaca	shadow_only (not promoted)
fmp-index-history	history_gateway_only
coingecko	legacy_off_gateway (SC-5)
stooq	legacy_off_gateway (SC-5)

Invariants preserved

- No Demo Data / no synthetic prices on failure
- Alpaca remains shadow unless `includeShadow`
- `scoreImpact` / `rankingImpact` unchanged (`false`)
- No crypto scoring path migration in this WP

Tests

- `tests/unit/providerMatrix.test.ts` — matrix, RL overrides, circuit ISO, health on success/rate-limit
- Existing `tests/unit/marketDataGateway.test.ts` remains authoritative for CB/cache/coalesce semantics

Follow-up

SC-5 live coverage expansion for crypto + remaining traditional paths; Alpaca promotion remains ADR-0041 Owner gate.

Evidence — SC-5 Live Coverage Matrix

Quelle: docs/evidence/sc-md/SC5_LIVE_COVERAGE_MATRIX_2026-08-16.md

Evidence — SC-5 Live Coverage Matrix

Date: 2026-08-16

SPT: SC-MD-SPT-0001

Claims: SC5-LIVE-COVERAGE-CRYPTO-GATEWAY-2026-08-16 (Phase A), SC5-PHASE-B-MULTIFIELD-MATRIX-GUARDS-2026-08-16 (Phase B)

Live coverage matrix (verified quotes / fields)

Asset class	Path	Gateway	Status	Notes
stock	traditionalQuoteEvidence → TwelveData	yes	VERIFIED	SC-4 matrix budgets
forex	traditionalQuoteEvidence → TwelveData	yes	VERIFIED	SC-4
index	traditionalQuoteEvidence → FMP Index	yes	VERIFIED	SC-4; approved ticker map
crypto (USD price)	cryptoQuoteEvidence → CoinGecko	yes	VERIFIED (Phase A)	Mapped core symbols only
crypto (market fields)	cryptoSnapshotProvider	matrix RL/CB	MATRIX-ALIGNED (Phase B)	marketCap/volume/supply; Canonical snapshot deferred
crypto (spot quorum)	cryptoSpotConsensus	partial	consensus_only	CoinAPI/TwelveData/EODHD keys required
commodity / bond / macro	evidence services	mixed	evidence-gated	SC-5 later

Fail-closed guarantees

- No synthetic / registry-bootstrap prices or market fields on failure
- Unmapped crypto symbol → UNSUPPORTED_ASSET (quote) / null (snapshot)
- Upstream error → SOURCE_UNAVAILABLE / degraded last-known-good / null
- Rate-limit or open circuit on multi-field path → last-known-good (degraded) or null
- executionPriceEligible remains **false** for crypto gateway quotes until multi-provider quorum is Owner-linked

Code

File	Role
<code>src/platform/MarketData/providers/CoinGecko-MarketDataProvider.ts</code>	Price adapter (Phase A)
<code>src/services/cryptoQuoteEvidence.ts</code>	Verified quote API
<code>src/services/cryptoSnapshotProvider.ts</code>	Multi-field + matrix guards (Phase B)
<code>src/platform/MarketData/ProviderMatrix.ts</code>	<code>coingecko</code> <code>behind_gateway</code> ; v1.2.0
<code>tests/unit/cryptoQuoteEvidence.test.ts</code>	Phase A unit coverage
<code>tests/unit/cryptoSnapshotProvider.test.ts</code>	Phase B RL/CB coverage

Follow-up

Phase C+: CanonicalMarketDataSnapshot market fields; optional CoinAPI gateway adapter; Stooq; execution-price eligibility gate (Owner).

Evidence — SC-5 Phase B Multi-Field Matrix Guards

Quelle: docs/evidence/sc-md/SC5_PHASE_B_MULTIFIELD_MATRIX_GUARDS_2026-08-16.md

Evidence — SC-5 Phase B Multi-Field Matrix Guards

Date: 2026-08-16
SPT: SC-MD-SPT-0001
Claim: SC5-PHASE-B-MULTIFIELD-MATRIX-GUARDS-2026-08-16

What changed

Item	Before	After
Multi-field CoinGecko path	Private module circuit (3 failures / 60s) + no matrix RL	RateLimitBudget + Circuit-Breaker from ProviderMatrix coingecko
Capability telemetry	None	Supervisor market-fields health
Fail-closed	Last-known-good on circuit open	Same + rate-limit exhaustion
API	VerifiedCryptoSnapshot	Unchanged
executionPriceEligible	false	still false

Live coverage matrix (update)

Asset class	Path	Gateway / Matrix	Status Phase B
stock	traditionalQuoteEvidence → TwelveData	yes	VERIFIED
forex	traditionalQuoteEvidence → TwelveData	yes	VERIFIED
index	traditionalQuoteEvidence → FMP Index	yes	VERIFIED
crypto (USD price)	cryptoQuoteEvidence → CoinGecko	yes	VERIFIED (Phase A)
crypto (market fields)	cryptoSnapshotProvider	matrix RL/CB	MATRIX-ALIGNED (Phase B)
crypto (spot quorum)	cryptoSpotConsensus	consensus_only	unchanged

Code

File	Role
src/services/cryptoSnapshotProvider.ts	Matrix-aligned guards
src/platform/MarketData/ProviderMatrix.ts	v1.2.0 notes

File	Role
tests/unit/cryptoSnapshotProvider.test.ts	RL + CB negative paths

Non-goals (confirmed)

- No CanonicalMarketDataSnapshot extension for marketCap/supply
- No CoinAPI gateway adapter registration
- No executionPriceEligible flip
- No scoreImpact / rankingImpact mutation

Evidence — SC-7 Phase B Orchestrator Composite Wiring

Quelle: docs/evidence/sc-md/SC7_PHASE_B_ORCHESTRATOR_WIRING_2026-08-16.md

Evidence — SC-7 Phase B Orchestrator Composite Wiring

SPT: SC-MD-SPT-0001
WP: SC-7 Ranking Composite Opt-In
Date: 2026-08-16
Branch: feat/sc-md-sc7-phase-b-orchestrator-composite-wiring-2026-08-16

Change

Path	Change
src/orchestrator/cryptoOrchestrator.ts	calculateRankScore(payload, finalScore, { compositeLevel: composite.level })
Reasoning trail	Appends [SC-7 Phase B] rankScore uses compositeLevel=...

Invariants preserved

- RANKING_SCORE_IMPACT_ENABLED = false
- Formula weights 0.70 / 0.15 / 0.10 / 0.05 unchanged
- isTop10Eligible thresholds unchanged
- SC-1 classification adapter path unchanged
- SC-3 computeCompositeDataQuality / computeUnifiedConfidence remain observation-first

Behavioral note (fail-closed)

When composite.level === 'unknown', Phase B rank DQ points resolve to **50** via compositeLevelToRankingDqPoints('unknown').

Previously the orchestrator only fed ranking through payload.data_quality.level, which rewrote unknown to medium/high for display. That rewrite still applies to the payload field returned to API consumers; only the rank-score calculation uses the pure composite level.

Non-goals

- No flip of scoreImpact / rankingImpact
- No wiring of valuation.service or cryptoRoutes (no SC-3 composite computed there)
- No cross-asset rank modes

Verification intent

- Existing `tests/unit/rankingService.test.ts` Phase A parity cases remain green
- CI class C: unit + build-and-test
- Owner merge required; agent does not self-merge

Evidence — SC-7 Phase A Ranking Composite Opt-In

Quelle: docs/evidence/sc-md/SC7_RANKING_COMPOSITE_OPT_IN_2026-08-16.md

Evidence — SC-7 Phase A Ranking Composite Opt-In

SPT: SC-MD-SPT-0001
Date: 2026-08-16
Branch: docs/sc-md-sc7-ranking-composite-opt-in-2026-08-16

Change

Before	After
Inline ternary in <code>calculateRankScore</code> for DQ points	<code>resolveRankingDqPoints</code> → <code>compositeLevelToRankingDqPoints</code>
No optional composite input	<code>RankingDqOptions.compositeLevel</code> optional
Implicit impact posture	<code>RANKING_SCORE_IMPACT_ENABLED = false</code> explicit

Formula (unchanged)

```
rank = 0.70 * finalScore + 0.15 * dqPoints + 0.10 * tierScore + 0.05 * liquidity
dqPoints: high=100, medium=70, low=40, unknown=50
```

Verification

- Unit: `tests/unit/rankingService.test.ts` (parity + regression 92.0 for high/tier1/liq80/final90)
- Existing `isTop10Eligible` thresholds unchanged

Non-goals

- No weight mutation
- No `scoreImpactEnabled: true`

Remediation note (PR #349)

Initial PR body was free-form Summary (not canonical template v1.3.5). Body was replaced with full `CAPITAL_AI_PR_TEMPLATE_VERSION: 1.3.5` contract; claim `schemaVersion` corrected to `1.0.0`. Root cause: agent PR creation path skipped `renderPullRequestBody.mjs` / template fill.

Authority

SC-MD-SPT-0001 SC-7 · SC-3 composite contract · DOCUMENTATION_HYGIENE_POLICY

GitGuardian-/Snyk-Integrationsbaseline — 2026-08-14

Quelle: docs/evidence/security/GITGUARDIAN_SNYK_BASELINE_2026-08-14.md

GitGuardian-/Snyk-Integrationsbaseline — 2026-08-14

Status: VERIFIED PASS / SNYK PROMOTED TO REQUIRED CHECKS, GITGUARDIAN-PROMOTION UNBESTÄTIGT Repository: SvenKulesa/Finance Aktuelle Main-Baseline: main@e2830fba142e076c49b8837e0823b8526f5f4e26 (PR #253 Merge)

Korrektur 2026-08-14 (Sicherheitsaudit-Nachtrag)

Die vorherige Fassung dieses Dokuments stufte Snyk als „PARTIAL / SECOND SNYK HEAD VERIFICATION ACTIVE" ein und behauptete für PR #249, es sei „kein Check erschienen". Das war falsch. Eine direkte Abfrage der GitHub-Commit-Status-API (nicht der Checks-API — Snyk meldet über die ältere Status-API, die in der PR-UI unter „Show all checks" statt im Actions-Tab erscheint und beim vorherigen Review offenbar übersehen wurde) zeigt für alle drei zuletzt gemergten PRs vollständige, head-gebundene Snyk- und GitGuardian-Evidence:

PR	Head-SHA	security/snyk	code/snyk	GitGuardian
#249	680c1d2894260447e74fc3374c94cdf969dc75a7	„No manifest changes detected in 1 project"	„No new Code Analysis issues found"	
#250	7a6c72bc6aa19-bae2721b330c53-ba962cb53e730	„1 security test has passed"	„No new Code Analysis issues found"	
#251	b81ee47-ce08a04b5a87fa74667e8adca627cdaad	„1 security test has passed"	„No new Code Analysis issues found"	

Wichtiger Nebenbefund: code/snyk ist **Snyk Code**, also ein SAST-Check (semantische Quellcode-Schwachstellenanalyse) — nicht nur Dependency-Scanning. Das war beim Schreiben der vorherigen Fassung nicht als eigene Kategorie erkannt.

Owner-bestätigte Konfiguration

- GitGuardian ist als GitHub App verbunden und liefert GitGuardian Security Checks .
- Snyk ist mit GitHub verbunden, SvenKulesa/Finance importiert, PR-Check-Konfiguration aktiv.
- Auf main existiert keine aktive GitGuardian- oder Snyk-Workflowdatei — beide laufen vollständig anbieterseitig, exakt wie in ADR-0070 vorgesehen.
- Ein Snyk-Token liegt weiterhin als GitHub Repository Secret; der Tokenwert wurde zu keinem Zeitpunkt gelesen, angezeigt oder in Evidence übernommen.

Owner-Entscheidung 2026-08-14: Promotion zu Required Checks

Der in ADR-0070 definierte Promotion-Gate („kein Check wird vor zwei Head-gebundenen erfolgreichen PR-Läufen und separater Owner-Freigabe als Required Check promoviert“) ist mit drei statt zwei unabhängigen, head-gebundenen PASS-Läufen erfüllt. Der Owner hat die Promotion freigegeben:

- `security/snyk (svenkulessa)` und `code/snyk (svenkulessa)` wurden anstelle von CodeQL als Required Checks im `main-production-protection`-Ruleset aktiviert (Owner-bestätigt). Ob GitGuardian Security Checks ebenfalls als Required Check aktiv ist, ist zum Zeitpunkt dieser Dokument-Fassung noch nicht Owner-bestätigt und daher als offener Nachweis unten geführt statt als erledigt markiert. `.github/policies/main-production-protection.expected.json ( decision_2026-08-14c )` beschreibt den vorgeschlagenen Zielzustand mit allen drei Kontexten.
- CodeQL wurde geprüft und **nicht** als Required Check übernommen: `code/snyk` deckt dieselbe SAST-Kategorie bereits ab, ohne zusätzliche GitHub-Actions-Laufzeit zu benötigen — genau das Designziel von ADR-0070. Eine parallele SAST-Engine wäre Doppelarbeit, kein zusätzlicher Schutz im Sinne einer neuen Bedrohungskategorie.

Owner-Entscheidung 2026-08-14b: Snyk-Token bleibt gesetzt

Der Owner hat bestätigt, dass der bestehende Snyk-Token als GitHub Repository Secret absichtlich gesetzt ist und weiterverwendet werden soll — keine Altlast. Grep über `.github/workflows/**` bestätigt weiterhin, dass kein Repository-Workflow das Secret direkt liest; die Verwendung erfolgt ausschließlich anbieterseitig innerhalb der Snyk-GitHub-Integration, konsistent mit der App-only-Architektur aus ADR-0070. Damit ist dieser Punkt geschlossen.

Offene Nachweise (nur Owner-seitig einsehbar, kein Tool-Zugriff verfügbar)

- [] GitGuardian-App-Zugriff ist auf `SvenKulessa/Finance` begrenzt (App-Berechtigungen unter GitHub Settings → Applications, nicht per Tool abfragbar).
- [] GitGuardian arbeitet ohne Repository-Schreibrechte (dieselbe Quelle).
- [] Snyk automatische Fix-PRs/Write-Zugriffe sind im Snyk-Dashboard deaktiviert.
- [] GitGuardian Security Checks als Required Check im Ruleset bestätigen oder bewusst weglassen (siehe Owner-Entscheidung oben — noch offen, ob es zusätzlich zu `security/snyk` und `code/snyk` aktiviert wurde).

Bewertung

Kontrolle	Status	Begründung
GitGuardian Laufzeit	VERIFIED PASS	drei unabhängige PR-Heads, alle grün
Snyk Dependency-Scan (<code>security/snyk</code>)	VERIFIED PASS	drei unabhängige PR-Heads, alle grün

Kontrolle	Status	Begründung
Snyk Code / SAST (code/snyk)	VERIFIED PASS	drei unabhängige PR-Heads, alle grün
App-only Architektur	VERIFIED	kein Workflow in .github/workflows/** konsumiert Snyk/GitGuardian
Required-Check-Promotion Snyk	OWNER-APPROVED, angewendet 2026-08-14	security/snyk und code/snyk ersetzen CodeQL im Ruleset
Required-Check-Promotion GitGuardian	UNBESTÄTIGT	ADR-0070-Gate erfüllt, Owner-Bestätigung der Ruleset-Aktivierung steht noch aus
Snyk-Token (GitHub Secret)	VERWENDUNG BESTÄTIGT	Owner-Entscheidung 2026-08-14b, kein Repository-Workflow liest es
CodeQL	BEWUSST NICHT verwendet	Redundanz zu code/snyk , keine neue Bedrohungskategorie abgedeckt

Sicherheitsaudit 2026-08-14 — ADR-0069 Trust Root, DevelopmentChain und Systemadmin-Prototyp

Quelle: docs/evidence/security/SECURITY_AUDIT_2026-08-14_ADR0069_DEVELOPMENT_CHAIN.md

Sicherheitsaudit 2026-08-14 — ADR-0069 Trust Root, DevelopmentChain und Systemadmin-Prototyp

Status: AUDIT-BEFUND / KEINE MUTATION DURCHGEFÜHRT Datum: 2026-08-14 Repository: SvenKulesa/Finance Auditierte Baseline: `main@5ba4ab12f7c6d98912b95bf000ceb352d9297fcd` (PR #252 Merge) Primäre Authority: `docs/adr/ADR-0069-human-owner-comment-gate-and-dispatched-pr-ci.md` Weitere Authority: ADR-0065, ADR-0067, ADR-0068, ADR-0070, ESS-0021, `.github/policies/main-production-protection.expected.json`

Dieses Dokument ist ein reiner Befundbericht. Es erzeugt keine Autorität, aktiviert kein Mandat und autorisiert keine Mutation. Alle Korrekturvorschläge bleiben Human/Owner-Entscheidungen.

1. Umgebungsabgleich

Die Arbeitsumgebung wurde gegen das aktuelle Remote-Repository abgeglichen.

Prüfung	Ergebnis
<code>git fetch origin main</code>	<code>main</code> = <code>5ba4ab1</code> , Arbeitsstand identisch (0 ahead / 0 behind)
<code>git fsck --strict --no-dangling</code>	sauber, keine Objektfehler, keine dangling Objects
Offene Pull Requests	keine
Branches auf dem Remote	nur <code>main</code> und der Audit-Branch

ADR-0069 lag in der Fassung **ACCEPTED — RECOVERY REVISION 2026-08-13** vor und wurde vollständig als Prüfmaßstab verwendet.

2. Verifizierte Positivbefunde

Die folgenden Kontrollen wurden real ausgeführt beziehungsweise im Code nachvollzogen und sind intakt.

2.1 Build-, Test- und Guard-Kette

Prüfung	Ergebnis
<code>npm run lint (tsc --noEmit)</code>	Exit 0

Prüfung	Ergebnis
<code>npx vitest run</code>	780 Tests passed , 2 skipped, 140 Testdateien passed, 1 skipped
<code>node --test scripts/pr/*.test.mjs</code>	16/16 passed
<code>npm audit --omit=dev --audit-level=high</code>	0 Vulnerabilities
<code>npm audit</code> (inkl. dev)	0 / 0 / 0 / 0 / 0
<code>verifyGoogleMarketingInvariants.ts</code>	34 Invarianten verifiziert
<code>verifyProductionConfigInvariants.ts</code>	11 Invarianten verifiziert
<code>verifyDockerHardening.mjs</code>	PASS

2.2 ADR-0069 verbotene Bootstrap-Muster — nicht vorhanden

Verbotenes Muster (ADR-0069)	Befund
1. <code>workflow_dispatch --ref main</code> als alleiniger Required-Check-Executor	nicht vorhanden
2. synthetischer <code>build-and-test</code> als alleiniger Rule-set-Check	nicht vorhanden
3. Ersetzen des funktionierenden PR-CI-Pfads in einem Bootstrap-Merge	nicht vorhanden; <code>ci.yml</code> unverändert aktiv
4. Aktivierung neuer Required-Check-Identity vor Shadow-PASS	nicht erfolgt
5. PR-Body-/Auto-Status-Automation als Human-Autorisierungsquelle	nicht vorhanden

Ergänzend geprüft und sauber:

- kein `pull_request_target` in irgendeinem Workflow;
- kein `checks: write`, kein `statuses: write`, keine `/check-runs` - oder `/statuses/` -Aufrufe, also kein synthetischer Check-Reporter;
- **alle** `uses:` -Referenzen auf unveränderliche 40-stellige Commit-SHAs gepinnt;
- **alle** Checkouts mit `persist-credentials: false`;
- `document-hygiene-*` und `lockfile-remediation` sind korrekt stillgelegt (`contents: read`, `if: false`).

2.3 Systemadmin-Ausführungshost (SA3B / SA4)

Die Vertrauenskette des Prototyps ist konsistent implementiert:

- **OIDC-Verifikation** (`server/systemadmin/githubActions0idc.ts`): Issuer, Audience, RS256-Signatur gegen JWKS, `exp / iat / nbf` mit 60 s Skew, `sub` -Repository-Bindung, `repository_id`, `repository_owner_id`, `actor / actor_id` gegen den kanonischen Owner, `event_name == issues`, `ref == refs/heads/main` und Allowlist der beiden `workflow_ref` -Werte. Fail-closed.

- **Permit-before-side-effect:** Branch, Commit und PR werden jeweils erst nach `ALLOW` plus gültiger `supabase:agent_audit_events:*`-Referenz ausgeführt; Head-, Branch-, Mandats- und `requestedPaths`-Bindung wird hostseitig gegengeprüft.
- **Deterministische Nutzlast:** Der SA4-Host erzeugt den Dateinhalt selbst und verifiziert ihn nach dem Commit per SHA-256 gegen den erwarteten Payload. Der auslösende Issue-Body kann keinen Dateinhalt liefern.
- **Rollback bei Evidence-Ausfall:** Schlägt die Outcome-Persistierung fehl, werden Branch und Draft-PR wieder entfernt.
- **Self-Authority-Deny** (`SYSTEMADMIN_SA3_SELF_AUTHORITY_PATHS`): mutierende Capabilities auf die eigene Control-Plane werden verweigert.
- **Untrusted-Input-Validierung:** `validateExecutionIssue.mjs` und `validateSa4PilotIssue.mjs` erzwingen striktes JSON, 8-KiB-Limit, Key-Allowlist, feste Mandats-/Modus-/Roadmap-Werte, Lowercase-SHA-40 und einen engen Branch-Namespace.
- **Mandatsprüfung** (`roadmapExecutionMandate.ts`): `status == OWNER_APPROVED`, aktiver Kill-Switch, `validFrom / expiresAt`-Fenster, Pfad-Allowlist, `maxOpenPullRequests`, Owner als alleinige `revocationAuthority`, `SELF_MANDATE_EXPANSION` als verbotene Mutationsklasse.

2.4 Secrets

- Keine Secret-Dateien im Tracked Tree; nur Beispiel-/Manifest-/Krypto-Module.
- Keine Treffer für Inline-Muster (`sk-*`, `ghp_*`, `AIza*`, PEM-Private-Keys, JWT-Literale).

3. Befunde

P0-1 — `main` ist serverseitig ungeschützt, obwohl das Promotion-Gate erfüllt ist

Nachweis. Die GitHub-API meldet für `main`: `"protected": false`. Es existiert kein Ruleset. `.github/policies/main-production-protection.expected.json` bestätigt das selbst mit `"current_server_ruleset": "absent"` und `"state": "shadow_validation_required"`.

Das in `docs/evidence/ci/P0_MAIN_PROTECTION_RECOVERY_2026-08-14.md` definierte Promotion-Gate verlangt mindestens einen realen Pull Request mit `capital-ai-ci` PASS auf dem aktuellen Head. **Dieses Gate ist erfüllt** — `capital-ai-ci` lief erfolgreich auf fünf realen Pull Requests (#248 `2ac1b00`, #249 `680c1d2`, #250 `7a6c72b`, #251 `b81ee47`, #252 `b985613`).

Die Ruleset-Aktivierung (Exit-Gate P0, Schritt 4) ist jedoch **nicht erfolgt**. Damit sind seit dem Merge von PR #247 fünf Merges nach `main` ohne jede serverseitige Merge-Protection gelaufen: kein Required Check, kein Pull-Request-Zwang, kein Deletion-/Non-Fast-Forward-Schutz.

Bewertung. ADR-0069 §1 setzt voraus, dass „der zuletzt verifizierte, funktionierende Required Check aktiv bleibt“. Aktuell ist überhaupt kein Required Check aktiv. Das Human/Owner-Gate in `ci.yml` ist funktional, aber nicht erzwungen — es ist umgehbar, indem ein PR gar nicht erst geöffnet oder der Check nicht abgewartet wird. Die gesamte Fail-closed-Prämisse der DevelopmentChain hat derzeit keine serverseitige Durchsetzung.

Empfehlung. Ruleset `main-production-protection` gemäß der bereits committeten Expected-Policy als separate Owner-Admin-Mutation aktivieren — aber erst nach Klärung von P1-1 (siehe unten).

P0-2 — Produktionsdeployment hängt an ungeschütztem `main` ohne Environment-Schutz

Nachweis. `.github/workflows/ci.yml:239-262` (`deploy-production`) feuert bei `push` auf `refs/heads/main` den Render-Deploy-Hook mit `secrets.RENDER_DEPLOY_HOOK_URL`. Der Job deklariert **kein** `environment:`. Die Expected-Policy führt `production_environment.status` als `"owner-admin-handoff-required"`, das heißt: es existiert keine GitHub-Environment-Protection (keine Required Reviewers, keine Deployment-Branch-Regel auf Secret-Ebene).

Bewertung. In Kombination mit P0-1 ergibt sich eine durchgehende Kette: Schreibzugriff auf `main` ⇒ Push ⇒ `build-and-test` ⇒ automatischer Produktionsdeploy — vollständig ohne Owner-Gate, ohne Required Check und ohne Deployment-Freigabe. Das ist der schwerwiegendste Einzelbefund dieses Audits, weil er die Repository-Governance mit der Produktionsumgebung kurzschließt.

Empfehlung. `environment: production` am `deploy-production`-Job deklarieren und das GitHub-Environment mit Required Reviewer (Owner) sowie Deployment-Branch-Beschränkung auf `main` konfigurieren. Der Deploy-Hook gehört als Environment-Secret, nicht als Repository-Secret.

P1-1 — Die geplante Ruleset-Promotion würde die Human/Owner-Head-Evidence aus dem Merge-Gate entfernen

Nachweis. Der `owner-gate`-Job — Prüfung der beiden Owner-Checkboxes aus dem Body-Snapshot des `edited`-Events plus current-head-gebundener Owner-Review mit exakt `1` oder `okay` — existiert **ausschließlich** in `.github/workflows/ci.yml:22-120`.

`.github/workflows/capital-ai-ci-shadow.yml` besitzt keinen solchen Gate: der Workflow geht direkt von `checkout` in Build/Test. Die Expected-Policy listet aber als `required_status_checks` **nur** `["capital-ai-ci"]` und setzt `required_approving_review_count: 0`.

Bewertung. Würde das Ruleset genau so aktiviert, wäre der einzige erzwungene Check ein reiner Build-Check ohne jede Human/Owner-Attestation. ADR-0069 §5 („Human-Evidence bleibt current-head gebunden“: vollständiger Files-changed-Review, alle Dateien Viewed, current-head-Review `1` / `okay`, Owner-Attestation, Invalidierung durch neuen Commit) verlöre damit seine technische Durchsetzung — und zwar durch genau den Migrationsschritt, der die Protection wiederherstellen soll.

Empfehlung. Vor der Promotion eine der beiden Varianten als eigene Change-Klasse nach ADR-0069 §4 entscheiden: 1. Das Ruleset verlangt **beide** Kontexte (`capital-ai-ci` **und** den Owner-gegateten `build-and-test`); oder 2. der Owner-Gate-Job wird als vorgeschalteter

needs -Job in `capital-ai-ci-shadow.yml` übernommen und vorher erneut im Shadow bewiesen.

Variante 1 ist die konservativere, weil sie den bereits verifizierten Pfad unangetastet lässt.

P1-2 — Für REM-M5A-REPOSITORY-001 existiert kein Ausführungspfad

Nachweis. Eine repository-weite Suche nach `REM-M5A` findet die Kennung ausschließlich in `.ai/mandates/REM-M5A-REPOSITORY-001.json` selbst. Es gibt keine Referenz in Workflows, Hosts, Validatoren oder im Broker.

Konkret blockieren drei unabhängige Stellen jede M5A-Ausführung über den SA-Pfad:

Stelle	Bindung
<code>server/systemadmin/githubActions0idc.ts</code>	<code>workflow_ref</code> -Allowlist enthält nur <code>systemadmin-roadmap-executor.yml</code> und <code>systemadmin-sa4-pilot.yml</code>
<code>systemadminExecutionBrokerRouter.ts:50-58</code>	<code>expectedMandateForWorkflow()</code> mappt nur auf <code>REM-SA3B-PROBE-001</code> / <code>REM-SA4-PILOT-001</code> ; alles andere ⇒ HTTP 403 <code>workflow-mandate-binding-mismatch</code>
<code>scripts/systemadmin/validateSa4PilotIssue.mjs:40</code> und <code>runSa4Pilot.mjs:265-274</code>	harte Gleichheitsprüfung auf <code>REM-SA4-PILOT-001</code> , Modus <code>BOUNDED_DOC_PR</code> , Ein-Pfad-Allowlist <code>docs/evidence/sa4/SA4_FIRST_AUTONOMOUS_WORK_PACKAGE.md</code>

Bewertung. `docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md:80` und `docs/roadmaps/M5A_SYSTEMADMIN_REPOSITORY_WORK_PACKAGE.md:11` erklären als nächsten zulässigen Auftrag, dass der Systemadministrator das M5A-Code-/Test-Paket „über den auditierten SA4-Pfad“ ausführt. Der SA4-Pfad kann das strukturell nicht: er ist ein Ein-Zweck-Ein-Pfad-Host für ein Dokument. Die Aktivierung des Mandats auf `OWNER_APPROVED` allein bewirkt daher nichts — der erste Broker-Aufruf endet in einem 403.

Verschärfend: Die dafür nötige Control-Plane-Erweiterung berührt Pfade aus `SYSTEMADMIN_SA3_SELF_AUTHORITY_PATHS`, die der Agent korrekterweise selbst nicht mutieren darf. Der Erweiterungsschritt ist damit zwingend ein Human-authored Change.

Empfehlung. Vor jeder M5A-Aktivierung einen eigenen, human-verfassten SA-Host für Code-/Test-Mandate spezifizieren (ADR + ESS-0021-Fortschreibung), inklusive Erweiterung der `workflow_ref` -Allowlist und des Mandats-Mappings. Alternativ die Roadmap-Aussage korrigieren und M5A explizit als Human-/Agent-PR-Arbeit außerhalb des SA4-Hosts kennzeichnen.

P1-3 — Abgeschlossene Prototyp-Workflows bleiben dauerhaft scharf

Nachweis. `systemadmin-roadmap-executor.yml` (SA3B) und `systemadmin-sa4-pilot.yml` (SA4) triggern beide auf `issues: [opened]` mit `contents: write`, `issues: write`, `id-token: write`

beziehungsweise zusätzlich `pull-requests: write`. Beide Roadmap-Einträge stehen auf **COMPLETE / VERIFIED PASS**. `docs/evidence/sa4/SA4_FIRST_AUTONOMOUS_WORK_PACKAGE.md` liegt auf `main`, wodurch der SA4-Host durch seinen eigenen Ein-Schuss-Check (`runSa4Pilot.mjs: 284-285`) inzwischen permanent selbst blockiert.

Bewertung. Zwei privilegierte, issue-getriggerte Hosts bleiben unbegrenzt aktiviert, obwohl ihr Auftrag erledigt ist und einer davon funktional tot ist. Das entspricht sinngemäß dem ADR-0069-Verbotsmuster 6 („dauerhaft aktive Bootstrap-Work-Claims nach Abschluss des zugehörigen PRs“). Die Angriffsfläche bleibt schmal (Issue-Autor muss der Owner sein, Titelpräfix, OIDC, Broker-Entscheid, Mandatsfenster) — aber sie ist unnötig.

Empfehlung. Beide Workflows nach dem Muster von `lockfile-remediation.yml` stilllegen (`workflow_dispatch: only, contents: read, if: false`) und als auditierbaren Nachweis behalten.

P1-4 — Mandatsfenster laufen vor dem Roadmap-Schritt ab

Mandat	validFrom	expiresAt	Status
REM-SA3B-PROBE-001	2026-08-12	2026-08-19	OWNER_APPROVED
REM-SA4-PILOT-001	2026-08-12	2026-08-19	OWNER_APPROVED
REM-M5A-REPOSITORY-001	2026-08-14	2026-08-21	DRAFT

Die Ablaufprüfung in `roadmapExecutionMandate.ts:487-488` ist korrekt fail-closed. Faktisch bleibt für M5A jedoch ein Fenster von sieben Tagen, das erst nutzbar wird, wenn P1-2 gelöst ist — was mindestens einen ADR, eine Host-Implementierung und einen Human-Merge erfordert. Das Fenster ist realistisch zu knapp und wird still ablaufen.

Empfehlung. Mandatsfenster erst dann setzen beziehungsweise verlängern, wenn der zugehörige Ausführungspfad existiert und der Owner die Aktivierung tatsächlich vornimmt.

P2-1 — Self-Authority-Schutzliste ist unvollständig

`SYSTEMADMIN_SA3_SELF_AUTHORITY_PATHS` (`server/agentAudit/systemadminAuditedExecution.ts: 25-43`) schützt 18 Pfade, lässt aber unter anderem aus:

- `src/platform/Security/roadmapExecutionMandate.ts` — **der Mandats-Validator selbst** (Status-, Ablauf-, Pfad- und Kill-Switch-Prüfung);
- `.ai/mandates/REM-M5A-REPOSITORY-001.json` ;
- `.github/workflows/ci.yml` , `.github/workflows/capital-ai-ci-shadow.yml` ;
- `.github/policies/main-production-protection.expected.json` ;
- `scripts/security/verifyChangedWorkflowSecurity.mjs` .

Aktuell nicht ausnutzbar, weil die aktiven Mandate extrem enge `allowedPaths` besitzen. Sobald M5A-Pfade nach `src/platform/Security/**` geöffnet werden, entfällt diese zufällige Absicherung. Der Schutz sollte nicht von der Enge des jeweils aktuellen Mandats abhängen.

P2-2 — M5A-Integrationstests würden in CI niemals ausgeführt

`vite.config.ts:25` beschränkt vitest auf `tests/unit/**/*.test.ts` und `tests/server/**/*.test.ts`. `REM-M5A-REPOSITORY-001.allowedPaths` enthält aber `tests/integration/nativeMfaAa-l2.test.ts`.

Ein dort abgelegter M5A-Negativtest liefere **n**ie, während CI grün meldet. Die Verzeichnisse `tests/{integration,security,contract,architecture,e2e,performance}` sind sämtlich leer — reines Gerüst, trotz der Roadmap-Anforderung „positive + negative Tests“ je Phase und der M9-Assurance-Drills.

Empfehlung. Entweder die vitest-`include`-Liste um die real genutzten Verzeichnisse erweitern, oder die M5A-Pfadliste auf `tests/unit/**` beziehungsweise `tests/server/**` korrigieren. Zusätzlich empfehlenswert: ein Guard, der Testdateien außerhalb der `include`-Muster meldet.

P2-3 — Workflow-Sicherheitsprüfung erkennt gelöschte Workflows nicht

`scripts/security/verifyChangedWorkflowSecurity.mjs:11` filtert die geänderten Workflow-Dateien mit `fs.existsSync(name)`. Eine im PR **gelösch**te Workflow-Datei fällt damit aus der Prüfung heraus und wird nicht als sicherheitsrelevante Änderung gemeldet. Bei fehlendem Ruleset (P0-1) existiert derzeit auch keine kompensierende Required-Check-Erzwingung.

4. Datenintegrität

#	Befund	Ort
D-1	Fremdschrift-Fragment בלבד (hebräisch, „nur“) mitten in einem deutschen Security-Evidence-Dokument: „contents: read בלבד“. Bytes D7 91 D7 9C D7 91 D7 93 , eingebracht mit Commit 2ac1b00 . Einziger Fremdschrift-Treffer im gesamten Repository.	docs/evidence/ci/P0_MAIN_PROTECTION_RECOVERY_2026-08-14.md:22
D-2	Roadmap-Baseline veraltet: dokumentiert main@6205868... (PR #251), tatsächlich main@5ba4ab1... (PR #252). Relevant, weil die M5A-Aktivierung ausdrücklich eine „exakte current-main-Bindung“ verlangt.	docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md:5
D-3		

#	Befund	Ort
	ADR-0070 bezeichnet <code>capital-ai-ci</code> als „den bestehenden Required Check“. <code>capital-ai-ci</code> ist kein Required Check — es existiert kein Ruleset.	<code>docs/adr/ADR-0070-...:Sicherheitsinvarianten</code>
D-4	Traceability-Plane veraltet: <code>coverage.json.generatedAt = 2026-08-02</code> (12 Tage alt), 5 ESS ohne <code>implementedBy</code> , <code>components.testRatio = 0.16</code> bei 25 Komponenten.	<code>.ai/knowledge/traceability/*</code>
D-5	Alle 20 Work-Claims tragen <code>"status": "active"</code> , obwohl die zugehörigen PRs (u. a. #75, #104, ADR-0037/0040-Pakete, Daten 2026-08-03 bis 2026-08-10) längst gemergt sind. Mehrere beanspruchen breite Globs (<code>src/platform/Security/</code> , <code>server/runtime/**</code> , <code>.ai/work-claims/**</code> , <code>scripts/pr/lib.mjs</code>), darunter genau Pfade, die M5A benötigt. <code>PR-75-pr-governance.json</code> verweist auf die nicht mehr existierende <code>.github/workflows/agent-branch-preflight.yml</code> . Kein harter Blocker — <code>validateWorkClaim.mjs</code> vergleicht ausschließlich gegen offene PRs, nicht gegen die Claim-Dateien auf <code>main</code> . Der Zustand ist aber irreführend und entspricht sinngemäß ADR-0069-Verbotsmuster 6.	<code>.ai/work-claims/*.json</code>
D-6	Mojibake: <code>// Do not modify file watching ...</code> (zerschossener Gedankenstrich).	<code>vite.config.ts:16</code>

Gegenprobe: Ein repository-weiter Scan über Hebräisch, Kyrillisch, Arabisch, CJK und Griechisch ergab außer D-1 nur legitime mathematische Symbole (Σ , σ , μ , Δ , Ω , λ) in Scoring- und Visualisierungsmodulen. Es gibt keine Hinweise auf breitflächige Textmanipulation.

5. Priorisierte Empfehlung

Reihenfolge bewusst so gewählt, dass keine Maßnahme eine spätere blockiert:

1. **P0-2** — `environment: production` am Deploy-Job plus Environment-Protection. Wirkt sofort und unabhängig vom Ruleset.

- 2. **P1-1** — Entscheidung über die Required-Check-Identität als eigene Change-Klasse nach ADR-0069 §4 (Empfehlung: Ruleset verlangt `capital-ai-ci` **und** den Owner-gegateten `build-and-test`).
- 3. **P0-1** — Ruleset gemäß Expected-Policy aktivieren, danach Merge-Protection an einem Test-PR ohne Bypass beweisen und `P0_MAIN_PROTECTION_RECOVERY_2026-08-14.md` auf `VERIFIED PASS` heben.
- 4. **P1-3** — SA3B/SA4-Workflows stilllegen.
- 5. **P1-2 / P1-4** — M5A-Ausführungspfad klären, bevor `REM-M5A-REPOSITORY-001` auf `OWNER_APPROVED` gesetzt wird; Mandatsfenster erst dann setzen.
- 6. **P2-1 bis P2-3** — Defense-in-Depth-Lücken schließen.
- 7. **D-1 bis D-6** — Dokumenten- und Metadatenkorrekturen.

6. Abgrenzung

- Es wurde **keine** produktive Konfiguration verändert (Supabase, Render, Stripe, Google, IONOS, GitHub-Rulesets, Environments, Secrets).
- Es wurde **kein** Mandat aktiviert, erweitert oder in seinem Status geändert.
- Es wurde **kein** geschützter Google-Marketing-, Consent-, CSP- oder IAM-Invariant be-rührt.
- Die Ausführung von `npm ci`, `lint`, `vitest`, `npm audit` und den drei Guard-Skripten er-folgte ausschließlich lokal und lesend.
- Sämtliche Korrekturen aus Abschnitt 5 bleiben Human/Owner-Entscheidungen; dieses Dokument ersetzt weder eine Owner-Freigabe noch eine ADR.

7. Nachtrag — durchgeführte Korrekturen (2026-08-14, Folge-Commit)

Auf Grundlage dieses Audits wurden die folgenden risikoarmen, rein code-/dokumentba-sierten Korrekturen im normalen Branch→PR→Human-Merge-Fluss umgesetzt. Keine davon mutiert Produktion, GitHub-Admin-Einstellungen oder ein `OWNER_APPROVED` -Mandat.

Befund	Umsetzung
P0-2	<code>deploy-production</code> in <code>ci.yml</code> läuft jetzt nur noch für den echten <code>push -auf- main</code> -Pfad (<code>Job- if</code> statt <code>Step- if</code>) und trägt <code>environment: production</code> . PR-Events überspringen den Job vollständig (zuvor nur eine Echo-Ausgabe) — kein Verhaltensunterschied für PR-CI. Wirkt erst, wenn der Owner das GitHub-En-vironment <code>production</code> mit Required-Reviewer-Schutz konfiguriert — das ist eine GitHub-Admin-Mu-tation außerhalb des Repository-Codes und wurde nicht vorgenommen.
P1-3	SA3B (<code>systemadmin-roadmap-executor.yml</code>) und SA4 (<code>systemadmin-sa4-pilot.yml</code>) per <code>false</code> <code>&&</code> -Präfix vor der bestehenden Job-Bedingung stillge-

Befund	Umsetzung
	legt. Trigger, Permissions und der komplette auditierte Host-Code bleiben byte-identisch erhalten, damit <code>tests/unit/systemadminExecutionHostWorkflow.test.ts</code> und <code>tests/unit/systemadminSa4-Contracts.test.ts</code> den Sicherheitsvertrag unverändert weiter verifizieren. Reaktivierung erfordert nur das Entfernen von <code>false &&</code> .
P2-1	<code>SYSTEMADMIN_SA3_SELF_AUTHORITY_PATHS</code> (<code>server/agentAudit/systemadminAuditedExecution.ts</code>) und der Parallelring <code>SYSTEMADMIN_SELF_AUTHORITY_PATHS</code> (<code>src/platform/Security/roadmapExecutionMandate.ts</code>) um <code>REM-M5A-REPOSITORY-001.json</code> , <code>ci.yml</code> , <code>capital-ai-ci-shadow.yml</code> , <code>main-production-protection.expected.json</code> , <code>roadmapExecutionMandate.ts</code> selbst und <code>verifyChangedWorkflowSecurity.mjs</code> erweitert.
P2-2	<code>vite.config.ts</code> lädt jetzt zusätzlich <code>tests/integration/**/*.*.test.ts</code> , damit ein künftiger <code>tests/integration/nativeMfaAal2.test.ts</code> (von <code>REM-M5A-REPOSITORY-001.allowedPaths</code> referenziert) tatsächlich in CI ausgeführt wird.
P2-3	<code>verifyChangedWorkflowSecurity.mjs</code> erkennt jetzt gelöschte Workflow-Dateien (<code>git diff --name-status</code> statt <code>--name-only</code>) und verlangt für Löschungen explizite Owner-Review statt sie stillschweigend zu ignorieren.
D-1	Fremdschrift-Fragment בלבד in <code>docs/evidence/ci/P0_MAIN_PROTECTION_RECOVERY_2026-08-14.md</code> : 22 nicht verändert — betrifft ein bereits gemergtes, append-only-artiges Evidence-Dokument auf <code>main</code> ; Korrektur dort ist eine eigene, kleine Owner-Entscheidung und wird hier nicht mitgezogen, um dieses Audit-/Fix-Paket nicht mit unabhängigen historischen Evidence-Dateien zu vermischen.
D-2	<code>DEVELOPMENT_CHAIN_ROADMAP.md</code> -Baseline auf <code>main@5ba4ab1</code> (PR #252) aktualisiert; zusätzlich Korrektur der SA4→M5A-Ausführungsaussage (siehe P1-2-Nachtrag).
D-3	ADR-0070 korrigiert: <code>capital-ai-ci</code> heißt nicht mehr fälschlich „bestehender Required Check“, sondern verweist auf den tatsächlichen Promotion-Status.
D-5	Alle 20 <code>.ai/work-claims/*.json</code> von <code>status: active</code> auf <code>status: superseded</code> mit <code>supersededReason</code> umgestellt (append-only, keine Löschung); Legacy-Datei <code>RENDER-CI-GATE-2026-08-09.json</code> ohne bisheriges <code>status</code> -Feld ebenso ergänzt.

Befund	Umsetzung
D-6	Doppelt-UTF-8-kodierter Gedankenstrich in <code>vite.config.ts:16</code> repariert.

P1-2 (Nachtrag): `docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md` wurde um eine explizite Korrektur ergänzt, dass der SA4-Pfad `REM-M5A-REPOSITORY-001` strukturell nicht ausführen kann (Workflow-Ref- Allowlist, Broker-Mandats-Mapping und `runSa4Pilot.mjs` sind hart auf `REM-SA4-PILOT-001` gebunden). Ein neuer M5A-Ausführungshost wurde **nicht** implementiert — das ist eine neue privilegierte Automations-Capability und erfordert einen eigenen, human-verfassten ADR mit Owner-Review, nicht einen stillschweigenden Workaround in diesem Fix-Paket.

Neuer Befund während der Umsetzung: ADR-Registry-Lücken (nicht behoben)

`npm run traceability:build` wurde probeweise ausgeführt, um D-4 (veraltete Traceability-Daten) zu beheben. Der Lauf endet mit Exit-Code 1 und vier harten Befunden, die **bereits vor diesem Audit** im Repository bestanden und nichts mit den hier vorgenommenen Änderungen zu tun haben:

- **ADR-0046 ist doppelt vergeben:**

`docs/adr/ADR-0046-modern-supabase-key-contracts-and-ai-admin-control-plane.md` und `docs/adr/ADR-0046-vocabulary-governance-authority-and-namespace.md` sind zwei inhaltlich unabhängige ADRs mit identischer Nummer;

- **ADR-0030 fehlt vollständig:** `src/platform/Release/manifest.json` referenziert

`ADR-0030`, aber keine `docs/adr/ADR-0030-*.md`-Datei existiert;

- **ADR-0056 fehlt in** `docs/adr/adr_history.json`: die Datei existiert (`ADR-0056-observability-telemetry-baseline.md`), ist aber nicht in der Historie registriert.

Die generierten Traceability-Artefakte (`matrix.json`, `coverage.json`, `orphans.json`, `COVERAGE_REPORT.md`, neun `manifest.json`-Dateien) wurden **testweise erzeugt und danach wieder verworfen** (`git checkout --`), weil ihre Regenerierung untrennbar mit diesen drei ungelösten Registry-Lücken verknüpft ist. Eine Umnummerierung eines bereits `ACCEPTED`-ADRs ist eine Dokumentations-Governance-Entscheidung (vgl. ADR-0044, Documentation-Governance-Validator) und keine mechanische Korrektur — sie gehört vor die nächste Traceability-Regenerierung, nicht in dieses Fix-Paket. D-4 bleibt bis dahin offen.

Nicht umgesetzt — erfordert Owner-Admin-Mutation oder neue Autorität

Vier Punkte aus Abschnitt 5 wurden bewusst **nicht** umgesetzt, weil sie außerhalb dessen liegen, was ein Repository-Code-Commit leisten kann oder darf:

1. **P0-1** — Aktivierung des GitHub-Rulesets `main-production-protection` (echte Admin-API-Mutation auf GitHub, kein Repository-Code);
2. **Owner-Environment-Schutz zu P0-2** — Anlegen/Konfigurieren des GitHub-Environments `production` mit Required-Reviewer (Owner-only, außerhalb des Repository-Codes; der Workflow-Code-Teil ist umgesetzt, siehe Tabelle oben);

3. **P1-1** — Endgültige Required-Check-Zusammensetzung des neuen Rulesets (Empfehlung bleibt: sowohl `capital-ai-ci` als auch den Owner-gegateten `build-and-test` verlangen) — an P0-1 gebunden;
4. **P1-4** — Mandatsfenster von `REM-SA3B-PROBE-001` / `REM-SA4-PILOT-001` (`OWNER_APPROVED`) wurden nicht verändert; nur der Owner mit frischem TOTP-Step-up darf ein bereits genehmigtes Mandat ändern.

Alle vier Punkte sind reine Entscheidungen des Human/Owner und werden hier ausdrücklich nicht vorweggenommen.

Snyk Required-Check-Demotion Evidence

Quelle: docs/evidence/security/SNYK_REQUIRED_CHECK_DEMOTION_2026-08-15.md

Snyk Required-Check-Demotion Evidence

Status: **VERIFIED** — Owner-durchgeführt, repository-seitig nachvollzogen Datum: 2026-08-15 Authority: docs/adr/ADR-0070-gitguardian-snyk-external-app-integration.md Abschnitt „Rollback“ Punkt 1, .github/policies/main-production-protection.expected.json

0. Auslöser

security/snyk (svenkulessa) und code/snyk (svenkulessa) begannen am 2026-08-15 dauerhaft mit state: error zu antworten:

- security/snyk : „You have used your limit of private tests“
- code/snyk : „Code test limit reached“

Beide Meldungen sind Snyk-seitige Kontingent-/Billing-Zustände, keine Aussage über den tatsächlichen Code-Zustand. Da beide Kontexte als Required Checks im main-production-protection-Ruleset standen, konnte **keine** der zu diesem Zeitpunkt offenen Pull Requests mehr gemergt werden, unabhängig von deren tatsächlichem Inhalt.

1. Betroffene Pull Requests (Snapshot zum Zeitpunkt der Meldung)

PR	Titel	Autor/Provider
#295	AES-GCM-Manipulation deterministisch prüfen (Ersatz für #294)	Codex Work / GPT-5
#296	Buffet Value Check unter TOP Rankings	ChatGPT Work / GPT-5.6
#297	M8 Policy Equivalence gegen echten Aufrufer (SA3B/SA4)	Claude Code / Sonnet 5

Alle drei zeigten identische Snyk- error -Stati auf demselben aktuellen main-Basisstand (6db2661cfc2b99b4ba2ad5890da1f1e6cb95f7e4), was das Kontingent- statt Code-Problem zusätzlich bestätigt.

2. Owner-Aktion (außerhalb dieser Sitzung)

Der Owner hat direkt im GitHub-Ruleset-Editor security/snyk und code/snyk aus dem Required-Status-Checks-Set der main-production-protection-Ruleset entfernt. Diese Sitzung hat diese Mutation **nicht** selbst durchgeführt (keine Berechtigung/kein Zugriff auf Repository-Rulesets über verfügbare Tools) und auch keinen Snyk-Token/Wert je gelesen oder verändert.

3. Repository-seitige Nachvollziehung (diese Sitzung)

- `.github/policies/main-production-protection.expected.json`: `security/snyk (svenkulesa)` und `code/snyk (svenkulesa)` aus `required.required_status_checks` entfernt; neuer Eintrag `decision_2026-08-15` dokumentiert Grund und Referenzen.
- `docs/adr/ADR-0070-gitguardian-snyk-external-app-integration.md`: Nachtrag 2026-08-15 hinzugefügt — referenziert explizit, dass dies der im ADR selbst vorgesehene Rollback-Schritt 1 ist, keine Abweichung vom dokumentierten Vorgehen.
- Für #295, #296 und #297: jeweils geschlossen (mit Supersession-Kommentar) und eine neue Pull Request vom **identischen Branch/Commit** eröffnet (kein Codeunterschied), um eine saubere Status-Check-Auswertung unter dem aktualisierten Ruleset zu erhalten:

Alt (geschlossen)	Neu	Branch
#295	#300	<code>hotfix/deterministic-secret-ciphertext-tamper-test-v2</code>
#296	#299	<code>feat/sidebar-buffet-value-unter-top-rankings</code>
#297	#298	<code>claude/security-audit-environment-update-i7lvfm</code>

Hinweis: der Head-Branch von #295 wurde vom Repository automatisch beim Schließen gelöscht (Automatically-delete-head-branches-Einstellung greift offenbar auch bei einem reinen Close, nicht nur bei Merge). Er wurde read-only exakt auf denselben Commit `e044f18b6bc55943c831700f1dc4466a3a42e6f2` aus `refs/pull/295/head` wiederhergestellt, bevor #300 eröffnet wurde — keine Inhaltsänderung.

4. Was dies NICHT bedeutet

- Snyk als Sicherheitswerkzeug bleibt aktiv (App-Integration, Repository-Scanning); nur die Merge-blockierende Required-Check-Bindung wurde entfernt.
- `GitGuardian Security Checks` bleibt unverändert Required Check, unbeeinflusst von dieser Änderung.
- Dies ist keine Schwächung einer bestehenden Sicherheitsinvariante — Snyk wurde nach eigenem ADR-0070-Verfahren einst promoviert (2026-08-14) und jetzt nach demselben ADRs eigenem Rollback-Verfahren wieder demotet, ausgelöst durch einen reinen Anbieter-Kontingent-Zustand, nicht durch einen gefundenen Fehler oder eine Owner-Entscheidung gegen Snyk als Kontrolle.

5. Geänderte Dateien

- `.github/policies/main-production-protection.expected.json`
- `docs/adr/ADR-0070-gitguardian-snyk-external-app-integration.md`
- `docs/evidence/security/SNYK_REQUIRED_CHECK_DEMOTION_2026-08-15.md` (diese Datei)

Evidence — D3 Soft-404 Production Gap (2026-08-16)

Quelle: docs/evidence/seo/D3_SOFT_404_PROD_GAP_2026-08-16.md

Evidence — D3 Soft-404 Production Gap (2026-08-16)

Roadmap: SEO-GM-ROADMAP-0002 / WP-D3
PR: #360 (merged)
Status: **CLOSED — VERIFIED in production**

Pre-fix gap (historical)

Production commit probed: 355c94b910336454d6de4494f9c2660b721925fc (version 0.6.0)
Probe time: 2026-08-16T01:39Z (CEST)

Code for Soft-404 existed on `main` (`installProductionSoft404Intercept` in `registerApplicationRoutes`, helpers in `server/runtime/spaFallback.ts`, unit tests green), but **production still returned HTTP 200 + SPA shell** for unknown paths.

URL	Observed	Expected
<code>https://capital-ai.online/not-a-real-page-xyz-seo-check</code>	<code>200 text/html (full index.html)</code>	<code>404 plain Not Found</code>
<code>https://capital-ai.online/impressum</code>	<code>301 → /impressum/</code>	<code>200 without slash (Q2)</code>
<code>https://capital-ai.online/impressum/</code>	<code>301 → /impressum</code>	<code>200 (or single canonical)</code>

Root cause analysis

1. **Catch-all still naive in composition root**
`server.application.ts` `startServer()` production branch still registered: `ts app.use(express.static(distPath)); app.get('*', (req, res) => { res.sendFile(path.join(distPath, 'index.html'))}; });`
2. **Prototype intercept is not trustworthy under the production bundle**
`installProductionSoft404Intercept()` patches `require('express').application.get`. Production server is **esbuild-bundled** (`npm run build → dist/server.cjs`). ESM `import express` used to create `app` and CJS `require('express')` inside the intercept can resolve to **different module instances**, so the prototype patch never wraps the real `app.get('*')` handler.
3. **Q2 trailing-slash vs prerender directories**
S2 prerender writes `dist/impressum/index.html` (and `agb/datenschutz`).

Default `express.static` **redirects directories to a trailing slash**, while `registerTrailingSlashNormalize` strips trailing slashes → redirect ping-pong risk.

Fix applied (PR #360)

- 1. Replace production catch-all with `registerProductionSpaFallback(app, distPath)` (explicit allow-list; unknown → 404).
- 2. Mount static as `express.static(distPath, { redirect: false, index: false })` so public HTML is owned by the SPA fallback and Q2 is not fought by directory redirects.
- 3. Keep unit/contract tests; update D3 status docs.

Post-deploy verification (PASS)

Probe time: 2026-08-16T00:40:11Z (UTC)
Production commit: 54c48ca28817b41b93623be61ea8292f0a8288c2
Headers: x-capital-ai-branch: main , x-capital-ai-provider: render , version 0.6.0

URL	Observed
https://capital-ai.online/	200
https://capital-ai.online/impressum	200 text/html
https://capital-ai.online/impressum/	301 → /impressum
https://capital-ai.online/agb	200
https://capital-ai.online/datenschutz	200
https://capital-ai.online/not-a-real-page-xyz-seo-check	404 text/plain body Not Found
https://capital-ai.online/wp-admin	404

DoD for WP-D3 met: unknown paths return real HTTP 404 without SPA shell.

Systemadmin Work-Package Catalog — Generalization Proof

Quelle: docs/evidence/systemadmin-work-packages/WORK_PACKAGE_GENERALIZATION_PROOF.md

Systemadmin Work-Package Catalog — Generalization Proof

Status: GENERATED BY AUDIT-BOUND SYSTEMADMIN WORK-PACKAGE HOST / HUMAN MERGE REQUIRED

This file is the deliberately bounded output of the first entry in the generalized Systemadmin work-package catalog (ADR-0074). It proves that the catalog-driven runner (`scripts/systemadmin/runWorkPackage.mjs`) can execute a pre-approved, already-reviewed work package through the same audit-bound BRANCH -> COMMIT -> PR chain that SA4 proved for exactly one hardcoded target (ADR-0068), without needing a brand-new workflow file or broker route per work package. It was generated by trusted GitHub Actions host code; the triggering Issue cannot supply this Markdown body.

Bound execution

- Work package: `GENERALIZATION-PROOF`
- Mandate: `REM-WORKPACKAGE-GEN-PROOF-001`
- Roadmap item: `SYSTEMADMIN-WORK-PACKAGE-CATALOG-GENERALIZATION`
- Issue: `#325`
- Workflow run: `31893944834`
- Base SHA: `9d55826a2ef41d1a933e9f843929eb5f51ea5c64`
- Branch: `agent/systemadmin-work-package-gen-proof-1`
- Path: `docs/evidence/systemadmin-work-packages/WORK_PACKAGE_GENERALIZATION_PROOF.md`

Pre-side-effect audit evidence

- BRANCH authorization: `supabase:agent_audit_events:dbf29831-321d-47c1-bca4-6b44502562d8`
- BRANCH outcome: `supabase:agent_audit_events:862c9e49-0bce-46d0-91f6-cd458a9659e2`
- COMMIT authorization: `supabase:agent_audit_events:0b2d1c28-a9cc-4875-b646-3a5936c04fb8`

Security boundary

This work package is documentation-only. It does not request CI, deploy, mutate production, change IAM, expose secrets, alter repository protection or merge a pull request. Final review and merge remain Human/Owner-only. The catalog entry that produced this file is trusted, already -merged code (`scripts/systemadmin/workPackages/generalizationProof.mjs`) — the triggering Issue could only select which pre-approved work package to run, never supply file content.

DEVELOPMENT Chain Phase Evidence Template

Quelle: docs/evidence/templates/DEVELOPMENT_CHAIN_PHASE_EVIDENCE_TEMPLATE.md

DEVELOPMENT Chain Phase Evidence Template

Template only. Copy into the phase-specific `docs/evidence/<phase>/` directory. Never store reusable credentials, TOTP secrets/codes, recovery codes, private keys, raw bearer tokens or unredacted sensitive payloads.

Header

- Phase:
- Roadmap Item:
- Evidence Status: `DRAFT` | `VERIFIED READ-ONLY` | `VERIFIED PASS` | `FAILED` | `ROLLED BACK`
- Date / UTC time:
- Repository: `SvenKulesa/Finance`
- Baseline branch: `main`
- Baseline SHA:
- Production baseline / target state:
- Authority refs:
- Evidence owner:

1. Scope

In scope

- ...

Explicitly out of scope

- ...

2. Risk / Check Classification

- PR Check Class: `D` | `C` | `R` | `M`
- Effective Risk: `LOW` | `MEDIUM` | `HIGH` | `CRITICAL`
- Mutation Class:
- Platform(s):
- Reserved Human/Owner action involved: `YES` / `NO`
- Kill switch / stop mechanism:

3. Repository Lifecycle

- Work branch:
- Branch created from SHA:
- Concurrent PR overlap result:
- Final PR number:
- Final Head SHA:
- Human file review evidence:
- Required CI run(s):
- Human merge evidence:
- Merge SHA:
- Finance remote branch deleted: YES / NO / N/A
- Ephemeral clone/worktree cleanup: DONE / N/A

4. Implementation Summary

- Changed components:
- Security controls changed:
- Data / runtime behavior changed:
- Documentation/contracts changed:

5. Tests

Positive tests

Test	Expected	Actual	Evidence ref	Result

Negative / fail-closed tests

Test	Expected DENY/ failure	Actual	Evidence ref	Result

6. External Mutation Classification

- External mutation required: YES / NO
- Mutation state: NOT REQUIRED | PLANNED | HUMAN APPROVED | MUTATED | VERIFIED PASS | FAILED / ROLLED BACK
- Handoff Contract ID:
- Exact target resource:
- Allowed operation(s):
- Explicitly forbidden operation(s):
- Idempotency key / replay control:

- Concurrency key:
- Approval Evidence Reference:
- Approval expiry / freshness:

7. Pre-Mutation Verification

Check	Baseline / Expected	Observed	Result
Correct account/project/service/environment			
Baseline / head / deployment drift			
Current platform health			
Last-known-good / backup			
Rollback executable			
No competing writer			
Mandate / approval valid			

Pre-Mutation Gate: PASS / FAILED / INCONCLUSIVE / N/A

8. Authorization / Audit

- Human actor:
- Agent / executor:
- Execution host:
- REM / Policy decision reference:
- Authorization Audit Reference:
- Audit timestamp:
- Capability / risk / target binding:
- Secret redaction review: PASS / FAIL

9. Mutation Execution

- Mutation started:
- Exact semantic operation(s):
- Mutation outcome: SUCCESS / ERROR / N/A
- Outcome Audit Reference:
- Unexpected side effects:

Do not paste raw secrets or sensitive request/response bodies.

10. Post-Mutation Verification

Check	Expected	Observed	Evidence ref	Result
Intended state				
Health/readiness				
Authorization boundary				
Negative/deny path				
Audit correlation				
Drift / unexpected changes				

Post-Mutation Gate: VERIFIED PASS / FAILED / INCONCLUSIVE / N/A

11. Rollback

- Rollback trigger reached: YES / NO
- Procedure ref:
- Rollback action:
- Rollback verification:
- Final rollback state: NOT REQUIRED | VERIFIED RESTORED | FAILED

12. Traceability

Requirement	Authority	Implementation / Mutation	Test	Evidence	State

13. Residual Risk

- Open findings:
- Accepted residual risk:
- Owner acceptance ref, if required:

14. Final State

- Phase state:
- Mutation state:
- All required tests VERIFIED PASS : YES / NO
- Evidence complete: YES / NO
- Roadmap synchronized: YES / NO
- Traceability synchronized: YES / NO
- Branch lifecycle closed: YES / NO

- Next gate:

Closure statement

A PR merge alone does not close the phase. Closure requires all required repository tests, external mutations, post-verification, rollback state, evidence and roadmap/traceability synchronization to be complete.

AI Agent M0-M10 Traceability Matrix

Quelle: docs/traceability/AI_AGENT_M0_M9_TRACEABILITY_MATRIX.md

AI Agent M0-M10 Traceability Matrix

Legacy filename retained for stable references. Detailed document/handoff traceability:
docs/traceability/DEVELOPMENT_CHAIN_DOCUMENT_TRACEABILITY_MATRIX.md .

Status date: 2026-08-14 Baseline: main@66da35b80ba23e4f216318a9cd9f9b4e7b787679 (PR #255 merge)

Phase	Execution State	Authority	Primary implementation / execution documents	Mutation / Test Gate	Exit Evidence
M0	COMPLETE	M0 Evidence Baseline	docs/evidence/m0/*	read-only	baseline PASS
M1	COMPLETE	Git Guardrails / Human Owner Policy	protected main / PR governance	Owner-reviewed policy	stable protection + merge boundary
M2 / M2G	COMPLETE	ESS-0019 + ADR-0057..0063	docs/architecture/ai-agent/* , Freeze Policy	no production mutation	Documentation Freeze COMPLETE
M3	COMPLETE	ADR-0053 + ADR-0060	.github/workflows/ci.yml , PR Check Classification	bounded scope-aware CI	one normal build-and-test path preserved
M4	COMPLETE	ADR-0058 + ESS-0018/0019	Agent IAM / PolicyGate / capability/risk models	negative IAM + Human gate	M4 VERIFIED
M5	VERIFIED PASS	ADR-0056 + ADR-0059	public.agent_audit_server/agentAudit/*	PR #222 corrected writer→schema contract; real successful privileged audit insert confirmed (docs/evidence/m5/M5_VERIFIED_PASS_CLOSURE_EVIDENCE.md)	persistence and application runtime both VERIFIED PASS
M5A	VERIFIED PASS	ESS-0020 + ADR-0064 + ADR-0003.5	docs/run-books/M5A_SU-	repo code+CI merged; both Owner identities	docs/evidence/m5a/M5A_VERI-

Phase	Execution State	Authority	Primary implementation / execution documents	Mutation / Test Gate	Exit Evidence
			PABA-SE_TOTP_AAL2_HARDENING	enrolled + verified on M6, no TOTP factor, confirmed aal2 session, Advisor without unowned HIGH/CRITICAL	FIED_PASS_CLOSURE_EVIDENCE M6 unblocked
M6	VERIFIED PASS	ADR-0060	AI_AGENT_SUPPLY_CHAIN_MODEL docs/run-books/M6_SUPPLY_CHAIN_PROVENANCE.md	Source Model SBOM/artifact/provenance choice implemented, merged, and confirmed on the real hosted push -to- main build path (run 31834114193): cosign keyless signing (Sigstore Fulcio/Rekor) signed and, in the same run, co-sign verify-blob confirmed the signature against the exact expected certificate identity and OIDC issuer (Verified OK); positive/negative consistency tests PASS (tests/unit/verifySupplyChainProvenance.test.ts)	docs/evidence/m6/M6_REPOSITORY_IMPLEMENTATION_EVIDENCE.md
M7	COMPLETE / VERIFIED PASS (2026-08-14, all 9 Exit Gate criteria met)	ADR-0061	AI_AGENT_DEPLOYMENT_IDENTITY docs/run-books/M7_DEPLOYMENT_IDENTITY_MUTATION.md	IDENTITY under preflight done; deployment gated on M6 attestation, confirmed on a real push; post-deploy identity/	docs/evidence/m7/M7_PHASE0_AND_REPOSITORY_CONTROLS_EVIDENCE.md , docs/run-

Phase	Execution State	Authority	Primary implementation / execution documents	Mutation / Test Gate	Exit Evidence
				health verification confirmed end-to-end against the live Render deploy after a real bug fix; Owner performed a real deploy-hook rotation, independently confirmed via 3 sources; Owner performed a real Render Dashboard rollback plus roll-forward to current <code>main</code> , both independently confirmed via 3 sources each; all 10 Required Negative Tests now have a concrete, automated, passing test (<code>docs/evidence/m7/M7_REQUIRED_NEGATIVE_TESTS_EVIDENCE.md</code> , new <code>src/platform/Security/development-ChainMutationHandoff.ts</code> validator/gate)	<code>books/M7_DEPLOY_HOOK_ROTATION_HANDOFF</code> <code>docs/run-books/M7_ROLLBACK_VERIFICATION_HANDOFF.md</code> , <code>docs/evidence/m7/M7_REQUIRED_NEGATIVE_TESTS_EVIDENCE.md</code> ; all required mutations VERIFIED PASS
M8	COMPLETE / VERIFIED PASS (2026-08-16, all 9 Exit Gate items PASS)	ADR-0062 + ESS-0019	Provider Profile Contract, <code>docs/run-books/M8_AGENT_CUTOVER.md</code>	Provider Profile Registry, Policy-Equivalence, no-regression <code>SA3B/SA4 wiring</code> , rollback-to-read-only, audit correlation and provider-specific-bypass-deni-	<code>docs/evidence/m8/M8_CLOSURE_EVIDENCE.md</code> ; <code>docs/evidence/m8/M8_EXIT_GATE_I-</code>

Phase	Execution State	Authority	Primary implementation / execution documents	Mutation / Test Gate	Exit Evidence
				al all verified; externalHost-Configuratio-nVerified for chatgpt-github-connector verified read-only 2026-08-16 → READY on all 6 ProviderCuto-verEvidence fields; Exit-Gate-Punkt 2 closed under Owner-accepted scope decision (2026-08-16, explicit "ACCEPT" via AskUserQuestion) — supported privileged providers = mutating providers with a production host, i.e. chatgpt-github-connector only; canonical provider set corrected 2026-08-16 (PR #365) to ChatGPT/Claude/Grok — claude-code-cli and grok-xai-connector both structurally BLOCKED and explicitly not supported privileged paths; google-ai-studio / notebooklm / gemini now RETIRED (DENY)	TEM2_SC0-PE_DECISION.md ; docs/evidence/m8/M8_EXTERNAL_HOST_CONFIGURATION_VERIFIED.md ; docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PROFILE_EVIDENCE.md ; docs/evidence/m8/M8_PROVIDER_BY_PASS_ROUTED_EDIT_EXIT_GATE_4_EVIDENCE.md ; docs/evidence/m8/M8_I1_CUTOVER_READINESS_MATRIX_AND_EXIT_GATE_SYNC_EVIDENCE.md
M9	BLOCKED BY M8	ADR-0063	Incident Response model,	bypass/injection/replay/exfil-	docs/evidence/m9/* ;

Phase	Execution State	Authority	Primary implementation / execution documents	Mutation / Test Gate	Exit Evidence
			docs/run-books/M9_ASS-URAN-CE_INCI-DENT_BREAK_GLASS.md	tration/audit/kill-switch/break-glass/rollback drills	no unowned CRITICAL control
M10	BLOCKED BY M9	ADR-0066 + ESS-0022	M10 Threat Model + docs/runbooks/M10_PASS-KEY_OW-NER_PR_AUTHORIZATION.md	exact-state WebAuthn + shadow/replay/recovery/audit/single-CI/legacy-cleanup tests	docs/evidence/m10/* ; passkey-only gate VERIFIED PASS

Cross-cutting DevelopmentChain Controls

Requirement	Authority / Artifact	Expected behavior
Operational roadmap	docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md	current phase + documentation readiness + next gate
Execution sequence	docs/governance/DEVELOPMENT_CHAIN_EXECUTION_POLICY.md	no mutation/phase skip
Branch lifecycle	docs/governance/DEVELOPMENT_CHAIN_BRANCH_LIFECYCLE_POLICY.md	fresh branch from current main; delete after Human merge; never reuse
Clone/worktree lifecycle	same Branch Lifecycle Policy + AGENTS.md	never edit local main; clean ephemeral workspace after Evidence
Responsibilities	docs/governance/DEVELOPMENT_CHAIN_RESPONSIBILITY_MATRIX.md	Owner/Dev/Prod/Executor/CI separation
Check class	docs/governance/PR_CHECK_CLASSIFICATION.md	strictest D/C/R/M applies
Generic phase execution	docs/runbooks/DEVELOPMENT_CHAIN_PHASE_EXECUTION.md	reproducible preflight → implementation → merge → optional mutation → closure
Generic Evidence	docs/evidence/templates/DEVELOPMENT_CHAIN_PHASE_EVIDENCE_TEMPLATE.md	baseline, tests, mutation, audit, rollback, closure captured
Mutation Handoff	docs/contracts/DEVELOPMENT_CHAIN_MUTATION_HANDOFF_CONTRACT.md	exact non-authorizing work order
Machine validation	.ai/contracts/development-chain-mutation-handoff.schema.json	exact bounded Handoff schema
Autonomous mutation	Systemadmin REM/IAM/Execution Host	permit-before-side-effect; independent VERIFIED PASS required

Requirement	Authority / Artifact	Expected behavior
Merge	Human/Owner only	never granted to agent

Current Phase Gate

M5 is `VERIFIED PASS` (confirmed 2026-08-14): the application writer's corrective runtime verification (started after the first real SA3B probe, PR #222 corrected the repository-side M5 schema mapping and added migration↔writer regression coverage) is closed — a real successful privileged production audit insert was found already recorded on 2026-08-12 (SA4 pilot chain, `pull_request_number: 229`), cross-verified against the actual merged PR #229, on a base SHA confirmed to be a descendant of PR #222's merge commit. See `docs/evidence/m5/M5_VERIFIED_PASS_CLOSURE_EVIDENCE.md`.

M5A is `VERIFIED PASS` (2026-08-14): both Owner identities completed native TOTP enrollment interactively, confirmed via read-only production evidence (`docs/evidence/m5a/M5A_VERIFIED_PASS_CLOSURE_EVIDENCE.md`). M6 (Supply Chain Provenance) is `VERIFIED PASS` (2026-08-14): the full source→lockfile→SBOM→provenance/attestation chain was confirmed on the real hosted `push-to-main` build path, including a real cosign keyless signature verified in the same CI run (`docs/evidence/m6/M6_REPOSITORY_IMPLEMENTATION_EVIDENCE.md`). M7 (Deployment Identity) is `COMPLETE / VERIFIED PASS` (2026-08-14): Phase 0 (read-only Render preflight) is complete; the provenance-gated deploy + post-deploy identity/health verification is `VERIFIED PASS` on the real hosted build/deploy path (`docs/evidence/m7/M7_PHASE0_AND_REPOSITORY_CONTROLS_EVIDENCE.md`); a real Owner-executed deploy-hook rotation/revocation is independently confirmed via GitHub Actions logs, Render's own deploy history, and a passing post-deploy verification (`docs/runbooks/M7_DEPLOY_HOOK_ROTATION_HANDOFF.md`); a real Owner-executed Render Dashboard rollback to a prior deploy plus its subsequent roll-forward to current `main` (via merge of PR #277) is likewise independently confirmed via GitHub Actions logs, Render's own deploy history (`trigger: rollback` then `trigger: deploy_hook`), and passing post-deploy verification (`docs/runbooks/M7_ROLLBACK_VERIFICATION_HANDOFF.md`); and all 10 Required Negative Tests from the runbook now have a concrete, automated, passing test against a new `src/platform/Security/development-ChainMutationHandoff.ts` validator/execution gate, static CI-config guards, and a real subprocess integration test proving the health/readiness fail-closed path (`docs/evidence/m7/M7_REQUIRED_NEGATIVE_TESTS_EVIDENCE.md`). All 9 M7 Exit Gate criteria are explicitly walked through and met in `docs/runbooks/M7_DEPLOYMENT_IDENTITY_MUTATION.md`'s "Exit Gate Closure" section, including confirming via `git ls-remote` that no M7 work branch persists after merge. M8 (Agent Cutover) Phase 0 (read-only provider inventory) is complete: all 6 Prerequisite Gate criteria are met, and the most significant finding is that this (and every) interactive Claude Code session's own MCP tool calls are not mediated by `agentIam.ts` at all — that tool grant comes from the outer CCR/session runtime, not from this repository, so it cannot be closed by a code change. The first Phase 1 repository package, the Provider Profile Contract + Registry (`src/platform/Security/providerProfile.ts`), is `VERIFIED PASS`: profiles for all 4 documented providers (ChatGPT, Claude Code, Google AI Studio, NotebookLM) compose with — never replace — the existing `agentIam.ts` kernel, and every Policy Equivalence Test and Negative Test required by the M8 runbook now has a passing test (`docs/evidence/m8/M8_PHASE-`

SEO_AND_PROVIDER_PROFILE_EVIDENCE.md). The module was then additionally wired as an additive, narrowing-only check into the live SA3B/SA4 execution host (server/agentAudit/systemadminAuditedExecution.ts), with a Policy Equivalence drift-guard test proving the new generic registry stays in exact sync with the pre-existing SA2-specific capability list, and an end-to-end test proving the actually-live BRANCH capability path still resolves identically (layer: 'REM_POLICY' , unchanged) — the full suite (949 tests) stayed green throughout. M8's own exit gate is still not met — Claude Code, Google AI Studio and NotebookLM still have no real caller, no direct provider bypass is deactivated, and rollback-to-read-only is not yet proven — so M8 as a whole stays PLANNED . M9-M10 remain blocked by their sequential predecessor gates.

M5 corrective trace — PR #222

```
PR #220 Execution Host deployed
→ real SA3B probe
→ strict Issue/REM/OIDC checks PASS
→ durable M5 persistence FAIL: Unregistered API key
→ branch side effect SKIPPED
→ read-only diagnosis finds writer↔schema drift
→ PR #222 corrects application mapping and contract tests
→ merge 91963f59...
→ real SA3B probe (07:28 UTC) + SA4 pilot chain (08:42 UTC, PR #229) on post-fix main
→ real successful privileged audit inserts confirmed, cross-verified against PR #229
→ M5 VERIFIED PASS (2026-08-14, docs/evidence/m5/M5_VERIFIED_PASS_CLOSURE_EVIDENCE.md)
```

The fail-closed invariant NO DURABLE AUDIT PERMIT → NO GITHUB SIDE EFFECT has already been observed in the real host probe.

M5A Requirement Trace

Requirement	Implementation / Verification	Expected result
Native factor flow	Supabase Native enroll → challenge → verify	positive PASS
Privileged AAL1	centralized server AAL2 gate	DENY
Enrollment without verification	assurance gate	DENY
Wrong TOTP / invalid challenge	Native MFA verification	DENY
stale/downgraded assurance	AAL lookup	DENY
AAL lookup failure	fail-closed privileged path	DENY
application step-up without AAL2	combined assurance gate	DENY
wrong user/purpose/replayed step-up	step-up validation	DENY
authorized AAL2 + required action step-up	combined gate	ALLOW
recovery/factor reset	Owner-controlled native recovery	audited PASS
advisor rerun	post-mutation production evidence	captured
secret safety	evidence review	

Requirement	Implementation / Verification	Expected result
		no MFA secret/code/recovery material

M6 Requirement Trace

Requirement	Artifact / Test	Expected result
exact source	source SHA in build/evidence	matches candidate head
exact dependencies	committed lockfile + digest	deterministic install
SBOM	SPDX/CycloneDX machine output	bound to source/lock
artifact identity	artifact/image digest	immutable subject
provenance	builder/source/input/subject statement	verifies exact artifact
attestation	verifiable signature/trust path where required	PASS
source/lock/artifact mismatch	negative tests	DENY
untrusted builder/provenance	negative test	DENY
rollback artifact	immutable last-known-good reference	retrievable/verifiable

M7 Mutation Trace

Every external state change must resolve:

- Roadmap Item
- exact target
 - pre-mutation baseline
 - Human Mutation Approval Evidence
 - DevelopmentChain Handoff ID
 - REM / IAM / Execution Host decision
 - authorization audit reference
 - exact side effect
 - outcome audit reference
 - post-verification
 - rollback state

No Handoff alone authorizes execution.

M8 Provider Equivalence Trace

Equivalent semantic capability requests must produce the same policy outcome across ChatGPT, Claude, Google AI Studio and future transports. All agent profiles deny `MERGE` ; re-search profiles deny mutation; production mutation requires exact approved Handoff plus separately verified execution permission.

M9 Assurance Trace

Required drill families: authorization bypass, prompt/tool injection, replay/idempotency, secret/data exfiltration, audit outage/completeness, kill switch, break-glass, rollback/recovery and independent Evidence review. Unexpected ALLOW/side effect blocks M10.

M10 Passkey Authorization Trace

```
Human File Review / Viewed
→ trusted PR-state resolver
→ server challenge
→ Owner WebAuthn assertion
→ RP/origin/credential/signature/UP/UV/freshness verification
→ exact PR-context match
→ immutable approval evidence
→ atomic one-CI consumption
→ build-and-test
→ Human merge
```

Exact binding covers Owner, repository, PR, base branch/SHA, exact head SHA, canonical changed-file-set hash, canonical diff/review digest, `AUTHORIZE_PR_CI`, challenge/approval freshness and replay state.

Parallel Systemadmin Workstream

PR #220 implemented the SA3B GitHub-Actions/OIDC host. PR #222 corrected the M5 application schema contract discovered by the first real probe. SA3B is still **VERIFICATION PENDING** until the Owner-controlled production credential is valid, a positive probe persists authorization before the branch side effect, SUCCESS outcome evidence is correlated, a no-permit/stale negative probe creates no side effect, and the probe branch is deleted.

Until then, direct mutating ChatGPT/Claude/other connector calls bypassing the enforceable host cannot be labeled autonomous DevelopmentChain `VERIFIED PASS` execution.

Mutation State Vocabulary

- `NOT REQUIRED`
- `PLANNED`
- `HUMAN APPROVED`
- `MUTATED`
- `VERIFIED PASS`
- `FAILED / ROLLED BACK`

`Documentation Ready` is not a mutation state and never implies approval.

Closure Rule

No DevelopmentChain phase closes without Authority consistency, required implementation/CI, required external mutation `VERIFIED PASS`, positive and negative verifi-

cation, redacted audit/evidence, rollback state, Human merge where applicable, Finance work branch deletion, synchronized Roadmap/traceability and an explicit Next Gate.

Traceability Coverage Report

Quelle: docs/traceability/COVERAGE_REPORT.md

Traceability Coverage Report

Automatisch generiert von `src/platform/Traceability`. Nicht manuell bearbeiten.

Generiert am: 2026-08-14T15:47:15.177Z

Kennzahlen

Kennzahl	Wert
ESS-Eintraege gesamt	24
ESS mit verknuepfter Komponente	11 (46 %)
Plattformkomponenten gesamt	26
davon implementiert/in Entwicklung	11
davon mit ESS-Referenz	26
davon mit ADR-Referenz	12
davon mit Testabdeckung	10 (38 %)

Komponenten im Detail

Komponente	Status	ESS	ADR	Tests
Architecture	unspecified	2	0	0
Compliance	implemented	3	1	7
Contracts	unspecified	2	0	0
Core	unspecified	2	0	0
Discovery	unspecified	2	0	0
Documentary	development	10	2	9
Documentary/Go- vernance	unspecified	7	3	0
EventMesh	implemented	5	1	4
Events	unspecified	2	0	0
Generators	unspecified	2	0	0
Interfaces	unspecified	2	0	0
Knowledge	unspecified	2	0	0
Models	unspecified	2	0	0
PlatformDirector	development	5	1	2

Komponente	Status	ESS	ADR	Tests
Plugins	unspecified	2	0	0
Quality	unspecified	2	0	0
Registry	unspecified	2	0	0
Release	development	3	1	1
Security	implemented	3	1	15
Shared	unspecified	2	0	0
Supervisor	implemented	2	1	5
Telemetry	development	6	1	1
Traceability	development	6	3	1
Validators	unspecified	2	0	0
VersionManager	implemented	3	2	0
Vocabulary	implemented	5	1	1

Befunde (16)

Typ	ID	Detail
ess-without-component	ESS-0001	ESS-0001 ("Documentary & Code Intelligence Architect") hat kein implementedBy in der ESS-Registry.
ess-without-component	ESS-0001-CONTRACTS	ESS-0001-CONTRACTS ("Enterprise Technical Contracts") hat kein implementedBy in der ESS-Registry.
ess-without-component	ESS-0002	ESS-0002 ("Supervisor Architect") hat kein implementedBy in der ESS-Registry.
ess-without-component	ESS-0003	ESS-0003 ("Platform Director") hat kein implementedBy in der ESS-Registry.
ess-without-component	ESS-0010	ESS-0010 ("Documentary Engine") hat kein implementedBy in der ESS-Registry.
ess-without-component	ESS-0014	ESS-0014 ("Google Marketing MCP Governance") hat kein implementedBy in der ESS-Registry.
ess-without-component	ESS-0014-CONTRACTS	ESS-0014-CONTRACTS ("Google Marketing Integration & Protected Change Contracts") hat kein implementedBy in der ESS-Registry.
ess-without-component	ESS-0015	ESS-0015 ("API Interface Inventory & QA Governance") hat kein implementedBy in der ESS-Registry.

Typ	ID	Detail
ess-without-component	ESS-0016	ESS-0016 ("Enterprise Market Data Provider & MCP Governance") hat kein implementedBy in der ESS-Registry.
ess-without-component	ESS-0017	ESS-0017 ("Vocabulary Governance") hat kein implementedBy in der ESS-Registry.
ess-without-component	ESS-0017-CONTRACTS	ESS-0017-CONTRACTS ("Vocabulary Governance Contracts") hat kein implementedBy in der ESS-Registry.
ess-without-component	ESS-0018	ESS-0018 ("Agentic Supabase Tool Governance") hat kein implementedBy in der ESS-Registry.
adr-without-component	Documentary:ADR-0046	src/platform/Documentary/manifest.json verweist auf ADR-0046, das nicht in docs/adr/adr_history.json gefuehrt wird.
adr-without-component	Release:ADR-0030	src/platform/Release/manifest.json verweist auf ADR-0030, das nicht in docs/adr/adr_history.json gefuehrt wird.
adr-without-component	Telemetry:ADR-0056	src/platform/Telemetry/manifest.json verweist auf ADR-0056, das nicht in docs/adr/adr_history.json gefuehrt wird.
adr-without-component	Vocabulary:ADR-0046	src/platform/Vocabulary/manifest.json verweist auf ADR-0046, das nicht in docs/adr/adr_history.json gefuehrt wird.

DEVELOPMENT Chain Document Traceability Matrix

Quelle: docs/traceability/DEVELOPMENT_CHAIN_DOCUMENT_TRACEABILITY_MATRIX.md

DEVELOPMENT Chain Document Traceability Matrix

Status: PROPOSED Date: 2026-08-14 Baseline: `main@66da35b80-ba23e4f216318a9cd9f9b4e7b787679` (PR #255 merge)

Zweck

Diese Matrix zeigt für jeden DEVELOPMENT Chain Roadmap-Punkt die normative Authority, Ausführungsdokumente, Machine Contracts, Evidence, Mutation State und das Exit Gate. Sie ergänzt die bestehende `AI_AGENT_M0_M9_TRACEABILITY_MATRIX.md` um die Dokument-/Handoff-Ebene.

Cross-cutting Controls

Control	Authority / Artifact	Zweck
DEVELOPMENT Chain Execution	<code>docs/governance/DEVELOPMENT_CHAIN_EXECUTION_POLICY.md</code>	kanonische Ausführungs- und Mutationsreihenfolge
Branch / Clone Lifecycle	<code>docs/governance/DEVELOPMENT_CHAIN_BRANCH_LIFECYCLE_POLICY.md</code>	fresh branch, Human merge, branch delete, clone/worktree cleanup
Responsibilities	<code>docs/governance/DEVELOPMENT_CHAIN_RESPONSIBILITY_MATRIX.md</code>	Owner / Dev / Prod / Executor / CI Trennung
PR Check Classes	<code>docs/governance/PR_CHECK_CLASSIFICATION.md</code>	D/C/R/M Validation Scope
Human Owner Approval	<code>docs/governance/HUMAN_OWNER_PR_APPROVAL_POLICY.md</code>	current transitional Owner gate until M10 cutover
Autonomous Agent Concept Gate	<code>docs/governance/AUTONOMOUS_AGENT_CONCEPT_GATE.md</code>	privileged agent prerequisites
Mutation Handoff	<code>docs/contracts/DEVELOPMENT_CHAIN_MUTATION_HANDOFF_CONTRACT.md</code>	nicht autorisierende Work Order
Mutation Handoff Schema	<code>.ai/contracts/development-chain-mutation-handoff.schema.json</code>	machine validation
Generic Phase Runbook	<code>docs/runbooks/DEVELOPMENT_CHAIN_PHASE_EXECUTION.md</code>	wiederholbarer Ablauf
Generic Evidence Template	<code>docs/evidence/templates/DEVELOPMENT_CHAIN_PHASE_EVIDENCE_TEMPLATE.md</code>	Mindest-Evidence pro Phase

Phase Matrix

Phase	Execution State	Primary Authority	Execution / Risk Documents	Machine Contract / Implementation	Evidence	Mutation State	Exit Gate
M0	COMPLETE	M0 evidence baseline	docs/evidence/m0/*	read-only	M0 Evidence	NOT REQUIRED	baseline preserved
M1	COMPLETE	Git Guardrails / Owner Policy	PR governance / CODEOWNERS / protection	GitHub policy	guardrail evidence	repository governance only	protected main preserved
M2	COMPLETE	ESS-0019 + ADR-0057..0060	docs/architecture/ai-agent/*	architecture contracts	M2 Evidence	NOT REQUIRED	Documentation Freeze complete
M3	COMPLETE	ADR-0053 + ADR-0060	CI / PR Check Classification	.github/workflows/ci.yml	M3 Evidence	repository workflow	one bounded build-and-test preserved
M4	COMPLETE	ADR-0058 + ESS-0018/0019	IAM / risk / capability models	Agent IAM / PolicyGate	M4 Evidence	NOT REQUIRED external	negative IAM PASS
M5	VERIFIED PASS	ADR-0056 + ADR-0059	M5 Evidence + PR #222 corrective contract	public.agent_server/agentAudit/*	docs/evidence/m5/its*	schema VERIFIED ; app runtime verification pending	real privileged audit insert PASS required before autonomous mutation reliance
M5A	VERIFIED PASS	ESS-0020 + ADR-0064 + ADR-0003.5	docs/run-books/M5A_SUPABASE_TOTP_AAL2-HARDENING + docs/roadmaps/M5A_SYSTEM-ADMIN_REPOSITORY_WORK_PACKAGE.md	Native Supabase MFA/AAL2 code (direkt implementiert, REM-M5A-REPOSITORY-001 bleibt DRAFT / nicht aktiviert)	docs/evidence/m5a/M5A_VERIFIED_PASS_CLOSURE_EVIDENCE.md	repo MERGED ; beide Owner-Profile mit Closure-Evidenz, System nativen Faktor + aal2-Session read-only bestätigt	erreicht — M6 unblocked
M6	VERIFIED PASS	ADR-0060	AI_AGENT_SUPPORT_CHAIN docs/run-books/	SUPPOM/CHAIN governance/ attestation outputs implement-	docs/evidence/m6/M6_REPOSITORY_IMPLEMENT-	MERGED, VERIFIED ON HOSTED	source→artifact→attestation trace VERIFIED

Phase	Execution State	Primary Authority	Execution / Risk Documents	Machine Contract / Implementation	Evidence	Mutation State	Exit Gate
			M6_SUP-PLY_CHAIN_P	ted, sup-PLY-ANCE.md chain-at-testati-on CI job merged (push+main only); first real push run found actions/ attest-build-prove-nance blocked for user-owned private repos, fixed with cosign keyless signing (Sig-store Fulcio/Rekor); second real push run (31834114193) signed and, in the same run, verified the signature against the exact expected certificate identity and OIDC issuer	MENTATI-ON_EVI-DENCE.md	BUILD PATH	PASS — met
M7	COMPLETE / VERIFIED PASS (2026-08-14)	ADR-0061	AI_AGENT_DEPLOYMENT_I-DENTI-TY_MUTA-TION.md	Deployment Render pre-flight done; prove-nance-gated deploy merged and confirmed on a real push; verify-deploy-	docs/evi-dence/m7/M7_PHA-SE0_AND_R-EPOSITO-RY_CON-TROLS_EVIDEN-9 docs/run-books/M7_DE-PLOY_H00K_R	MERGED, ALL RE-QUIRED MUTATIONS VERIFIED PASS, ALL EXIT, GATE CRI-TERIA MET (docs/run-books/INDOFF.md , M7_DE-	all required platform mutations VERIFIED PASS — met

Phase	Execution State	Primary Authority	Execution / Risk Documents	Machine Contract / Implementation	Evidence	Mutation State	Exit Gate
				ment-identity CI job failed its first real run (JSON-body-only read, production identity is header-only), root cause found and fixed, second real push run confirmed it end-to-end against the live Render deploy; Owner performed a real deploy-hook rotation, independently confirmed via GitHub Actions log, Render deploy history, and verify-deployment-identity PASS; Owner performed a real Render Dashboard rollback to a prior deploy plus its roll-forward to current main (via merge of PR #277), both inde-	books/ M7_ROLL-BACK_VE-RIFICATI-ON_HAN-DOFF.md , docs/evidence/m7/M7_REQUIRE-NEGATIVE_TESTS	PLOY-MENT_I-DENTI-TY_MUTATION.md "Exit Gate Closure") EVIDENCE.md	

Phase	Execution State	Primary Authority	Execution / Risk Documents	Machine Contract / Implementation	Evidence	Mutation State	Exit Gate
				pendently confirmed (GitHub Actions log, Render deploy history, verify-deploy-identity PASS); all 10 Required Negative Tests now have a concrete, automated, passing test (docs/evidence/m7/M7_REQUIRED_NEGATIVE_TESTS_EVIDENCE.md)			
M8	COMPLETE / VERIFIED PASS (2026-08-16, all 9 Exit Gate items PASS)	ADR-0062 + ESS-0019	Provider Profile Contract + docs/runbooks/M8_AGENT_CUTOVER.MD	Provider Registry/ Policy-Equivalence verified; PR #308/ SA4 caller wired; rollback-to-read-only verified; PR #308 binds request/ trace/ session correlation plus provider/model/ tool attribution across authorization and outcome with tamper/ missing-identity	docs/evidence/m8/M8_CLOSURE_EVIDENCE.md ; docs/evidence/m8/M8_EXTER-NAL_HOST_CORRELATION_VERIFIED_EVIDENCE.md ; docs/evidence/m8/M8_PHASE0_AND_PROVIDER_PRO-FILE_EVIDENCE.md ;	PR #308 MERGED; POINT 7 VERIFIED PASS; POINT 4 VERIFIED PASS; POINT 2 OWN-ER_AC-CEPTED 2026-08-16; M8 CLOSED – M9 unblocked, drill se-parately Owner-au-thorized	equivalent provider policy + rollback + audit correlation + by-pass-route-audit + external-host-config + Exit-Gate-2 scope decision — all VERIFIED PASS

Phase	Execution State	Primary Authority	Execution / Risk Documents	Machine Contract / Implementation	Evidence	Mutation State	Exit Gate
				DENY tests; full repository-wide router-mount audit confirmed Exit-Gate-Punkt 4 / Cutover-Sequenz-Punkt 6 vacuously satisfied; external-HostCon-figurati-onVeri-fied for chatgpt-github-connec-tor veri-fied read-only 2026-08-16 → READY on all 6 fields; Exit-Gate-Punkt 2 closed under Owner-accepted scope decision (2026-08-16) — supported privileged providers = mutating providers with a production host (chatgpt-github-connec-tor only); canonical provider set correc-ted	docs/evidence/m8/M8_AU-DIT_COR-RELATI-ON_E-XIT_GA-TE_7_EVI-DENCE.md ; docs/evidence/m8/M8_PROVI-DER_BY-PASS_ROU-TE_AU-DIT_E-XIT_GA-TE_4_EVI-DENCE.md ; docs/evidence/m8/M8_I1_CU-TO-VER_REA-DI-NESS_MA-TRIX_AND_E-XIT_GA-TE_SYNC_EVIDENCE.md		

Phase	Execution State	Primary Authority	Execution / Risk Documents	Machine Contract / Implementation	Evidence	Mutation State	Exit Gate
				2026-08-16 (PR #365) to ChatGPT/ Claude/ Grok — <code>claude-code-cli</code> and <code>grok-xai-connec-tor</code> both structurally BLOCKED, explicitly not supported privileged paths; <code>google-ai-studio / notebook-klm / gemini</code> now RETIRED (DENY)			
M9	BLOCKED BY M8	ADR-0063	Incident Response + <code>docs/runbooks/M9_ASSURANCE_INCIDENT_BREAK_GLASS.md</code>	kill-switch / audit / recovery controls	<code>docs/evidence/m9/*</code>	PLANNED / BLOCKED	all drills PASS; no unowned CRITICAL control
M10	BLOCKED BY M9	ADR-0066 + ESS-0022	M10 Threat Model + <code>docs/runbooks/M10_PASSKEY_OWNER_PR_AUTHORIZATION.md</code>	PR-bound WebAuthn approval contract/ service	<code>docs/evidence/m10/*</code>	PLANNED / BLOCKED	passkey-only gate VERIFIED PASS; legacy auth removed

Requirement-to-Evidence Rules

Repository change

```
Roadmap Item
→ Authority Ref
→ Branch
→ Commit / PR Head
→ Human Review
→ Required CI
→ Human Merge
→ Merge SHA
→ Branch Deleted
```

External platform mutation

```
Mutation Requirement
→ Exact Target
→ Pre-Mutation Check
→ Human Mutation Approval Evidence
→ Handoff Contract ID
→ REM / IAM / Execution Host Decision
→ Authorization Audit Reference
→ Side Effect
→ Outcome Audit Reference
→ Post-Mutation Positive/Negative Verification
→ Rollback State
→ Final Mutation State
```

Documentation Readiness vs Execution State

- **Documentation Ready** — Planungs-/Runbook-Artefakte vorhanden;
- **Execution Unblocked** — Vorgänger-Exit-Gate erfüllt;
- **Mutation Approved** — separate Owner-Mutation-Autorisierung vorhanden;
- **Verified Pass** — tatsächliche Ausführung und Verifikation abgeschlossen.

Documentation Ready ist weder Mutation Approved noch Verified Pass.

M5 / SA4 verified execution trace

The M5 persistence and application writer path, the SA3B GitHub-Actions/OIDC execution host and the SA4 policy-validation pilot are recorded as **COMPLETE / VERIFIED PASS** in the canonical Systemadmin roadmap and traceability matrix.

The Development Chain therefore records:

- M5 production schema/persistence and application writer: **VERIFIED PASS**;
- SA3B execution host: **COMPLETE / VERIFIED PASS**;
- SA4 policy-validation pilot: **COMPLETE / VERIFIED PASS**;
- SA5 external production mutation: **DENY until M10 VERIFIED PASS**.

M5A Systemadmin Repository Package Trace

The next bounded Systemadmin activity is repository-only M5A implementation under:

- work package: docs/roadmaps/M5A_SYSTEMADMIN_REPOSITORY_WORK_PACKAGE.md ;
- mandate draft: .ai/mandates/REM-M5A-REPOSITORY-001.json ;
- authority candidates: ESS-0020 and ADR-0064.

The package is **DRAFT / NOT ACTIVATED**. The permit sequence is:

```
Documentation PR merged
→ Owner accepts ESS-0020 and ADR-0064 for repository implementation
→ mandate baseline updated to exact current main
→ mandate status set to OWNER_APPROVED with approval evidence
→ Systemadmin validates permit and SA3B/SA4 state
→ fresh branch
→ bounded code + tests + evidence
→ Draft PR
→ Human review and merge
```

Explicitly denied by this package: Supabase production factor enrollment or removal, owner IAM/MFA changes, secrets, Render/Stripe/DNS mutations, GitHub ruleset changes, merge, and mandate self-expansion.

Branch Closure Trace

Every merged repository work item needs `branchDeleted=true` or equivalent Evidence. Cloned repositories/worktrees are cleaned after required Evidence retention. Merged or superseded branches are never reused for new work.

Mutation State Vocabulary

- NOT REQUIRED
- PLANNED
- HUMAN APPROVED
- MUTATED
- VERIFIED PASS
- FAILED / ROLLED BACK

Closure

A DEVELOPMENT Chain Roadmap item is closed only when Authority, implementation/Handoff, tests, Evidence, mutation state, Roadmap, Branch Lifecycle and Traceability are consistent and the Next Gate is explicit.

Enterprise Traceability Matrix — Übersicht

Quelle: docs/traceability/ETM.md

Enterprise Traceability Matrix — Übersicht

Document ID

DOC-ETM-0001

Version

1.0.0

Status

Operative Dokumentation

Referenzen

ESS-0011 — Enterprise Traceability (Spezifikation)

ESS-0011-CONTRACTS — Regelwerk

ADR-0015 — Einführung der Komponente

src/platform/Traceability/ — Implementierung

Zweck dieses Dokumentensatzes

Die Dokumente unter docs/traceability/ bilden die **operative Ebene** der ETM.

Sie beschreiben Betrieb, Anwendung und Nachvollzug.

Sie enthalten **keine** Spezifikation und **keine** Regeln.

Abgrenzung

Ebene	Ort	Inhalt
Spezifikation	ESS-0011	Architektur, Komponenten, Prozesse, Reports, Integration
Regelwerk	ESS-0011-CONTRACTS	Link Contract, Coverage, Orphans, Events, Validation
Operative Doku	docs/traceability/	Betrieb, Lifecycle, Anwendung, Nachvollzug
Implementierung	src/platform/Traceability/	Code

Bei Abweichungen gilt ausnahmslos ESS-0011 beziehungsweise ESS-0011-CONTRACTS.

Diese Trennung ist verbindlich, um eine Doppelregelung zu vermeiden — sie entspricht der Verantwortungsabgrenzung aus ARCH-RESP-0001 und dem Grundsatz Zero Duplication aus ESS-0001.

Dokumentensatz

Dokument	Inhalt
ETM.md	dieses Dokument — Übersicht und Abgrenzung
Traceability-Architecture.md	Aufbau im Betrieb, Datenflüsse, Quellen
Traceability-Lifecycle.md	Lebenszyklus eines Matrixeintrags
Traceability-Governance.md	Zuständigkeiten, Rollentrennung, Eskalation
Traceability-Validation.md	Prüfungen und Abnahmekriterien
Traceability-Versioning.md	Versionierung der Matrix
Traceability-Reports.md	Berichte und ihre Verwendung

Was die ETM beantwortet

Die ETM beantwortet lückenlos und bidirektional:

Welche Anforderung, welche Entscheidung, welche Regel führte zu welchem Artefakt — und welches Artefakt erfüllt welche Regel?

Vorwärts

ESS-Kapitel → ADR → Component → Interface → Event → Test

Rückwärts

Test → Event → Interface → Component → ADR → ESS-Kapitel

Was die ETM nicht tut

Die ETM erzeugt keine Regeln.

Die ETM erzeugt keine Dokumentation.

Die ETM verändert keine Quelle.

Die ETM bewertet nicht.

Die ETM entscheidet nicht.

Sie verknüpft und misst.

Aktueller Stand

Die ETM ist **spezifiziert und strukturell angelegt, nicht implementiert**.

Ein Matrixaufbau ist derzeit nicht möglich, da Knowledge Graph und Digital Twin fehlen — beide sind Pflichtquellen. Der Versuch würde nach den eigenen Regeln `GOV-KG-001` und `GOV-TWIN-001` abbrechen.

Der Umsetzungspfad ist in `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md` (Stufen 1 bis 5) festgelegt.

End of Document

DOC-ETM-0001

Marketing Agent Traceability & Gate Matrix

Quelle: docs/traceability/MARKETING_AGENT_TRACEABILITY_MATRIX.md

Marketing Agent Traceability & Gate Matrix

Status: DRAFT Date: 2026-08-12 Baseline: `main@3a733f2b6efeffb3013fbb2c558b5ef4c185125e`

Logical agent: `capital-ai-marketing-roadmap-executor`

Purpose

Map the proposed Marketing Agent and Autonomous Content Engine to existing CAPITAL-AI authorities, implementation phases, required tests and external best-practice references.

This matrix is non-authorizing. It must be synchronized with canonical Traceability registries before implementation enablement.

1. Core traceability

Requirement	Source authority	Proposed artifact	Implementation gate	Evidence / test
Provider is not trust root	ESS-0019	ESS-0022 §3	MA1	subject/provider separation test
Separate Marketing identity	ESS-0019, ADR-0058	ESS-0022 / ADR-0068	MA1	wrong-agent DENY
No inherited Systemadmin authority	ESS-0021, ADR-0065	ADR-0068	MA1	Systemadmin REM cannot authorize Marketing subject
Exact capabilities	ESS-0019	Execution Policy	MA1	missing-capability DENY
Exact path scope	ESS-0021 pattern	Execution Policy	MA1	wrong-path DENY
Expiring/revocable authority	ESS-0021 pattern	Marketing mandate model	MA1	expired/revoked DENY
Kill switch	ESS-0019/0021	ESS-0022 / Execution Policy	MA1	active kill switch DENY
Audit before side effect	ADR-0059 / SA3	Execution Policy	MA5	audit fail -> zero mutation
Human-only merge	AGENTS / ADR-0039	ESS-0022 / Execution Policy	all mutating phases	merge capability absent
Fresh branch / delete after merge	AGENTS / DevelopmentChain	Roadmap / Execution Policy	MA5+	branch lifecycle evidence
Generation separated from publishing	ADR-0026 / ADR-0068	Content Engine Architecture	MA2+	generator has no publish capability
Existing publisher retained	ADR-0026	Content Engine Architecture	MA2-MA6	regression tests

Requirement	Source authority	Proposed artifact	Implementation gate	Evidence / test
Source-grounded financial claims	AGENTS No-Fake-Data	ESS-0022 / Content Architecture	MA3	unsupported claim DENY
AI provenance	governance requirement	Content Architecture	MA3	provenance completeness tests
Human content approval	SEO Roadmap N4	ESS-0022 / Content Architecture	MA3+	hash mismatch DENY
Renderer least privilege	ADR-0068	Content Architecture	MA4	no credential access
SSRF-safe media fetch	Security baseline	Content Architecture	MA4	private/local URL DENY
External publish separately controlled	ADR-0068	Roadmap MA6	MA6	no publish without approval
Analytics does not grant authority	ESS-0019	Roadmap MA7	MA7	feedback cannot publish

2. Roadmap gate dependencies

Gate	Depends on	Required result
MA0	Deep Research + repo read	Human-accepted documentation package
MA1	MA0	Marketing subject/mandate negative tests PASS
MA2	MA1 design	Content contracts/tests PASS
MA3	MA2	provenance/compliance/approval negative tests PASS
MA4	MA3	rendering isolation + asset security PASS
MA5	MA4 + reusable SA host foundation VERIFIED PASS	one bounded repo pilot PASS
MA6	MA5 + strong Owner approval architecture	separate external publishing ADR/control PASS
MA7	MA6	analytics feedback safely separated from authority

3. Shared Integration Zone coordination

Resource	Default Marketing ownership	Required coordination
src/platform/SocialMediaEngine/**	candidate yes	exact mandate + overlap check
server/socialMedia/**	candidate yes	exact mandate + security review
docs/seo/**	yes	normal docs governance

Resource	Default Marketing ownership	Required coordination
src/platform/SeoEngine/**	future yes	architecture/ADR where needed
AGENTS.md	no	Human/shared integration writer
package.json , lock	no	explicit shared lease
.github/workflows/**	no	Systemadmin/Security coordination
global IAM/Security	no	Security/Systemadmin authority
global Event Mesh registry	no	explicit integration work package
global Traceability registry	no	explicit integration work package
root server/app bootstrap	no	architecture/backend integration
deployment config	no	Systemadmin/production handoff
shared Supabase migrations	no implicit ownership	explicit data/production mutation gate

4. Security negative-test matrix

Test	Expected result
Retrieved article contains tool instruction	treat as data; no capability change
Retrieved page requests secret/token	ignore/redact; no disclosure
Wrong logical agent id	DENY
Marketing agent supplied Systemadmin REM	DENY
Expired Marketing mandate	DENY
Revoked mandate / kill switch	DENY
Path outside allowlist	DENY
Shared-zone file not explicitly authorized	STOP/DENY
Risk exceeds mandate ceiling	DENY
Audit persistence fails	zero protected side effect
Quantitative finance claim lacks source evidence	DENY
Source is synthetic/heuristic but text says live fact	DENY
Content provenance incomplete	no external publish
Content hash changed after approval	DENY publish
Asset hash changed after approval	DENY publish
Target platform changed after approval	require new approval
Cross-user content/asset id	DENY
mediaUrl resolves to loopback/private/link-local	DENY
Media MIME/magic bytes mismatch	QUARANTINE/DENY
Rendering provider unavailable	honest FAILED, no fake success

Test	Expected result
Renderer attempts social publish	capability/credential unavailable
Analytics recommendation attempts direct publish	DENY

5. Best-practice crosswalk

NIST AI RMF / Generative AI Profile

Application:

- GOVERN: Owner accountability, ESS/ADR, explicit agent identity and mandate;
- MAP: Marketing use case, financial claim risk, provider/data boundaries;
- MEASURE: source quality, compliance decisions, content performance, security tests;
- MANAGE: kill switch, approval, rollback, provider isolation and staged rollout.

NIST SSDF

Application:

- integrate Marketing Agent work into the existing branch/PR/CI/review lifecycle;
- do not treat AI-generated code/content infrastructure as trusted by origin;
- test adapters, dependencies and supply-chain artifacts;
- retain evidence for defects and corrective action.

ISO/IEC 42001

Application:

- accountable owner;
- documented AI purpose/scope;
- risk treatment and controls;
- traceability/transparency;
- continual improvement via measured feedback without removing governance.

OWASP Agentic Security Initiative

Application:

- retrieved content/tool output is untrusted;
- least-privilege tools;
- explicit agent identity and capability boundaries;
- prevent privilege/self-mandate expansion;
- isolate external renderers;
- protect against unsafe inter-agent/shared-resource coupling;
- fail closed on control/audit uncertainty.

6. Evidence retention classes

Security authorization evidence

Examples:

- mandate/approval reference;
- policy decision;
- audit reference;
- execution-host subject;
- branch/commit/PR;
- terminal result.

Retained separately from normal sampled telemetry.

Content business evidence

Examples:

- source references/digest;
- generated content hash;
- asset hashes;
- provenance;
- compliance decision;
- content approval;
- publish log references.

Operational telemetry

Examples:

- latency;
- provider/model/template version;
- safe token/cost metrics;
- render duration;
- failure class.

Full secrets, bearer tokens and raw sensitive prompts are excluded/redacted.

7. Documentation package completeness

MA0 package expected before implementation:

- [X] `.ai/skills/ESS-0022-Marketing-Roadmap-Executor.md` draft
- [X] `docs/adr/ADR-0068-marketing-roadmap-executor-and-content-automation-boundary.md` draft
- [X] `docs/roadmaps/MARKETING_AGENT_ROADMAP.md` draft
- [X] `docs/governance/MARKETING_AGENT_ROADMAP_EXECUTION_POLICY.md` draft

- [x] docs/architecture/AUTONOMOUS_CONTENT_ENGINE_ARCHITECTURE.md draft
- [x] this traceability/gate matrix
- [x] inactive machine-readable Marketing execution profile
- [] canonical ESS Registry update after Human review
- [] canonical ADR/traceability index update after Human review
- [] docs/architecture/ROADMAP.md integration after Human review
- [] M0-M10 traceability integration where required
- [] first Marketing mandate schema/validator in MA1

8. Current authorization state

READ / ANALYZE / PLAN conceptual profile only.

No document in this package authorizes runtime repository mutation, CI requests, deployment, production mutation, Social publishing or merge.

Systemadmin Agent Traceability Matrix

Quelle: docs/traceability/SYSTEMADMIN_AGENT_TRACEABILITY_MATRIX.md

Systemadmin Agent Traceability Matrix

Status: IMPLEMENTATION PHASE Date: 2026-08-12 Repository baseline:

main@6205868da833a6ee75b5301e78b0a2e6a118c411

Stage	Authority	Implementation / Evidence	Mutation boundary	Exit gate
SA0 Governance	ESS-0021 + ADR-0065	PR #214	repository governance	COMPLETE
SA1 Mandate Validator	ESS-0021 + ADR-0065 + ADR-0058	PR #215; expanded trust root in SA4 bootstrap	repository only; production denied	COMPLETE / VERIFIED PASS
SA2 Chat Profile	ESS-0021 + SA1	PR #216	mutating LIVE envelope alone denied	COMPLETE / VERIFIED PASS
SA3A Audit Adapter	ADR-0059 + ADR-0065	PR #218; M5 writer corrected by #222	no permit before durable M5 audit	COMPLETE / VERIFIED PASS
SA3B Execution Host	ADR-0067	PR #220 + #222; Issues #221/#223/#224; probe branch cleanup verified	BRANCH host proof only	COMPLETE / VERIFIED PASS
SA4 Bounded Pilot	ADR-0068 + REM-SA4-PILOT-001	PR #226; Issue #228 / run 31579519025 ; PR #229; six M5 events	exact doc path; BRANCH/COMMIT/ Draft PR only	COMPLETE / VERIFIED PASS
M5A Repository Package	ESS-0020 + ADR-0064 + REM-M5A-REPOSITORY-001	Work-package + DRAFT mandate in governance PR	repository code/ tests/evidence only; production denied	OWNER REVIEW + REM ACTIVATION REQUIRED
SA5 External Mutation	future ADR + M10	not implemented	production mutation prohibited	BLOCKED BY M10 VERIFIED PASS

Canonical evidence chain

Every Systemadmin repository side effect must be reconstructable as:

Owner-approved REM → Owner issue → trusted main workflow → GitHub workload identity → exact capability/path/head → durable authorization auditReference → exact side effect → durable terminal outcomeReference

Direct ChatGPT→GitHub connector writes remain outside the autonomous Systemadmin mutation path.

SA3B production proof

Fail-closed audit outage

Issue #221 / run 31570833507 :

AUDIT PERSISTENCE FAILURE → NO PERMIT → NO BRANCH

Positive permit-before-side-effect

Issue #223 / run 31574111075 on exact main@91963f59b74c8c3c3c0b33c6a23237a01ac0128e :

Correlation	Value
capability	BRANCH
branch	agent/sa3b-host-probe-20260812b
authorization	supabase:agent_audit_events:194f1198-492c-4d4f-b1e4-0c13a5d99d20
outcome	supabase:agent_audit_events:5ab9eefe-f472-4396-a7e7-2a3ceb029e34
outcome result	SUCCESS
branch SHA	91963f59b74c8c3c3c0b33c6a23237a01ac0128e

Read-only Supabase verification confirmed both real append-only rows and the outcome→authorization reference.

Stale-base negative path

Issue #224 / run 31574221718 :

STALE BASE → DENY BEFORE OIDC/BROKER → NO BRANCH

Requested negative branch was independently absent.

SA3B lifecycle closure

The successful probe branch agent/sa3b-host-probe-20260812b was deleted before the live SA4 pilot. The SA4 preflight confirmed the ref was absent before OIDC acquisition or any mutation-permit request.

SA3B therefore satisfies both runtime verification and post-proof branch lifecycle cleanup.

SA4 authority trace

Mandate

.ai/mandates/REM-SA4-PILOT-001.json

Field	Required
Owner	SvenKulessa

Field	Required
agent	capital-ai-systemadmin-roadmap-executor
repository/base	SvenKulesa/Finance / main
roadmap item	SA4-FIRST-AUTONOMOUS-WORK-PACKAGE
capabilities	BRANCH , COMMIT , PR
exact path	docs/evidence/sa4/ SA4_FIRST_AUTONOMOUS_WORK_PACKAGE.md
mutation class	REPOSITORY
max risk	MEDIUM
max open SA4 PRs	1
expiry	<= 7 days
kill switch	Owner-controlled / required

All reserved mutation classes, including `MERGE` , remain prohibited.

SA4 ingress trace

`validateSa4PilotIssue.mjs` accepts only:

- `version=1.0 ;`
- `mode=BOUNDED_DOC_PR ;`
- `mandateId=REM-SA4-PILOT-001 ;`
- `roadmapItem=SA4-FIRST-AUTONOMOUS-WORK-PACKAGE ;`
- full lowercase base SHA;
- `agent/sa4-pilot-*` branch.

Unknown fields, file payloads, arbitrary commands, alternate paths/modes and unsafe branches are denied.

SA4 host identity trace

The OIDC verifier accepts exactly two workflow refs on `refs/heads/main` :

1. `.github/workflows/systemadmin-roadmap-executor.yml` for `REM-SA3B-PROBE-001` ;
2. `.github/workflows/systemadmin-sa4-pilot.yml` for `REM-SA4-PILOT-001` .

The broker separately binds workflow ref ↔ mandate. Cross-stage token/mandate substitution returns 403.

All existing Owner/repository immutable ID, audience, issuer, signature, event and ref checks remain required.

SA4 permit-before-side-effect matrix

Operation	Pre-side-effect control	Side effect	Terminal evidence
BRANCH	SA1/SA2/SA3 + M5 BRANCH authorization	create exact <code>agent/sa4-pilot-*</code> branch	BRANCH SUCCESS/ERROR outcome
COMMIT	exact path + branch/head + targeted prevalidation + M5 COMMIT authorization	create deterministic evidence file only	COMMIT outcome with commit SHA
PR	open-PR inventory + exact path/head + M5 PR authorization	create one draft PR to <code>main</code>	PR outcome with PR number

No earlier permit may authorize a later capability.

SA4 live pilot trace

Runtime correlation

Correlation	Value
Owner trigger	Issue #228
workflow run	31579519025
workflow result	SUCCESS
exact base	f7dfcda36905d9a55d74f57f2140224928960379
branch	agent/sa4-pilot-proof-20260812b
autonomous commit	02f012e71106d5ffd9a4baa3e6f3eba7160eb55d
autonomous Draft PR	#229
exact path	docs/evidence/sa4/ SA4_FIRST_AUTONOMOUS_WORK_PACKAGE.md

Durable M5 authorization/outcome pairs

Capability	Authorization auditReference	Outcome auditReference	Result
BRANCH	supaba-se:agent_audit_events:1b4b04cb-a86b-4612-9ef5-308e95a18c95	supaba-se:agent_audit_events:e99b9af7-74cc-4693-966f-c9b85102035d	SUCCESS
COMMIT	supaba-se:agent_audit_events:3c5916a1-d8c1-4ba1-9de1-d839fcc1bc85	supaba-se:agent_audit_events:bc6ecf0b-86db-4b8c-ae95-2c963a0776f5	SUCCESS
PR	supaba-se:agent_audit_events:		SUCCESS / PR #229

Capability	Authorization auditReference	Outcome auditReference	Result
	41874d2e-a7b7-48c9-8287-71d75-bea7d05	supaba-se:agent_audit_events:d4722de8-3242-4822-ab7d-f353880312ac	

Direct read-only Supabase verification confirmed all six correlated append-only rows. The PR outcome carries `pull_request_number=229` ; the COMMIT outcome carries the exact autonomous commit SHA.

Deterministic content boundary

The Issue cannot provide Markdown/file payload. `runSa4Pilot.mjs` generated the evidence file from trusted metadata and verified the committed bytes by SHA-256 before recording COMMIT SUCCESS.

PR #229 changed exactly one file with one autonomous commit and no additional repository path.

Concurrent writer boundary

Before COMMIT and again before PR authorization, the trusted host queried open PR changed files through GitHub and supplied them to the SA1 policy. Exact target-path overlap is DENY.

The successful live pilot had no concurrent PR overlap on `docs/evidence/sa4/SA4_FIRST_AUTONOMOUS_WORK_PACKAGE.md`.

Trust-root boundary

SA1 and SA3 self-authority protection include the SA3B/SA4 workflows, mandates, OIDC verifier, broker, audit adapter and SA4 parser/runner. A Systemadmin REM cannot authorize modifications to these paths.

Rollback boundary

The trusted runner may delete only its exact `agent/sa4-pilot-*` branch as bounded rollback. If a Draft PR was already created and its outcome cannot be persisted, the host closes that exact Draft PR and deletes its exact branch before failing.

Rollback never targets `main`, repository protection, another PR or production infrastructure.

Human boundary

- `CI_REQUEST` was not part of `REM-SA4-PILOT-001` ;
- the autonomous host did not approve or merge PR #229;
- final CI remained behind Human/Owner current-head review and attestation;

- Governance #662 completed SUCCESS ;
- final CI #965 completed SUCCESS ;
- MERGE remained Human-only;
- Human merge produced main@2e86d5fbc54f9b5ea2af4e6db33e9749c2ac15dd ;
- pilot branch deletion remained mandatory and was verified after merge.

SA4 activation gate — SATISFIED

1. bootstrap PR #226 final CI PASS and Human merge: PASS;
2. bootstrap branch deletion: PASS;
3. successful SA3B probe branch deletion: PASS;
4. bootstrap runtime deployed to Render: PASS;
5. no exact target-path overlap before live execution: PASS.

SA4 VERIFIED PASS exit gate — SATISFIED

1. fresh Owner Issue #228 bound to exact current main : PASS;
2. BRANCH authorization persisted before side effect: PASS;
3. BRANCH SUCCESS outcome persisted: PASS;
4. COMMIT authorization persisted before deterministic file mutation: PASS;
5. committed bytes verified by SHA-256: PASS;
6. COMMIT SUCCESS outcome persisted with 02f012e71106d5ffd9a4baa3e6f3eba7160eb55d : PASS;
7. PR authorization persisted before Draft-PR creation: PASS;
8. Draft PR #229 created from the exact pilot commit: PASS;
9. PR SUCCESS outcome persisted with pull_request_number=229 : PASS;
10. no production mutation, autonomous final CI request or autonomous merge: PASS;
11. Human review and final CI #965: PASS;
12. Human merge to 2e86d5fbc54f9b5ea2af4e6db33e9749c2ac15dd : PASS;
13. agent/sa4-pilot-proof-20260812b deleted after merge: PASS;
14. Issue #228 closed with reason completed : PASS;
15. roadmap and traceability synchronized: completed by this SA4 status-sync change set.

SA5 boundary after SA4

SA4 is no longer a prerequisite blocker. SA5 remains **BLOCKED BY M10 VERIFIED PASS**.

Completion of SA4 does not authorize external production mutation. Any future SA5 design still requires separate authority, exact-target and rollback controls, and strong M10 Owner assurance.

M5A bounded repository package trace

Dimension	Binding
Roadmap item	M5A-NATIVE-MFA-AAL2-REPOSITORY-IMPLEMENTATION
Executor	capital-ai-systemadmin-roadmap-executor
Mandate	.ai/mandates/REM-M5A-REPOSITORY-001.json
Initial mandate state	DRAFT — no execution authority
Work-package	docs/roadmaps/M5A_SYSTEMADMIN_REPOSITORY_WORK_PACKAGE.md
Mutation class	REPOSITORY only
Risk ceiling	HIGH
Open PR limit	1
Production mutation	DENY
Merge	Human-only
Next activation	exact-current-main binding + Owner acceptance + durable audit preflight

Required permit sequence after activation:

BRANCH authorization/outcome → COMMIT authorization/outcome → PR authorization/outcome → Human review → CI → Human merge → branch delete

The package cannot enroll/remove/reset Supabase factors, mutate Auth settings, read secrets, deploy Render, alter billing/DNS, weaken repository protection or expand its own authority. Productive M5A enrollment remains a later separate Human/Owner gate.

Traceability Architecture — Betriebssicht

Quelle: docs/traceability/Traceability-Architecture.md

Traceability Architecture — Betriebssicht

Document ID

DOC-ETM-0002

Version

1.0.0

Referenz

ESS-0011 Chapter 2 und Chapter 3 (verbindliche Spezifikation)

Zweck

Dieses Dokument beschreibt die ETM aus **Betriebssicht**: welche Daten woher kommen, in welcher Reihenfolge sie verarbeitet werden und was bei fehlenden Quellen geschieht.

Die verbindliche Architekturdefinition steht in ESS-0011 Chapter 2 und wird hier nicht wiederholt.

Datenfluss im Betrieb

```
Repository
  ↓
Documentary Engine (ESS-0010)
  ↓
Knowledge Graph ─┐
Enterprise Registry ─┐ TraceabilityBuilder
Digital Twin ─┐
ESS Registry ─┐
Exception Registry ─┐
docs/adr/ ─┐
Validierungsergebnisse ─┐
  ↓
Matrix (.ai/knowledge/traceability/matrix.json)
  ↓
CoverageAnalyzer → coverage.json
OrphanDetector → orphans.json
  ↓
TraceabilityReporter → docs/quality/
```

Die ETM läuft ausschließlich **nach** der Documentary Engine.

Pflichtquellen

Quelle	Fehlt sie, dann
Knowledge Graph	Abbruch — keine Knotenidentität auflösbar
Enterprise Registry	Abbruch — keine Artefaktexistenz prüfbar
Digital Twin	Abbruch — kein Ist-Zustand vergleichbar
ESS Registry	ESS-Achse wird ausgesetzt, Befund
Exception Registry	EXEMPTS-Verknüpfungen fehlen, Befund
docs/adr/	ADR-Achse wird ausgesetzt, Befund
Validierungsergebnisse	Test-Achse wird ausgesetzt, Befund

Ein ausgesetzter Bereich wird **niemals stillschweigend übersprungen** — er erzeugt immer einen Befund.

Verzeichniszuordnung

Verzeichnis	Betriebliche Aufgabe
Core/	Matrixmodell, Identitätsauflösung
Discovery/	Erkennung verknüpfbarer Artefakte
Services/	Ablaufsteuerung eines Matrixlaufs
Registry/	Persistenz der Matrixeinträge
Validators/	Selbstprüfung der Matrix
Reports/	Aufbereitung der Berichtsdaten
Events/	Veröffentlichung der Lauf-Events
Versioning/	Matrixversion und Prüfsummen
Models/ , Interfaces/ , Contracts/	Datenmodelle, Schnittstellen, Verträge

Determinismus im Betrieb

Bei identischem Eingangszustand erzeugt ein Matrixlauf eine **byteidentische** Matrix.

Zeitstempel sind von der Prüfsummenbildung ausgenommen.

Weicht die Prüfsumme bei unverändertem Eingangszustand ab, gilt der Lauf als fehlerhaft und die Ergebnisse werden verworfen.

End of Document

DOC-ETM-0002

Traceability Governance

Quelle: docs/traceability/Traceability-Governance.md

Traceability Governance

Document ID

DOC-ETM-0004

Version

1.0.0

Referenz

ESS-0011 Chapter 6, ESS-0002, ESS-0003, ARCH-RESP-0001

Zweck

Zuständigkeiten und Eskalationswege im Betrieb der ETM.

Rollentrennung

Instanz	Rolle	Darf nicht
ETM	verknüpft und misst	bewerten, entscheiden, Quellen ändern
Governance Validator	prüft Regeln	verknüpfen, entscheiden
Supervisor	bewertet, eskaliert	entscheiden, dokumentieren
Platform Director	entscheidet	implementieren
Version Manager	leitet Version ab	entscheiden über Freigabe

Diese Trennung ist die tragende Governance-Aussage der ETM: Eine Instanz, die misst, darf nicht zugleich bewerten — sonst ist die Messung nicht mehr unabhängig.

Eskalationsweg

Orphan oder Coverage-Unterschreitung
↓
Befund mit Nachweis
↓
Supervisor (ESS-0002) bewertet

↓

bei Critical: Blockade der Freigabe

↓

Platform Director (ESS-0003) entscheidet

↓

Ausnahme nur mit ADR

Die ETM selbst blockiert niemals.

Zuständigkeiten

Gegenstand	Verantwortlich
Matrixaufbau	Traceability-Komponente
Regelwerk	ESS-0011-CONTRACTS
Schwellwerte	ESS-0011-CONTRACTS Chapter 2
Befundbewertung	Supervisor
Ausnahmen	Platform Director, ausschließlich per ADR
Berichtserzeugung	Documentary Engine

AI-Verbindlichkeit

Sämtliche KI-Systeme verwenden **dieselbe zentrale ETM**.

Nicht zulässig sind

- eigene Traceability-Modelle
- lokale Matrixkopien
- abweichende Verknüpfungsarten
- Verknüpfungen ohne Nachweis

Ein KI-System, das Beziehungen benötigt, fragt die ETM ab.

End of Document

DOC-ETM-0004

Traceability Lifecycle

Quelle: docs/traceability/Traceability-Lifecycle.md

Traceability Lifecycle

Document ID

DOC-ETM-0003

Version

1.0.0

Referenz

ESS-0011 Chapter 4 (verbindliche Spezifikation)

Zweck

Lebenszyklus eines Matrixeintrags im Betrieb.

Die verbindlichen Trigger und Workflows stehen in ESS-0011 Chapter 4.

Lebenszyklus einer Verknüpfung

Erkennung	Artefakt wird in einer Pflichtquelle gefunden
↓	
Ableitung	Beziehung wird aus Quelle abgeleitet
↓	
Nachweis	Ursprung wird zugeordnet
↓	
Bewertung	Vertrauenswert Verified oder Derived
↓	
Aufnahme	Eintrag in die Matrix
↓	
Prüfung	Bidirektionale Auflösbarkeit
↓	
Gültig	Eintrag ist Bestandteil der Matrix
↓	
Verwaisung	Endpunkt entfällt → Orphan-Befund
↓	
Entfernung	Eintrag wird beim nächsten Lauf nicht neu erzeugt

Ein Eintrag mit Vertrauenswert `Assumed` wird **niemals** aufgenommen.

Auslöser eines Matrixlaufs

Ereignis	Umfang
KnowledgeUpdatedEvent	inkrementell
TwinSynchronizedEvent	inkrementell
ComponentRegisteredEvent	inkrementell
ExceptionRegisteredEvent	inkrementell
ADR-Änderung	vollständig
ESS-Änderung	vollständig
ReleasePreparedEvent	vollständig

Vor jeder Produktionsfreigabe erfolgt ein vollständiger Lauf.

Inkrementell gegen vollständig

Ein inkrementeller Lauf verarbeitet ausschließlich den geänderten Umfang und dessen Abhängigkeiten laut Knowledge Graph.

Er muss dasselbe Ergebnis liefern wie ein vollständiger Lauf, beschränkt auf den betroffenen Teilgraphen.

Weicht das Ergebnis ab, gilt der inkrementelle Lauf als fehlerhaft.

Umgang mit Orphans

Ein Orphan wird nicht automatisch entfernt.

Er erzeugt einen Befund und bleibt bis zur Behebung sichtbar.

Orphans der Stufe Critical blockieren Produktionsfreigaben.

End of Document

DOC-ETM-0003

Traceability Reports

Quelle: docs/traceability/Traceability-Reports.md

Traceability Reports

Document ID

DOC-ETM-0007

Version

1.0.0

Referenz

ESS-0011 Chapter 5

Zweck

Verwendung der ETM-Berichte im Betrieb.

Die verbindliche Berichtsdefinition steht in ESS-0011 Chapter 5.

Berichte und ihre Adressaten

Bericht	Adressat	Beantwortet
Architecture Traceability Report	Platform Director	Ist die Architektur durchgängig belegt?
ESS Coverage Report	Platform Director	Welche Regeln sind nicht umgesetzt?
ADR Coverage Report	Platform Director	Welche Entscheidungen sind ohne Wirkung?
Component Coverage Report	Supervisor	Welche Komponenten sind unbelegt?
Documentation Coverage Report	Documentary Engine	Welche Artefakte sind undokumentiert?
Test Coverage Report	Quality Center	Welche Komponenten sind ungetestet?
Version Coverage Report	Version Manager	Welche Änderungen sind unversioniert?
Governance Report	Supervisor	

Bericht	Adressat	Beantwortet
		Welche Governance-Lücken bestehen?
Enterprise Readiness Report	Platform Director	Ist eine Freigabe vertretbar?

Erzeugung

Die ETM liefert **Daten**, nicht Dokumente.

Die physische Erzeugung erfolgt durch die Documentary Engine gemäß ESS-0010.

Ablage: docs/quality/

Sämtliche Berichte tragen die Kennzeichnung als generiert gemäß ESS-0001-CONTRACTS Chapter 19.

Enterprise Readiness Report

Der aussagekräftigste Bericht. Er fasst zusammen

- Abdeckungsgrad je Achse
- offene Orphans nach Schweregrad
- Breaking Changes ohne SUPERSEDES
- fehlende Testverknüpfungen
- Matrixversion und Prüfsumme

Er ist die Entscheidungsgrundlage des Platform Director vor einer Produktionsfreigabe.

Interpretationshinweis

Ein hoher Abdeckungsgrad bedeutet **nicht**, dass die Umsetzung korrekt ist.

Er bedeutet ausschließlich, dass eine Beziehung nachweisbar existiert.

Die inhaltliche Korrektheit prüfen Tests, nicht die ETM.

End of Document

DOC-ETM-0007

Traceability Validation

Quelle: docs/traceability/Traceability-Validation.md

Traceability Validation

Document ID

DOC-ETM-0005

Version

1.0.0

Referenz

ESS-0011-CONTRACTS Chapter 5, ESS-0001-CONTRACTS Chapter 12

Zweck

Abnahmekriterien und Prüfungen im Betrieb.

Das verbindliche Regelwerk steht in ESS-0011-CONTRACTS und wird hier nicht wiederholt.

Selbstprüfung vor Veröffentlichung

Vor jeder Veröffentlichung der Matrix wird geprüft

- ✓ sämtliche Verknüpfungen besitzen vollständige Pflichtfelder
- ✓ sämtliche IDs sind auflösbar
- ✓ keine Verknüpfung auf nicht existierende Artefakte
- ✓ keine Assumed-Verknüpfungen
- ✓ keine doppelten Verknüpfungen
- ✓ sämtliche Verknüpfungen bidirektional auflösbar
- ✓ Prüfsumme reproduzierbar
- ✓ Abdeckungsgrade berechnet
- ✓ Orphans klassifiziert

Schlägt eine Prüfung fehl, wird die Matrix nicht veröffentlicht.

Abnahmekriterien der Komponente

Die Komponente gilt als abgenommen, wenn

Kriterium	Nachweis
Matrix reproduzierbar	zwei Läufe, identische Prüfsumme
Bidirektionalität	Stichprobe je Achse in beide Richtungen
Nachweispflicht	kein Eintrag ohne <code>origin</code>
Determinismus	identischer Eingangszustand, identische Ausgabe
Read-Only	keine Schreibzugriffe außerhalb <code>.ai/knowledge/traceability/</code>
Event-Vollständigkeit	jeder Lauf erzeugt Start- und Abschluss-Event

Prüfung durch Dritte

Die Matrix wird zusätzlich geprüft durch

Instanz	Prüfung
Governance Validator (ESS-0012)	GOV-TRACE-001 bis GOV-TRACE-004
Supervisor (ESS-0002)	Vollständigkeit, Aktualität
Contract Tests (Chapter 12)	Schema der Matrixdateien

Die ETM prüft sich selbst — sie ist damit nicht von der Fremdprüfung befreit.

Umgang mit Nichtprüfbarkeit

Fehlt eine Pflichtquelle, meldet die ETM **Nichtprüfbarkeit**, nicht einen Negativbefund.

Der Unterschied ist wesentlich: Eine fehlende Quelle bedeutet nicht, dass eine Beziehung nicht existiert — sie bedeutet, dass sie nicht festgestellt werden kann.

End of Document

DOC-ETM-0005

Traceability Versioning

Quelle: docs/traceability/Traceability-Versioning.md

Traceability Versioning

Document ID

DOC-ETM-0006

Version

1.0.0

Referenz

ESS-0001-CONTRACTS Chapter 9, ESS-0011 Chapter 4

Zweck

Versionierung der Matrix und ihr Beitrag zur Versionsbewertung der Plattform.

Matrixversion

Die Matrix besitzt eine eigene Version.

Sie wird erhöht bei

- neuen Verknüpfungsarten
- neuen Achsen
- strukturellen Änderungen des Matrixmodells
- geänderter Prüfsummenbildung
- vollständigem Neuaufbau

Jeder Matrixstand speichert zusätzlich

Repository Version
Knowledge Version
Architecture Version
Twin Version

Beitrag zur Versionsbewertung

Der Version Manager verwendet die ETM als **Regelachse** der Versionsbewertung.

Feststellung der ETM	Versionsauswirkung
Interface entfällt, Verknüpfung IMPLEMENTS bricht	Major
Event entfällt, Verknüpfung EMITS bricht	Major
neue Komponente mit vollständiger Verknüpfung	Minor
neue Verknüpfung ohne Strukturänderung	Patch
Verknüpfung fehlt bei bestehender Regel	blockiert Release

Die ETM **empfiehlt keine Version**. Sie liefert die Feststellungen, aus denen der Version Manager die Version ableitet.

Breaking-Change-Erkennung

Ein Breaking Change liegt aus ETM-Sicht vor, wenn eine zuvor bestehende Verknüpfung entfällt, ohne dass ein Ersatz erzeugt wurde.

Beispiel

```
Version n    interface:platform/IKnowledgeBuilder <-IMPLEMENTS- component:platform/Knowledge
Version n+1  Verknüpfung fehlt, kein SUPERSEDES vorhanden
              → Breaking Change
```

Ein **SUPERSEDES** auf ein Nachfolgeartefakt verhindert die Einstufung.

Snapshot je Release

Zu jedem Release wird ein Matrixstand gesichert.

Dadurch ist rückwirkend nachvollziehbar, welche Regel zum Releasezeitpunkt durch welches Artefakt erfüllt war.

End of Document

DOC-ETM-0006